



01-DARS · 14 TADAN

Web qanday ishlaydi

Internet, mijoz-server modeli, DNS, HTTP protokoli, so'rov va javoblar, status kodlari hamda brauzer sahifani qanday chizishi.

Boshlang'ich · ~60 daqiqa

Kod yozishni boshlashdan oldin eng muhim savolga javob topamiz: **brauzerga** `google.com` **deb yozib** `Enter` **bosganingizda aslida nima sodir bo'ladi?** Bu savolga chuqur javob bera olgan dasturchi frontend, backend, tarmoq va xavfsizlik muammolarining aksariyatini tez tushunadi. Ushbu dars — qolgan barcha darslarning poydevori.

Internet va Web — farqi nimada?

Ko'pchilik bu ikki so'zni sinonim deb o'ylaydi, lekin ular turli narsalar:

- **Internet** — butun dunyodagi kompyuterlarni bir-biriga bog'lovchi *jismoniy va mantiqiy tarmoq* (kabellar, routerlar, Wi-Fi, mobil tarmoqlar, TCP/IP protokollari). U 1960–70-yillarda ARPANET loyihasidan boshlangan.
- **Web (World Wide Web)** — Internet *ustida* ishlaydigan xizmatlardan biri: bir-biriga havolalar orqali bog'langan hujjatlar (sahifalar) tizimi. Uni 1989-yilda Tim Berners-Li CERN'da ixtiro qilgan va uchta asosiy texnologiyani yaratgan: **HTML** (hujjat tili), **URL** (manzil) va **HTTP** (uzatish protokoli).

Internet ustida boshqa xizmatlar ham bor: elektron pochta (SMTP), fayl uzatish (FTP), onlayn o'yinlar, video qo'ng'iroqlar va hokazo. Web — ulardan faqat bittasi, lekin eng mashhuri.

Ma'lumot tarmoqda qanday harakatlanadi?

Siz yuborgan har qanday ma'lumot (rasm, matn, video) kichik bo'laklarga — **paketlarga** bo'linadi. Har bir paketda qabul qiluvchining **IP-manzili** yoziladi va paketlar routerlar orqali turli yo'llar bilan manzilga yetib boradi, u yerda yana birlashtiriladi. Bu jarayonni protokollar qatlamlari boshqaradi:

Qatlam	Protokol misollari	Vazifasi

Ilova (Application)	HTTP, HTTPS, DNS, SMTP, WebSocket	Dasturlar nima haqida "gaplashishini" belgilaydi
Transport	TCP, UDP	Ma'lumot to'liq va to'g'ri tartibda yetib borishini kafolatlaydi (TCP)
Tarmoq (Internet)	IP (IPv4, IPv6)	Paketni qaysi kompyuterga yetkazishni belgilaydi
Kanal (Link)	Ethernet, Wi-Fi	Ma'lumotni jismoniy muhit orqali uzatadi



Veb-dasturchi uchun nima muhim?

Siz asosan eng yuqori qatlamda — **HTTP** bilan ishlaysiz. Lekin TCP ulanish o'rnatish vaqt olishini, IP-manzil nima ekanini va DNS qanday ishlashini bilish sahifa sekin yuklanishi yoki "server topilmadi" kabi muammolarni tushunishda juda yordam beradi.

Mijoz-server (Client-Server) modeli

Web **mijoz-server** arxitekturasiga asoslangan. Bunda ikki tomon bor:

- **Mijoz (client)** — so'rov yuboruvchi dastur. Ko'pincha bu brauzer, lekin mobil ilova, `curl` buyrug'i yoki boshqa server ham mijoz bo'lishi mumkin.
- **Server** — so'rovlarni kutib turadigan va ularga javob qaytaradigan dastur (va u ishlayotgan kompyuter). Server 24/7 yoqilgan bo'ladi va bir vaqtda minglab mijozlarga xizmat qiladi.



Mijoz (brauzer)

— 1. So'rov (Request) →



Server

← 2. Javob (Response) —



Mijoz

Muloqotni har doim mijoz boshlaydi; server faqat javob beradi

Bu modelning muhim xususiyati: **muloqotni doim mijoz boshlaydi**. Server o'zi mijozga "birinchi bo'lib" xabar yubora olmaydi (oddiy HTTP'da). Real vaqtdagi chatlar uchun WebSocket kabi qo'shimcha texnologiyalar ishlatilishining sababi ham shu.

Statik va dinamik sahifalar

- **Statik sayt** — server diskdagi tayyor fayllarni (`.html` , `.css` , rasmlar) o'zgartirmasdan yuboradi. Har bir foydalanuvchi bir xil sahifani ko'radi. Masalan: portfolio, hujjatlar sayti (aynan siz o'qiyotgan ushbu qo'llanma ham statik sayt).
- **Dinamik sayt** — server har bir so'rov uchun javobni dastur orqali *hosil qiladi*: ma'lumotlar bazasiga murojaat qiladi, foydalanuvchini aniqlaydi, hisob-kitob qiladi. Masalan: Instagram lentasi har bir odam uchun boshqacha.

URL tuzilishi

URL (Uniform Resource Locator) — internetdagi resursning to'liq manzili. Uni qismlarga ajratib ko'ramiz:

Matn

`https://shop.example.uz:443/products/phones?brand=samsung&sort=price#reviews`

schema

host

port

yo'l (path)

so'rov satri (query)

fragment

Qism	Misol	Ma'nosi
Sxema (protocol)	<code>https</code>	Qaysi protokol bilan ulanish. <code>https</code> = shifrlangan HTTP
Host (domen)	<code>shop.example.uz</code>	Server nomi. <code>shop</code> — subdomen, <code>example</code> — domen, <code>uz</code> — yuqori darajali domen (TLD)
Port	<code>443</code>	Serverdagi "eshik" raqami. HTTP uchun standart 80, HTTPS uchun 443 (yozilmasa ham shu nazarda tutiladi)
Yo'l (path)	<code>/products/phones</code>	Server ichidagi aniq resurs
Query	<code>?brand=samsung&sort=price</code>	Qo'shimcha parametrlar: <code>kalit=qiymat</code> juftliklari, <code>&</code> bilan ajratiladi
Fragment	<code>#reviews</code>	Sahifa ichidagi bo'lim. Serverga yuborilmaydi , faqat brauzerda ishlaydi

Full-stack qo'llanma

3 / 341



URL kodlash (encoding)

URL'da bo'sh joy, kirill harflari yoki maxsus belgilar bevosita yozilmaydi — ular `%` bilan kodlanadi. Masalan, bo'sh joy `%20` ga, `o'` esa `o%27` ga aylanadi. JavaScript'da buning uchun `encodeURIComponent("salom dunyo")` funksiyasi bor.

DNS — internetning telefon kitobi

Kompyuterlar bir-birini **IP-manzil** orqali topadi (masalan `142.250.74.46`). Odamlar uchun esa `google.com` kabi nomlarni eslab qolish oson. **DNS (Domain Name System)** — domen nomini IP-manzilga aylantiruvchi tizim.

DNS so'rovi bosqichma-bosqich

- Brauzer keshi** — brauzer bu domenni yaqinda ko'rganmi? Ha bo'lsa, tayyor IP ishlatiladi.
- Operatsion tizim keshi** va `hosts` fayli tekshiriladi.
- Rekursiv resolver** — odatda internet provayderingiz yoki `8.8.8.8` (Google), `1.1.1.1` (Cloudflare) serveri. U javobni topish uchun quyidagilardan so'raydi:
- Root server** → "`.com` zonasini qaysi server shug'ullanadi?"
- TLD server** (`.com`) → "`google.com` uchun mas'ul (authoritative) server qaysi?"
- Authoritative server** → "`google.com` ning IP-manzili: `142.250.74.46`".
- Javob keshlanadi (TTL — "yashash vaqti" davomida) va brauzerga qaytariladi.

Asosiy DNS yozuvlari

Yozuv	Vazifasi	Misol
A	Domen → IPv4 manzil	<code>example.uz</code> → <code>93.184.216.34</code>
AAAA	Domen → IPv6 manzil	<code>example.uz</code> → <code>2606:2800:220:1::</code>
CNAME	Domen → boshqa domen (taxallus)	<code>www.example.uz</code> → <code>example.uz</code>
MX	Pochta serveri	<code>example.uz</code> → <code>mail.example.uz</code>

TXT

Ixtiyoriy matn (domen egaligini tasdiqlash
va h.k.)

"v=spf1 include:_spf.google.com
~all"

Sayingizni hostingga joylaganingizda (14-dars) aynan shu yozuvlarni domen sozlamalarida kiritasiz.

HTTP protokoli chuqur

HTTP (HyperText Transfer Protocol) — mijoz va server o'rtasidagi muloqot qoidalari. U **matnli** (HTTP/1.1 da) va **holatsiz (stateless)** protokol: har bir so'rov mustaqil, server oldingi so'rovni "eslamaydi". Foydalanuvchi tizimga kirganini eslab qolish uchun cookie va tokenlar ishlatiladi (11-dars).

HTTP so'rovi (Request) tuzilishi

HTTP

```
POST /api/users HTTP/1.1
Host: api.example.uz
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: application/json
Content-Length: 46

{"name": "Aziza", "email": "aziza@mail.uz"}
```

1. **Boshlang'ich qator:** *metod* (`POST`), *yo'l* (`/api/users`), *versiya* (`HTTP/1.1`).
2. **Sarlavhalar (headers):** `Nom: qiymat` ko'rinishidagi meta-ma'lumotlar.
3. **Bo'sh qator** — sarlavhalar tugaganini bildiradi.
4. **Tana (body)** — yuborilayotgan ma'lumot (GET so'rovlarida odatda bo'lmaydi).

HTTP javobi (Response) tuzilishi

```
HTTP

HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /api/users/57
Set-Cookie: session=abc123; HttpOnly; Secure
Cache-Control: no-store
Date: Sat, 03 Oct 2026 10:15:00 GMT

{"id": 57, "name": "Aziza", "email": "aziza@mail.uz"}
```

HTTP metodlari

Metod	Ma'nosi	Tana bormi?	Xavfsiz?	Idempotent?
GET	Resursni o'qish	Yo'q	Ha	Ha
POST	Yangi resurs yaratish / amal bajarish	Ha	Yo'q	Yo'q
PUT	Resursni to'liq almashtirish	Ha	Yo'q	Ha
PATCH	Resursni qisman yangilash	Ha	Yo'q	Odatda yo'q
DELETE	Resursni o'chirish	Odatda yo'q	Yo'q	Ha
HEAD	GET kabi, lekin faqat sarlavhalar	Yo'q	Ha	Ha
OPTIONS	Server qaysi metodlarni qabul qilishini so'rash (CORS)	Yo'q	Ha	Ha

- **Xavfsiz (safe)** — serverdagi ma'lumotni o'zgartirmaydi.
- **Idempotent** — bir xil so'rovni 1 marta yoki 10 marta yuborish natijasi bir xil. Masalan, `DELETE /users/5` ni ikki marta yuborsangiz ham foydalanuvchi bir marta o'chadi. `POST /orders` ni ikki marta yuborsangiz esa ikkita buyurtma yaratiladi — shuning uchun "To'lash" tugmasini ikki marta bosish xavfli.

Status kodlari

Server javobining birinchi qatoridagi uch xonali raqam so'rov natijasini bildiradi. Birinchi raqam — guruh:

Kod	Ma'nosi	Qachon ishlatiladi
2xx — Muvaffaqiyat		
200 OK	Hammasi joyida	GET, PUT, PATCH muvaffaqiyatli
201 Created	Yangi resurs yaratildi	POST orqali foydalanuvchi qo'shildi
204 No Content	Bajarildi, qaytaradigan narsa yo'q	DELETE muvaffaqiyatli
3xx — Yo'naltirish		
301 Moved Permanently	Manzil butunlay o'zgargan	http:// → https://
302 Found	Vaqtincha boshqa manzil	Login'dan keyin bosh sahifaga
304 Not Modified	Keshdagi nusxa hali yangi	Brauzer faylni qayta yuklamaydi
4xx — Mijoz xatosi		
400 Bad Request	So'rov noto'g'ri	Email maydoni bo'sh yuborildi
401 Unauthorized	Tizimga kirilmagan	Token yo'q yoki eskirgan
403 Forbidden	Kirilgan, lekin ruxsat yo'q	Oddiy foydalanuvchi admin sahifasiga
404 Not Found	Resurs topilmadi	/users/9999 mavjud emas
409 Conflict	Holat ziddiyati	Bu email allaqachon ro'yxatdan o'tgan
422 Unprocessable Entity	Ma'lumot validatsiyadan o'tmadi	Parol juda qisqa

429 Too Many Requests	Juda ko'p so'rov	Rate limit oshib ketdi
5xx — Server xatosi		
500 Internal Server Error	Serverda kutilmagan xato	Kodda bug
502 Bad Gateway	Proksi orqadagi serverdan javob ololmadi	Node.js ilova o'chib qolgan, Nginx ishlayapti
503 Service Unavailable	Server vaqtincha band	Texnik ishlar, haddan tashqari yuklama



Eslab qolish uchun

4xx — "sen xato qilding" (mijoz), 5xx — "men xato qildim" (server). API yozganda to'g'ri status kod qaytarish — professional backend dasturchining belgisi.

Muhim sarlavhalar (headers)

Sarlavha	Yo'nalish	Vazifasi
Content-Type	Ikkalasi	Tana formati: text/html, application/json, multipart/form-data
Authorization	So'rov	Kimligingizni isbotlash: Bearer <token>
Cookie / Set-Cookie	So'rov / Javob	Brauzerda kichik ma'lumot saqlash (sessiya)
Cache-Control	Javob	Javobni qancha vaqt keshlash mumkin
Location	Javob	Yo'naltirish manzili yoki yangi resurs manzili
Access-Control-Allow-Origin	Javob	CORS: qaysi saytlar bu API'ga murojaat qila oladi
User-Agent	So'rov	Mijoz dasturi haqida ma'lumot

HTTPS va TLS

Oddiy HTTP'da ma'lumot **ochiq matn** sifatida uzatiladi — Wi-Fi tarmog'idagi har qanday odam parolingizni o'qiy oladi. **HTTPS** = HTTP + **TLS** shifrlash. TLS uchta narsani kafolatlaydi:

- **Maxfiylik** — ma'lumotni yo'lda hech kim o'qiy olmaydi;
- **Yaxlitlik** — ma'lumotni yo'lda hech kim o'zgartira olmaydi;
- **Haqiqiylik** — siz haqiqatan `bank.uz` bilan gaplashyapsiz (SSL sertifikat orqali).

Bugungi kunda har qanday sayt HTTPS'da bo'lishi shart. Let's Encrypt bepul sertifikat beradi.

HTTP versiyalari

- **HTTP/1.1** (1997) — matnli, bitta ulanishda so'rovlar ketma-ket ketadi.
- **HTTP/2** (2015) — ikkilik format, bitta ulanishda ko'plab so'rovlar parallel (multiplexing), sarlavhalar siqiladi.
- **HTTP/3** (2022) — TCP o'rniga QUIC (UDP asosida), mobil tarmoqlarda tezroq.

Dasturchi sifatida siz yozadigan kod deyarli o'zgarmaydi — metodlar, status kodlar va sarlavhalar ma'nosi bir xil.

Brauzer sahifani qanday chizadi?

Server HTML qaytargandan keyin brauzer ichida murakkab jarayon boshlanadi — buni **Critical Rendering Path** deyiladi:

1. **HTML parsing → DOM**. Brauzer HTML matnini o'qib, teglar daraxtini — **DOM (Document Object Model)** ni quradi.
2. **CSS parsing → CSSOM**. Barcha CSS qoidalaridan uslublar daraxti quriladi.
3. **Render tree**. DOM va CSSOM birlashtiriladi; ko'rinmas elementlar (`display:none` , `<head>`) tashlab yuboriladi.
4. **Layout (reflow)**. Har bir elementning ekrandagi aniq o'lchami va joylashuvi hisoblanadi.
5. **Paint**. Piksellar chiziladi: ranglar, matn, soyalar, rasmlar.
6. **Composite**. Qatlamlar birlashtirilib ekranga chiqariladi.



Nega bu muhim?

`<script>` tegi HTML parsing'ni **to'xtatib qo'yadi** — brauzer skriptni yuklab, bajarib bo'lgunicha qolgan HTML'ni o'qimaydi. Shuning uchun skriptlarni `<body>` oxiriga qo'yish yoki `defer` atributidan foydalanish tavsiya qilinadi. CSS esa chizishni bloklaydi — shuning uchun u `<head>` ichida bo'ladi.

To'liq zanjir: Enter bosilgandan sahifa chiqqungacha

1. Brauzer URL'ni tahlil qiladi.
2. **DNS** orqali domen IP-manzilga aylantiriladi.
3. Server bilan **TCP ulanish** o'rnatiladi (3 bosqichli "qo'l siqish": SYN → SYN-ACK → ACK).
4. HTTPS bo'lsa — **TLS handshake**: shifrlash kalitlari kelishiladi, sertifikat tekshiriladi.
5. **HTTP so'rov** yuboriladi: `GET / HTTP/1.1`.
6. Server so'rovni qayta ishlaydi (balki bazaga murojaat qiladi) va **HTML javob** qaytaradi.
7. Brauzer HTML'ni o'qiy boshlaydi, unda CSS, JS, rasm havolalarini ko'rib, ular uchun **qo'shimcha so'rovlar** yuboradi.
8. DOM + CSSOM → Layout → Paint → sahifa ekranda.
9. JavaScript bajarilib, sahifa interaktiv bo'ladi.

Frontend, Backend va Full-stack

	Frontend	Backend
Qayerda ishlaydi	Foydalanuvchi brauzerida	Serverda
Texnologiyalar	HTML, CSS, JavaScript, React/Vue/Angular	Node.js, Python, PHP, Go, Java + SQL/NoSQL bazalar
Vazifasi	Interfeys, animatsiya, foydalanuvchi bilan muloqot	Ma'lumot saqlash, biznes-mantiq, xavfsizlik, API
Kodni kim ko'radi	Har kim (DevTools orqali)	Faqat siz



Oltin qoida

Frontend kodiga **hech qachon ishonmang**. Foydalanuvchi brauzerdagi JavaScript'ni o'zgartirishi, tekshiruvlarni chetlab o'tishi va serverga istalgan so'rovni yuborishi mumkin. Narx, ruxsatlar, validatsiya — barchasi **serverda** qayta tekshirilishi shart.

Amaliy misollar

MISOL 1

DevTools orqali real saytning so'rovlarini tahlil qilish

Har bir veb-dasturchining eng yaqin do'sti — brauzerning **Developer Tools** paneli.

1. Chrome'da istalgan saytni oching (masalan, `https://kun.uz`).
2. `F12` (yoki `Ctrl + Shift + I`) bosib va **Network** yorlig'iga o'ting.
3. Sahifani `Ctrl + R` bilan yangilang.
4. Ro'yxatdagi birinchi so'rovni (sahifaning o'zi, turi `document`) bosib.

Siz quyidagilarni ko'rasiz:

- **Headers** yorlig'i — Request URL, Request Method (`GET`), Status Code (`200`), Remote Address (serverning IP-manzili va porti), barcha so'rov va javob sarlavhalari.
- **Response** — server qaytargan xom HTML.
- **Timing** — vaqt qayerga ketgani: DNS Lookup, Initial connection (TCP), SSL, Waiting for server response (TTFB), Content Download.

Pastki satrda jami so'rovlar soni (odatda 50–150 ta!), yuklangan hajm va **DOMContentLoaded / Load** vaqtlari ko'rsatiladi. **Type** ustuni bo'yicha saralab, sahifa nechta rasm, skript va CSS yuklayotganini ko'ring.



"Disable cache" belgisini qo'yib, sahifani qayta yuklang — keshsiz yuklanish qancha uzoq davom etishini taqqoslang. Network panelidagi "Throttling" orqali sekin 3G internetni ham sinab ko'rish mumkin.

`curl` — terminaldan HTTP so'rov yuboruvchi dastur. U Windows 10+, macOS va Linux'da o'rnatilgan bo'ladi. Brauzer "yashiradigan" narsalarni ko'rishning eng yaxshi usuli.

Terminal

```
# 1. Oddiy GET so'rov – javob tanasini chiqaradi
curl https://jsonplaceholder.typicode.com/users/1

# 2. -i : javob sarlavhalarini ham ko'rsatish
curl -i https://jsonplaceholder.typicode.com/users/1

# 3. -v : to'liq jarayon (DNS, TCP, TLS, so'rov va javob sarlavhalari)
curl -v https://example.com 2>&1 | head -40

# 4. Yo'naltirishni ko'rish (301) va unga ergashish (-L)
curl -I http://github.com
curl -IL http://github.com

# 5. POST so'rov: JSON yuborish
curl -X POST https://jsonplaceholder.typicode.com/posts \
  -H "Content-Type: application/json" \
  -d '{"title": "Salom", "body": "Birinchi post", "userId": 1}'

# 6. Mavjud bo'lmagan resurs – 404 ni ko'rish
curl -i https://jsonplaceholder.typicode.com/users/9999
```

5-buyruq natijasi:

JSON

```
{
  "title": "Salom",
  "body": "Birinchi post",
  "userId": 1,
  "id": 101
}
```

E'tibor bering: server `201 Created` qaytardi va yangi resursga `id` berdi. Aynan shunday API'ni siz 9-darsda o'zingiz yozasiz.

MISOL 3 DNS so'rovini qo'lda tekshirish

Terminal

```
# Domen IP-manzilini bilish (Windows, macOS, Linux)
nslookup github.com

# Batafsil (macOS / Linux): A, MX, TXT yozuvlari
dig github.com A +short
dig gmail.com MX +short
dig google.com TXT +short

# Ma'lum DNS server orqali so'rash (Cloudflare 1.1.1.1)
nslookup kun.uz 1.1.1.1

# Paket serverga qaysi routerlar orqali borayotganini ko'rish
tracert google.com      # macOS / Linux
tracert google.com      # Windows
```

Tajriba: o'z kompyuteringizdagi `hosts` faylini oching (Windows:

`C:\Windows\System32\drivers\etc\hosts`, macOS/Linux: `/etc/hosts`) va administrator huquqi bilan quyidagi qatorni qo'shing:

Matn

```
127.0.0.1   mening-saytim.test
```

Endi brauzerda `http://mening-saytim.test` manzili sizning o'z kompyuteringizga (`localhost`) yo'naladi — chunki `hosts` fayli DNS'dan oldin tekshiriladi. Tajribadan keyin qatorni o'chirib qo'ying.

MISOL 4 20 qatorda o'z HTTP serveringizni yozish

Server — bu sehr emas, shunchaki portni tinglab turuvchi dastur. Buni isbotlash uchun Node.js'ning o'rnatilgan `http` moduli bilan oddiy server yozamiz (Node.js 8-darsda batafsil o'rganiladi; hozir faqat o'rnatish).

server.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  // Har bir kelgan so'rovni terminalga chiqaramiz
  console.log(`${req.method} ${req.url}`);
  console.log('Sarlavhalar:', req.headers);

  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
    res.end('<h1>Salom! Bu mening birinchi serverim</h1><a  
href="/api/vaqt">Vaqtni bilish</a>');
  } else if (req.url === '/api/vaqt') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ vaqt: new Date().toISOString() }));
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end('404 – Sahifa topilmadi');
  }
});

server.listen(3000, () => console.log('Server http://localhost:3000 da  
ishlayapti'));
```

Ishga tushirish:

Terminal

```
node server.js
# Boshqa terminalda:
curl -i http://localhost:3000/api/vaqt
curl -i http://localhost:3000/yoq-sahifa
```

Brauzerda `http://localhost:3000` ni oching va server terminaliga qarang: siz bitta sahifa ochgan bo'lsangiz ham, brauzer `/favicon.ico` uchun ham avtomatik so'rov yuborganini ko'rasiz. Endi siz mijoz (brauzer) va server o'rtasidagi muloqotni ikkala tomondan ham ko'radingiz.

Mashqlar



Mustaqil ish

1. `https://www.youtube.com/watch?v=dQw4w9WgXcQ&t=42s` URL'ini qismlarga ajrating: sxema, host, subdomen, yo'l, query parametrlari.
2. DevTools orqali uchta turli saytda nechta so'rov yuborilishini va sahifa necha soniyada yuklanishini solishtiring. Eng ko'p joy egallagan fayl turi qaysi?
3. `curl -i` orqali `http://google.com` ga so'rov yuboring. Qanday status kod va `Location` sarlavhasi qaytdi? Nega?
4. 4-misoldagi serverga `/api/salom?ism=Ali` marshrutini qo'shing, u `{"xabar": "Salom, Ali!"}` qaytarsin. (Maslahat: `new URL(req.url, 'http://localhost').searchParams.get('ism')`)
5. Quyidagi holatlar uchun to'g'ri status kodni tanlang: (a) parol noto'g'ri; (b) foydalanuvchi boshqa birovning buyurtmasini o'chirmoqchi; (c) yangi maqola yaratildi; (d) ma'lumotlar bazasi bilan aloqa uzildi.

▼ 5-savol javobi

(a) `401` ; (b) `403` (yoki ma'lumotni yashirish uchun `404`); (c) `201` ; (d) `500` yoki `503` .



Xulosa

- Internet — tarmoq; Web — uning ustidagi HTML + URL + HTTP tizimi.
- Mijoz so'rov yuboradi, server javob qaytaradi; muloqotni doim mijoz boshlaydi.
- DNS domen nomini IP-manzilga aylantiradi; natijalar keshlanadi.
- HTTP so'rovi: metod + yo'l + sarlavhalar + tana. Javob: status kod + sarlavhalar + tana.
- Status kodlar: 2xx muvaffaqiyat, 3xx yo'naltirish, 4xx mijoz xatosi, 5xx server xatosi.
- Brauzer HTML → DOM, CSS → CSSOM, keyin Layout va Paint orqali sahifani chizadi.

- Serverda doim qayta tekshiring — frontendga ishonib bo'lmaydi.

☐ Darsni o'qib bo'ldim



02-DARS · 14 TADAN

HTML — sahifa tuzilishi

Teglar, atributlar, semantik HTML, formalar, jadvallar, media, accessibility va SEO uchun meta-teglar.

Boshlang'ich · ~90 daqiqa

HTML (HyperText Markup Language) — veb-sahifaning skeleti. U dasturlash tili emas, balki **belgilash (markup) tili**: biz matnning qaysi qismi sarlavha, qaysisi paragraf, qaysisi havola yoki tugma ekanini brauzerga tushuntiramiz. Yaxshi yozilgan HTML — tez ishlaydigan, qidiruv tizimlarida yuqorida chiqadigan va ko'zi oqiz foydalanuvchilar ham bemaolol ishlata oladigan saytning asosi.

Element, teg va atribut

HTML hujjati **elementlardan** tashkil topadi. Element odatda ochuvchi teg, mazmun va yopuvchi tegdan iborat:

HTML

```
<a href="https://kun.uz" target="_blank" title="Yangiliklar">Kun.uz saytiga  
o'tish</a>
```

←! — | ochuvchi teg | | atributlar | | mazmun
| | yopuvchi | →

- **Teg** — burchak qavslar ichidagi nom: `<p>`, `</p>`.
- **Atribut** — elementga qo'shimcha ma'lumot: `nom="qiymat"`. Har doim ochuvchi tegda yoziladi.
- **Bo'sh (void) elementlar** — mazmuni bo'lmaydi va yopilmaydi: `<img>`, `<br>`, `<input>`, `<meta>`, `<link>`, `<hr>`.
- **Ichma-ich joylashish (nesting)** — elementlar to'g'ri tartibda yopilishi kerak: `<p><b>matn</b></p>` ✓, `<p><b>matn</p></b>` ✗.

Global atributlar

Istalgan elementda ishlatiladigan atributlar:

Atribut	Vazifasi
id	Sahifada yagona identifikator. CSS, JS va #havoLa uchun
class	Bir yoki bir nechta sinf nomi (bo'sh joy bilan). CSS uslublari uchun asosiy vosita
style	Inline CSS (kamdan-kam ishlatilg)
title	Sichqoncha ustiga olib borilganda chiqadigan izoh
hidden	Elementni yashiradi
data-*	O'z ma'lumotingizni saqlash: data-product-id="42" → JS'da el.dataset.productId
lang	Mazmun tili: lang="uz"
tabindex	Klaviatura (Tab) bilan fokus tartibi

HTML hujjatining asosiy tuzilishi

Har bir HTML sahifa quyidagi skeletdan boshlanadi. VS Code'da bo'sh .html faylga ! yozib Tab bossangiz, u avtomatik yaratiladi.

```

<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Sahifa haqida 150-160 belgili qisqa tavsif
(Google natijalarida chiqadi)">
  <title>Sahifa sarlavhasi – Sayt nomi</title>
  <link rel="icon" href="/favicon.ico">
  <link rel="stylesheet" href="css/style.css">
  <script src="js/app.js" defer></script>
</head>
<body>
  <!-- Foydalanuvchi ko'radigan barcha narsa shu yerda -->
</body>
</html>

```

- `<!DOCTYPE html>` — brauzerga "bu zamonaviy HTML5 hujjat" deydi. Usiz brauzer eski "quirks mode"ga o'tadi va CSS g'alati ishlaydi.
- `<html lang="uz">` — ildiz element; `lang` ekran o'quvchilar va tarjimonlar uchun muhim.
- `<head>` — ko'rinmaydigan meta-ma'lumotlar: kodlash, sarlavha, CSS, skriptlar.
- `<meta charset="UTF-8">` — o'zbekcha `o'`, `g'`, kirill harflari va emoji to'g'ri chiqishi uchun. Har doim birinchi qatorlarda.
- `<meta name="viewport">` — telefonlarda sahifa kichraytirilmasdan, ekran kengligida ko'rsatilishi uchun. Usiz responsiv dizayn ishlamaydi.
- `defer` — skript HTML to'liq o'qilgandan keyin bajariladi va parsing'ni bloklamaydi.

Matn bilan ishlash

Sarlavhalar

`<h1>` dan `<h6>` gacha. Ular hujjatning **mazmun tuzilishini** (kitobning mundarijasi kabi) bildiradi, shrift o'lchamini emas.

- Sahifada odatda **bitta** `<h1>` bo'ladi — sahifaning asosiy mavzusi.
- Darajalarni sakrab o'tmang: `h2` dan keyin to'g'ridan-to'g'ri `h4` emas.

- Matnni katta qilish uchun sarlavha tegi ishlatmang — buning uchun CSS bor.

Matn elementlari

HTML

```
<p>Paragraf – matnning asosiy bloki.</p>

<p>
  <strong>Muhim matn</strong> (semantik qalin) va <b>shunchaki qalin</b>.<br>
  <em>Urg'u berilgan matn</em> (semantik kursiv) va <i>atama yoki chet
  so'z</i>.<br>
  <mark>Belgilangan</mark>, <small>mayda izoh</small>, <del>o'chirilgan</del>,
  <ins>qo'shilgan</ins>.<br>
  H<sub>2</sub>O va E = mc<sup>2</sup><br>
  <code>console.log()</code> – kod, <kbd>Ctrl</kbd> + <kbd>C</kbd> –
  klaviatura.<br>
  <abbr title="HyperText Markup Language">HTML</abbr> – qisqartma.
</p>

<blockquote cite="https://example.uz/manba">
  <p>Dasturlash – bu fikrlashni o'rgatadigan san'at.</p>
  <footer>– <cite>Noma'lum muallif</cite></footer>
</blockquote>

<pre>
  Formatlangan matn:
    bo'sh joylar    saqlanadi
</pre>

<hr> <!-- mavzu almashuvi -->
```



strong va b farqi

`<strong>` — "bu matn muhim" degan *ma'no* beradi (ekran o'quvchi ovozini o'zgartiradi). `<b>` — faqat vizual qalinlik. Xuddi shunday `<em>` (urg'u) va `<i>` (stil).

Ro'yxatlar

HTML

```
<!-- Tartibsiz ro'yxat -->
<ul>
  <li>Non</li>
  <li>Sut</li>
</ul>

<!-- Tartibli ro'yxat: type="1|a|A|i|I", start, reversed -->
<ol start="3">
  <li>Uchinchi qadam</li>
  <li>To'rtinchi qadam
    <ul>
      <li>Ichki ro'yxat</li>
    </ul>
  </li>
</ol>

<!-- Ta'riflar ro'yxati -->
<dl>
  <dt>HTML</dt>
  <dd>Sahifa tuzilishi</dd>
  <dt>CSS</dt>
  <dd>Sahifa ko'rinishi</dd>
</dl>
```

Havolalar

HTML

```
<a href="https://developer.mozilla.org">Tashqi havola (absolyut URL)</a>
<a href="/about.html">Saytning ildizidan (root-relative)</a>
<a href="../index.html">Joriy papkadan bir yuqoriga (relative)</a>
<a href="#aloqa">Sahifadagi id="aloqa" elementiga</a>
<a href="mailto:info@example.uz?subject=Savol">Email yozish</a>
<a href="tel:+998901234567">Qo'ng'iroq qilish</a>
<a href="/files/narxlar.pdf" download>PDF ni yuklab olish</a>

<!-- Yangi oynada ochish: rel="noopener noreferrer" xavfsizlik uchun -->
<a href="https://github.com" target="_blank" rel="noopener
noreferrer">GitHub</a>
```

Rasmlar va media

HTML

```


<!-- Rasm + izoh -->
<figure>
    
    <figcaption>1-rasm. Oylik savdo hajmi</figcaption>
</figure>

<!-- Ekran o'lchamiga qarab turli rasm (responsive images) -->


<!-- Zamonaviy format + zaxira -->
<picture>
    <source srcset="hero.avif" type="image/avif">
    <source srcset="hero.webp" type="image/webp">
    
</picture>

<video src="intro.mp4" controls poster="poster.jpg" width="640" muted>
    Brauzeringiz video tegini qo'llab-quvvatlamaydi.
</video>

<audio src="podcast.mp3" controls></audio>

<iframe src="https://www.youtube.com/embed/VIDEO_ID" title="YouTube video"
    width="560" height="315" allowfullscreen loading="lazy"></iframe>
```



alt atributi majburiy

`alt` — rasm yuklanmasa yoki foydalanuvchi ko'ra olmasa (ekran o'quvchi) o'qiladigan matn. U rasmda *nima borligini* tasvirlashi kerak, "rasm" so'zini emas. Faqat bezak uchun ishlatilgan rasmlarga `alt=""` (bo'sh) qo'ying. `width` va `height` berilsa, rasm yuklanguncha joy band qilinadi va sahifa "sakramaydi".

Semantik HTML

Semantik — "ma'noli" degani. 2000-yillarda saytlar faqat `<div>` lardan qurilgan ("div sho'rva"). HTML5 sahifa qismlarining *vazifasini* bildiruvchi teglar qo'shdi:

Teg	Ma'nosi
<code><header></code>	Sahifa yoki bo'limning kirish qismi: logo, sarlavha, navigatsiya
<code><nav></code>	Asosiy navigatsiya havolalari bloki
<code><main></code>	Sahifaning asosiy, noyob mazmuni (sahifada bitta)
<code><article></code>	Mustaqil, alohida tarqatsa bo'ladigan mazmun: maqola, post, mahsulot kartasi, izoh
<code><section></code>	Mavzu bo'yicha guruhlangan qism (odatda sarlavhasi bilan)
<code><aside></code>	Asosiy mazmunga bilvosita aloqador: yon panel, reklama, "o'xshash maqolalar"
<code><footer></code>	Pastki qism: mualliflik, aloqa, havolalar
<code><time></code>	Sana/vaqt: <code><time datetime="2026-10-03">3-oktabr</time></code>
<code><address></code>	Aloqa ma'lumotlari
<code><div></code> , <code></code>	Ma'nosiz konteynerlar — faqat uslub berish uchun (blok va qator ichi)

Nega semantika muhim?

1. **SEO** — Google sahifaning qaysi qismi asosiy mazmun ekanini tushunadi.

2. **Accessibility (a11y)** — ekran o'quvchi dasturlar "navigatsiyaga o'tish", "asosiy mazmunga o'tish" imkonini beradi.

3. **O'qilishi** — kodni ko'rgan dasturchi tuzilmani darhol tushunadi.

Blok va qator ichi (inline) elementlar

- **Blok** elementlar (`div`, `p`, `h1-h6`, `ul`, `section`, `article`, `form`) yangi qatordan boshlanadi va butun kenglikni egallaydi.
- **Inline** elementlar (`span`, `a`, `strong`, `em`, `img`, `code`) matn oqimida qatorda turadi va faqat kerakli kenglikni egallaydi.

Bu xatti-harakatni CSS'dagi `display` xususiyati bilan o'zgartirish mumkin (3-dars).

Jadvallar

Jadvallar faqat **jadval ko'rinishidagi ma'lumotlar** uchun (narxlar, dars jadvali, statistika). Sahifa maketini jadval bilan qurish — 1990-yillar usuli, hozir CSS Grid/Flexbox ishlatiladi.

```

<table>
  <caption>2026-yil 1-chorak savdo hisoboti</caption>
  <thead>
    <tr>
      <th scope="col">Oy</th>
      <th scope="col">Buyurtmalar</th>
      <th scope="col">Daromad (so'm)</th>
    </tr>
  </thead>
  <tbody>
    <tr><th scope="row">Yanvar</th><td>120</td><td>36 000 000</td></tr>
    <tr><th scope="row">Fevral</th><td>145</td><td>43 500 000</td></tr>
    <tr><th scope="row">Mart</th><td>190</td><td>57 000 000</td></tr>
  </tbody>
  <tfoot>
    <tr><th scope="row">Jami</th><td>455</td><td>136 500 000</td></tr>
  </tfoot>
</table>

<!-- Kataknlarni birlashtirish -->
<table>
  <tr><th colspan="2">Ism va familiya</th><th rowspan="2">Yosh</th></tr>
  <tr><td>Ali</td><td>Valiyev</td></tr>
</table>

```

Formalar — foydalanuvchidan ma'lumot olish

Formalar — frontend va backend o'rtasidagi ko'prik. Ro'yxatdan o'tish, qidiruv, buyurtma — barchasi forma orqali.

HTML

```
<form action="/api/register" method="POST">
  <label for="email">Email</label>
  <input type="email" id="email" name="email" required
placeholder="siz@mail.uz">

  <label for="password">Parol</label>
  <input type="password" id="password" name="password" required minlength="8">

  <button type="submit">Ro'yxatdan o'tish</button>
</form>
```

- `action` — ma'lumot yuboriladigan URL. `method` — `GET` (ma'lumot URL'ga qo'shiladi, qidiruv uchun) yoki `POST` (tanada yuboriladi, maxfiy ma'lumotlar uchun).
- `name` — **eng muhim atribut**: serverga `name=qiymat` ko'rinishida yuboriladi. `name` siz maydon yuborilmaydi!
- `<label for="id">` — yozuvni maydonga bog'laydi: yozuv bosilganda maydon fokuslanadi va ekran o'quvchi uni o'qiydi.

Input turlari

type	Ko'rinishi / vazifasi
<code>text</code>	Oddiy matn
<code>email</code>	Email formatini tekshiradi; telefonda <code>@</code> bor klaviatura chiqadi
<code>password</code>	Belgilar yashiriladi
<code>number</code>	Raqam; <code>min</code> , <code>max</code> , <code>step</code>
<code>tel</code> , <code>url</code> , <code>search</code>	Telefon, manzil, qidiruv
<code>date</code> , <code>time</code> , <code>datetime-local</code>	Sana/vaqt tanlagich
<code>checkbox</code>	Bir nechtasini belgilash mumkin
<code>radio</code>	Bir guruhdan (bir xil <code>name</code>) faqat bittasi

<code>file</code>	Fayl tanlash; <code>accept="image/*"</code> , <code>multiple</code>
<code>range</code> , <code>color</code>	Slayder, rang tanlagich
<code>hidden</code>	Ko'rinmas maydon (masalan, mahsulot id)

Brauzerning o'rnatilgan validatsiyasi

- `required` — majburiy maydon
- `minlength`, `maxlength` — matn uzunligi
- `min`, `max` — son/sana chegarasi
- `pattern="[0-9]{9}"` — regex bo'yicha tekshirish



Brauzer validatsiyasi faqat foydalanuvchiga qulaylik uchun. Uni DevTools orqali osongina o'chirish mumkin. **Server tomonda albatta qayta tekshiring** (9-dars).

Accessibility (a11y) va SEO asoslari

Accessibility — hamma uchun qulay sayt

- Semantik teglardan foydalaning; tugma uchun `<button>`, havola uchun `<a>` (`<div onclick>` emas — u klaviaturadan ishlamaydi).
- Har bir rasmda mazmunli `alt`, har bir input'da `<label>`.
- Sarlavhalar ierarxiyasini buzming.
- Faqat ikonkadan iborat tugmalarga `aria-label` qo'shing: `<button aria-label="Yopish">x</button>`.
- Saytni sichqonchasiz, faqat `Tab`, `Enter`, `Space` bilan ishlatib ko'ring.

SEO va ijtimoiy tarmoqlar uchun meta-teglar

HTML

```
<head>
  <title>iPhone 16 Pro – narxi va xususiyatlari | TexnoShop</title>
  <meta name="description" content="iPhone 16 Pro: 256 GB, kafolat 1 yil,
Toshkent bo'ylab bepul yetkazib berish.">
  <link rel="canonical" href="https://texnoshop.uz/iphone-16-pro">

  <!-- Telegram, Facebook va boshqalarda havola "kartochkasi" uchun (Open
Graph) -->
  <meta property="og:title" content="iPhone 16 Pro – TexnoShop">
  <meta property="og:description" content="Kafolat bilan, bepul yetkazib
berish">
  <meta property="og:image" content="https://texnoshop.uz/img/iphone-og.jpg">
  <meta property="og:url" content="https://texnoshop.uz/iphone-16-pro">
  <meta property="og:type" content="product">
</head>
```

Amaliy misollar

MISOL 1

Shaxsiy portfolio sahifasi (to'liq semantik tuzilma)

Dasturchi rezyumesi o'rnida ishlatiladigan bir sahifali sayt. Faqat HTML — dizaynni keyingi darsda qo'shamiz.

```

<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Dilnoza Karimova – Full-stack
dasturchi portfoliosi">
  <title>Dilnoza Karimova – Full-stack dasturchi</title>
</head>
<body>
  <header>
    <a href="/" aria-label="Bosh sahifa">DK</a>
    <nav>
      <ul>
        <li><a href="#haqida">Men haqimda</a></li>
        <li><a href="#loyihalar">Loyihalar</a></li>
        <li><a href="#tajriba">Tajriba</a></li>
        <li><a href="#aloqa">Aloqa</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section id="haqida">
      
      <h1>Dilnoza Karimova</h1>
      <p>Toshkentlik <strong>full-stack dasturchi</strong>. React va
Node.js bilan
      tez va qulay veb-ilovalar yarataman.</p>
      <a href="cv.pdf" download>Rezyumeni yuklab olish (PDF)</a>
    </section>

    <section id="loyihalar">
      <h2>Loyihalar</h2>

      <article>
        <h3>Kitob do'koni</h3>
        
        <p>Onlayn kitob do'koni: savat, to'lov va admin panel.</p>
      </article>
    </section>
  </main>

```

```

        <li>React</li><li>Express</li><li>PostgreSQL</li>
    </ul>
    <a href="https://github.com/dilnoza/bookshop" target="_blank"
rel="noopener">Kod</a>
    <a href="https://bookshop.example.uz" target="_blank"
rel="noopener">Demo</a>
</article>

<article>
    <h3>Ob-havo ilovasi</h3>
    <p>Shahar bo'yicha 7 kunlik ob-havo prognozi (ochiq API asosida).
</p>
</article>
</section>

<section id="tajriba">
    <h2>Ish tajribasi</h2>
    <ol reversed>
        <li>
            <h3>Junior Full-stack dasturchi – <span>IT Park rezidenti</span>
</h3>
            <p><time datetime="2025-03">2025-mart</time> – hozirgacha</p>
</li>
<li>
            <h3>Frontend stajyor – <span>Startup</span></h3>
            <p><time datetime="2024-06">2024-iyun</time> – <time
datetime="2025-02">2025-fevral</time></p>
</li>
</ol>
</section>
</main>

<footer id="aloqa">
    <address>
        <a href="mailto:dilnoza@example.uz">dilnoza@example.uz</a> ·
        <a href="https://t.me/dilnoza_dev">Telegram</a> ·
        <a href="https://github.com/dilnoza">GitHub</a>
    </address>
    <p><small>&copy; 2026 Dilnoza Karimova</small></p>
</footer>
</body>
</html>

```

Tahlil: navigatsiya havolalari `#id` orqali sahifa ichidagi bo'limlarga olib boradi; har bir loyiha alohida `<article>`; tajriba teskari tartibli ro'yxat (`reversed`); aloqa ma'lumotlari `<address>` ichida. Sahifani CSS'siz ochsangiz ham u mantiqan to'liq o'qiladi — bu yaxshi HTML belgisi.

register.html

```
<form action="/api/enroll" method="POST" enctype="multipart/form-data">
  <fieldset>
    <legend>Shaxsiy ma'lumotlar</legend>

    <label for="fullname">To'liq ism *</label>
    <input id="fullname" name="fullname" type="text" required
minlength="3"
      autocomplete="name" placeholder="Masalan: Jasur Toshmatov">

    <label for="phone">Telefon *</label>
    <input id="phone" name="phone" type="tel" required
      pattern="\+998[0-9]{9}" placeholder="+998901234567"
      title="Format: +998 va 9 ta raqam">

    <label for="birth">Tug'ilgan sana</label>
    <input id="birth" name="birth" type="date" min="1960-01-01" max="2015-
12-31">

    <label for="avatar">Rasm (ixtiyoriy)</label>
    <input id="avatar" name="avatar" type="file" accept="image/png,
image/jpeg">
  </fieldset>

  <fieldset>
    <legend>Kurs tanlovi</legend>

    <label for="course">Yo'nalish *</label>
    <select id="course" name="course" required>
      <option value="">- Tanlang -</option>
      <optgroup label="Dasturlash">
        <option value="frontend">Frontend</option>
        <option value="backend">Backend</option>
        <option value="fullstack" selected>Full-stack</option>
      </optgroup>
      <optgroup label="Dizayn">
        <option value="uiux">UI/UX</option>
      </optgroup>
    </select>
```

```

    <p>O'qish shakli *</p>
    <label><input type="radio" name="format" value="offline" required>
Oflayn</label>
    <label><input type="radio" name="format" value="online">
Onlayn</label>

    <p>Qaysi kunlar qulay?</p>
    <label><input type="checkbox" name="days" value="du-chor-ju"> Du /
Chor / Ju</label>
    <label><input type="checkbox" name="days" value="se-pay-shan"> Se /
Pay / Shan</label>

    <label for="level">Hozirgi bilim darajangiz: <output
id="levelOut">3</output>/10</label>
    <input id="level" name="level" type="range" min="0" max="10" value="3"
oninput="levelOut.value = this.value">

    <label for="about">O'zingiz haqingizda</label>
    <textarea id="about" name="about" rows="4" maxlength="500"
placeholder="Nima uchun dasturlashni o'rganmoqchisiz?">
</textarea>
</fieldset>

<label>
    <input type="checkbox" name="agree" required>
    <a href="/oferta" target="_blank">Ommaviy oferta</a> shartlariga
roziman
</label>

<input type="hidden" name="source" value="landing-oktabr">

<button type="submit">Yuborish</button>
<button type="reset">Tozalash</button>
</form>

```

Muhim jihatlar:

- `enctype="multipart/form-data"` — fayl yuborilayotgan bo'lsa majburiy.
- `<fieldset>` + `<legend>` — maydonlarni mantiqiy guruhlaydi.
- Radio tugmalar bir xil `name="format"` ga ega — shuning uchun faqat bittasi tanlanadi.
- `autocomplete="name"` — brauzer saqlangan ma'lumotni avtomatik taklif qiladi.
- Bo'sh `value=""` li birinchi `<option>` + `required` = foydalanuvchi albatta tanlashi kerak.

Forma yuborilganda server quyidagi kabi ma'lumot oladi:

```
fullname=Jasur ...&phone=%2B998 ...&course=fullstack&format=online&days=du-chor-ju&...
```

blog/js-closure.html

```
<body>
  <header>... </header>

  <main>
    <nav aria-label="Breadcrumb">
      <ol>
        <li><a href="/">Bosh sahifa</a></li>
        <li><a href="/blog">Blog</a></li>
        <li aria-current="page">Closure nima?</li>
      </ol>
    </nav>

    <article>
      <header>
        <h1>JavaScript'da closure nima va u nima uchun kerak?</h1>
        <p>
          Muallif: <a href="/authors/sardor" rel="author">Sardor
Aliyev</a> .
          <time datetime="2026-09-28T14:30">28-sentabr, 2026</time> .
          8 daqiqa o'qish
        </p>
      </header>

      <section>
        <h2>Kirish</h2>
        <p>Closure – funksiyaning o'zi yaratilgan muhitni <em>eslab
qolishi</em>...</p>
      </section>

      <section>
        <h2>Misol</h2>
        <pre><code>function counter() {
  let n = 0;
  return () => ++n;
}</code></pre>
        <figure>
          
```

```

        <figcaption>Closure sxemasi</figcaption>
      </figure>
    </section>

    <footer>
      <p>Teglar: <a href="/tag/javascript" rel="tag">javascript</a>, <a
href="/tag/funksiyalar" rel="tag">funksiyalar</a></p>
    </footer>
  </article>

  <section aria-labelledby="comments-title">
    <h2 id="comments-title">Izohlar (2)</h2>
    <article>
      <h3>Bekzod</h3>
      <time datetime="2026-09-29">29-sentabr</time>
      <p>Juda tushunarli yozilgan, rahmat!</p>
    </article>
  </section>
</main>

<aside>
  <h2>O'xshash maqolalar</h2>
  <ul>
    <li><a href="/blog/scope">Scope va hoisting</a></li>
    <li><a href="/blog/this">this kalit so'zi</a></li>
  </ul>
</aside>

<footer>...</footer>
</body>

```

Izohlar ham `<article>` — chunki har bir izoh mustaqil mazmun birligi. `aria-current="page"` ekran o'quvchiga "hozir shu sahifadasiz" deydi. `<time datetime>` esa qidiruv tizimlariga maqola sanasini mashina o'qiy oladigan formatda beradi.

menu.html

```

<main>
  <h1>"Bahor" milliy taomlar restorani – Menu</h1>

  <nav aria-label="Menu bo'limlari">
    <a href="#birinchi">Birinchi taomlar</a> |
    <a href="#ikkinchi">Ikkinchi taomlar</a> |
    <a href="#ichimliklar">Ichimliklar</a>
  </nav>

  <section id="birinchi">
    <h2>Birinchi taomlar</h2>
    <table>
      <thead>
        <tr><th scope="col">Taom</th><th scope="col">Hajm</th><th
scope="col">Narx</th></tr>
      </thead>
      <tbody>
        <tr>
          <th scope="row">
            <figure>
              
              <figcaption>Sho'rva</figcaption>
            </figure>
          </th>
          <td>400 ml</td>
          <td><data value="28000">28 000 so'm</data></td>
        </tr>
        <tr>
          <th scope="row">Mastava</th>
          <td>400 ml</td>
          <td><data value="25000">25 000 so'm</data></td>
        </tr>
      </tbody>
    </table>
  </section>

  <section id="ikkinchi">

```

```

<h2>Ikkinchi taomlar</h2>
<dl>
  <dt>Toshkent oshi <mark>Xit!</mark></dt>
  <dd>Devzira guruch, qo'y go'shti, sariq sabzi, bedana tuxumi – <data
value="45000">45 000 so'm</data></dd>
  <dt>Manti (5 dona)</dt>
  <dd>Qo'y go'shti va piyoz – <data value="35000">35 000 so'm</data>
</dd>
</dl>
</section>

<p><a href="#">↑ Yuqoriga qaytish</a></p>
</main>

```

`<data value>` — narxni odamlar uchun chiroyli, dasturlar uchun esa toza raqam ko'rinishida saqlaydi. Bu JavaScript bilan savatcha summasini hisoblashda qulay.

Mashqlar



Mustaqil ish

- O'zingiz uchun 1-misoldagi kabi portfolio sahifasi yarating — kamida 3 ta loyiha, ko'nikmalar ro'yxati va aloqa bo'limi bilan.
- Kitob do'koni uchun "Buyurtma berish" formasini yarating: kitob tanlash (`select`), soni (`number` , 1–10), yetkazish usuli (`radio`), manzil (`textarea`), yetkazish sanasi (`date`).
- O'z dars jadvalingizni `<table>` bilan yozing, katta tanaffusni `colspan` bilan birlashtiring.
- Sahifangizni validator.w3.org orqali tekshiring va barcha xatolarni tuzating.
- Chrome DevTools → **Lighthouse** → "Accessibility" va "SEO" bo'yicha audit o'tkazing. 90+ ball olishga harakat qiling.



Xulosa

- HTML — mazmun va tuzilma; ko'rinish CSS'ga, xatti-harakat JS'ga tegishli.
- `<!DOCTYPE>`, `charset`, `viewport` — har bir sahifada majburiy.

- Semantik teglar (`header`, `nav`, `main`, `article`, `section`, `aside`, `footer`) SEO va accessibility uchun muhim.
- Formalarda `name`, `label`, to'g'ri `type` va validatsiya atributlarini ishlating; serverda baribir qayta tekshiring.
- Rasmlarda doim `alt`, `width/height` va kerak bo'lsa `loading="lazy"`.



03-DARS · 14 TADAN

CSS — dizayn va maket

Selektorlar, kaskad va spetsifiklik, box model, Flexbox, Grid, pozitsiyalash, responsiv dizayn, CSS o'zgaruvchilari va animatsiyalar.

Boshlang'ich · ~120 daqiqa

CSS (Cascading Style Sheets) HTML elementlarining ko'rinishini boshqaradi: rang, shrift, o'lcham, joylashuv, animatsiya. HTML — uyning g'ishtlari bo'lsa, CSS — uning bo'yog'i, mebeli va xonalar rejasi. CSS'ni "oson" deb o'ylash xato: kaskad, spetsifiklik va maket algoritmlarini tushunmasdan yozilgan CSS tezda boshqarib bo'lmaydigan tartibsizlikka aylanadi. Bu darsda CSS'ni *tizimli* tushunamiz.

CSS'ni ulash usullari

HTML

```
←!— 1. Tashqi fayl (eng to'g'ri usul — keshlanadi, qayta ishlatiladi) →  
<link rel="stylesheet" href="css/style.css">  
  
←!— 2. Ichki <style> bloki (bitta sahifaga xos uslublar) →  
<style>  
  h1 { color: navy; }  
</style>  
  
←!— 3. Inline (eng yuqori ustuvorlik, lekin qo'llab-quvvatlash qiyin) →  
<p style="color: red; font-weight: bold;">Diqqat!</p>
```

Qoida sintaksisi

```
CSS

/* selektor { xususiyat: qiymat; } */
.card-title {
  color: #1e293b;          /* deklaratsiya */
  font-size: 1.25rem;
  margin-bottom: 8px;
}
```

Selektorlar

Selektor	Misol	Nimani tanlaydi
Universal	*	Barcha elementlar
Teg	p	Barcha <p> lar
Sinf (class)	.btn	class="btn" bo'lganlar
ID	#header	id="header" (yagona)
Atribut	input[type="email"], a[href^="https"]	Atribut qiymati bo'yicha (^= boshlanadi, \$= tugaydi, *= o'z ichiga oladi)
Avlod (descendant)	nav a	nav ichidagi istalgan chuqurlikdagi a
Bevosita farzand	ul > li	Faqat birinchi darajali li
Qo'shni aka- uka	h2 + p	h2 dan keyingi birinchi p
Umumiy aka- uka	h2 ~ p	h2 dan keyingi barcha p lar

Psevdo-sinflar va psevdo-elementlar

CSS

```
/* Holat bo'yicha (psevdo-sinflar — bitta ikki nuqta) */
a:hover          { color: crimson; }          /* sichqoncha ustida */
button:active    { transform: scale(.97); }    /* bosilgan payt */
input:focus      { outline: 2px solid royalblue; }
input:invalid    { border-color: red; }        /* validatsiyadan o'tmagan */
input:disabled   { opacity: .5; }
a:visited        { color: purple; }

/* Tuzilma bo'yicha */
li:first-child   { font-weight: bold; }
li:last-child    { border-bottom: none; }
tr:nth-child(even) { background: #f5f5f5; }    /* juft qatorlar */
li:nth-child(3n + 1) { color: teal; }          /* 1, 4, 7, ... */
p:not(.lead)     { font-size: 15px; }
.card:has(img)   { padding-top: 0; }           /* ichida img bor kartalar */

/* Psevdo-elementlar — ikki ikki nuqta: mavjud bo'lmagan "virtual" element yaratadi */
.required::after { content: " *"; color: red; }
blockquote::before { content: "“"; font-size: 3em; }
p::first-line    { font-weight: 600; }
::selection      { background: gold; }
input::placeholder { color: #999; }
```

Kaskad, spetsifiklik va meros

Bitta elementga bir nechta qoida to'g'ri kelsa, qaysi biri g'olib bo'ladi? Brauzer quyidagi tartibda hal qiladi (bu "Cascading" so'zining ma'nosi):

1. **Muhimlik (importance)** — `!important` belgilangan deklaratsiya ustun turadi.
2. **Spetsifiklik** — aniqroq selektor yutadi.
3. **Manba tartibi** — spetsifiklik teng bo'lsa, **keyinroq yozilgan** qoida yutadi.

Spetsifiklikni hisoblash

Spetsifiklik uchta raqam bilan o'lchanadi: (ID, Sinf, Teg).

Selektor	(ID, Sinf/atribut/psevdo-sinf, Teg/psevdo-element)
p	(0, 0, 1)
.card p	(0, 1, 1)
.card .title:hover	(0, 3, 0)
#main .card p	(1, 1, 1)
style="..."	har qanday selektordan yuqori

Raqlamlar chapdan o'ngga solishtiriladi: (1,0,0) har doim (0,15,0) dan katta.

CSS

```
#app p      { color: blue; } /* (1,0,1) – G'OLIB */
.text.big   { color: green; } /* (0,2,0) */
p           { color: red; }  /* (0,0,1) */
```

 **Amaliy maslahat**

Uslublar uchun deyarli faqat **sinflardan** foydalaning. ID selektorlari va `!important` spetsifiklikni keskin oshiradi va keyinchalik uslubni o'zgartirishni qiyinlashtiradi. Spetsifiklikni past va bir tekis saqlash — toza CSS'ning siri.

Meros (inheritance)

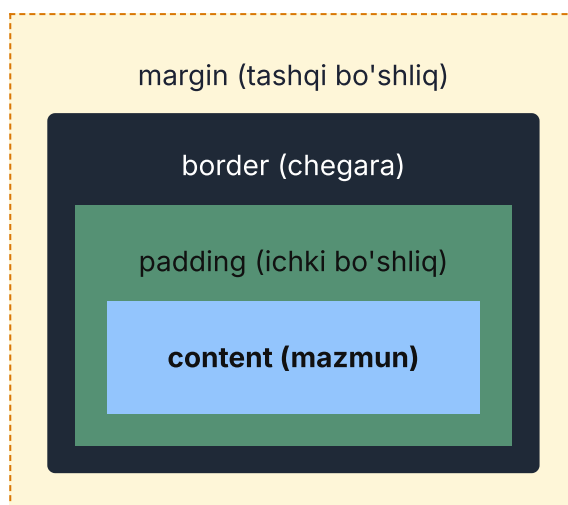
Ba'zi xususiyatlar ota elementdan farzandlarga avtomatik o'tadi: asosan **matn bilan bog'liqlar** — `color`, `font-family`, `font-size`, `line-height`, `text-align`. O'lcham va joylashuv xususiyatlari (`margin`, `padding`, `border`, `width`) meros qilinmaydi. Shuning uchun shriftni bir marta `body` ga berish yetarli.

CSS

```
body { font-family: "Inter", sans-serif; color: #222; line-height: 1.6; }  
button { font: inherit; } /* tugmalar odatda shriftni meros olmaydi –  
majburlaymiz */
```

Box model — har bir element bu quti

Brauzer har bir elementni to'rtburchak quti sifatida chizadi. Quti to'rt qatlamdan iborat:



CSS

```
.box {  
  width: 300px;  
  padding: 20px; /* ichki bo'shliq: barcha tomonlar */  
  border: 2px solid #333;  
  margin: 16px auto; /* tepa/past 16px, chap/o'ng auto = gorizontal  
markazlash */  
}  
  
/* Qisqa yozuv tartibi: tepa → o'ng → past → chap (soat strelkasi bo'yicha) */  
.a { padding: 10px 20px 30px 40px; }  
.b { padding: 10px 20px; } /* tepa/past 10, chap/o'ng 20 */
```

box-sizing — eng muhim bitta qator

Standart holatda (`content-box`) `width: 300px` faqat mazmunga tegishli: yuqoridagi quti haqiqatda `300 + 2×20 + 2×2 = 344px` keng bo'ladi. Bu hisob-kitobni qiyinlashtiradi. `border-box` da esa `width` padding va border'ni o'z ichiga oladi:

CSS

```
/* Deyarli har bir loyihaning birinchi qatorlari */
*, *::before, *::after {
  box-sizing: border-box;
}
```



Margin collapse

Ikki blok element ustma-ust tursa, ularning vertikal marginlari **qo'shilmaydi**, balki kattasi olinadi: `margin-bottom: 30px` va `margin-top: 20px` orasidagi masofa 50 emas, **30px** bo'ladi. Flex va Grid ichida bu hodisa yuz bermaydi.

display xususiyati

Qiymat	Xatti-harakat
<code>block</code>	Yangi qatordan, butun kenglik; width/height ishlaydi
<code>inline</code>	Qator ichida; width/height va vertikal margin ishlamaydi
<code>inline-block</code>	Qator ichida, lekin width/height ishlaydi (tugmalar uchun)
<code>none</code>	Element umuman chizilmaydi va joy egallamaydi
<code>flex</code> / <code>grid</code>	Zamonaviy maket tizimlari (pastda batafsil)

O'lchov birliklari va ranglar

Birlik	Nimaga nisbatan	Qachon ishlatish

px	Mutlaq piksel	Chegaralar, soyalar, kichik detallar
rem	html shrift o'lchami (odatda 16px)	Shrift o'lchamlari, bo'shliqlar — foydalanuvchi sozlamalarini hurmat qiladi
em	Joriy elementning shrift o'lchami	Matnga proporsional padding (tugmalar)
%	Ota element o'lchami	Kengliklar
vw / vh	Ekran kengligi / balandligining 1%	To'liq ekranli bloklar; mobilda dvh ishonchliroq
fr	Grid'dagi bo'sh joy ulushi	Grid ustunlari
ch	"0" belgisining kengligi	Matn qatori uzunligi: max-width: 65ch

CSS

```
.title {
  /* clamp(minimum, ideal, maksimum) — ekranga moslashuvchi shrift */
  font-size: clamp(1.5rem, 4vw, 3rem);
}
.sidebar { width: calc(100% - 2rem); }

/* Ranglar */
.a { color: #4f46e5; } /* HEX */
.b { color: rgb(79 70 229); } /* RGB */
.c { color: rgb(79 70 229 / 50%); } /* shaffoflik bilan */
.d { color: hsl(243 75% 59%); } /* ton, to'yinganlik, yorqinlik */
.e { background: linear-gradient(135deg, #4f46e5, #0ea5e9); }
```

Tipografiya

CSS

```
@import url('https://fonts.googleapis.com/css2?
family=Inter:wght@400;600;700&display=swap');

body {
  font-family: "Inter", system-ui, -apple-system, "Segoe UI", sans-serif; /*
zaxira shriftlar */
  font-size: 1rem;
  line-height: 1.6;          /* birliksiz — shrift o'lchamiga nisbatan */
  color: #1f2937;
}
h1 { font-weight: 700; letter-spacing: -0.02em; }
.muted { color: #6b7280; }
.truncate {                /* uzun matnni "..." bilan kesish */
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
.uppercase { text-transform: uppercase; letter-spacing: .08em; font-size:
.75rem; }
```

Flexbox — bir o'lchamli maket

Flexbox elementlarni **bir yo'nalish bo'yicha** (qator yoki ustun) joylashtirish uchun. Navigatsiya, tugmalar guruhi, kartaning ichki qismi, markazlash — barchasi Flexbox bilan oson.

Ikki tushuncha: **flex-konteyner** (`display: flex` berilgan ota) va **flex-elementlar** (uning bevosita farzandlari). Shuningdek ikki o'q: **asosiy o'q (main axis)** — `flex-direction` yo'nalishi, va **kesishgan o'q (cross axis)** — unga perpendikulyar.

Konteyner xususiyatlari

CSS

```
.container {
  display: flex;
  flex-direction: row;          /* row | row-reverse | column | column-reverse */
  flex-wrap: wrap;              /* sig'masa keyingi qatorga o'tish */
  justify-content: space-between; /* ASOSIY o'q bo'yicha: flex-start | center | flex-end | space-between | space-around | space-evenly */
  align-items: center;          /* KESISHGAN o'q bo'yicha: stretch | flex-start | center | flex-end | baseline */
  gap: 16px;                    /* elementlar orasidagi masofa */
}
```

Element xususiyatlari

CSS

```
.item {
  flex-grow: 1;      /* bo'sh joydan qancha ulush olish (0 – olmaydi) */
  flex-shrink: 1;    /* joy yetmasa qanchalik qisqarish (0 – qisqarmaydi) */
  flex-basis: 200px; /* boshlang'ich o'lcham */
  /* qisqa yozuv: */
  flex: 1 1 200px;
}
.sidebar { flex: 0 0 260px; } /* qat'iy 260px, cho'zilmaydi, qisqarmaydi */
.main     { flex: 1; }        /* qolgan barcha joyni egallaydi */
.special { align-self: flex-end; order: -1; } /* alohida tekislash va tartibni o'zgartirish */
```



Markazlashning eng oson usuli

`.parent { display: flex; justify-content: center; align-items: center; min-height: 100vh; }` — element ekranning qoq o'rtasiga joylashadi. Grid bilan esa yanada qisqa:
`display: grid; place-items: center;`

CSS Grid — ikki o'lchamli maket

Grid qator va ustunlarni bir vaqtda boshqaradi. Sahifa maketi, rasm galereyasi, mahsulotlar katalogi, dashboard — Grid'ning ishi.

CSS

```
.grid {
  display: grid;
  grid-template-columns: 200px 1fr 1fr;      /* 3 ustun: 200px va qolgani teng
ikkiga */
  grid-template-rows: auto 1fr auto;
  gap: 20px;                                /* row-gap va column-gap */
}

/* repeat() va minmax() */
.gallery {
  display: grid;
  grid-template-columns: repeat(4, 1fr);      /* 4 ta teng
ustun */
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr)); /* ✨ media
query'siz responsiv! */
}

/* Element bir nechta katakni egallashi */
.featured {
  grid-column: 1 / 3;      /* 1-chiziqdan 3-chiziqqacha = 2 ustun */
  grid-column: span 2;     /* yoki: 2 ustun egallash */
  grid-row: span 2;
}
```

Nomlangan hududlar (grid-template-areas)

CSS

```
.page {
  display: grid;
  grid-template-columns: 240px 1fr;
  grid-template-rows: 64px 1fr 48px;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  min-height: 100vh;
}

.page > header { grid-area: header; }
.page > aside { grid-area: sidebar; }
.page > main { grid-area: main; }
.page > footer { grid-area: footer; }

@media (max-width: 768px) {
  .page {
    grid-template-columns: 1fr;
    grid-template-areas: "header" "main" "sidebar" "footer";
  }
}
```

Vazifa	Flexbox	Grid
Yo'nalish	Bir o'lchamli (qator yoki ustun)	Ikki o'lchamli (qator va ustun)
Yondashuv	Mazmundan kelib chiqadi (content-first)	Maketdan kelib chiqadi (layout-first)
Eng yaxshi joyi	Navbar, tugmalar, kartaning ichi	Sahifa skeleti, galereya, katalog

Ular raqobatchi emas — odatda birga ishlatiladi: sahifa Grid bilan, uning ichidagi komponentlar Flexbox bilan.

Pozitsiyalash (position)

Qiymat	Xatti-harakat	Nimaga nisbatan
static	Standart oqim	—
relative	Oqimda qoladi, lekin siljitish mumkin; absolute farzandlar uchun "langar"	O'zining asl joyiga
absolute	Oqimdan chiqadi (joy egallamaydi)	Eng yaqin pozitsiyalangan (static bo'lmagan) ajdodga
fixed	Oqimdan chiqadi, scroll'da qimirlamaydi	Ekranga (viewport)
sticky	Scroll ma'lum nuqtaga yetguncha oddiy, keyin "yopishib" qoladi	Scroll konteyneriga

CSS

```
.card { position: relative; }
.card .badge {                      /* kartaning o'ng yuqori burchagidagi "Yangi"
belgisi */
  position: absolute;
  top: 12px; right: 12px;
}
.header { position: sticky; top: 0; z-index: 10; }
.chat-button { position: fixed; bottom: 24px; right: 24px; }
.modal-backdrop { position: fixed; inset: 0; background: rgb(0 0 0 / .5); } /*
inset: 0 = top/right/bottom/left: 0 */
```

z-index — ustma-ust tushgan elementlardan qaysi biri yuqorida bo'lishi. Faqat pozitsiyalangan (yoki flex/grid farzandi) elementlarda ishlaydi.

Responsiv dizayn

Bugun trafikning 60%+ qismi telefonlardan keladi. **Responsiv dizayn** — sayt har qanday ekranga moslashishi.

Mobile-first yondashuv

Avval eng kichik ekran uchun oddiy uslub yoziladi, keyin `min-width` media so'rovlari bilan katta ekranlar uchun kengaytiriladi.

CSS

```
/* Standart: telefon (bitta ustun) */
.products { display: grid; grid-template-columns: 1fr; gap: 16px; }

/* Planshet: 640px va undan keng */
@media (min-width: 640px) {
  .products { grid-template-columns: repeat(2, 1fr); }
}

/* Noutbuk: 1024px va undan keng */
@media (min-width: 1024px) {
  .products { grid-template-columns: repeat(4, 1fr); }
}

/* Boshqa foydali media so'rovlar */
@media (prefers-color-scheme: dark) { /* tizim tungi rejimda */ }
@media (prefers-reduced-motion: reduce) { * { animation: none !important;
transition: none !important; } }
@media (hover: hover) { .card:hover { transform: translateY(-4px); } } /* faqat
sichqonchali qurilmalarda */
@media print { nav, footer { display: none; } }
```



Responsivlik uchun nazorat ro'yxati

`viewport` meta-tegi bor · rasmlarga `max-width: 100%` · qat'iy `width: 1200px` o'rniga `max-width · rem` va `clamp()` · Grid'da `auto-fill + minmax` · DevTools'dagi "Toggle device toolbar" (`Ctrl + Shift + M`) bilan sinash.

CSS o'zgaruvchilari (Custom Properties)

CSS

```
:root {
  --color-primary: #4f46e5;
  --color-text: #1f2937;
  --color-bg: #ffffff;
  --radius: 12px;
  --space: 8px;
}
.btn {
  background: var(--color-primary);
  border-radius: var(--radius);
  padding: calc(var(--space) * 1.5) calc(var(--space) * 3);
}
/* Mahalliy qayta belgilash */
.btn.danger { --color-primary: #dc2626; }
```

Sass o'zgaruvchilaridan farqli, CSS o'zgaruvchilari **ishlash vaqtida** (runtime) ishlaydi: ularni JavaScript bilan o'zgartirish, media query ichida qayta belgilash va meros qilib berish mumkin. Bu tungi rejim yasashning eng qulay usuli (4-misol).

Transition, transform va animatsiya

CSS

```
.btn {
  transition: background-color .2s ease, transform .2s ease;
}

.btn:hover {
  background-color: #4338ca;
  transform: translateY(-2px) scale(1.02); /* siljitish, kattalashtirish,
  rotate(45deg), skew() */
}

/* Kalit kadrlar bilan animatsiya */
@keyframes spin {
  from { transform: rotate(0deg); }
  to   { transform: rotate(360deg); }
}

.loader {
  width: 40px; height: 40px;
  border: 4px solid #e5e7eb;
  border-top-color: #4f46e5;
  border-radius: 50%;
  animation: spin .8s linear infinite;
}

@keyframes fadeUp {
  from { opacity: 0; transform: translateY(20px); }
  to   { opacity: 1; transform: none; }
}

.card { animation: fadeUp .5s ease both; }
```



Unumdorlik

`transform` va `opacity` ni animatsiya qilish arzon (faqat Composite bosqichi). `width`, `top`, `margin` ni animatsiya qilish esa har kadrda Layout'ni qayta hisoblaydi va sekinlashtiradi (1-darsdagi rendering bosqichlarini eslang).

Amaliy misollar

MISOL 1 Responsiv navigatsiya paneli (Flexbox)

navbar.html

```
<header class="navbar">
  <a class="logo" href="/">Texno<span>Shop</span></a>
  <input type="checkbox" id="nav-toggle" class="nav-toggle">
  <label for="nav-toggle" class="burger" aria-label="Menyuni
ochish">≡</label>
  <nav class="nav-links">
    <a href="/" class="active">Bosh sahifa</a>
    <a href="/catalog">Katalog</a>
    <a href="/sale">Chegirmalar</a>
    <a href="/contact">Aloqa</a>
    <a href="/login" class="btn">Kirish</a>
  </nav>
</header>
```



```

*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; }

.navbar {
  position: sticky; top: 0; z-index: 100;
  display: flex;
  align-items: center;
  justify-content: space-between;
  flex-wrap: wrap;
  padding: 12px 24px;
  background: #fff;
  box-shadow: 0 2px 10px rgb(0 0 0 / .06);
}
.logo { font-size: 1.4rem; font-weight: 800; color: #111; text-decoration: none; }
.logo span { color: #4f46e5; }

.nav-links { display: flex; align-items: center; gap: 24px; }
.nav-links a { color: #374151; text-decoration: none; font-weight: 500; position: relative; }
.nav-links a:not(.btn)::after { /* hover'da pastdan chiziq "o'sib" chiqadi */
  content: ""; position: absolute; left: 0; bottom: -4px;
  width: 0; height: 2px; background: #4f46e5; transition: width .25s;
}
.nav-links a:hover::after, .nav-links a.active::after { width: 100%; }
.btn { background: #4f46e5; color: #fff !important; padding: 8px 18px; border-radius: 8px; }

.nav-toggle, .burger { display: none; }

@media (max-width: 768px) {
  .burger { display: block; font-size: 1.6rem; cursor: pointer; }
  .nav-links {
    display: none; /* yopiq */
    flex-direction: column;
    align-items: flex-start;
    width: 100%;
    padding-top: 16px;
    gap: 14px;
  }
  .nav-toggle:checked ~ .nav-links { display: flex; } /* checkbox

```

```
belgilansa — ochiladi */  
}
```

Hiyla: JavaScript'siz mobil menyuni! Yashirin `checkbox` va uning `label` i "burger" tugmasi vazifasini bajaradi, `:checked ~ .nav-links` selektori esa menyuni ko'rsatadi. 5-darsda buni JavaScript bilan ham qilamiz.

MISOL 2 Mahsulot kartalari katalogi (Grid + Flexbox)

catalog.html

```
<section class="catalog">  
  <article class="product">  
    <span class="badge">—20%</span>  
      
    <div class="product-body">  
      <h3 class="product-title">Samsung Galaxy A55 8/256 GB, ko'k  
rang</h3>  
      <div class="price">  
        <span class="new">4 799 000 so'm</span>  
        <span class="old">5 999 000</span>  
      </div>  
      <button class="add">Savatga</button>  
    </div>  
  </article>  
  <!-- ...yana kartalar... -->  
</section>
```

```

.catalog {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(240px, 1fr));
  gap: 24px;
  padding: 24px;
  max-width: 1200px;
  margin: 0 auto;
}

.product {
  position: relative;
  display: flex;                /* karta ichi - ustun bo'yicha flex */
  flex-direction: column;
  background: #fff;
  border: 1px solid #e5e7eb;
  border-radius: 14px;
  overflow: hidden;            /* rasm burchaklari ham yumaloq bo'lishi
uchun */
  transition: transform .2s, box-shadow .2s;
}

.product:hover { transform: translateY(-4px); box-shadow: 0 12px 30px
rgb(0 0 0 / .08); }

.product img { width: 100%; aspect-ratio: 4 / 3; object-fit: cover;
display: block; }

.badge {
  position: absolute; top: 12px; left: 12px;
  background: #ef4444; color: #fff; font-weight: 700; font-size: .8rem;
  padding: 4px 8px; border-radius: 6px;
}

.product-body { display: flex; flex-direction: column; gap: 10px; padding:
16px; flex: 1; }

.product-title {
  margin: 0; font-size: 1rem; line-height: 1.4;
  display: -webkit-box; -webkit-line-clamp: 2; -webkit-box-orient:
vertical; overflow: hidden; /* 2 qatordan oshsa "... " */
}

.price { display: flex; align-items: baseline; gap: 8px; margin-top: auto;
} /* margin-top:auto - pastga itaradi */

.new { font-size: 1.15rem; font-weight: 800; }

.old { color: #9ca3af; text-decoration: line-through; font-size: .9rem; }

.add {
  border: 0; border-radius: 10px; padding: 10px; cursor: pointer;

```

```
background: #4f46e5; color: #fff; font: inherit; font-weight: 600;
}
.add:hover { background: #4338ca; }
```

Asosiy g'oyalar: `repeat(auto-fill, minmax(240px, 1fr))` media query'siz ham ustunlar sonini ekranga moslaydi; `aspect-ratio` + `object-fit: cover` turli o'lchamdagi rasmlarni bir xil ko'rinishga keltiradi; `margin-top: auto` narx va tugmani doim karta pastiga itarib, barcha kartalarni tekis qiladi.

MISOL 3 Landing sahifa: hero bo'lim va admin panel maketi

HTML

```
<section class="hero">
  <div class="hero-text">
    <h1>Dasturlashni <span>noldan</span> o'rganing</h1>
    <p>6 oyda birinchi ishingizga tayyorlaymiz. Amaliy loyihalar va
mentorlar bilan.</p>
    <div class="actions">
      <a class="btn primary" href="#">Bepul sinov darsi</a>
      <a class="btn ghost" href="#">Dastur bilan tanishish</a>
    </div>
  </div>
  
</section>

<div class="dashboard">
  <header>Header</header>
  <aside>Sidebar</aside>
  <main>
    <div class="stats">
      <div class="stat">Foydalanuvchilar<b>12 480</b></div>
      <div class="stat">Buyurtmalar<b>3 215</b></div>
      <div class="stat">Daromad<b>$48k</b></div>
    </div>
  </main>
</div>
```

```

.hero {
  display: grid;
  grid-template-columns: 1.1fr 1fr;
  align-items: center;
  gap: 48px;
  min-height: 80vh;
  padding: clamp(24px, 6vw, 96px);
  background: radial-gradient(circle at top right, #eef2ff, #fff 60%);
}

.hero h1 { font-size: clamp(2rem, 5vw, 3.6rem); line-height: 1.1; margin:
0 0 16px; }

.hero h1 span {
  background: linear-gradient(90deg, #4f46e5, #06b6d4);
  -webkit-background-clip: text; background-clip: text; color:
transparent;
}

.hero p { font-size: 1.15rem; color: #4b5563; max-width: 50ch; }
.actions { display: flex; flex-wrap: wrap; gap: 12px; margin-top: 24px; }
.btn { padding: 14px 24px; border-radius: 12px; font-weight: 700; text-
decoration: none; }
.btn.primary { background: #4f46e5; color: #fff; }
.btn.ghost { border: 2px solid #c7d2fe; color: #4f46e5; }
.hero img { width: 100%; height: auto; }

@media (max-width: 860px) {
  .hero { grid-template-columns: 1fr; text-align: center; }
  .hero p { margin-inline: auto; }
  .actions { justify-content: center; }
  .hero img { order: -1; max-width: 360px; margin: 0 auto; } /* mobilda
rasm tepada */
}

/* Admin panel maketi */
.dashboard {
  display: grid;
  grid-template: "header header" 64px
                  "aside main" 1fr / 240px 1fr; /* qisqa yozuv:
hududlar + qatorlar / ustunlar */
  min-height: 100vh;
}

.dashboard > header { grid-area: header; background: #111827; color: #fff;
}

```

```
.dashboard > aside { grid-area: aside; background: #1f2937; color: #fff;
}
.dashboard > main { grid-area: main; background: #f3f4f6; padding:
24px; }
.stats { display: grid; grid-template-columns: repeat(auto-fit,
minmax(180px, 1fr)); gap: 16px; }
.stat { background: #fff; padding: 20px; border-radius: 12px; color:
#6b7280; }
.stat b { display: block; font-size: 1.8rem; color: #111827; }

@media (max-width: 768px) {
  .dashboard { grid-template: "header" 64px "main" 1fr / 1fr; }
  .dashboard > aside { display: none; }
}
```

theme.html

```
<!DOCTYPE html>
<html lang="uz">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Tungi rejim</title>
<style>
  :root {
    --bg: #ffffff;
    --surface: #f3f4f6;
    --text: #111827;
    --muted: #6b7280;
    --primary: #4f46e5;
  }
  /* Tizim tungi rejimda bo'lsa (foydalanuvchi qo'lda tanlamagan bo'lsa)
  */
  @media (prefers-color-scheme: dark) {
    :root:not([data-theme="light"]) {
      --bg: #0f172a; --surface: #1e293b; --text: #e2e8f0; --muted:
#94a3b8; --primary: #818cf8;
    }
  }
  /* Foydalanuvchi qo'lda tungi rejimni tanlagan */
  :root[data-theme="dark"] {
    --bg: #0f172a; --surface: #1e293b; --text: #e2e8f0; --muted: #94a3b8;
--primary: #818cf8;
  }

  body {
    margin: 0; min-height: 100vh;
    display: grid; place-items: center;
    background: var(--bg); color: var(--text);
    font-family: system-ui, sans-serif;
    transition: background .3s, color .3s;
  }

  .card { background: var(--surface); padding: 32px; border-radius: 16px;
max-width: 360px; }
  .card p { color: var(--muted); }
```

```

    button {
      background: var(--primary); color: #fff; border: 0;
      padding: 10px 18px; border-radius: 10px; font: inherit; cursor:
pointer;
    }
  </style>
</head>
<body>
  <div class="card">
    <h2>Salom! 🙌</h2>
    <p>Barcha ranglar CSS o'zgaruvchilarida. Mavzuni almashtirish uchun
faqat
      o'zgaruvchilar qayta belgilanadi.</p>
    <button id="toggle">🌙 Tungi rejim</button>
  </div>
  <script>
    const btn = document.getElementById('toggle');
    btn.addEventListener('click', () => {
      const root = document.documentElement;
      const dark = root.dataset.theme === 'dark';
      root.dataset.theme = dark ? 'light' : 'dark';
      btn.textContent = dark ? '🌙 Tungi rejim' : '☀️ Kunduzgi rejim';
    });
  </script>
</body>
</html>

```

Bu aynan siz o'qiyotgan sayt ishlatgan usul (o'ng yuqoridagi 🌑 tugmasi). Komponentlar hech qachon aniq rangni bilmaydi — faqat `var(--text)` kabi "ma'noli" o'zgaruvchilarni. Natijada yangi mavzu qo'shish bir necha qator bilan hal bo'ladi.

Mashqlar



Mustaqil ish

1. Flexbox Froggy va Grid Garden o'yinlarini to'liq o'ting.

- 2-darsdagi portfolio sahifangizga dizayn bering: sticky navbar, hero bo'lim, loyihalar Grid'da, mobil versiya.
- Quyidagi selektorlarning spetsifikligini hisoblang: `nav ul li a`, `.menu a: hover`, `#top .menu > li`.
- Narxlar jadvali (pricing) yarating: 3 ta tarif kartasi, o'rtadagisi "Mashhur" belgisi bilan kattaroq; mobilda ustma-ust.
- Faqat CSS bilan yuklanish animatsiyasi (spinner) va "skeleton" karta (kulrang miltillovchi bloklar) yasang.

▼ 3-savol javobi

`nav ul li a` → (0,0,4); `.menu a: hover` → (0,2,1); `#top .menu > li` → (1,1,1).

✓ Xulosa

- Kaskad tartibi: `!important` → spetsifiklik → manba tartibi. Asosan sinflardan foydalaning.
- Har bir element — quti: content + padding + border + margin. `box-sizing: border-box` ni unutmang.
- Flexbox — bir o'lchamli, Grid — ikki o'lchamli maket; ular birga ishlatiladi.
- Mobile-first + `min-width` media query + `rem` / `clamp()` = responsiv sayt.
- CSS o'zgaruvchilari mavzular (theme) va izchil dizayn tizimi uchun asos.
- Animatsiyada `transform` va `opacity` dan foydalaning.



04-DARS · 14 TADAN

JavaScript asoslari

O'zgaruvchilar, ma'lumot turlari, operatorlar, shartlar, sikllar, funksiyalar, scope va closure, massiv va obyekt metodlari, sinflar, modullar va xatolar bilan ishlash.

Boshlang'ich–O'rta · ~150 daqiqa

JavaScript — webning yagona dasturlash tili: u brauzerda ishlaydi va sahifani "jonlantiradi". Node.js tufayli esa serverda ham ishlaydi — ya'ni full-stack dasturchi **bitta til** bilan frontend va backendni yoza oladi. Bu dars eng uzun va eng muhim darslardan biri: keyingi barcha mavzular shu yerdagi tushunchalarga tayanadi.

JavaScript'ni ishga tushirish

- **Brauzer konsoli:** `F12` → Console. Tez tajribalar uchun ideal.
- **HTML ichida:** `<script src="app.js" defer></script>`
- **Node.js:** `node app.js` (terminalda)

JavaScript

```
console.log('Salom, dunyo!');           // oddiy chiqarish
console.table([ { ism: 'Ali', yosh: 20 }, { ism: 'Vali', yosh: 22 } ]); // jadval
ko'rinishida
console.error('Xato xabari');           // qizil rangda
```

O'zgaruvchilar: let, const, var

JavaScript

```
const PI = 3.14159;    // qayta tayinlab bo'lmaydi
let yosh = 25;         // qayta tayinlash mumkin
yosh = 26;             // ✓
// PI = 3;             // ✗ TypeError: Assignment to constant variable

const user = { ism: 'Aziz' };
user.izm = 'Anvar';    // ✓ const – havolani qotiradi, obyekt ichini emas!
// user = {};         // ✗
```

	var	let	const
Ko'rinish doirasi	Funksiya	Blok { }	Blok { }
Qayta tayinlash	Ha	Ha	Yo'q
Hoisting	Ha, undefined bilan	Ha, lekin TDZ (e'londan oldin ishlatib bo'lmaydi)	Ha, TDZ



Qoida

Doim `const` dan boshlang. Qiymat o'zgarishi kerak bo'lsagina `let` ga almashtiring. `var` ni umuman ishlatmang — u eski kodlarda uchraydi va kutilmagan xatolarga sabab bo'ladi.

Nomlash: `camelCase` (`userName`, `totalPrice`), konstantalar uchun ba'zan `UPPER_CASE`, sinflar uchun `PascalCase`. Nom ma'noli bo'lsin: `x` emas, `kunlikDaromad`.

Ma'lumot turlari

JavaScript'da 8 ta tur bor: 7 ta **primitiv** va **obyekt**.

Tur	Misol	typeof
-----	-------	--------

String (satr)	'salom', "dunyo", `shablon \${x}`	"string"
Number (son)	42, 3.14, -7, NaN, Infinity	"number"
BigInt	9007199254740993n	"bigint"
Boolean	true, false	"boolean"
undefined	qiymat berilmagan o'zgaruvchi	"undefined"
null	"ataylab bo'sh" qiymat	"object" (tarixiy xato!)
Symbol	Symbol('id') — noyob kalit	"symbol"
Object	{}, [], funksiyalar, Date, Map ...	"object" / "function"

Primitiv va havola (reference) turlar

Bu farqni tushunish ko'plab xatolarning oldini oladi:

JavaScript

```
// Primitivlar QIYMAT bo'yicha nusxalanadi
let a = 10;
let b = a;
b = 20;
console.log(a); // 10 — o'zgarmadi

// Obyektlar HAVOLA bo'yicha nusxalanadi
const user1 = { ism: 'Ali' };
const user2 = user1; // ikkalasi BITTA obyektga ishora qiladi
user2.ism = 'Vali';
console.log(user1.ism); // 'Vali' — o'zgardi!

// Haqiqiy nusxa olish
const copy = { ...user1 }; // sayoz (shallow) nusxa
const deep = structuredClone(user1); // chuqur (deep) nusxa

// Taqqoslash ham havola bo'yicha
console.log({ a: 1 } === { a: 1 }); // false — ikki xil obyekt
```

Satrlar bilan ishlash

JavaScript

```
const ism = 'Malika';
const salom = `Salom, ${ism}! Bugun ${new Date().getFullYear()}-yil.`; //
template literal

const s = '  JavaScript Dasturlash  ';
s.length; // 25
s.trim(); // 'JavaScript Dasturlash'
s.toUpperCase(); // ' JAVASCRIPT DASTURLASH '
s.includes('Script'); // true
s.trim().startsWith('Java'); // true
s.indexOf('a'); // 3
s.trim().slice(0, 10); // 'JavaScript'
'a,b,c'.split(','); // ['a', 'b', 'c']
'ha'.repeat(3); // 'hahaha'
'5'.padStart(3, '0'); // '005'
'salom dunyo'.replaceAll('o', '0'); // 'sa10m duny0'
```

Sonlar va Math

JavaScript

```
0.1 + 0.2; // 0.30000000000000004 (!) – kasr sonlar ikkilik
tizimda aniq emas
(0.1 + 0.2).toFixed(2); // '0.30'
Number('42'); // 42
Number('42px'); // NaN
parseInt('42px'); // 42
parseFloat('3.5kg'); // 3.5
Number.isNaN(NaN); // true
Math.round(4.5); // 5
Math.floor(4.9); // 4
Math.ceil(4.1); // 5
Math.max(3, 7, 2); // 7
Math.floor(Math.random() * 10) + 1; // 1..10 oralig'ida tasodifiy butun son
(1234567.891).toLocaleString('uz-UZ'); // '1 234 567,891'
```



Pul bilan ishlaganda kasr sonlardan saqlaning: summani **tiyinlarda (butun son)** saqlang (`1999 so'm 50 tiyin → 199950`) yoki serverda aniq o'nlik turlaridan (SQL'dagi `NUMERIC`) foydalaning.

Turlarni o'zgartirish (type coercion)

JavaScript

```
'5' + 3;      // '53' - + operatori satr bo'lsa birlashtiradi
'5' - 3;      // 2    - boshqa operatorlar songa aylantiradi
'5' * '2';    // 10
true + 1;     // 2

// Falsy qiymatlar (shartda false deb hisoblanadi) - atigi 8 ta:
// false, 0, -0, 0n, '', null, undefined, NaN
// Qolgan HAMMASI truthy: '0', 'false', [], {}, ' '

// = va ===
0 = '';      // true 🤖 (tur o'zgartiriladi)
0 === '';    // false ✅ (tur ham tekshiriladi)
null = undefined; // true
null === undefined; // false
```



Har doim `===` va `!==` dan foydalaning. `=` ning tur o'zgartirish qoidalari chalkash va xatolarga olib keladi.

Operatorlar

JavaScript

```
// Arifmetik: + - * / % (qoldiq) ** (daraja)
10 % 3;      // 1
2 ** 10;     // 1024

// Qisqa tayinlash
let x = 5;
x += 2; x *= 3; x++; // 22

// Mantiqiy
true && false; // false (VA)
true || false; // true  (YOKI)
!true;         // false (EMAS)

// Qisqa tutashuv (short-circuit) – real kodda juda ko'p ishlatiladi
const ism = kiritilganIsm || 'Mehmon'; // chap tomon falsy bo'lsa – o'ng
const soni = kiritilganSon ?? 10;      // FAQAT null/undefined bo'lsa –
o'ng (0 ni saqlaydi!)
isLoggedIn && showDashboard();         // chap true bo'lsa bajaradi

// Ixtiyoriy zanjir (optional chaining)
const shahar = user?.address?.city;    // address yo'q bo'lsa xato emas,
undefined
user.sayHi?.();                        // metod bo'lsa chaqiradi

// Uchlik (ternary) operator
const status = yosh ≥ 18 ? 'katta' : 'voyaga yetmagan';
```

Shartlar va sikllar

JavaScript

```
// if / else if / else
const ball = 78;
if (ball ≥ 86) {
  console.log("A'lo");
} else if (ball ≥ 71) {
  console.log('Yaxshi');
} else if (ball ≥ 56) {
  console.log('Qoniqarli');
} else {
  console.log('Qoniqarsiz');
}

// switch – bitta qiymatni ko'p variantga solishtirish
switch (kun) {
  case 'shanba':
  case 'yakshanba':
    console.log('Dam olish kuni');
    break; // break unutilsa, keyingi case ham bajariladi!
  default:
    console.log('Ish kuni');
}

// Sikllar
for (let i = 1; i ≤ 5; i++) console.log(i);

const mevalar = ['olma', 'nok', 'uzum'];
for (const meva of mevalar) console.log(meva); // massiv QIYMATLARI

const narxlar = { olma: 12000, nok: 15000 };
for (const kalit in narxlar) console.log(kalit, narxlar[kalit]); // obyekt KALITLARI

let n = 3;
while (n > 0) { console.log(n); n--; }

// break – siklni to'xtatish, continue – keyingi iteratsiyaga o'tish
for (let i = 1; i ≤ 10; i++) {
  if (i % 2 === 0) continue;
  if (i > 7) break;
}
```



```
console.log(i); // 1, 3, 5, 7  
}
```

Funksiyalar

Funksiya — nomlangan, qayta ishlatiladigan kod bloki. JavaScript'da funksiyalar **birinchi darajali qiymatlar** (first-class): ularni o'zgaruvchiga saqlash, argument sifatida berish va funksiyadan qaytarish mumkin.

```
// 1. Function declaration – hoisting tufayli e'londan oldin ham chaqirish
mumkin
function qoshish(a, b) {
  return a + b;
}

// 2. Function expression
const ayirish = function (a, b) {
  return a - b;
};

// 3. Arrow function (strelkali)
const kopaytirish = (a, b) => a * b; // bitta ifoda – return avtomatik
const kvadrat = x => x * x; // bitta parametr – qavs
ixtiyoriy
const yaratish = (ism) => ({ ism, sana: Date.now() }); // obyekt qaytarish –
qavsga o'rang

// Standart parametrlar va rest parametrlar
function salomlash(ism = 'Mehmon', ...qolganlar) {
  return `Salom, ${ism}! Yana ${qolganlar.length} kishi bor.`;
}
salomlash(); // 'Salom, Mehmon! Yana 0 kishi bor.'
salomlash('Ali', 'Vali', 'Gani'); // 'Salom, Ali! Yana 2 kishi bor.'

// Callback – funksiyani argument sifatida berish
function takrorla(n, amal) {
  for (let i = 0; i < n; i++) amal(i);
}
takrorla(3, i => console.log(`${i}-qadam`));
```



Arrow funksiyaning farqi

Arrow funksiyalarda o'zining `this`, `arguments` yo'q — ular tashqi muhitnikini oladi. Shuning uchun obyekt metodlari uchun oddiy funksiya, callback'lar uchun arrow funksiya qulay.

Scope, hoisting va closure

Scope — ko'rinish doirasi

JavaScript

```
const global = 'hamma joyda ko\'rinadi';

function tashqi() {
  const tashqiOzgaruvchi = 'tashqi funksiyada';

  function ichki() {
    const ichkiOzgaruvchi = 'faqat ichkida';
    console.log(global, tashqiOzgaruvchi, ichkiOzgaruvchi); // ✅ hammasi
    ko\'rinadi
  }
  ichki();
  // console.log(ichkiOzgaruvchi); // ❌ ReferenceError
}

if (true) {
  let blokda = 1;
  var funksiyada = 2;
}
// console.log(blokda); // ❌ - let blok bilan cheklangan
console.log(funksiyada); // 2 - var blokni "teshib" chiqadi
```

Ichki doira tashqi doiradagi o'zgaruvchilarni ko'radi, aksincha emas. Bu **leksik scope** deyiladi: o'zgaruvchining ko'rinishi kod *qayerda yozilganiga* bog'liq, qayerda chaqirilganiga emas.

Closure (yopilish)

Closure — funksiya o'zi yaratilgan muhitdagi o'zgaruvchilarni, hatto o'sha muhit tugaganidan keyin ham *eslab qolishi*.

JavaScript

```
function hisoblagichYarat() {  
  let son = 0; // tashqaridan to'g'ridan-to'g'ri o'zgartirib  
  bo'lmaydi (xususiy)  
  return {  
    oshir: () => ++son,  
    kamaytir: () => --son,  
    qiymat: () => son,  
  };  
}  
  
const h1 = hisoblagichYarat();  
h1.oshir(); h1.oshir();  
console.log(h1.qiymat()); // 2  
  
const h2 = hisoblagichYarat(); // yangi, mustaqil "son"  
console.log(h2.qiymat()); // 0
```

Closure'lar hamma joyda: hodisa ishlovchilari, `setTimeout`, React hook'lari, middleware'lar. Ularni tushunish — JavaScript'ni tushunish demak.

Massivlar va ularning metodlari

JavaScript

```
const arr = [10, 20, 30];  
arr.push(40); // oxiriga qo'shish → [10,20,30,40]  
arr.pop(); // oxiridan olish  
arr.unshift(5); // boshiga qo'shish  
arr.shift(); // boshidan olish  
arr.includes(20); // true  
arr.indexOf(30); // 2  
arr.slice(0, 2); // [10, 20] — nusxa, asl massiv o'zgarmaydi  
arr.splice(1, 1); // 1-indeksdan 1 ta elementni O'CHIRADI (asl massivni  
o'zgartiradi)  
arr.join(' - '); // '10 - 30'  
[...arr, 50]; // spread bilan yangi massiv  
Array.from({ length: 5 }, (_, i) => i * 2); // [0, 2, 4, 6, 8]
```

Yuqori tartibli (higher-order) metodlar — eng muhimlari

Bu metodlar asl massivni o'zgartirmaydi va real loyihalarda `for` sikllarining o'rnini deyarli to'liq egallaydi.

```

const mahsulotlar = [
  { nomi: 'Noutbuk', narx: 9_500_000, toifa: 'texnika', soni: 3 },
  { nomi: 'Sichqoncha', narx: 150_000, toifa: 'texnika', soni: 0 },
  { nomi: 'Kitob', narx: 85_000, toifa: 'kitob', soni: 12 },
  { nomi: 'Monitor', narx: 2_800_000, toifa: 'texnika', soni: 5 },
];

// map – har bir elementni o'zgartirib YANGI massiv qaytaradi (uzunlik bir xil)
const nomlar = mahsulotlar.map(m ⇒ m.nomi);
// ['Noutbuk', 'Sichqoncha', 'Kitob', 'Monitor']

// filter – shartga mos elementlar
const mavjud = mahsulotlar.filter(m ⇒ m.soni > 0);

// find / findIndex – birinchi mos element (yoki undefined / -1)
const kitob = mahsulotlar.find(m ⇒ m.toifa === 'kitob');

// some / every – kamida bittasi / hammasi shartga mosmi
mahsulotlar.some(m ⇒ m.soni === 0); // true
mahsulotlar.every(m ⇒ m.narx > 0); // true

// reduce – massivni BITTA qiymatga "yig'ish"
const omborQiymati = mahsulotlar.reduce((jami, m) ⇒ jami + m.narx * m.soni,
0);
// 9_500_000*3 + 0 + 85_000*12 + 2_800_000*5 = 43_520_000

// Toifalar bo'yicha guruhlash (reduce bilan)
const guruhlar = mahsulotlar.reduce((acc, m) ⇒ {
  (acc[m.toifa] ??= []).push(m.nomi);
  return acc;
}, {});
// { texnika: ['Noutbuk','Sichqoncha','Monitor'], kitob: ['Kitob'] }

// sort – DIQQAT: asl massivni o'zgartiradi. toSorted() – nusxa qaytaradi
const arzondan = [...mahsulotlar].sort((a, b) ⇒ a.narx - b.narx);
const alifbo = mahsulotlar.toSorted((a, b) ⇒ a.nomi.localeCompare(b.nomi));

// Zanjir (chaining)
const texnikaNomlari = mahsulotlar
  .filter(m ⇒ m.toifa === 'texnika' && m.soni > 0)

```

```
.sort((a, b) => b.narx - a.narx)
.map(m => `${m.nomi} - ${m.narx.toLocaleString()} so'm`);
```

Obyektlar

JavaScript

```
const talaba = {
  ism: 'Nodira',
  yosh: 21,
  'sevimli fan': 'Matematika', // bo'sh joyli kalit - qo'shtirnoqda
  manzil: { shahar: 'Buxoro', kocha: 'Mustaqillik 5' },
  tanishtir() { // metod
    return `Men ${this.izm}, ${this.yosh} yoshdaman`;
  },
};

talaba.izm; // nuqta orqali
talaba['sevimli fan']; // qavs orqali
const kalit = 'yosh';
talaba[kalit]; // dinamik kalit
talaba.kurs = 3; // yangi xususiyat qo'shish
delete talaba.kurs; // o'chirish
'izm' in talaba; // true

Object.keys(talaba); // ['izm', 'yosh', 'sevimli fan', 'manzil', 'tanishtir']
Object.values({ a: 1, b: 2 }); // [1, 2]
Object.entries({ a: 1, b: 2 }); // [['a', 1], ['b', 2]]
Object.fromEntries(['a', 1]); // { a: 1 }

// Qisqa yozuv
const ism = 'Jamshid', yosh = 30;
const odam = { ism, yosh }; // { ism: 'Jamshid', yosh: 30 }

// Spread bilan birlashtirish / yangilash (immutable usul)
const yangilangan = { ...talaba, yosh: 22, manzil: { ...talaba.manzil, shahar: 'Toshkent' } };
```

Destructuring

JavaScript

```
const { ism, yosh, manzil: { shahar }, kurs = 1 } = talaba; // standart
qiymat bilan

const { ism: talabaIsmi } = talaba; // boshqa
nomga

const [birinchi, ikkinchi, ...qolgan] = [10, 20, 30, 40];
let p = 1, q = 2;
[p, q] = [q, p]; // qiymatlarni almashtirish

// Funksiya parametrlarida — juda ko'p ishlatiladi (ayniqsa React'da)
function kartaChiz({ nomi, narx, chegirma = 0 }) {
  return `${nomi}: ${narx * (1 - chegirma)} so'm`;
}
kartaChiz({ nomi: 'Kitob', narx: 50000, chegirma: 0.1 });
```

JSON

JSON — obyektlarni matn ko'rinishida uzatish formati. Frontend va backend aynan JSON orqali "gaplashadi".

JavaScript

```
const obj = { ism: 'Ali', ballar: [90, 85], faol: true };
const matn = JSON.stringify(obj); // '{"ism":"Ali","ballar":
[90,85],"faol":true}'
const chiroyli = JSON.stringify(obj, null, 2); // 2 bo'shliq bilan formatlangan
const qaytaObj = JSON.parse(matn); // yana obyekt
// JSON'da: kalitlar faqat qo'shtirnoqda, funksiya va undefined bo'lmaydi, izoh
yozib bo'lmaydi
```


Sinflar (class) va OOP

JavaScript

```
class Hisob {
  #balans = 0; // # – haqiqiy xususiy (private) maydon
  static bankNomi = 'Ishonch Bank'; // sinfga tegishli, obyektga emas

  constructor(egasi, boshlangich = 0) {
    this.egasi = egasi;
    this.#balans = boshlangich;
    this.tarix = [];
  }

  get balans() { // getter – xususiyat kabi o'qiladi:
    hisob.balans
    return this.#balans;
  }

  toldirish(summa) {
    if (summa ≤ 0) throw new Error('Summa musbat bo\'lishi kerak');
    this.#balans += summa;
    this.tarix.push({ tur: 'kirim', summa, sana: new Date() });
    return this; // zanjir uchun
  }

  yechish(summa) {
    if (summa > this.#balans) throw new Error('Mablag\' yetarli emas');
    this.#balans -= summa;
    this.tarix.push({ tur: 'chiqim', summa, sana: new Date() });
    return this;
  }
}

// Meros (inheritance)
class JamgarmaHisob extends Hisob {
  constructor(egasi, foiz) {
    super(egasi); // ota konstruktorini chaqirish majburiy
    this.foiz = foiz;
  }

  foizQosh() {
    return this.toldirish(this.balans * this.foiz / 100);
  }
}
```

```

}

const h = new JamgarmaHisob('Laylo', 12);
h.toldirish(1_000_000).foizQosh();
console.log(h.balans);           // 1120000
// console.log(h.#balans);      // ❌ SyntaxError – tashqaridan kirib bo'lmaydi
h instanceof Hisob;             // true

```

Xatolar bilan ishlash

JavaScript

```

function yoshniTekshir(yosh) {
  if (typeof yosh !== 'number' || Number.isNaN(yosh)) {
    throw new TypeError('Yosh son bo\'lishi kerak');
  }
  if (yosh < 0 || yosh > 150) {
    throw new RangeError('Yosh 0 va 150 oraliq\'ida bo\'lishi kerak');
  }
  return true;
}

try {
  yoshniTekshir(-5);
} catch (err) {
  console.error(err.name, err.message); // RangeError Yosh 0 va 150
  oraliq\'ida...
} finally {
  console.log('Bu blok har doim bajariladi (resurslarni tozalash uchun)');
}

// O'z xato sinfigiz
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
  }
}

```

Modullar (ES Modules)

Katta dasturni fayllarga bo'lish uchun modullar ishlatiladi.

utils/format.js

```
// Nomli eksport (bir nechta bo'lishi mumkin)
export const valyuta = (son) => `${son.toLocaleString('uz-UZ')} so'm`;
export function sanaFormat(d) {
  return new Date(d).toLocaleDateString('uz-UZ');
}

// Standart eksport (faylda bitta)
export default function slugify(matn) {
  return matn.toLowerCase().trim().replace(/[a-z0-9]+/g, '-');
}
```

main.js

```
import slugify, { valyuta, sanaFormat as sana } from './utils/format.js';
import * as fmt from './utils/format.js';

console.log(valyuta(150000)); // '150 000 so'm'
console.log(slugify('Salom Dunyo')); // 'salom-dunyo'
```

Brauzerda modullarni ishlatish: `<script type="module" src="main.js"></script>`. Node.js'da esa `package.json` ga `"type": "module"` qo'shiladi (8-dars).

Amaliy misollar

MISOL 1 Internet-do'kon savatchasi hisob-kitobi

Real e-commerce mantiq: chegirmalar, promokod, yetkazib berish narxi va QQS.

```

const savat = [
  { id: 1, nomi: 'Simsiz qu'loqchin', narx: 450_000, soni: 2, chegirma: 0.1 },
  { id: 2, nomi: 'Telefon g\'ilofi', narx: 60_000, soni: 3, chegirma: 0 },
  { id: 3, nomi: 'Quvvatlagich 65W', narx: 280_000, soni: 1, chegirma: 0.15 },
];

const PROMOKODLAR = {
  BAHOR10: { tur: 'foiz', qiymat: 10 },
  MINUS50K: { tur: 'summa', qiymat: 50_000, minSumma: 500_000 },
};

const BEPUL_YETKAZISH_CHEGARASI = 1_000_000;
const YETKAZISH_NARXI = 25_000;

function qatorSummasi({ narx, soni, chegirma = 0 }) {
  return Math.round(narx * (1 - chegirma)) * soni;
}

function promokodQoll(summa, kod) {
  if (!kod) return { summa, xabar: null };
  const promo = PROMOKODLAR[kod.trim().toUpperCase()];
  if (!promo) return { summa, xabar: 'Promokod topilmadi' };
  if (promo.minSumma && summa < promo.minSumma) {
    return { summa, xabar: `Promokod ${promo.minSumma.toLocaleString()} so'mdan yuqori xaridga amal qiladi` };
  }
  const yangi = promo.tur === 'foiz'
    ? Math.round(summa * (1 - promo.qiymat / 100))
    : summa - promo.qiymat;
  return { summa: Math.max(yangi, 0), xabar: 'Promokod qo\'llandi ✅' };
}

function hisobla(savat, promokod) {
  const qatorlar = savat.map(m => ({ ...m, jami: qatorSummasi(m) }));
  const oraliq = qatorlar.reduce((s, q) => s + q.jami, 0);
  const tejaldi = savat.reduce((s, m) => s + m.narx * m.soni, 0) - oraliq;
  const { summa: promodan, xabar } = promokodQoll(oraliq, promokod);
  const yetkazish = promodan >= BEPUL_YETKAZISH_CHEGARASI ? 0 : YETKAZISH_NARXI;
  const jami = promodan + yetkazish;

```

```

const mahsulotSoni = savat.reduce((s, m) => s + m.soni, 0);

return { qatorlar, oraliq, tejalldi, promokodXabari: xabar, yetkazish,
jami, mahsulotSoni };
}

const f = (n) => `${n.toLocaleString('uz-UZ')} so'm`;
const natija = hisobla(savat, 'bahor10');

console.table(natija.qatorlar.map(({ nomi, soni, jami }) => ({ nomi, soni,
jami: f(jami) })));
console.log(`Mahsulotlar: ${natija.mahsulotSoni} ta`);
console.log(`Oraliq summa: ${f(natija.oralik)} (chegirmalar:
-${f(natija.tejalldi)})`);
console.log(natija.promokodXabari);
console.log(`Yetkazish: ${natija.yetkazish ? f(natija.yetkazish) :
'BEPUL'}`);
console.log(`To'lovga: ${f(natija.jami)}`);

```

Nimani o'rgandik: `map` / `reduce` bilan hisob-kitob, destructuring, standart parametrlar, obyektidan qidiruv-jadval (lookup table) sifatida foydalanish, sof funksiyalar (bir xil kirish → bir xil chiqish), raqamli ajratgich (`450_000`). Bu kod 13-darsda React interfeysiga ulanadi.

grades.js

```
const talabalar = [
  { ism: 'Akmal', baholar: { matematika: 92, fizika: 78, ingliz: 85 } },
  { ism: 'Barno', baholar: { matematika: 67, fizika: 88, ingliz: 95 } },
  { ism: 'Davron', baholar: { matematika: 45, fizika: 52, ingliz: 60 } },
  { ism: 'Gulnoza', baholar: { matematika: 98, fizika: 94, ingliz: 91 } },
  { ism: 'Erkin', baholar: { matematika: 73, fizika: 39, ingliz: 70 } },
];

const ortacha = (sonlar) => sonlar.reduce((a, b) => a + b, 0) /
sonlar.length;
const yaxlitla = (n) => Math.round(n * 10) / 10;

function daraja(ball) {
  if (ball ≥ 86) return "A'lo";
  if (ball ≥ 71) return 'Yaxshi';
  if (ball ≥ 56) return 'Qoniqarli';
  return 'Qoniqarsiz';
}

// 1. Har bir talabaning o'rtacha bahosi va darajasi
const reyting = talabalar
  .map(({ ism, baholar }) => {
    const ort = yaxlitla(ortacha(Object.values(baholar)));
    return { ism, ortacha: ort, daraja: daraja(ort) };
  })
  .sort((a, b) => b.ortacha - a.ortacha)
  .map((t, i) => ({ orin: i + 1, ...t }));

console.table(reyting);

// 2. Fanlar bo'yicha statistika
const fanlar = Object.keys(talabalar[0].baholar);
const fanStatistika = fanlar.map(fan => {
  const ballar = talabalar.map(t => t.baholar[fan]);
  const eng = talabalar.reduce((best, t) => (t.baholar[fan] >
best.baholar[fan] ? t : best));
  return {
    fan,
```

```

    ortacha: yaxlitla(ortacha(ballar)),
    max: Math.max(...ballar),
    min: Math.min(...ballar),
    eng_yaxshi: eng.ism,
    yiqilganlar: talabalar.filter(t => t.baholar[fan] < 56).map(t =>
t.ism).join(', ') || '-',
  };
});
console.table(fanStatistika);

// 3. Darajalar bo'yicha taqsimot: { "A'lo": 1, Yaxshi: 2, ... }
const taqsimot = reyting.reduce((acc, { daraja }) => {
  acc[daraja] = (acc[daraja] || 0) + 1;
  return acc;
}, {});
console.log(taqsimot);

// 4. Stipendiya oladiganlar: barcha fanlardan 71+
const stipendiya = talabalar
  .filter(t => Object.values(t.baholar).every(b => b ≥ 71))
  .map(t => t.ism);
console.log('Stipendiya:', stipendiya); // ['Akmal', 'Gulnoza']

```

MISOL 3 Closure bilan amaliy vositalar: memoize, once va debounce

Professional kodda tez-tez uchraydigan uchta yordamchi funksiya — barchasi closure asosida.

```
// 1. memoize — og'ir hisob natijasini keshlash
function memoize(fn) {
  const kesh = new Map(); // closure ichida saqlanadi
  return function (...args) {
    const kalit = JSON.stringify(args);
    if (kesh.has(kalit)) return kesh.get(kalit);
    const natija = fn(...args);
    kesh.set(kalit, natija);
    return natija;
  };
}

const fib = memoize(n => (n < 2 ? n : fib(n - 1) + fib(n - 2)));
console.time('fib');
console.log(fib(90)); // 2880067194370816000 — bir zumda (keshsiz yillab
hisoblanardi)
console.timeEnd('fib');

// 2. once — funksiyanı faqat bir marta bajarish (masalan "To'lash"
tugmasi)
function once(fn) {
  let bajarildi = false, natija;
  return (...args) => {
    if (!bajarildi) {
      bajarildi = true;
      natija = fn(...args);
    }
    return natija;
  };
}

const tolash = once((summa) => { console.log(`${summa} so'm to'landi`);
return 'OK'; });
tolash(100000); // "100000 so'm to'landi"
tolash(100000); // hech narsa — ikkinchi marta bajarilmaydi

// 3. debounce — ketma-ket chaqiruvlarda faqat oxirgisini bajarish
// (qidiruv maydoniga yozilganda har harf uchun emas, yozib bo'lgach
so'rov yuborish)
function debounce(fn, kutish = 300) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
```



```
    timer = setTimeout(() => fn(...args), kutish);  
  };  
}  
const qidir = debounce((soz) => console.log('Serverga so\'rov:', soz),  
500);  
qidir('j'); qidir('ja'); qidir('jav'); qidir('java');  
// 500ms dan keyin faqat bitta: "Serverga so'rov: java"
```

Uchala funksiya ham ichki holatni (`kesh`, `bajarildi`, `timer`) closure orqali saqlaydi. `debounce` ni 6-darsda real qidiruv bilan ishlatamiz.

library.js

```
class KitobMavjudEmasError extends Error {
  constructor(isbn) {
    super(`ISBN ${isbn} bo'yicha bo'sh nusxa yo'q`);
    this.name = 'KitobMavjudEmasError';
  }
}

class Kitob {
  constructor({ isbn, nomi, muallif, nusxalar = 1 }) {
    Object.assign(this, { isbn, nomi, muallif });
    this.jami = nusxalar;
    this.bosh = nusxalar;
  }
  get mavjudmi() { return this.bosh > 0; }
  toString() { return `${this.nomi} - ${this.muallif} (${this.bosh}/${this.jami})`; }
}

class Kutubxona {
  #kitoblar = new Map(); // isbn → Kitob
  #ijaralar = []; // { isbn, oquvchi, berilgan, muddat }
  static IJARA_KUNI = 14;
  static JARIMA_KUNIGA = 2000;

  qoshish(malumot) {
    const mavjud = this.#kitoblar.get(malumot.isbn);
    if (mavjud) {
      mavjud.jami += malumot.nusxalar ?? 1;
      mavjud.bosh += malumot.nusxalar ?? 1;
    } else {
      this.#kitoblar.set(malumot.isbn, new Kitob(malumot));
    }
    return this;
  }

  qidirish(soz) {
    const q = soz.toLowerCase();
    return [...this.#kitoblar.values()].filter(
```

```

        k ⇒ k.nomi.toLowerCase().includes(q) ||
k.muallif.toLowerCase().includes(q)
    );
}

berish(isbn, oquvchi, sana = new Date()) {
    const kitob = this.#kitoblar.get(isbn);
    if (!kitob) throw new Error(`Bunday kitob yo'q: ${isbn}`);
    if (!kitob.mavjudmi) throw new KitobMavjudEmasError(isbn);
    kitob.bosh--;
    const muddat = new Date(sana);
    muddat.setDate(muddat.getDate() + Kutubxona.IJARA_KUNI);
    this.#ijaralar.push({ isbn, oquvchi, berilgan: sana, muddat });
    return muddat;
}

qaytarish(isbn, oquvchi, sana = new Date()) {
    const i = this.#ijaralar.findIndex(r ⇒ r.isbn === isbn && r.oquvchi
=== oquvchi);
    if (i === -1) throw new Error('Bunday ijara topilmadi');
    const [ijara] = this.#ijaralar.splice(i, 1);
    this.#kitoblar.get(isbn).bosh++;
    const kechikish = Math.ceil((sana - ijara.muddat) / 86_400_000);
    return kechikish > 0 ? kechikish * Kutubxona.JARIMA_KUNIGA : 0;
}

get hisobot() {
    const kitoblar = [...this.#kitoblar.values()];
    return {
        nomlar: kitoblar.length,
        jamiNusxa: kitoblar.reduce((s, k) ⇒ s + k.jami, 0),
        ijarada: this.#ijaralar.length,
        tugaganlar: kitoblar.filter(k ⇒ !k.mavjudmi).map(String),
    };
}

// Foydalanish
const lib = new Kutubxona()
    .qoshish({ isbn: '978-1', nomi: "O'tkan kunlar", muallif: 'Abdulla
Qodiriy', nusxalar: 2 })
    .qoshish({ isbn: '978-2', nomi: 'Sariq devni minib', muallif:
"Xudoyberdi To'xtaboyev" })
    .qoshish({ isbn: '978-3', nomi: 'Clean Code', muallif: 'Robert Martin'

```

```
});

lib.berish('978-2', 'Sherzod', new Date('2026-09-01'));
try {
  lib.berish('978-2', 'Madina');
} catch (e) {
  if (e instanceof KitobMavjudEmasError) console.warn('⚠️', e.message);
  else throw e; // kutilmagan xatoni yashirmaymiz
}

const jarima = lib.qaytarish('978-2', 'Sherzod', new Date('2026-09-20'));
console.log(`Jarima: ${jarima} so'm`); // 5 kun kechikish → 10000 so'm
console.log(lib.qidirish('qodiriy').map(String));
console.log(lib.hisobot);
```

Ko'rib chiqilgan tushunchalar: xususiy maydonlar (`#`), statik konstantalar, getter, o'z xato sinfi va `instanceof` bilan tekshirish, `Map`, sanalar bilan hisob, metodlar zanjiri. 10-darsda xuddi shu mantiqni ma'lumotlar bazasiga o'tkazamiz.

Mashqlar



Mustaqil ish

- `sozlarSoni(matn)` funksiyasini yozing: matndagi har bir so'z necha marta uchrashini obyekt ko'rinishida qaytarsin (katta-kichik harf farqsiz, tinish belgilarisiz).
- Massivdagi takroriy elementlarni olib tashlang — ikki usulda: `filter + indexOf` va `new Set()`.
- 1-misolga "bir nechta promokod qo'llab bo'lmaydi" va "chegirmali mahsulotga promokod amal qilmaydi" qoidalarini qo'shing.
- `throttle(fn, ms)` funksiyasini yozing: `fn` har `ms` millisoniyada ko'pi bilan bir marta bajarilsin. U `debounce` dan nimasi bilan farq qiladi?
- `TodoList` sinfini yozing: `add`, `toggle(id)`, `remove(id)`, `filter('all'|'done'|'active')` metodlari va `stats` getteri bilan.

▼ 1-mashq yechimi

JavaScript

```
const sozlarSoni = (matn) =>
  matn.toLowerCase()
    .replace(/^[^p{L}\p{N}\s'-]/gu, '')
    .split(/\s+/)
    .filter(Boolean)
    .reduce((acc, soz) => ({ ...acc, [soz]: (acc[soz] || 0) + 1 }),
  {});

sozlarSoni('Bir, ikki. Bir uch! bir'); // { bir: 3, ikki: 1, uch: 1 }
```

✓ Xulosa

- `const` > `let`; `var` ishlatmang. Har doim `≡`.
- Primitivlar qiymat bo'yicha, obyektlar havola bo'yicha nusxalanadi.
- `??`, `?.`, destructuring va spread — zamonaviy JS'ning kundalik vositalari.
- `map`, `filter`, `reduce`, `find`, `some` / `every` — ma'lumotlar bilan ishlashning asosi.
- Closure — funksiya o'z muhitini eslab qoladi; xususiy holat va yordamchi funksiyalar uchun ishlatiladi.
- Sinflar, `#private` maydonlar, `extends`; xatolarni `throw` / `try...catch` bilan boshqaring.



05-DARS · 14 TADAN

DOM va hodisalar

DOM daraxti, elementlarni topish va o'zgartirish, hodisalar (events), bubbling va delegatsiya, formalar bilan ishlash, localStorage.

O'rta · ~120 daqiqa

Oldingi darsda JavaScript tilini o'rgandik. Endi uni sahifa bilan bog'laymiz. **DOM (Document Object Model)** — brauzer HTML'dan quradigan va JavaScript orqali boshqarish mumkin bo'lgan **obyektlar daraxti**. Tugma bosilganda oyna ochilishi, forma tekshirilishi, ro'yxatga element qo'shilishi — barchasi DOM bilan ishlash.

DOM daraxti

Quyidagi HTML:

HTML

```
<body>
  <h1 id="title">Vazifalar</h1>
  <ul class="list">
    <li>Non olish</li>
    <li>Dars qilish</li>
  </ul>
</body>
```

brauzerda shunday daraxtga aylanadi:

Matn

```
document
├── html
│   ├── head
│   └── body
│       ├── h1#title
│       │   └── "Vazifalar" (matn tuguni)
│       └── ul.list
│           ├── li
│           │   └── "Non olish"
│           └── li
│               └── "Dars qilish"
```

- Daraxtdagi har bir narsa — **tugun (node)**: element tugunlari, matn tugunlari, izoh tugunlari.
- `document` — daraxtning ildizi va DOM bilan ishlashning kirish nuqtasi.
- Elementlar o'rtasida oilaviy munosabatlar bor: ota (parent), farzand (child), aka-uka (sibling).



DOM ≠ HTML manba kodi

DOM — "jonli" tuzilma. JavaScript uni o'zgartirsa, sahifa darhol yangilanadi, lekin serverdagi HTML fayl o'zgarmaydi. DevTools'dagi **Elements** paneli manba kodni emas, aynan joriy DOM'ni ko'rsatadi.

Elementlarni topish

JavaScript

```
// Zamonaviy va universal usul – CSS selektorlari bilan
const title = document.querySelector('#title');           // birinchi mos
// element yoki null
const firstItem = document.querySelector('.list li');
const items = document.querySelectorAll('.list li');      // NodeList (barcha
// moslar)
items.forEach(li => console.log(li.textContent));
const arr = [...items];                                   // haqiqiy massivga
// aylantirish

// Klassik usullar
document.getElementById('title');
document.getElementsByClassName('list'); // "jonli" HTMLCollection
document.getElementsByTagName('li');

// Element ichidan qidirish
const list = document.querySelector('.list');
list.querySelectorAll('li');

// Daraxt bo'ylab harakatlanish
const li = list.firstElementChild;
li.parentElement;           // ul
li.nextElementSibling;      // ikkinchi li
li.closest('.list');         // eng yaqin mos ajdod (o'zi ham kiradi) – juda
// foydali!
list.children;               // farzand elementlar
```


Elementlarni o'zgartirish

Mazmun

JavaScript

```
const el = document.querySelector('#title');

el.textContent = 'Yangi sarlavha';           // faqat matn – XAVFSIZ
el.innerHTML = 'Bugungi <em>vazifalar</em>'; // HTML sifatida talqin qilinadi
– EHTIYOT BO'LING

// ❌ Hech qachon foydalanuvchi kiritgan ma'lumotni innerHTML ga qo'ymang:
const izoh = '<img src=x onerror="alert(document.cookie)">';
// el.innerHTML = izoh; // XSS hujumi! Skript bajariladi
el.textContent = izoh; // ✅ shunchaki matn sifatida ko'rinadi
```



XSS (Cross-Site Scripting)

`innerHTML` ga tekshirilmagan ma'lumot qo'yish — webdagi eng keng tarqalgan zaifliklardan biri. Hujumchi izoh, ism yoki qidiruv so'zi orqali o'z skriptini boshqa foydalanuvchilar brauzerida bajarishi mumkin. Foydalanuvchi ma'lumoti uchun doim `textContent` dan foydalaning (14-darsda batafsil).

Atributlar va sinflar

JavaScript

```
const img = document.querySelector('img');
img.src = 'yangi.jpg'; // standart atributlar xususiyat
sifatida
img.alt = 'Tavsif';
img.setAttribute('loading', 'lazy');
img.getAttribute('width');
img.removeAttribute('title');
img.hasAttribute('alt');

// data-* atributlar → dataset
// <button data-product-id="42" data-price="15000">
btn.dataset.productId; // '42' (har doim satr!)
Number(btn.dataset.price); // 15000

// Sinflar – uslubni o'zgartirishning to'g'ri usuli
el.classList.add('active');
el.classList.remove('hidden');
el.classList.toggle('open'); // bor bo'lsa olib tashlaydi, yo'q
bo'lsa qo'shadi
el.classList.toggle('dark', isDark); // ikkinchi argument: majburan
qo'shish/olib tashlash
el.classList.contains('active');

// Inline uslublar (kamdan-kam – asosan dinamik qiymatlar uchun)
el.style.width = `${progress}%`;
el.style.setProperty('--accent', '#e11d48'); // CSS o'zgaruvchisi
getComputedStyle(el).fontSize; // haqiqiy hisoblangan uslub
```

Element yaratish, qo'shish va o'chirish

JavaScript

```
const li = document.createElement('li');
li.textContent = 'Yangi vazifa';
li.className = 'task';
li.dataset.id = crypto.randomUUID();

const list = document.querySelector('.list');
list.append(li);           // oxiriga (bir nechta tugun va satr qabul qiladi)
list.prepend(li);         // boshiga
list.before(el); list.after(el); // ro'yxatdan oldin/keyin
li.replaceWith(boshqaEl);
li.remove();              // o'chirish

// HTML satrdan joylash (ishonchli, statik shablon uchun)
list.insertAdjacentHTML('beforeend', '<li class="task">Statik matn</li>');

// <template> – HTML'da tayyor shablon
// <template id="card-tpl"><article class="card"><h3></h3><p></p></article>
// </template>
const tpl = document.querySelector('#card-tpl');
const card = tpl.content.cloneNode(true);
card.querySelector('h3').textContent = 'Sarlavha';
document.body.append(card);
```



Unumdorlik: ko'p elementni bir yo'la qo'shing

Siklda har bir elementni alohida DOM'ga qo'shish har safar Layout'ni qayta hisoblashi mumkin. 1000 ta elementni avval `DocumentFragment` ga yoki massivga yig'ib, keyin bir marta qo'shing:

```
list.append(...elements) yoki list.replaceChildren(...elements) .
```

Hodisalar (Events)

Hodisa — brauzerda sodir bo'lgan narsa: bosish, klavish bosilishi, forma yuborilishi, sahifa yuklanishi, scroll. Biz hodisaga **tinglovchi (listener)** — funksiya biriktiramiz.

```
const btn = document.querySelector('#save');

function saqlash(event) {
  console.log('Hodisa turi:', event.type);           // 'click'
  console.log('Bosilgan element:', event.target);
  console.log('Koordinatalar:', event.clientX, event.clientY);
}

btn.addEventListener('click', saqlash);
btn.removeEventListener('click', saqlash); // olib tashlash uchun AYNI
funksiya kerak

// Variantlar
btn.addEventListener('click', saqlash, { once: true }); // faqat bir marta
window.addEventListener('scroll', onScroll, { passive: true }); // scroll
unumdorligi uchun

// ❌ Eskicha usul (bitta hodisaga faqat bitta ishlovchi):
// btn.onclick = saqlash;
// <button onclick="saqlash()"> - HTML va JS aralashib ketadi
```

Ko'p ishlatiladigan hodisalar

Toifa	Hodisalar
Sichqoncha	click, dblclick, contextmenu, mouseenter / mouseleave, mousemove
Pointer (sichqoncha + sensor)	pointerdown, pointermove, pointerup
Klaviatura	keydown, keyup (event.key : 'Enter', 'Escape', 'a')
Forma	submit, input (har o'zgarishda), change (yakunlanganda), focus, blur, reset
Hujjat/oyina	DOMContentLoaded, load, resize, scroll, beforeunload
Boshqalar	dragstart / drop, copy / paste, animationend, transitionend

Standart xatti-harakatni to'xtatish

JavaScript

```
form.addEventListener('submit', (e) => {
  e.preventDefault();      // sahifa qayta yuklanmasin – ma'lumotni JS orqali
                             yuboramiz
});

link.addEventListener('click', (e) => {
  e.preventDefault();      // havolaga o'tmasin
});

document.addEventListener('keydown', (e) => {
  if ((e.ctrlKey || e.metaKey) && e.key === 's') {
    e.preventDefault();    // brauzerning "Saqlash" oynasi o'rniga o'z
                             funksiyamiz
    saqlash();
  }
});
```

Hodisa tarqalishi: capturing va bubbling

Ichma-ich elementlardan biri bosilganda hodisa uch bosqichdan o'tadi:

1. **Capturing** — `window` dan maqsad elementga qarab pastga tushadi;
2. **Target** — maqsad elementga yetadi;
3. **Bubbling** — maqsad elementdan `window` gacha yuqoriga "ko'tariladi" (pufakchadek).

HTML

```
<div id="karta">
  <button id="tugma">Bos</button>
</div>
<script>
  karta.addEventListener('click', () => console.log('2. karta'));
  tugma.addEventListener('click', () => console.log('1. tugma'));
  document.body.addEventListener('click', () => console.log('3. body'));
  // Tugma bosilsa: 1. tugma → 2. karta → 3. body
</script>
```

- `event.target` — hodisa **haqiqatda** sodir bo'lgan element (bosilgan tugma).
- `event.currentTarget` — tinglovchi **biriktirilgan** element.
- `event.stopPropagation()` — ko'tarilishni to'xtatadi (kamdan-kam va ehtiyotkorlik bilan ishlatish).

Hodisa delegatsiyasi (event delegation)

Bubbling tufayli 100 ta tugmaning har biriga tinglovchi qo'yish shart emas — **ularning umumiy otasiga bittasini** qo'yish yetarli. Bu usul dinamik qo'shilgan elementlar uchun ham avtomatik ishlaydi.

JavaScript

```
// ❌ Har bir tugmaga alohida — yangi qo'shilgan elementlar ishlamaydi
document.querySelectorAll('.delete-btn').forEach(btn =>
  btn.addEventListener('click', ...)
);

// ✅ Delegatsiya — bitta tinglovchi
document.querySelector('#todo-list').addEventListener('click', (e) => {
  const btn = e.target.closest('button[data-action]');
  if (!btn) return; // tugmadan boshqa joy
  bosildi
  const item = btn.closest('li');
  const id = item.dataset.id;

  if (btn.dataset.action === 'delete') deleteTodo(id);
  if (btn.dataset.action === 'edit') editTodo(id);
});
```

Formalar bilan ishlash

JavaScript

```
const form = document.querySelector('#signup');

form.addEventListener('submit', (e) => {
  e.preventDefault();

  // 1-usul: alohida maydonlar
  const email = form.elements.email.value.trim();

  // 2-usul: FormData — barcha maydonlarni bir yo'la
  const data = Object.fromEntries(new FormData(form));
  // { email: '...', password: '...', course: 'fullstack' }

  // Bir xil name'li checkbox'lar uchun
  const days = new FormData(form).getAll('days');

  console.log(data, days);
  form.reset();
});

// Real vaqtda kuzatish
form.elements.password.addEventListener('input', (e) => {
  const len = e.target.value.length;
  hint.textContent = len < 8 ? `Yana ${8 - len} ta belgi kerak` : '✅ Yaxshi';
});

// checkbox va select
checkbox.checked;           // true / false
select.value;           // tanlangan option value
select.selectedOptions[0].text;
```

Brauzer xotirasi: localStorage va sessionStorage

JavaScript

```
// Faqat SATR saqlaydi — obyektlar uchun JSON kerak
localStorage.setItem('theme', 'dark');
localStorage.getItem('theme');           // 'dark' (yoki yo'q bo'lsa null)
localStorage.removeItem('theme');
localStorage.clear();

const todos = [{ id: 1, text: 'Dars', done: false }];
localStorage.setItem('todos', JSON.stringify(todos));
const saved = JSON.parse(localStorage.getItem('todos') ?? '[]');

// sessionStorage — xuddi shunday, lekin brauzer yorlig'i yopilganda o'chadi
```

	localStorage	sessionStorage	Cookie
Hajm	~5–10 MB	~5 MB	~4 KB
Muddati	Abadiy (o'chirilguncha)	Yorliq yopilguncha	Belgilangan muddat
Serverga yuboriladimi	Yo'q	Yo'q	Ha, har so'rovda
JS'dan o'qish	Ha	Ha	Ha (HttpOnly bo'lmasa)

 localStorage'ga maxfiy ma'lumot (parol, karta raqami) saqlamang — sahifadagi har qanday skript (shu jumladan XSS orqali kiritilgan) uni o'qiy oladi.

Amaliy misollar

MISOL 1

To'liq Todo ilovasi (CRUD, filtr, localStorage)

Klassik, lekin DOM'ning deyarli barcha tushunchalarini qamrab oladigan loyiha. Arxitektura muhim: **holat (state)** alohida massivda saqlanadi, DOM esa har doim shu holatdan **chiziladi (render)**. Bu React'ning asosiy g'oyasi (12-dars).

```
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vazifalar</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <h1>📝 Vazifalar</h1>
    <form id="todo-form" autocomplete="off">
      <input name="text" placeholder="Yangi vazifa..." required
maxlength="120">
      <button>Qo'shish</button>
    </form>

    <div class="filters" role="tablist">
      <button data-filter="all" class="active">Barchasi</button>
      <button data-filter="active">Faol</button>
      <button data-filter="done">Bajarilgan</button>
    </div>

    <ul id="todo-list"></ul>

    <footer class="footer">
      <span id="counter"></span>
      <button id="clear-done">Bajarilganlarni tozalash</button>
    </footer>
  </main>
  <script src="app.js"></script>
</body>
</html>
```

```

// ===== Holat (state) =====
const STORAGE_KEY = 'todos-v1';
let todos = load();
let filter = 'all';

function load() {
  try { return JSON.parse(localStorage.getItem(STORAGE_KEY)) ?? []; }
  catch { return []; }
}

function save() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(todos));
}

// ===== Amallar: holatni o'zgartirib, keyin render =====
function addTodo(text) {
  todos.unshift({ id: crypto.randomUUID(), text, done: false, createdAt:
Date.now() });
  update();
}

function toggleTodo(id) {
  todos = todos.map(t => (t.id === id ? { ...t, done: !t.done } : t));
  update();
}

function deleteTodo(id) {
  todos = todos.filter(t => t.id !== id);
  update();
}

function editTodo(id, text) {
  todos = todos.map(t => (t.id === id ? { ...t, text } : t));
  update();
}

function update() { save(); render(); }

// ===== Render: holatdan DOM chizish =====
const list = document.querySelector('#todo-list');
const counter = document.querySelector('#counter');

function render() {
  const visible = todos.filter(t =>
    filter === 'all' ? true : filter === 'done' ? t.done : !t.done
  );

```

```

list.replaceChildren(...visible.map(createItem));

if (visible.length === 0) {
  const empty = document.createElement('li');
  empty.className = 'empty';
  empty.textContent = 'Hozircha vazifa yo\'q 🤖';
  list.append(empty);
}

const left = todos.filter(t => !t.done).length;
counter.textContent = `${left} ta faol vazifa`;

document.querySelectorAll('[data-filter]').forEach(b =>
  b.classList.toggle('active', b.dataset.filter === filter)
);
}

function createItem(todo) {
  const li = document.createElement('li');
  li.dataset.id = todo.id;
  li.classList.toggle('done', todo.done);

  const checkbox = document.createElement('input');
  checkbox.type = 'checkbox';
  checkbox.checked = todo.done;
  checkbox.dataset.action = 'toggle';

  const span = document.createElement('span');
  span.className = 'text';
  span.textContent = todo.text; // ✅ textContent – XSS'dan
himoya
  span.title = 'Tahrirlash uchun ikki marta bosish';

  const del = document.createElement('button');
  del.dataset.action = 'delete';
  del.setAttribute('aria-label', 'O\'chirish');
  del.textContent = 'x';

  li.append(checkbox, span, del);
  return li;
}

// ===== Hodisalar =====
document.querySelector('#todo-form').addEventListener('submit', (e) => {

```

```

    e.preventDefault();
    const input = e.target.elements.text;
    const text = input.value.trim();
    if (!text) return;
    addTodo(text);
    input.value = '';
    input.focus();
  });

  // Delegatsiya: bitta tinglovchi barcha elementlar uchun
  list.addEventListener('click', (e) => {
    const li = e.target.closest('li[data-id]');
    if (!li) return;
    const action = e.target.dataset.action;
    if (action === 'toggle') toggleTodo(li.dataset.id);
    if (action === 'delete') deleteTodo(li.dataset.id);
  });

  // Ikki marta bosib tahrirlash
  list.addEventListener('dblclick', (e) => {
    const span = e.target.closest('.text');
    if (!span) return;
    const li = span.closest('li');
    const input = document.createElement('input');
    input.value = span.textContent;
    input.className = 'edit';
    span.replaceWith(input);
    input.focus();

    const finish = (saveIt) => {
      const value = input.value.trim();
      if (saveIt && value) editTodo(li.dataset.id, value);
      else render();
    };

    input.addEventListener('keydown', (ev) => {
      if (ev.key === 'Enter') finish(true);
      if (ev.key === 'Escape') finish(false);
    });
    input.addEventListener('blur', () => finish(true), { once: true });
  });

  document.querySelector('.filters').addEventListener('click', (e) => {
    const btn = e.target.closest('[data-filter]');
    if (!btn) return;

```

```
filter = btn.dataset.filter;
render();
});

document.querySelector('#clear-done').addEventListener('click', () => {
  todos = todos.filter(t => !t.done);
  update();
});

render();
```

```

body { font-family: system-ui, sans-serif; background: #f1f5f9; margin: 0; }

.app { max-width: 520px; margin: 48px auto; background: #fff; padding: 28px; border-radius: 16px; box-shadow: 0 10px 30px rgb(0 0 0 / .06); }

#todo-form { display: flex; gap: 8px; }
#todo-form input { flex: 1; padding: 12px; border: 1px solid #cbd5e1; border-radius: 10px; font: inherit; }
button { font: inherit; cursor: pointer; }
#todo-form button { padding: 0 18px; border: 0; border-radius: 10px; background: #4f46e5; color: #fff; }

.filters { display: flex; gap: 6px; margin: 16px 0; }
.filters button { border: 1px solid #e2e8f0; background: #fff; padding: 6px 12px; border-radius: 999px; }
.filters button.active { background: #eef2ff; border-color: #4f46e5; color: #4f46e5; }

#todo-list { list-style: none; padding: 0; margin: 0; }
#todo-list li { display: flex; align-items: center; gap: 10px; padding: 12px 4px; border-bottom: 1px solid #f1f5f9; }
#todo-list li .text { flex: 1; }
#todo-list li.done .text { text-decoration: line-through; color: #94a3b8; }
#todo-list li button { border: 0; background: none; color: #ef4444; opacity: .5; }
#todo-list li:hover button { opacity: 1; }

.edit { flex: 1; padding: 6px; font: inherit; }
.empty { justify-content: center; color: #94a3b8; }
.footer { display: flex; justify-content: space-between; margin-top: 16px; color: #64748b; font-size: 14px; }
.footer button { border: 0; background: none; color: #64748b; text-decoration: underline; }

```

MISOL 2 Modal oyna (accessibility bilan)

HTML'ning o'rnatilgan `<dialog>` elementi fokusni boshqarish va `Esc` bilan yopishni o'zi bajaradi.

```

<button id="open-modal">Buyurtmani tasdiqlash</button>

<dialog id="confirm-modal" aria-labelledby="modal-title">
  <form method="dialog">
    <h2 id="modal-title">Buyurtmani tasdiqlaysizmi?</h2>
    <p>Jami: <b>1 105 200 so'm</b>. Buyurtma 2 kun ichida yetkaziladi.</p>
    <menu>
      <button value="cancel">Bekor qilish</button>
      <button value="confirm" class="primary">Tasdiqlash</button>
    </menu>
  </form>
</dialog>

<p id="status"></p>

<style>
  dialog { border: 0; border-radius: 16px; padding: 28px; max-width:
420px; box-shadow: 0 20px 60px rgb(0 0 0 / .25); }
  dialog::backdrop { background: rgb(15 23 42 / .55); backdrop-filter:
blur(3px); }
  dialog[open] { animation: pop .2s ease; }
  @keyframes pop { from { opacity: 0; transform: scale(.95); } }
  menu { display: flex; justify-content: flex-end; gap: 8px; padding: 0; }
  .primary { background: #4f46e5; color: #fff; border: 0; padding: 8px
16px; border-radius: 8px; }
</style>

<script>
  const modal = document.querySelector('#confirm-modal');
  const status = document.querySelector('#status');

  document.querySelector('#open-modal').addEventListener('click', () =>
modal.showModal());

  // Fon (backdrop) bosilganda yopish
  modal.addEventListener('click', (e) => {
    if (e.target === modal) modal.close('cancel');
  });

  // Qaysi tugma bilan yopilganini bilish
  modal.addEventListener('close', () => {
    status.textContent = modal.returnValue === 'confirm'

```



```
    ? '✅ Buyurtma qabul qilindi!'  
    : 'Buyurtma bekor qilindi.';  
  });  
</script>
```

`<form method="dialog">` ichidagi tugma bosilganda dialog avtomatik yopiladi va tugmaning `value` si `modal.returnValue` ga yoziladi. `showModal()` orqa fondagi sahifani "faolsizlantiradi" — klaviatura fokusi oynadan tashqariga chiqmaydi.

HTML

```
<form id="register" novalidate>
  <div class="field">
    <label for="username">Foydalanuvchi nomi</label>
    <input id="username" name="username">
    <small class="error"></small>
  </div>
  <div class="field">
    <label for="email">Email</label>
    <input id="email" name="email" type="email">
    <small class="error"></small>
  </div>
  <div class="field">
    <label for="password">Parol</label>
    <input id="password" name="password" type="password">
    <div class="meter"><span></span></div>
    <small class="error"></small>
  </div>
  <div class="field">
    <label for="confirm">Parolni tasdiqlang</label>
    <input id="confirm" name="confirm" type="password">
    <small class="error"></small>
  </div>
  <button type="submit">Ro'yxatdan o'tish</button>
</form>

<style>
  .field { margin-bottom: 14px; display: grid; gap: 4px; }
  input { padding: 10px; border: 2px solid #cbd5e1; border-radius: 8px; }
  .field.invalid input { border-color: #ef4444; }
  .field.valid input { border-color: #22c55e; }
  .error { color: #ef4444; min-height: 1.2em; }
  .meter { height: 6px; background: #e2e8f0; border-radius: 3px; overflow:
hidden; }
  .meter span { display: block; height: 100%; width: 0; transition: width
.3s, background .3s; }
</style>

<script>
```

```

const form = document.querySelector('#register');

// Har bir maydon uchun qoidalar: xato matnini yoki null qaytaradi
const rules = {
  username: (v) =>
    !v ? 'Majburiy maydon' :
    !/^[a-z0-9_]{3,20}$/i.test(v) ? '3-20 ta lotin harf, raqam yoki _' :
    null,
  email: (v) =>
    !v ? 'Majburiy maydon' :
    !/^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/i.test(v) ? 'Email noto\'g\'ri' :
    null,
  password: (v) =>
    v.length < 8 ? 'Kamida 8 ta belgi' :
    !/\d/.test(v) || !/[A-Za-z]/.test(v) ? 'Harf va raqam bo\'lishi
shart' : null,
  confirm: (v, all) => v !== all.password ? 'Parollar mos emas' : null,
};

function validateField(name) {
  const values = Object.fromEntries(new FormData(form));
  const error = rules[name](values[name] ?? '', values);
  const field = form.elements[name].closest('.field');
  field.querySelector('.error').textContent = error ?? '';
  field.classList.toggle('invalid', !!error);
  field.classList.toggle('valid', !error);
  return !error;
}

function passwordStrength(p) {
  let score = 0;
  if (p.length ≥ 8) score++;
  if (p.length ≥ 12) score++;
  if (/[A-Z]/.test(p) && /[a-z]/.test(p)) score++;
  if (/\\d/.test(p)) score++;
  if (/^[^A-Za-z0-9]/.test(p)) score++;
  return score; // 0..5
}

form.addEventListener('input', (e) => {
  const name = e.target.name;
  validateField(name);
  if (name === 'password') {
    if (form.elements.confirm.value) validateField('confirm');
  }
});

```

```

    const s = passwordStrength(e.target.value);
    const bar = form.querySelector('.meter span');
    bar.style.width = `${s * 20}%`;
    bar.style.background = ['#ef4444', '#ef4444', '#f59e0b', '#eab308',
'#22c55e', '#16a34a'][s];
  }
});

form.addEventListener('submit', (e) => {
  e.preventDefault();
  const ok = Object.keys(rules).map(validateField).every(Boolean);
  if (!ok) {
    form.querySelector('.invalid input').focus();
    return;
  }
  const { confirm, ...data } = Object.fromEntries(new FormData(form));
  console.log('Serverga yuboriladi:', data); // 6-darsda fetch() bilan
yuboramiz
});
</script>

```

Diqqat qiling: barcha maydonlar uchun **bitta** `input` tinglovchisi (delegatsiya); qoidalar ma'lumot sifatida (`rules` obyekti) saqlangan — yangi maydon qo'shish uchun faqat bitta qoida yozish kifoya. `novalidate` esa brauzerning standart xabarlarini o'chirib, o'zimiznikini ko'rsatishga imkon beradi.

HTML

```

<div class="tabs" data-tabs>
  <div role="tablist">
    <button role="tab" aria-selected="true" aria-controls="p1"
id="t1">Tavsif</button>
    <button role="tab" aria-selected="false" aria-controls="p2" id="t2"
tabindex="-1">Xususiyatlar</button>
    <button role="tab" aria-selected="false" aria-controls="p3" id="t3"
tabindex="-1">Sharhlar (12)</button>
  </div>
  <section role="tabpanel" id="p1" aria-labelledby="t1">Mahsulot
tavsifi... </section>
  <section role="tabpanel" id="p2" aria-labelledby="t2" hidden>Texnik
xususiyatlar... </section>
  <section role="tabpanel" id="p3" aria-labelledby="t3" hidden>Xaridorlar
sharhlari... </section>
</div>

<h2>Ko'p beriladigan savollar</h2>
<div class="faq">
  <details><summary>Yetkazib berish qancha vaqt oladi?</summary>
<p>Toshkent bo'ylab 1 kun, viloyatlarga 2-4 kun.</p></details>
  <details><summary>Mahsulotni qaytarish mumkinmi?</summary><p>Ha, 10 kun
ichida, qadoq buzilmagan bo'lsa.</p></details>
  <details><summary>Qanday to'lov usullari bor?</summary><p>Click, Payme,
Uzum, naqd pul.</p></details>
</div>

<script>
  // Tablar: sichqoncha va klaviatura (← →) bilan
  document.querySelectorAll('[data-tabs]').forEach((root) => {
    const tabs = [...root.querySelectorAll('[role="tab"]')];

    function select(tab) {
      tabs.forEach((t) => {
        const active = t ≡ tab;
        t.setAttribute('aria-selected', active);
        t.tabIndex = active ? 0 : -1;
        root.querySelector('#' + t.getAttribute('aria-controls')).hidden =

```

```

!active;
  });
  tab.focus();
}

root.addEventListener('click', (e) => {
  const tab = e.target.closest('[role="tab"]');
  if (tab) select(tab);
});

root.addEventListener('keydown', (e) => {
  const i = tabs.indexOf(document.activeElement);
  if (i === -1) return;
  if (e.key === 'ArrowRight') select(tabs[(i + 1) % tabs.length]);
  if (e.key === 'ArrowLeft') select(tabs[(i - 1 + tabs.length) %
tabs.length]);
});
});

// Akkordeon: bir vaqtda faqat bitta savol ochiq bo'lsin
const faq = document.querySelector('.faq');
faq.addEventListener('toggle', (e) => {
  if (!e.target.open) return;
  faq.querySelectorAll('details[open]').forEach((d) => {
    if (d !== e.target) d.open = false;
  });
}, true); // toggle hodisasi bubbling qilmaydi – capturing bosqichida
ushlaymiz
</script>

```

Bu yerda ARIA atributlari (`role`, `aria-selected`, `aria-controls`) ekran o'quvchilarga komponent tuzilishini tushuntiradi, `tabIndex` esa "roving tabindex" usuli bilan klaviatura navigatsiyasini to'g'ri qiladi. Akkordeonda `toggle` hodisasi ko'tarilmagani uchun `addEventListener` ning uchinchi argumenti `true` (capturing) ishlatilgan.

Mashqlar



Mustaqil ish

1. Todo ilovasiga quyidagilarni qo'shing: muddat (`date`), muhimlik darajasi (rangli belgi), drag & drop bilan tartiblash.
2. Rasm galereyasi: kichik rasmlar bosilganda katta ko'rinishda (lightbox) ochilsin, `←` `→` bilan almashsin, `Esc` bilan yopilsin.
3. "Xarid kalkulyatori": mahsulotlar ro'yxati, har birida +/- tugmalari, jami summa real vaqtda yangilansin (4-dars 1-misol funksiyalaridan foydalaning).
4. Sahifa pastga scroll qilinganda paydo bo'ladigan "↑ Yuqoriga" tugmasini yasang.
5. Kalit so'z bo'yicha jadval qatorlarini filtrlaydigan qidiruv maydonini yarating; mos kelgan qism `<mark>` bilan belgilansin (XSS'dan ehtiyot bo'ling!).



Xulosa

- DOM — HTML'dan qurilgan jonli obyektlar daraxti; `querySelector` / `closest` bilan topiladi.
- Foydalanuvchi ma'lumoti uchun `textContent` ; `innerHTML` — XSS xavfi.
- Uslubni `classList` orqali o'zgartiring; ma'lumotni `data-*` va `dataset` da saqlang.
- Hodisalar bubbling qiladi — delegatsiya bilan bitta tinglovchi ko'plab elementga xizmat qiladi.
- Formalarda `preventDefault()` + `FormData` .
- Holat (state) → render arxitekturasida kodni tartibli qiladi va React'ga tayyorlaydi.



06-DARS · 14 TADAN

Asinxron JavaScript va Fetch API

Event loop, callback, Promise, async/await, fetch bilan API'ga so'rov yuborish, xatolarni boshqarish, parallel so'rovlar va so'rovni bekor qilish.

O'rta · ~120 daqiqa

Veb-ilova doimo "kutadi": serverdan javobni, foydalanuvchi bosishini, faylning o'qilishini, taymerni. Agar har bir kutish butun dasturni to'xtatib qo'ysa, sahifa qotib qolardi. **Asinxron dasturlash** — uzoq davom etadigan amallarni boshlab, natijasini kutayotganda boshqa ishlarni davom ettirish san'ati. Frontend'ni backend bilan bog'lash aynan shu bilimga tayanadi.

Sinxron va asinxron kod

JavaScript

```
console.log('1. Boshlandi');

setTimeout(() => {
  console.log('3. Taymer ishladi (0 ms bo\'lsa ham!)');
}, 0);

console.log('2. Tugadi');

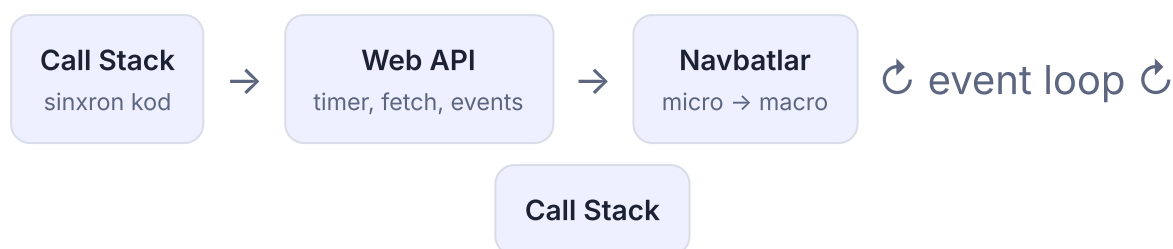
// Natija: 1 → 2 → 3
```

Nega **0 ms** taymer oxirida bajarildi? Javob — **event loop** da.

Event loop — JavaScript qanday ishlaydi

JavaScript **bir oqimli (single-threaded)**: bir vaqtda faqat bitta kod qatorini bajaradi. Lekin brauzer (yoki Node.js) unga yordamchi mexanizmlar beradi:

- **Call Stack (chaqiruvlar steki)** — hozir bajarilayotgan funksiyalar.
- **Web API'lar** — brauzerning o'zi bajaradigan ishlar: taymerlar, tarmoq so'rovlari, hodisalar. Ular JS oqimini band qilmaydi.
- **Macrotask navbati** — `setTimeout`, `setInterval`, hodisa ishlovchilari callback'lari.
- **Microtask navbati** — Promise callback'lari (`.then`, `await` dan keyingi kod). **Ustuvorligi yuqori.**



Stek bo'shagach, avval BARCHA microtask'lar, keyin BITTA macrotask bajariladi, so'ng sahifa qayta chiziladi

Event loop algoritmi: (1) stekdagi sinxron kodni oxirigacha bajar → (2) microtask navbatini to'liq bo'shat → (3) kerak bo'lsa sahifani qayta chiz → (4) macrotask navbatidan bittasini olib, 1-qadamga qayt.

JavaScript

```
console.log('A');
setTimeout(() => console.log('B - macrotask'), 0);
Promise.resolve().then(() => console.log('C - microtask'));
queueMicrotask(() => console.log('D - microtask'));
console.log('E');

// Natija: A, E, C, D, B
```



Event loop'ni bloklamang

Og'ir sinxron hisob (masalan, 5 soniyalik sikl) davomida sahifa butunlay qotadi — tugmalar ishlamaydi, animatsiya to'xtaydi. Node.js serverida esa bunday kod **barcha** foydalanuvchilarning so'rovlarini to'xtatib qo'yadi. Og'ir hisoblar uchun Web Worker (brauzer) yoki worker_threads (Node.js) ishlatiladi.

Callback'lar va "callback hell"

Asinxronlikning eng qadimiy usuli — natija tayyor bo'lganda chaqiriladigan funksiya (callback) berish:

JavaScript

```
function getUser(id, callback) {
  setTimeout(() => callback(null, { id, name: 'Ali' }), 500);
}

// Ketma-ket bog'liq amallar → "piramida" (callback hell)
getUser(1, (err, user) => {
  if (err) return console.error(err);
  getOrders(user.id, (err, orders) => {
    if (err) return console.error(err);
    getOrderDetails(orders[0].id, (err, details) => {
      if (err) return console.error(err);
      getShipping(details.shippingId, (err, shipping) => {
        // ... va hokazo — o'qish, xatolarni boshqarish juda qiyin
      });
    });
  });
});
```

Promise

Promise (va'da) — kelajakda tayyor bo'ladigan natijani ifodalovchi obyekt. U uch holatdan birida bo'ladi:

- **pending** — kutilmoqda;
- **fulfilled** — muvaffaqiyatli bajarildi (qiymat bilan);
- **rejected** — xato bilan tugadi (sabab bilan).

Holat bir marta o'zgaradi va boshqa o'zgarmaydi ("settled").

JavaScript

```
// Promise yaratish
function kutish(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

function tangaTashlash() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      Math.random() > 0.5
        ? resolve('Gerb 🦅')
        : reject(new Error('Raqam chiqdi 🎲'));
    }, 1000);
  });
}

// Promise'ni ishlatish
tangaTashlash()
  .then((natija) => console.log('Yutdingiz:', natija))
  .catch((xato) => console.log('Yutqazdingiz:', xato.message))
  .finally(() => console.log('O\'yin tugadi'));
```

Zanjir (chaining)

`.then()` har doim **yangi Promise** qaytaradi. Callback'dan qaytarilgan qiymat keyingi `.then` ga o'tadi; agar Promise qaytarilsa — u tugashini kutadi. Bu callback piramidasini tekis zanjirga aylantiradi:

JavaScript

```
getUser(1)
  .then(user => getOrders(user.id))
  .then(orders => getOrderDetails(orders[0].id))
  .then(details => getShipping(details.shippingId))
  .then(shipping => console.log(shipping))
  .catch(err => console.error('Zanjirning istalgan joyidagi xato shu yerga keladi:', err));
```

Promise kombinadorlari

Metod	Qachon tugaydi	Natija	Qo'llanilishi
<code>Promise.all</code>	Hammasi bajarilganda yoki birinchi xatoda	Natijalar massivi	Bir-biriga bog'liq bo'lmagan, hammasi kerak bo'lgan so'rovlar
<code>Promise.allSettled</code>	Hammasi tugaganda (xato bo'lsa ham)	<code>{status, value reason}</code> massivi	Ba'zilar xato bo'lsa ham davom etish
<code>Promise.race</code>	Birinchisi tugaganda (muvaffaqiyat yoki xato)	Birinchi natija	Timeout qo'yish
<code>Promise.any</code>	Birinchi muva ffaqiyatlisida	Birinchi muvaffaqiyatli qiymat	Bir nechta zaxira serverdan eng tezi

async / await

`async/await` — Promise ustidagi "sintaktik shakar": asinxron kodni xuddi sinxron koddek yuqoridan pastga o'qiladigan qiladi.

```
async function buyurtmaMalumoti(userId) {
  try {
    const user = await getUser(userId);           // Promise tugashini KUTADI
    (lekin oqimni bloklamaydi)
    const orders = await getOrders(user.id);
    const details = await getOrderDetails(orders[0].id);
    return details;                               // async funksiya DOIM Promise
    qaytaradi
  } catch (err) {
    console.error('Xato:', err.message);
    throw err;                                   // yuqoriga uzatish
  }
}

buyurtmaMalumoti(1).then(d => console.log(d));
// yoki boshqa async funksiya ichida:
// const d = await buyurtmaMalumoti(1);
```

- `await` faqat `async` funksiya ichida (yoki ES modulning yuqori darajasida) ishlatiladi.
- `await` funksiyaning "pauza"ga qo'yadi, lekin event loop boshqa ishlarni davom ettiradi.
- Xatolar oddiy `try...catch` bilan ushlanadi.

Ketma-ket va parallel bajarish

JavaScript

```
// ❌ SEKIN: bir-biriga bog'liq emas, lekin ketma-ket kutilyapti (1s + 1s + 1s = 3s)
const users = await fetchUsers();
const products = await fetchProducts();
const stats = await fetchStats();

// ✅ TEZ: parallel (eng uzuni = 1s)
const [users2, products2, stats2] = await Promise.all([
  fetchUsers(),
  fetchProducts(),
  fetchStats(),
]);

// ⚠️ forEach ichida await ishlamaydi (kutmaydi)!
ids.forEach(async id ⇒ await save(id)); // ❌ — hammasi "uchib ketadi"
for (const id of ids) await save(id); // ✅ ketma-ket
await Promise.all(ids.map(id ⇒ save(id))); // ✅ parallel
```

Fetch API — serverga so'rov yuborish

`fetch()` — brauzerda (va Node.js 18+ da) HTTP so'rov yuborishning standart usuli. U Promise qaytaradi.

JavaScript

```
// GET so'rov
const response = await fetch('https://jsonplaceholder.typicode.com/users');
console.log(response.status); // 200
console.log(response.ok); // true (status 200-299 bo'lsa)
console.log(response.headers.get('content-type'));
const users = await response.json(); // tanani JSON sifatida o'qish (bu ham Promise!)
// boshqa formatlar: response.text(), response.blob(), response.formData()
```

```
// POST so'rov – JSON yuborish
const res = await fetch('https://jsonplaceholder.typicode.com/posts', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${token}`,
  },
  body: JSON.stringify({ title: 'Salom', body: 'Matn', userId: 1 }),
});
const created = await res.json();

// PUT / PATCH / DELETE
await fetch(`/api/posts/${id}`, { method: 'PATCH', headers: {...}, body:
JSON.stringify({ title: 'Yangi' }) });
await fetch(`/api/posts/${id}`, { method: 'DELETE' });

// Fayl yuklash – FormData (Content-Type ni o'zingiz qo'ymang!)
const fd = new FormData();
fd.append('avatar', fileInput.files[0]);
await fetch('/api/upload', { method: 'POST', body: fd });

// Query parametrlar
const url = new URL('https://api.example.uz/products');
url.searchParams.set('q', 'telefon');
url.searchParams.set('page', 2);
await fetch(url); // https://api.example.uz/products?q=telefon&page=2
```



Eng ko'p uchraydigan xato

`fetch` faqat **tarmoq xatosida** (internet yo'q, server topilmadi, CORS) reject bo'ladi. Server `404` yoki `500` qaytarsa ham Promise **muvaffaqiyatli** tugaydi! Shuning uchun `response.ok` ni doim tekshiring.

Qayta ishlatiladigan API yordamchisi

api.js

```
export class ApiError extends Error {
  constructor(status, message, data) {
    super(message);
    this.name = 'ApiError';
    this.status = status;
    this.data = data;
  }
}

const BASE_URL = 'http://localhost:3000/api';

export async function api(path, { method = 'GET', body, headers, ...rest } = {}) {
  const token = localStorage.getItem('token');
  const res = await fetch(BASE_URL + path, {
    method,
    headers: {
      ...(body && { 'Content-Type': 'application/json' }),
      ...(token && { Authorization: `Bearer ${token}` }),
      ...headers,
    },
    body: body ? JSON.stringify(body) : undefined,
    ...rest,
  });

  // 204 No Content — tana yo'q
  const data = res.status === 204 ? null : await res.json().catch(() => null);

  if (!res.ok) {
    throw new ApiError(res.status, data?.message || res.statusText, data);
  }
  return data;
}

// Foydalanish:
// const posts = await api('/posts');
// const post = await api('/posts', { method: 'POST', body: { title: 'Salom' } });
```


Bu yordamchini 13-darsdagi full-stack loyihada ishlatamiz.

So'rovni bekor qilish va timeout

JavaScript

```
const controller = new AbortController();

fetch('/api/big-report', { signal: controller.signal })
  .then(r => r.json())
  .catch(err => {
    if (err.name === 'AbortError') console.log('So\`rov bekor qilindi');
    else throw err;
  });

controller.abort(); // masalan, foydalanuvchi "Bekor qilish" tugmasini bosdi

// Timeout — zamonaviy qisqa usul
await fetch('/api/slow', { signal: AbortSignal.timeout(5000) }); // 5 soniyadan keyin bekor
```

CORS haqida qisqacha

Brauzer xavfsizlik uchun **Same-Origin Policy** ni qo'llaydi: `http://localhost:5173` dagi sahifa `http://localhost:3000` dagi API'ning javobini o'qiy olmaydi (port farq qilsa ham — boshqa "origin"). Ruxsat berish **server** zimmasida: u `Access-Control-Allow-Origin` sarlavhasini qaytarishi kerak. Konsolda "blocked by CORS policy" xatosini ko'rsangiz, frontendni emas, backendni sozlang (9-darsda `cors` middleware).

Amaliy misollar

MISOL 1 Foydalanuvchilar katalogi: yuklanish, xato va bo'sh holatlar

Har bir ma'lumot yuklaydigan interfeys **to'rt holatni** ko'rsatishi kerak: yuklanmoqda, xato, bo'sh, ma'lumot.

```

<main>
  <h1>Jamoa a'zolari</h1>
  <button id="reload"><img alt="refresh icon" data-bbox="345 140 365 155"/> Qayta yuklash</button>
  <div id="status" role="status"></div>
  <div id="users" class="grid"></div>
</main>

<style>
  .grid { display: grid; grid-template-columns: repeat(auto-fill,
minmax(240px, 1fr)); gap: 16px; }
  .user { border: 1px solid #e2e8f0; border-radius: 12px; padding: 16px; }
  .user h3 { margin: 0 0 4px; }
  .user small { color: #64748b; }
  .skeleton { height: 110px; border-radius: 12px; background: linear-
gradient(90deg, #f1f5f9 25%, #e2e8f0 50%, #f1f5f9 75%);
    background-size: 200% 100%; animation: shimmer 1.2s
infinite; }
  @keyframes shimmer { to { background-position: -200% 0; } }
  .error { color: #b91c1c; background: #fef2f2; padding: 12px; border-
radius: 8px; }
</style>

<script type="module">
  const box = document.querySelector('#users');
  const status = document.querySelector('#status');

  function showSkeleton(n = 6) {
    box.replaceChildren(...Array.from({ length: n }, () => {
      const d = document.createElement('div');
      d.className = 'skeleton';
      return d;
    }));
  }

  function userCard(u) {
    const el = document.createElement('article');
    el.className = 'user';
    const h3 = document.createElement('h3');
    h3.textContent = u.name;
    const email = document.createElement('a');
    email.href = `mailto:${u.email}`;
    email.textContent = u.email;
  }

```

```

    const company = document.createElement('small');
    company.textContent = `${u.company.name} · ${u.address.city}`;
    el.append(h3, email, document.createElement('br'), company);
    return el;
}

async function loadUsers() {
    showSkeleton();
    status.textContent = 'Yuklanmoqda...';
    try {
        const res = await
fetch('https://jsonplaceholder.typicode.com/users', {
    signal: AbortSignal.timeout(8000),
});
        if (!res.ok) throw new Error(`Server xatosi: ${res.status}`);
        const users = await res.json();

        if (users.length === 0) {
            box.replaceChildren();
            status.textContent = 'Hozircha a\'zolar yo\'q.';
            return;
        }
        box.replaceChildren(...users.map(userCard));
        status.textContent = `${users.length} ta a'zo topildi`;
    } catch (err) {
        box.replaceChildren();
        const msg = document.createElement('p');
        msg.className = 'error';
        msg.textContent = err.name === 'TimeoutError'
            ? 'Server javob bermadi. Internetni tekshiring.'
            : `Xatolik: ${err.message}`;
        box.append(msg);
        status.textContent = '';
    }
}

document.querySelector('#reload').addEventListener('click', loadUsers);
loadUsers();
</script>

```

DevTools → Network → **Offline** rejimini yoqib, xato holatini sinab ko'ring. "Slow 3G" bilan skeleton animatsiyasini ko'rasiz.

MISOL 2**Ob-havo ilovasi: debounce qidiruv va ketma-ket so'rovlar**

Open-Meteo — kalitsiz (API key talab qilmaydigan) bepul ob-havo API. Avval shahar nomidan koordinata topamiz (geocoding), keyin shu koordinata bo'yicha ob-havoni olamiz — **ikkinchi so'rov birinchisiga bog'liq**.

```

<div class="weather">
  <input id="city" placeholder="Shahar nomi: Samarqand, Tashkent..."
autocomplete="off">
  <ul id="suggestions"></ul>
  <section id="result"></section>
</div>

<script type="module">
  const input = document.querySelector('#city');
  const list = document.querySelector('#suggestions');
  const result = document.querySelector('#result');

  const debounce = (fn, ms) => { let t; return (...a) => {
clearTimeout(t); t = setTimeout(() => fn(...a), ms); }; };

  let controller; // eski so'rovni bekor qilish uchun

  async function searchCities(query) {
    if (query.length < 2) return list.replaceChildren();
    controller?.abort(); // foydalanuvchi yozishda davom
etsa — eski so'rov keraksiz
    controller = new AbortController();

    const url = new URL('https://geocoding-api.open-meteo.com/v1/search');
    url.search = new URLSearchParams({ name: query, count: 5, language:
'en' });

    try {
      const res = await fetch(url, { signal: controller.signal });
      const { results = [] } = await res.json();
      list.replaceChildren(...results.map((c) => {
        const li = document.createElement('li');
        li.textContent = `${c.name}, ${c.country}`;
        li.tabIndex = 0;
        li.addEventListener('click', () => showWeather(c));
        return li;
      }));
    } catch (err) {
      if (err.name !== 'AbortError') console.error(err);
    }
  }
}

```

```

const WEATHER_CODES = { 0: '☀️ Ochiq', 1: '☁️ Asosan ochiq', 2: '☀️
Qisman bulutli', 3: '☁️ Bulutli',
  45: '🌫️ Tuman', 61: '🌧️ Yengil yomg\'ir', 63: '🌧️ Yomg\'ir', 71: '🌧️
Qor', 95: '🌨️ Momaqalldiroq' };

async function showWeather(city) {
  list.replaceChildren();
  input.value = city.name;
  result.textContent = 'Yuklanmoqda...';

  const url = new URL('https://api.open-meteo.com/v1/forecast');
  url.search = new URLSearchParams({
    latitude: city.latitude,
    longitude: city.longitude,
    current: 'temperature_2m,wind_speed_10m,weather_code',
    daily: 'temperature_2m_max,temperature_2m_min',
    timezone: 'auto',
  });

  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error(res.statusText);
    const data = await res.json();
    const { temperature_2m: t, wind_speed_10m: w, weather_code: code } =
data.current;

    const days = data.daily.time.map((day, i) =>
      `${new Date(day).toLocaleDateString('uz-UZ', { weekday: 'short',
day: 'numeric' })}: ` +
      `${Math.round(data.daily.temperature_2m_min[i])}°...
${Math.round(data.daily.temperature_2m_max[i])}°`
    );

    result.innerHTML = '';
    const h2 = document.createElement('h2');
    h2.textContent = `${city.name}: ${Math.round(t)}°C
${WEATHER_CODES[code] ?? ''}`;
    const p = document.createElement('p');
    p.textContent = `Shamol: ${w} km/soat`;
    const ul = document.createElement('ul');
    days.forEach((d) => { const li = document.createElement('li');
li.textContent = d; ul.append(li); });
    result.append(h2, p, ul);
  } catch (err) {

```

```
    result.textContent = `Ob-havoni olishda xato: ${err.message}`;  
  }  
}  
  
input.addEventListener('input', debounce((e) =>  
  searchCities(e.target.value.trim()), 400));  
</script>
```

Ikki muhim usul: `debounce` har harf uchun so'rov yubormaslikni ta'minlaydi; `AbortController` esa eski, endi keraksiz so'rovni bekor qiladi — aks holda sekin kelgan eski javob yangi natijaning ustiga yozilib qolishi mumkin ("race condition").

dashboard.js

```
const BASE = 'https://jsonplaceholder.typicode.com';

async function getJSON(path) {
  const res = await fetch(BASE + path);
  if (!res.ok) throw new Error(`${path} → ${res.status}`);
  return res.json();
}

async function loadDashboard() {
  console.time('dashboard');

  // To'rtta mustaqil so'rov PARALLEL; bittasi xato bo'lsa ham qolganlari
  ko'rsatiladi
  const results = await Promise.allSettled([
    getJSON('/users'),
    getJSON('/posts'),
    getJSON('/comments'),
    getJSON('/this-does-not-exist'), // ataylab xato
  ]);

  const [users, posts, comments, broken] = results.map(r =>
    r.status === 'fulfilled' ? r.value : null
  );

  results
    .filter(r => r.status === 'rejected')
    .forEach(r => console.warn('⚠️ Vidget yuklanmadi:',
      r.reason.message));

  if (users && posts) {
    // Eng faol 3 ta muallif
    const postCount = posts.reduce((acc, p) => {
      acc[p.userId] = (acc[p.userId] ?? 0) + 1;
      return acc;
    }, {});
    const top = users
      .map(u => ({ ism: u.name, postlar: postCount[u.id] ?? 0 }))
      .sort((a, b) => b.postlar - a.postlar)
```



```

        .slice(0, 3);
        console.table(top);
    }

    console.log({
        foydalanuvchilar: users?.length ?? '-',
        postlar: posts?.length ?? '-',
        izohlar: comments?.length ?? '-',
    });

    console.timeEnd('dashboard'); // ~ eng sekin so'rov vaqti, yig'indisi
    emas
}

loadDashboard();

```

`Promise.all` ishlatilganda bitta buzilgan vidjet butun sahifani "o'ldirardi". `allSettled` esa har bir natijani alohida tekshirishga imkon beradi — real dashboard'larda aynan shunday qilinadi.

MISOL 4

Ishonchli so'rov: timeout, qayta urinish (retry) va eksponensial kutish

Mobil internet uzilib qolishi yoki server vaqtincha band (`503`) bo'lishi mumkin. Professional ilovalar bunday holatlarda so'rovni bir necha marta, har safar uzoqroq kutib, takrorlaydi.

fetchWithRetry.js

```
const sleep = (ms) => new Promise(r => setTimeout(r, ms));

const RETRYABLE = new Set([408, 429, 500, 502, 503, 504]);

export async function fetchWithRetry(url, options = {}, {
  retries = 3,
  timeout = 8000,
  baseDelay = 500,
  onRetry = () => {},
} = {}) {
  for (let attempt = 0; ; attempt++) {
    try {
      const res = await fetch(url, { ...options, signal:
AbortSignal.timeout(timeout) });

      if (res.ok) return res;

      // 4xx xatolarni (404, 401...) takrorlashdan foyda yo'q
      if (!RETRYABLE.has(res.status) || attempt ≥ retries) {
        throw Object.assign(new Error(`HTTP ${res.status}`), { status:
res.status });
      }

      // Server "Retry-After" sarlavhasini yuborgan bo'lsa, unga amal
qilamiz
      const retryAfter = Number(res.headers.get('Retry-After')) * 1000;
      await wait(attempt, retryAfter);
    } catch (err) {
      const network = err.name === 'TypeError' || err.name ===
'TimeoutError';
      if (!network || attempt ≥ retries) throw err;
      await wait(attempt);
    }
  }

  async function wait(attempt, forced) {
    // 500ms, 1000ms, 2000ms ... + tasodifiy "jitter" (hamma bir vaqtda
qayta urinmasligi uchun)
    const delay = forced || baseDelay * 2 ** attempt + Math.random() *
200;
    onRetry({ attempt: attempt + 1, delay: Math.round(delay) });
    await sleep(delay);
  }
}
```

```

}

// Foydalanish
try {
  const res = await fetchWithRetry('https://httpbin.org/status/503', {}, {
    retries: 3,
    onRetry: ({ attempt, delay }) => console.log(`${attempt}-qayta
urinish, ${delay}ms kutamiz...`),
  });
  console.log(await res.text());
} catch (err) {
  console.error('Barcha urinishlar muvaffaqiyatsiz:', err.message);
}

```



Faqat **idempotent** so'rovlarni (GET, PUT, DELETE) avtomatik takrorlang. `POST /payments` ni qayta yuborish ikki marta to'lovga olib kelishi mumkin (1-darsdagi idempotentlik tushunchasini eslang). Bunday holatlar uchun serverlar "Idempotency-Key" sarlavhasidan foydalanadi.

Mashqlar



Mustaqil ish

1. Quyidagi kodning natijasini oldindan ayting, keyin tekshiring:

JavaScript

```

setTimeout(() => console.log(1));
Promise.resolve().then(() => { console.log(2); setTimeout(() =>
console.log(3)); });
(async () => { console.log(4); await null; console.log(5); })();
console.log(6);

```

2. `promiseWithTimeout(promise, ms)` funksiyasini `Promise.race` bilan yozing.

3. GitHub API (`https://api.github.com/users/USERNAME` va `/repos`) orqali profil kartasi yasang: avatar, ism, followerlar va eng ko'p yulduzli 5 ta repo.
4. "Cheksiz scroll": JSONPlaceholder'dan postlarni 10 tadan yuklang (`?_page=2&_limit=10`), sahifa oxiriga yetganda keyingisini (`IntersectionObserver` bilan).
5. 5-darsdagi ro'yxatdan o'tish formasini `https://jsonplaceholder.typicode.com/users` ga POST qiladigan qiling; yuborish davomida tugmani o'chirib, "Yuborilmoqda..." deb ko'rsating.

▼ 1-savol javobi

`4, 6, 2, 5, 1, 3`. Sinxron: 4 va 6. Microtask'lar: 2 va 5. Macrotask'lar: 1, keyin 2-qadamda qo'shilgan 3.

✓ Xulosa

- JavaScript bir oqimli; asinxronlik event loop, Web API'lar va navbatlar orqali ishlaydi. Microtask'lar macrotask'lardan oldin.
- Promise — kelajakdagi natija: pending → fulfilled/rejected.
- `async/await` + `try/catch` — asinxron kodni o'qilishi oson qiladi.
- Mustaqil so'rovlarni `Promise.all` / `allSettled` bilan parallel bajaring.
- `fetch` HTTP xatolarda reject qilmaydi — `response.ok` ni tekshiring.
- `AbortController` — bekor qilish va timeout; retry faqat idempotent so'rovlar uchun.



07-DARS · 14 TADAN

Git va GitHub

Versiyalarni boshqarish: commit, branch, merge, konfliktlarni hal qilish, remote repozitoriya, Pull Request, jamoaviy ish oqimi va GitHub Pages.

Boshlang'ich–O'rta · ~90 daqiqa

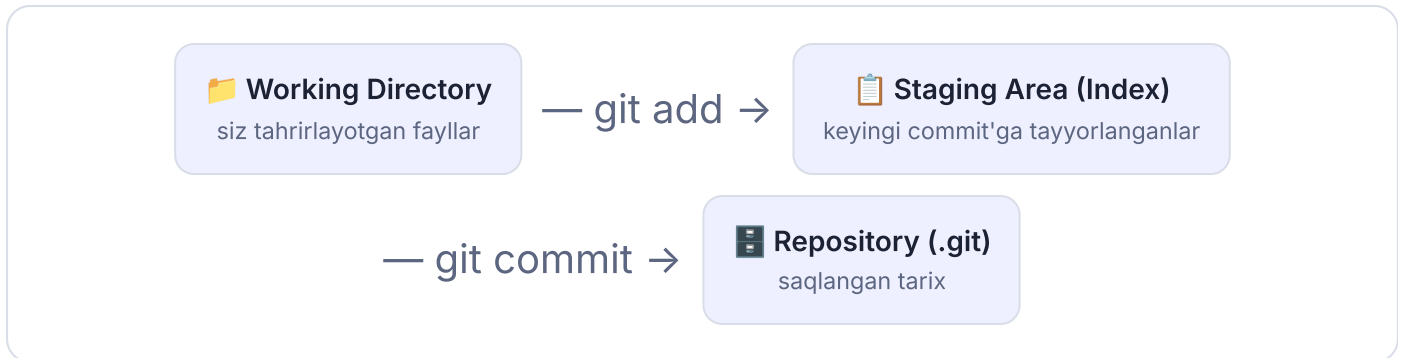
Tasavvur qiling: loyihangiz ishlab turibdi, siz yangi funksiya qo'shdingiz va hammasi buzildi. Qaysi o'zgarish sababchi? Kecha ishlagan versiyani qanday qaytarish mumkin? Uch kishi bitta faylni bir vaqtda qanday tahrirlaydi? **Git** — bu savollarning barchasiga javob beradigan **versiyalarni boshqarish tizimi (VCS)**. Bugun Git'ni bilmaydigan dasturchini ishga olishmaydi. Siz hozir o'qiyotgan qo'llanma ham Git orqali GitHub'ga joylangan.

Git va GitHub — farqi

- **Git** — kompyuteringizda ishlaydigan dastur. Loyiha tarixini saqlaydi. Internetsiz ham ishlaydi. 2005-yilda Linus Torvalds (Linux yaratuvchisi) yozgan.
- **GitHub** — Git repozitoriylarini internetda saqlash va jamoa bo'lib ishlash platformasi (Pull Request, Issues, Actions, Pages). Muqobillari: GitLab, Bitbucket.

Git'ning asosiy tushunchalari

Uch hudud



- **Working directory** — oddiy papka va fayllaringiz.
- **Staging area** — "savat": qaysi o'zgarishlar keyingi suratga (commit'ga) kirishini tanlaysiz. Bu bir nechta faylni o'zgartirib, ularni mantiqiy jihatdan alohida commit'larga bo'lish imkonini beradi.
- **Repository** — `.git` yashirin papkasi: barcha commit'lar, branch'lar va tarix shu yerda.

Commit

Commit — loyihaning ma'lum paytdagi to'liq **surati (snapshot)**. Har bir commit'da: noyob identifikator (SHA-1 xesh, masalan `a3f9c2e`), muallif, sana, xabar va ota commit'ga havola bor. Commit'lar zanjiri — loyiha tarixi.

Branch (tarmoq)

Branch — commit'ga ko'rsatkich (yorliq). Yangi branch yaratish — tarixdan "nusxa" olib, asosiy kodga tegmasdan tajriba qilish demak. `HEAD` — hozir qaysi branch'da turganingizni bildiruvchi maxsus ko'rsatkich.

Matn

```
      C4 — C5      ← feature/login
      /
C1 — C2 — C3 — C6  ← main  ← HEAD
```

O'rnatish va sozlash

Terminal

```
# Versiyani tekshirish
git --version

# Bir martalik sozlash (har bir commit'da ism va email yoziladi)
git config --global user.name "Ismingiz Familiyangiz"
git config --global user.email "siz@example.com"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"      # VS Code muharrir sifatida

git config --list    # sozlamalarni ko'rish
```

Asosiy buyruqlar

Terminal

```
# Yangi repozitoriya
git init

# Holat: qaysi fayllar o'zgargan, qaysilari staging'da
git status
git status -s          # qisqa ko'rinish

# O'zgarishlarni ko'rish
git diff               # working directory vs staging
git diff --staged      # staging vs oxirgi commit

# Staging'ga qo'shish
git add index.html     # bitta fayl
git add src/           # papka
git add .              # barcha o'zgarishlar
git add -p             # o'zgarishlarni bo'lakma-bo'lak tanlash (juda foydali!)

# Commit
git commit -m "Bosh sahifaga navigatsiya qo'shildi"
git commit -am "Xabar" # kuzatilayotgan fayllarni add + commit (yangi fayllar kirmaydi)

# Tarix
git log
git log --oneline --graph --all    # chiroyli daraxt ko'rinishi
git log -p index.html              # fayl o'zgarishlari tarixi
git show a3f9c2e                   # bitta commit tafsilotlari
git blame app.js                   # har bir qatorni kim, qachon o'zgartirgan
```

Yaxshi commit xabari

 Yomon

 Yaxshi

fix

Login formasida bo'sh parol bilan yuborish xatosi tuzatildi

`update``Mahsulot kartasiga chegirma belgisi qo'shildi``asdasd``feat(cart): promokod qo'llash funksiyasi`

Ko'plab jamoalar **Conventional Commits** uslubidan foydalanadi: `feat:` (yangi funksiya), `fix:` (xato tuzatish), `docs:`, `style:`, `refactor:`, `test:`, `chore:`. Qoida: bitta commit — bitta mantiqiy o'zgarish.

.gitignore

Ba'zi fayllar repozitoriyaga **hech qachon** tushmasligi kerak:

`.gitignore`

```
# Bog'liqliklar (npm install bilan qayta tiklanadi)
node_modules/

# Maxfiy ma'lumotlar — PAROLLAR, KALITLAR!
.env
.env.local
*.pem

# Build natijalari
dist/
build/

# Log va tizim fayllari
*.log
.DS_Store
Thumbs.db

# Muharrir sozlamalari
.vscode/
.idea/
```



Maxfiy kalitni push qilib qo'ydingizmi?

Faylni o'chirib, yangi commit qilish yetarli emas — kalit tarixda qoladi. GitHub'dagi botlar ochiq repozitoriyalardagi kalitlarni bir necha daqiqada topadi. **Darhol kalitni bekor qiling (revoke) va yangisini yarating.**

Branch'lar bilan ishlash

Terminal

```
git branch                # branch'lar ro'yxati (* – joriy)
git branch feature/cart   # yangi branch yaratish
git switch feature/cart   # unga o'tish (eski usul: git checkout
feature/cart)
git switch -c feature/search # yaratish + o'tish bir buyruqda

# ... o'zgarishlar, commit'lar ...

git switch main           # asosiy branch'ga qaytish
git merge feature/cart    # feature/cart ni main ga birlashtirish
git branch -d feature/cart # birlashtirilgan branch'ni o'chirish
```

Merge turlari

- **Fast-forward** — `main` da yangi commit bo'lmagan bo'lsa, Git shunchaki ko'rsatkichni oldinga suradi. Tarix to'g'ri chiziq.
- **Uch tomonlama merge** — ikkala branch'da ham yangi commit'lar bo'lsa, Git yangi **merge commit** yaratadi.
- **Rebase** — `git rebase main` — sizning commit'laringizni `main` ning oxiriga "ko'chirib o'tkazadi", tarix chiziqli bo'ladi. **Qoida:** boshqalar bilan bo'lishilgan (push qilingan) branch'ni rebase qilmang.

Konfliktlar (merge conflict)

Ikki branch bitta faylning **bitta qatorini** turlicha o'zgartirsa, Git qaysi birini tanlashni bilmaydi va sizdan so'raydi:

index.html

```
<<<<<<< HEAD
<h1>Xush kelibsiz, aziz mijoz!</h1>
=====
<h1>Bizning do'konga xush kelibsiz!</h1>
>>>>>> feature/new-header
```

Hal qilish tartibi:

1. Faylni oching; `<<<<<< HEAD` va `=====` orasida — sizning versiyangiz, `=====` va `>>>>>>` orasida — birlashtirilayotgan branch versiyasi.
2. Keraklisini qoldiring (yoki ikkalasini birlashtiring) va **barcha belgilarni o'chiring**.
3. `git add index.html` — "konflikt hal qilindi" deb belgilash.
4. `git commit` — merge'ni yakunlash.

VS Code konfliktlarni rangli ko'rsatadi va "Accept Current / Incoming / Both" tugmalarini taklif qiladi.

Bekor qilish: `git merge --abort`.

Xatolarni tuzatish va orqaga qaytish

Vaziyat	Buyruq
Fayldagi saqlanmagan o'zgarishlarni bekor qilish	<code>git restore app.js</code>
Faylni staging'dan chiqarish (o'zgarish qoladi)	<code>git restore --staged app.js</code>
Oxirgi commit xabarini o'zgartirish / unutilgan faylni qo'shish (push qilinmagan bo'lsa)	<code>git commit --amend</code>
Oxirgi commit'ni bekor qilish, o'zgarishlarni saqlab qolish	<code>git reset --soft HEAD~1</code>
Push qilingan commit'ni xavfsiz bekor qilish (teskari commit yaratadi)	<code>git revert a3f9c2e</code>
Ishni vaqtincha chetga qo'yish (boshqa branch'ga o'tish uchun)	<code>git stash</code> → keyin <code>git stash pop</code>
Eski versiyadagi faylni ko'rish	<code>git show a3f9c2e:src/app.js</code>
"Yo'qolgan" commit'ni topish	<code>git reflog</code>



`git reset --hard` va `git push --force` — ma'lumotni qaytarib bo'lmas darajada o'chirishi mumkin bo'lgan buyruqlar. Umumiy branch'larda `revert` dan foydalaning; force push kerak bo'lsa `--force-with-lease` xavfsizroq.

Remote: GitHub bilan ishlash

Terminal

```
# Mavjud repozitoriyani yuklab olish
git clone https://github.com/oybekalloyorov/web_fullstak.git
cd web_fullstak

# Lokal repozitoriyani GitHub'ga ulash
git remote add origin https://github.com/USERNAME/REPO.git
git remote -v                # ulangan remote'lar

# Yuborish
git push -u origin main      # -u: keyingi safar shunchaki "git push"
git push origin feature/cart # boshqa branch

# Olish
git fetch origin             # o'zgarishlarni yuklab olish
(birlashtirmasdan)
git pull                     # fetch + merge
git pull --rebase            # fetch + rebase (tarix toza qoladi)
```

Autentifikatsiya

GitHub parol bilan push qilishni qabul qilmaydi. Ikki usul bor:

- **HTTPS + Personal Access Token** (Settings → Developer settings → Tokens) yoki GitHub CLI: `gh auth login`.
- **SSH kalit:** `ssh-keygen -t ed25519 -C "siz@example.com"`, so'ng `~/.ssh/id_ed25519.pub` mazmunini GitHub → Settings → SSH keys ga qo'shing. Remote: `git@github.com:USERNAME/REPO.git`.

Jamoaviy ish oqimi: GitHub Flow

1. `main` — doim ishlaydigan, deploy qilinadigan kod.
2. Har bir vazifa uchun **yangi branch**: `feature/payment`, `fix/login-error`.
3. Kichik, mantiqiy commit'lar; muntazam `push`.
4. GitHub'da **Pull Request (PR)** ochish: nima va nima uchun o'zgartirilganini yozish.
5. **Code review**: jamoadoshlar kodni ko'rib chiqadi, izoh qoldiradi; avtomatik testlar (CI) ishlaydi.

6. Tasdiqlangach — `main` ga merge va branch'ni o'chirish.

7. Deploy.



Fork va open-source

Boshqa birovning loyihasiga hissa qo'shish uchun: repozitoriyaning **Fork** qiling (o'z akkauntingizga nusxa) → clone → branch → o'zgarish → push (o'z fork'ingizga) → asl loyihaga Pull Request oching.

Amaliy misollar

MISOL 1 Noldan loyiha yaratish va GitHub'ga joylash

Terminal

```
# 1. Loyiha papkasi
mkdir portfolio && cd portfolio
git init

# 2. Fayllar
echo "# Mening portfoliom" > README.md
echo "node_modules/" > .gitignore
touch index.html style.css

# 3. Birinchi commit
git status          # 4 ta "untracked" fayl
git add .
git commit -m "Loyiha tuzilmasi yaratildi"

# 4. GitHub'da bo'sh repozitoriya yarating (README'siz!), so'ng:
git branch -M main
git remote add origin https://github.com/USERNAME/portfolio.git
git push -u origin main

# 5. Keyingi ish sikli
#   (index.html ga navigatsiya qo'shdingiz)
git add index.html
git commit -m "feat: sarlavha va navigatsiya qo'shildi"
#   (style.css ga dizayn)
git add style.css
git commit -m "style: navbar uchun flex maket"
git push

git log --oneline
# 9c1e4b2 style: navbar uchun flex maket
# 5a7d310 feat: sarlavha va navigatsiya qo'shildi
# 1f0a2c8 Loyiha tuzilmasi yaratildi
```

Siz ushbu qo'llanmaning repozitoriyasini ham aynan shunday boshlagansiz: `git init` → `git add README.md` → `git commit` → `git remote add origin` → `git push -u origin main`.

MISOL 2 Feature branch, Pull Request va code review

Terminal

```
# main yangi ekaniga ishonch hosil qilamiz
git switch main
git pull

# Vazifa: "Aloqa formasi" – yangi branch
git switch -c feature/contact-form

# ... contact.html yaratildi ...
git add contact.html
git commit -m "feat(contact): aloqa formasi sahifasi"

# ... validatsiya qo'shildi ...
git add js/contact.js
git commit -m "feat(contact): email va telefon validatsiyasi"

git push -u origin feature/contact-form
# Terminal PR yaratish havolasini ko'rsatadi:
# remote: Create a pull request for 'feature/contact-form' on GitHub by
# visiting:
# remote:
https://github.com/USERNAME/portfolio/pull/new/feature/contact-form
```

GitHub'da PR tavsifiga namuna:

Matn

```
## Nima o'zgardi
- /contact sahifasi va aloqa formasi qo'shildi
- Email va +998 formatidagi telefon raqami tekshiriladi

## Qanday tekshirish mumkin
1. /contact.html ni oching
2. Noto'g'ri email kiriting – qizil xabar chiqishi kerak

## Skrinshotlar
(rasm)

Closes #12
```

Reviewer izoh qoldirdi: "Telefon maydoniga `autocomplete="tel"` qo'shing". Siz shu branch'da tuzatasiz:

Terminal

```
git add contact.html
git commit -m "fix(contact): telefon maydoniga autocomplete qo'shildi"
git push                                # PR avtomatik yangilanadi

# PR merge qilingach:
git switch main
git pull
git branch -d feature/contact-form
git push origin --delete feature/contact-form # (GitHub'da "Delete
branch" tugmasi ham bor)
```

Closes #12 — PR merge qilinganda 12-raqamli issue avtomatik yopiladi.

MISOL 3 Konfliktni qadamma-qadam hal qilish

Konfliktni o'z kompyuteringizda ataylab yaratib, hal qilishni mashq qilamiz:

Terminal

```
mkdir conflict-demo && cd conflict-demo && git init
echo "Narx: 100 000 so'm" > price.txt
git add . && git commit -m "Boshlang'ich narx"

# Birinchi dasturchi: chegirma
git switch -c discount
echo "Narx: 80 000 so'm (chegirma!)" > price.txt
git commit -am "Chegirmali narx"

# Ikkinchi dasturchi (main'da): narx oshdi
git switch main
echo "Narx: 120 000 so'm" > price.txt
git commit -am "Narx oshirildi"

# Birlashtirish
git merge discount
# CONFLICT (content): Merge conflict in price.txt
# Automatic merge failed; fix conflicts and then commit the result.

git status          # both modified: price.txt
cat price.txt
```

Fayl mazmuni:

Matn

```
<<<<<< HEAD
Narx: 120 000 so'm
=====
Narx: 80 000 so'm (chegirma!)
>>>>>> discount
```

Jamoa bilan kelishib, yangi narxga chegirma qo'llaymiz. Faylni shunday tahrirlaymiz:

Matn

```
Narx: 96 000 so'm (120 000 dan 20% chegirma!)
```

Terminal

```
git add price.txt
git commit -m "Merge discount: yangi narxga chegirma qo'llandi"
git log --oneline --graph
# *    d4e5f6a Merge discount: yangi narxga chegirma qo'llandi
# | \
# | * b2c3d4e Chegirmali narx
# * | c3d4e5f Narx oshirildi
# | /
# * a1b2c3d Boshlang'ich narx
```

MISOL 4 Saytni GitHub Pages'da bepul joylash

Statik sayt (HTML/CSS/JS) uchun eng oson hosting — GitHub Pages. Ushbu qo'llanma ham shu usulda joylanadi.

1. Repozitoriyaning ildizida `index.html` bo'lsin va `main` ga push qiling.
2. GitHub'da repozitoriya → **Settings** → **Pages**.
3. **Source**: "Deploy from a branch", **Branch**: `main`, papka: `/ (root)` → Save.
4. 1–2 daqiqadan so'ng sayt `https://USERNAME.github.io/REPO/` manzilida ochiladi.

Avtomatlashtirilgan variant — GitHub Actions bilan har push'da deploy:

.github/workflows/pages.yml

```
name: Deploy to GitHub Pages
on:
  push:
    branches: [main]
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v3
        with:
          path: .
      - id: deployment
        uses: actions/deploy-pages@v4
```

Bunda Settings → Pages → Source sifatida "GitHub Actions" tanlanadi. O'z domeningiz bo'lsa (masalan `portfolio.uz`), Pages sozlamasida "Custom domain" ga yozing va domen DNS'ida `CNAME` yozuvini `USERNAME.github.io` ga yo'naltiring (1-darsdagi DNS bo'limi).

Mashqlar



Mustaqil ish

1. Oldingi darslarda yozgan portfolio va Todo ilovangizni alohida repozitoriyalarga joylang va GitHub Pages orqali e'lon qiling.
2. Learn Git Branching interaktiv darslarining "Main" bo'limini o'ting.

3. 3-misoldagi konfliktni takrorlang, lekin bu safar `git merge` o'rniga `git rebase main` bilan. Tarixda qanday farq bor?
4. Do'stingiz bilan bitta repozitoriyada ishlang: har biringiz alohida branch'da bitta faylni o'zgartiring, PR oching, bir-biringizning kodini review qiling.
5. Ataylab xato commit qiling va uni (a) `--amend`, (b) `reset --soft`, (c) `revert` bilan tuzating. Qaysi biri push'dan keyin xavfsiz?

✓ Xulosa

- Git — lokal versiyalar tizimi; GitHub — uning ustidagi hamkorlik platformasi.
- Ish sikli: o'zgartirish → `git add` → `git commit` → `git push`.
- Har bir vazifa — alohida branch; `main` doim ishlaydigan holatda.
- Konflikt — qo'rqinchli emas: belgilarni o'chirib, to'g'ri variantni qoldirib, `add` + `commit`.
- `.env` va `node_modules` — doim `.gitignore` da.
- Pull Request + code review — professional jamoaviy ishning standarti.



Node.js va npm

Node.js nima, modullar tizimi (CommonJS va ESM), npm va package.json, fs, path, events, http modullari, muhit o'zgaruvchilari va CLI dasturlar.

O'rta · ~100 daqiqa

Shu paytgacha JavaScript faqat brauzerda ishladi. **Node.js** — JavaScript'ni brauzerdan tashqarida, ya'ni kompyuter yoki serverda ishga tushiruvchi muhit (runtime). U tufayli siz bilgan til bilan web-serverlar, API'lar, CLI dasturlar va avtomatlashtirish skriptlarini yoza olasiz. Bu dars — backend olamiga kirish eshigi.

Node.js nima va qanday ishlaydi?

- 2009-yilda Ryan Dahl yaratgan. Ichida Google Chrome'ning **V8** JavaScript dvigateli ishlaydi.
- **libuv** kutubxonasi orqali fayl tizimi, tarmoq, taymerlar bilan **bloklanmaydigan (non-blocking)** tarzda ishlaydi.
- Brauzerdagi kabi **event loop** modeliga asoslangan (6-dars): bitta oqim minglab ulanishlarga xizmat qiladi, chunki kutish paytida (bazadan javob, fayl o'qish) boshqa so'rovlarni qayta ishlaydi.

Brauzer va Node.js farqi

Brauzer		Node.js
Global obyekt	<code>window</code>	<code>global</code> (umumiy: <code>globalThis</code>)
DOM (<code>document</code>)	Bor	Yo'q
Fayl tizimi	Yo'q (xavfsizlik)	To'liq kirish (<code>fs</code>)

Tarmoq	<code>fetch</code> , WebSocket	<code>fetch</code> , <code>http</code> , TCP/UDP serverlar
Muhit	Har xil brauzer, versiyani tanlay olmaysiz	Versiyani o'zingiz boshqarasiz



Node.js qachon yaxshi tanlov?

I/O ko'p bo'lgan ilovalar: REST API, real vaqtli chat (WebSocket), streaming, mikroservislar. Og'ir CPU hisoblari (video kodlash, katta ML modellari) uchun esa boshqa tillar yoki worker'lar mos.

O'rnatish va birinchi dastur

nodejs.org dan **LTS** versiyani o'rnatish. Bir nechta versiya bilan ishlash uchun `nvm` (Node Version Manager) qulay.

Terminal

```
node --version      # v22.x.x
npm --version       # 10.x.x

node                # REPL – interaktiv rejim (chiqish: .exit yoki Ctrl+C ikki
marta)
> 2 + 2
4

node app.js         # faylni bajarish
node --watch app.js # fayl o'zgarganda avtomatik qayta ishga tushirish (Node
18.11+)
```

app.js

```
console.log('Salom, Node.js!');
console.log('Node versiyasi:', process.version);
console.log('Operatsion tizim:', process.platform);
console.log('Joriy papka:', process.cwd());
console.log('Buyruq argumentlari:', process.argv.slice(2)); // node app.js
ali 25 → ['ali', '25']
```

Modullar tizimi

Node.js'da har bir fayl — alohida modul. Ikki tizim mavjud:

CommonJS (an'anaviy)

math.js

```
function qoshish(a, b) { return a + b; }
const PI = 3.14159;

module.exports = { qoshish, PI };
```

index.js

```
const { qoshish, PI } = require('./math'); // .js kengaytmasi ixtiyoriy
const fs = require('fs');                 // o'rnatilgan modul
const express = require('express');        // npm paketi
console.log(qoshish(2, 3));
```

ES Modules (zamonaviy, tavsiya etiladi)

```
package.json

{
  "type": "module"
}
```

```
math.js

export const qoshish = (a, b) => a + b;
export default class Kalkulyator { /* ... */ }
```

```
index.js

import Kalkulyator, { qoshish } from './math.js'; // ESM'da kengaytma
MAJBURIY
import fs from 'node:fs/promises'; // "node:" prefiksi –
o'rnatilgan modul ekanini aniq ko'rsatadi
import express from 'express';

const config = JSON.parse(await fs.readFile('./config.json', 'utf8')); // top-
level await
```

	CommonJS	ES Modules
Import	<code>require()</code>	<code>import</code>
Eksport	<code>module.exports</code>	<code>export</code>
Yuklanishi	Sinxron	Asinxron
Top-level await	Yo'q	Bor
<code>__dirname</code>	Bor	<code>import.meta.dirname</code> (Node 20.11+)

Ushbu qo'llanmaning qolgan qismida **ES Modules** ishlatamiz.

npm va package.json

npm (Node Package Manager) — dunyodagi eng katta dasturiy kutubxonalar ombori (2 milliondan ortiq paket) va ularni boshqaruvchi vosita.

Terminal

```
npm init -y                # package.json yaratish

npm install express        # dependency (ilova ishlashi uchun kerak)
npm i express              # qisqa yozuv
npm install -D nodemon eslint # devDependency (faqat ishlab chiqishda)
npm install -g npm-check-updates # global (buyruq sifatida)

npm install                # package.json dagi barcha paketlarni o'rnatish
(clone'dan keyin)

npm ci                    # lock-fayl bo'yicha aniq o'rnatish (CI/server
uchun)

npm uninstall lodash

npm outdated              # eskirgan paketlar
npm audit                 # xavfsizlik zaifliklari
npx create-vite@latest    # paketni o'rnatmasdan bir martalik bajarish
```

package.json

```
{
  "name": "kitob-api",
  "version": "1.0.0",
  "description": "Kitoblar do'koni uchun REST API",
  "type": "module",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "node --watch src/server.js",
    "test": "node --test",
    "lint": "eslint src"
  },
  "dependencies": {
    "express": "^5.1.0",
    "pg": "^8.13.0"
  },
  "devDependencies": {
    "eslint": "^9.0.0"
  },
  "engines": {
    "node": "≥ 20"
  }
}
```

- **scripts** — `npm run dev`, `npm test`, `npm start` orqali ishga tushiriladigan buyruqlar.
- **Semantik versiyalash (SemVer)** — `MAJOR.MINOR.PATCH`: `5.1.0`. MAJOR — mos kelmaydigan o'zgarishlar, MINOR — yangi funksiyalar, PATCH — xato tuzatishlar.
- `^5.1.0` — 5.x.x ichida yangilanishga ruxsat; `~5.1.0` — faqat 5.1.x; `5.1.0` — aynan shu versiya.
- **package-lock.json** — har bir paketning aniq o'rnatilgan versiyasini qayd qiladi. **Git'ga commit qiling** — jamoadagi hamma bir xil versiyalarni oladi.
- **node_modules/** — o'rnatilgan paketlar. **Hech qachon commit qilmang** (`.gitignore`).



Paket tanlashda ehtiyot bo'ling

Har bir paket — boshqa bironing kodi serveringizda ishlashi demak. Haftalik yuklanishlar soni, oxirgi yangilanish sanasi, GitHub'dagi faollik va `npm audit` natijasiga qarang. Nomi o'xshash soxta paketlardan (typosquatting: `expres`, `loadsh`) saqlaning.

O'rnatilgan asosiy modullar

fs — fayl tizimi

JavaScript

```
import fs from 'node:fs/promises';
import { existsSync, createReadStream } from 'node:fs';

// O'qish va yozish
const matn = await fs.readFile('eslatma.txt', 'utf8');
await fs.writeFile('chiqish.txt', 'Salom\n'); // yaratadi yoki
ustidan yozadi
await fs.appendFile('log.txt', `${new Date().toISOString()} - kirish\n`);

// JSON fayl
const data = JSON.parse(await fs.readFile('data.json', 'utf8'));
await fs.writeFile('data.json', JSON.stringify(data, null, 2));

// Papkalar
await fs.mkdir('uploads/2026', { recursive: true });
const fayllar = await fs.readdir('uploads');
const info = await fs.stat('data.json'); // info.size, info.mtime,
info.isDirectory()
await fs.rename('eski.txt', 'yangi.txt');
await fs.rm('temp', { recursive: true, force: true });

if (!existsSync('config.json')) console.log('Config topilmadi');
```



`fs.readFileSync` kabi sinxron funksiyalar event loop'ni bloklaydi. Ularni faqat dastur ishga tushayotganda (konfiguratsiya o'qish) yoki CLI skriptlarda ishlating; server so'rovlarini qayta ishlashda — doim asinxron versiyalar.

path — fayl yo'llari

JavaScript

```
import path from 'node:path';

path.join('uploads', '2026', 'rasm.png'); // 'uploads/2026/rasm.png'
// Windows'da '\\'

path.resolve('src', 'app.js'); // to'liq absolyut yo'l
path.extname('rasm.final.png'); // '.png'
path.basename('/a/b/rasm.png'); // 'rasm.png'
path.basename('/a/b/rasm.png', '.png'); // 'rasm'
path.dirname('/a/b/rasm.png'); // '/a/b'

// Joriy fayl papkasi (ESM)
const __dirname = import.meta.dirname;
const publicDir = path.join(__dirname, 'public');
```

Yo'llarni hech qachon satr qo'shish bilan (`'uploads/' + name`) yasamang — `path.join` Windows/Linux farqini va ortiqcha `/` larni o'zi hal qiladi.

events — hodisalar

JavaScript

```
import { EventEmitter } from 'node:events';

class Buyurtmalar extends EventEmitter {
  yarat(buyurtma) {
    // ... bazaga saqlash ...
    this.emit('yaratildi', buyurtma);
  }
}

const orders = new Buyurtmalar();
orders.on('yaratildi', (b) => console.log(`✉️ ${b.email} ga tasdiq xati yuborildi`));
orders.on('yaratildi', (b) => console.log(`📦 Omborga xabar: ${b.mahsulot}`));
orders.once('yaratildi', () => console.log(`🎉 Birinchi buyurtma!`));

orders.yarat({ email: 'ali@mail.uz', mahsulot: 'Noutbuk' });
```

Node.js'ning ko'plab qismlari (server, oqimlar) `EventEmitter` ustiga qurilgan. Bu usul kodni bir-biridan mustaqil qismlarga ajratadi.

Streams — oqimlar

1 GB faylni `readFile` bilan o'qish 1 GB xotira talab qiladi. **Stream** esa ma'lumotni kichik bo'laklarda (chunk) qayta ishlaydi:

JavaScript

```
import { createReadStream, createWriteStream } from 'node:fs';
import { createGzip } from 'node:zlib';
import { pipeline } from 'node:stream/promises';

// Katta log faylni siqish — xotira sarfi bir necha KB
await pipeline(
  createReadStream('access.log'),
  createGzip(),
  createWriteStream('access.log.gz'),
);
```

process va muhit o'zgaruvchilari

Parollar, API kalitlari, port raqami kabi sozlamalar kodda emas, **muhit o'zgaruvchilarida (environment variables)** saqlanadi.

.env

```
PORT=3000
DATABASE_URL=postgres://user:parol@localhost:5432/kitoblar
JWT_SECRET=juda-uzun-va-tasodifiy-maxfiy-kalit
NODE_ENV=development
```

JavaScript

```
// Node 20.6+ da .env faylni o'rnatilgan holda yuklash:
// node --env-file=.env src/server.js
// (eski versiyalarda: npm i dotenv → import 'dotenv/config';)

const PORT = Number(process.env.PORT) || 3000;
const isProd = process.env.NODE_ENV === 'production';

if (!process.env.JWT_SECRET) {
  console.error('✗ JWT_SECRET o\'rnatilmagan');
  process.exit(1); // xato kodi bilan chiqish
}

process.on('SIGINT', () => { // Ctrl+C
  console.log('Server to\'xtatilmoqda...');
  process.exit(0);
});
```

Repozitoriyaga `.env` emas, uning namunasini — `.env.example` ni (qiymatlarsiz) commit qiling.

http moduli — freymvorksiz server

Express'ga o'tishdan oldin, uning ostida nima borligini ko'raylik:

```

import http from 'node:http';

const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);

  // So'rov tanasini o'qish — stream bo'laklarini yig'amiz
  let body = '';
  for await (const chunk of req) body += chunk;

  console.log(req.method, url.pathname, url.searchParams.toString(), body);

  res.setHeader('Content-Type', 'application/json; charset=utf-8');

  if (req.method === 'GET' && url.pathname === '/api/salom') {
    const ism = url.searchParams.get('ism') ?? 'Mehmon';
    res.end(JSON.stringify({ xabar: `Salom, ${ism}!` }));
  } else if (req.method === 'POST' && url.pathname === '/api/echo') {
    try {
      const data = JSON.parse(body);
      res.statusCode = 201;
      res.end(JSON.stringify({ olindi: data }));
    } catch {
      res.statusCode = 400;
      res.end(JSON.stringify({ xato: 'JSON noto\'g\'ri' }));
    }
  } else {
    res.statusCode = 404;
    res.end(JSON.stringify({ xato: 'Topilmadi' }));
  }
});

server.listen(3000, () => console.log('http://localhost:3000'));

```

Ko'rib turganingizdek: marshrutlash (`if/else`), tanani o'qish, JSON'ni parse qilish, xatolar — hammasi qo'lda. Marshrutlar 50 taga yetsa, bu kod boshqarib bo'lmaydigan bo'ladi. Aynan shu muammoni **Express** hal qiladi (9-dars).

Amaliy misollar

MISOL 1

Yuklamalar papkasini tartiblovchi CLI dastur

"Downloads" papkasidagi yuzlab fayllarni turiga qarab papkalarga ajratadigan foydali skript.


```

#!/usr/bin/env node
import fs from 'node:fs/promises';
import path from 'node:path';

const TOIFALAR = {
  Rasmlar: ['.jpg', '.jpeg', '.png', '.gif', '.webp', '.svg'],
  Hujjatlar: ['.pdf', '.doc', '.docx', '.xls', '.xlsx', '.ppt', '.pptx', '.txt'],
  Videolar: ['.mp4', '.mkv', '.avi', '.mov'],
  Musiqa: ['.mp3', '.wav', '.flac'],
  Arxivlar: ['.zip', '.rar', '.7z', '.tar', '.gz'],
  Dasturlar: ['.exe', '.msi', '.dmg', '.deb'],
};

// Kengaytma → toifa (tez qidirish uchun teskari jadval)
const kengaytmaToifa = Object.fromEntries(
  Object.entries(TOIFALAR).flatMap(([toifa, exts]) ⇒ exts.map(e ⇒ [e, toifa]))
);

const [, , papka = '.', ...flags] = process.argv;
const sinov = flags.includes('--dry-run'); // faqat ko'rsatish, ko'chirmaslik

async function tartibla(dir) {
  const elementlar = await fs.readdir(dir, { withFileTypes: true });
  const hisobot = {};

  for (const el of elementlar) {
    if (!el.isFile() || el.name.startsWith('.')) continue;

    const toifa = kengaytmaToifa[path.extname(el.name).toLowerCase()] ?? 'Boshqalar';
    const manzilPapka = path.join(dir, toifa);
    let manzil = path.join(manzilPapka, el.name);

    // Bir xil nomli fayl bo'lsa: rasm.png → rasm (1).png
    for (let i = 1; await mavjudmi(manzil); i++) {
      const { name, ext } = path.parse(el.name);
      manzil = path.join(manzilPapka, `${name} (${i})${ext}`);
    }
  }
}

```

```

    if (!sinov) {
      await fs.mkdir(manzilPapka, { recursive: true });
      await fs.rename(path.join(dir, el.name), manzil);
    }
    hisobot[toifa] = (hisobot[toifa] ?? 0) + 1;
    console.log(`${sinov ? '[sinov] ' : ''}${el.name} → ${toifa}/`);
  }
  return hisobot;
}

async function mavjudmi(p) {
  try { await fs.access(p); return true; } catch { return false; }
}

try {
  const natija = await tartibla(path.resolve(papka));
  console.log(`\n📊 Natija:`);
  console.table(natija);
} catch (err) {
  console.error(`❌ Xato: ${err.code === 'ENOENT' ? 'papka topilmadi' : err.message}`);
  process.exit(1);
}

```

Terminal

```

node organize.js ~/Downloads --dry-run # avval nima bo'lishini ko'ramiz
node organize.js ~/Downloads          # haqiqiy ko'chirish

```

`--dry-run` rejimi — xavfli amallarni bajaruvchi har qanday skriptda bo'lishi kerak bo'lgan yaxshi odat.

MISOL 2 Statik fayl serveri (http + fs + path)

Shaxsiy saytingizni (2–3-darslar) brauzerda `http://localhost:8080` orqali ochadigan server. Unda real serverlar duch keladigan muammolar — MIME turlari, 404 va xavfsizlik — hal qilingan.

```

import http from 'node:http';
import fs from 'node:fs/promises';
import { createReadStream } from 'node:fs';
import path from 'node:path';

const ROOT = path.resolve(process.argv[2] ?? 'public');
const PORT = Number(process.env.PORT) || 8080;

const MIME = {
  '.html': 'text/html; charset=utf-8',
  '.css': 'text/css; charset=utf-8',
  '.js': 'text/javascript; charset=utf-8',
  '.json': 'application/json',
  '.png': 'image/png',
  '.jpg': 'image/jpeg',
  '.svg': 'image/svg+xml',
  '.ico': 'image/x-icon',
  '.woff2': 'font/woff2',
};

const server = http.createServer(async (req, res) => {
  const started = Date.now();
  res.on('finish', () =>
    console.log(`${req.method} ${req.url} → ${res.statusCode} (${Date.now() - started}ms)`));
  try {
    const { pathname } = new URL(req.url, 'http://localhost');
    let filePath = path.join(ROOT, decodeURIComponent(pathname));

    // 🚫 Path traversal hujumidan himoya: ../../etc/passwd
    if (filePath !== ROOT && !filePath.startsWith(ROOT + path.sep)) {
      res.writeHead(403).end('Forbidden');
      return;
    }

    let stat = await fs.stat(filePath).catch(() => null);
    if (stat?.isDirectory()) {
      filePath = path.join(filePath, 'index.html');
      stat = await fs.stat(filePath).catch(() => null);
    }
  }
});

```

```

    if (!stat) {
      res.writeHead(404, { 'Content-Type': 'text/html; charset=utf-8' });
      res.end('<h1>404</h1><p>Sahifa topilmadi</p>');
      return;
    }

    res.writeHead(200, {
      'Content-Type': MIME[path.extname(filePath)] ?? 'application/octet-
stream',
      'Content-Length': stat.size,
      'Cache-Control': 'public, max-age=3600',
    });
    createReadStream(filePath).pipe(res); // stream – katta fayllar ham
xotirani to'ldirmaydi
  } catch (err) {
    console.error(err);
    res.writeHead(500).end('Server xatosi');
  }
});

server.listen(PORT, () => console.log(`📁 ${ROOT} →
http://localhost:${PORT}`));

```

Terminal

```

node static-server.js ./portfolio
curl -i http://localhost:8080/
curl -i --path-as-is http://localhost:8080/../../etc/passwd # 403
qaytishi kerak

```

MISOL 3 JSON faylga asoslangan "mini ma'lumotlar bazasi" moduli

Haqiqiy bazaga (10-dars) o'tishdan oldin, ma'lumotni JSON faylda saqlovchi qayta ishlatiladigan modul yozamiz. U 9-darsdagi Express API'da ishlatiladi.

```

import fs from 'node:fs/promises';
import path from 'node:path';
import { randomUUID } from 'node:crypto';

export class JsonStore {
  #file;
  #queue = Promise.resolve(); // yozuvlarni navbatga qo'yish uchun

  constructor(file) {
    this.#file = path.resolve(file);
  }

  async #read() {
    try {
      return JSON.parse(await fs.readFile(this.#file, 'utf8'));
    } catch (err) {
      if (err.code === 'ENOENT') return []; // fayl hali yo'q
      throw err;
    }
  }

  // Bir vaqtdagi ikki yozuv bir-birini o'chirib yubormasligi uchun -
  // ketma-ket bajaramiz
  #write(mutator) {
    this.#queue = this.#queue.then(async () => {
      const items = await this.#read();
      const result = mutator(items);
      const tmp = `${this.#file}.tmp`;
      await fs.mkdir(path.dirname(this.#file), { recursive: true });
      await fs.writeFile(tmp, JSON.stringify(items, null, 2));
      await fs.rename(tmp, this.#file); // atomar almashtirish -
      // fayl hech qachon yarim yozilmaydi
      return result;
    });
    return this.#queue;
  }

  async all({ filter = () => true, sort } = {}) {
    const items = (await this.#read()).filter(filter);
    return sort ? items.sort(sort) : items;
  }
}

```

```

async get(id) {
  return (await this.#read()).find(i => i.id === id) ?? null;
}

create(data) {
  return this.#write(items => {
    const item = { id: randomUUID(), ...data, createdAt: new
Date().toISOString() };
    items.push(item);
    return item;
  });
}

update(id, patch) {
  return this.#write(items => {
    const i = items.findIndex(x => x.id === id);
    if (i === -1) return null;
    items[i] = { ...items[i], ...patch, id, updatedAt: new
Date().toISOString() };
    return items[i];
  });
}

remove(id) {
  return this.#write(items => {
    const i = items.findIndex(x => x.id === id);
    if (i === -1) return false;
    items.splice(i, 1);
    return true;
  });
}
}

```

try.js

```
import { JsonStore } from './src/db/jsonStore.js';

const books = new JsonStore('data/books.json');
const b = await books.create({ title: 'Clean Code', author: 'Robert Martin', price: 180000 });
await books.update(b.id, { price: 165000 });
console.log(await books.all({ sort: (a, b) => a.price - b.price }));

// Bir vaqtda 10 ta yozuv – navbat tufayli birortasi ham yo'qolmaydi
await Promise.all(Array.from({ length: 10 }, (_, i) => books.create({ title: `Kitob ${i}` })));
console.log((await books.all()).length); // 11
```



Navbatsiz versiyada parallel `create` chaqiruvlari bir xil eski faylni o'qib, bir-birining natijasini ustidan yozib yuborardi — klassik **race condition**. Haqiqiy ma'lumotlar bazalari bu muammoni tranzaksiyalar bilan hal qiladi (10-dars).

MISOL 4

Valyuta kursi CLI: tashqi API, argumentlar va kesh

O'zbekiston Markaziy banki ochiq API'sidan kurslarni olib, summani konvertatsiya qiladi. Natija 1 soat keshlanadi.

```

#!/usr/bin/env node
import fs from 'node:fs/promises';
import os from 'node:os';
import path from 'node:path';
import { parseArgs } from 'node:util';

const { values, positionals } = parseArgs({
  allowPositionals: true,
  options: {
    to: { type: 'string', short: 't', default: 'UZS' },
    fresh: { type: 'boolean', default: false },
    help: { type: 'boolean', short: 'h' },
  },
});

if (values.help || positionals.length < 2) {
  console.log(`Foydalanish: kurs <summa> <valyuta> [--to UZS] [--fresh]
Misollar:
  node kurs.js 100 USD
  node kurs.js 1000000 UZS --to EUR`);
  process.exit(0);
}

const [summaStr, from] = positionals;
const summa = Number(summaStr);
const CACHE = path.join(os.tmpdir(), 'cbu-rates.json');
const TTL = 60 * 60 * 1000;

async function kurslarniOl() {
  if (!values.fresh) {
    try {
      const { savedAt, rates } = JSON.parse(await fs.readFile(CACHE,
'utf8'));
      if (Date.now() - savedAt < TTL) return rates;
    } catch { /* kesh yo'q — davom etamiz */ }
  }
  const res = await fetch('https://cbu.uz/uz/arkhiv-kursov-valyut/json/',
{
    signal: AbortSignal.timeout(10_000),
  });
  if (!res.ok) throw new Error(`CBU API: ${res.status}`);
  const list = await res.json();

```



```

// [{ Ccy: 'USD', Rate: '12650.15', Nominal: '1', Date: '03.10.2026',
... }]

const rates = { UZS: 1 };
for (const r of list) rates[r.Ccy] = Number(r.Rate) / Number(r.Nominal);
await fs.writeFile(CACHE, JSON.stringify({ savedAt: Date.now(), rates
}));
return rates;
}

try {
  if (!Number.isFinite(summa) || summa < 0) throw new Error('Summa
noto\'g\'ri');
  const rates = await kurslarniOl();
  const a = from.toUpperCase(), b = values.to.toUpperCase();
  if (!rates[a] || !rates[b]) throw new Error(`Noma'lum valyuta:
${!rates[a] ? a : b}`);

  const natija = (summa * rates[a]) / rates[b];
  const f = (n, c) => new Intl.NumberFormat('uz-UZ', {
maximumFractionDigits: 2 }).format(n) + ' ' + c;
  console.log(`💰 ${f(summa, a)} = ${f(natija, b)}`);
  console.log(` 1 ${a} = ${f(rates[a] / rates[b], b)}`);
} catch (err) {
  console.error('❌', err.message);
  process.exitCode = 1;
}

```

Terminal

```

node kurs.js 100 USD
node kurs.js 5000000 UZS --to EUR
node kurs.js 50 EUR -t USD --fresh

```

`package.json` ga `"bin": { "kurs": "./kurs.js" }` qo'shib, `npm link` qilsangiz, buyruq butun tizimda `kurs 100 USD` ko'rinishida ishlaydi.

Mashqlar



Mustaqil ish

1. `wc.js` yozing: berilgan fayldagi qatorlar, so'zlar va belgilar sonini chiqarsin (`node wc.js matn.txt`).
2. Papkadagi barcha `.jpg` fayllar nomini `2026-10-03_001.jpg` formatiga o'zgartiruvchi skript (fayl yaratilgan sana bo'yicha).
3. 8-darsdagi `http` serveriga `GET /api/notes` va `POST /api/notes` marshrutlarini qo'shing, ma'lumot 3-misoldagi `JsonStore` da saqlansin.
4. `EventEmitter` asosida oddiy "Pomodoro taymer" yozing: `start`, `tick`, `break`, `done` hodisalari bilan.
5. Loyihangizga `.env` va `.env.example` qo'shing; PORT o'rnatilmagan bo'lsa ogohlantirish chiqaring.



Xulosa

- Node.js = V8 + libuv: JavaScript serverda, bloklanmaydigan I/O bilan.
- ES Modules (`"type": "module"`, `import/export`) — zamonaviy standart.
- `package.json` — loyiha pasporti; `package-lock.json` ni commit qiling, `node_modules` ni — yo'q.
- `fs/promises`, `path`, `events`, `streams` — eng ko'p ishlatiladigan modullar.
- Maxfiy sozlamalar — `.env` va `process.env` da.
- Sof `http` bilan server yozish mumkin, lekin katta ilovalar uchun freymvork kerak — keyingi dars Express.



09-DARS · 14 TADAN

Express.js va REST API

Express asoslari, marshrutlash, middleware zanjiri, REST API dizayni, validatsiya, xatolarni markazlashgan boshqarish, CORS va loyiha tuzilmasi (MVC).

O'rta · ~140 daqiqa

Express — Node.js uchun eng mashhur minimalistik web-freymvork. U oldingi darsda qo'lda yozgan narsalarimizni — marshrutlash, JSON'ni o'qish, statik fayllar, xatolarni boshqarish — oddiy va tushunarli API bilan beradi. Bu darsda frontend ilovalar "gaplashadigan" haqiqiy **REST API** qurishni o'rganamiz.

Birinchi Express ilova

Terminal

```
mkdir kitob-api && cd kitob-api
npm init -y
npm pkg set type=module
npm install express
```

src/server.js

```
import express from 'express';

const app = express();
app.use(express.json()); // JSON tanani avtomatik o'qish (req.body)

app.get('/', (req, res) => {
  res.send('Kitob API ishlayapti 📖');
});

app.get('/api/salom/:ism', (req, res) => {
  res.json({ xabar: `Salom, ${req.params.ism}!` });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`🚀 http://localhost:${PORT}`));
```

8-darsdagi 35 qatorlik `http` serverini Express 10 qatorda bajardi. Endi har bir qismni chuqurroq ko'ramiz.

Marshrutlash (Routing)

Marshrut = **HTTP metod** + **yo'l** + **ishlovchi funksiya(lar)**.

JavaScript

```
app.get('/api/books', handler); // o'qish
app.post('/api/books', handler); // yaratish
app.put('/api/books/:id', handler); // to'liq almashtirish
app.patch('/api/books/:id', handler); // qisman yangilash
app.delete('/api/books/:id', handler); // o'chirish
app.all('/api/secret', handler); // istalgan metod

// Bitta yo'l uchun bir nechta metod
app.route('/api/books/:id')
  .get(getBook)
  .patch(updateBook)
  .delete(deleteBook);
```

So'rov (req) obyektidan ma'lumot olish

JavaScript

```
// GET /api/users/42/orders?status=paid&page=2
app.get('/api/users/:userId/orders', (req, res) => {
  req.params;           // { userId: '42' }           - yo'l parametrlari
  (doim satr!)
  req.query;            // { status: 'paid', page: '2' } - query parametrlar
  (doim satr!)
  req.body;            // POST/PUT tanasi (express.json() kerak)
  req.headers;         // barcha sarlavhalar (kichik harfda)
  req.get('authorization');
  req.method;          // 'GET'
  req.path;            // '/api/users/42/orders'
  req.ip;              // mijoz IP manzili
  req.cookies;         // cookie-parser middleware bilan
});
```

Javob (res) usullari

JavaScript

```
res.json({ id: 1 });           // JSON + Content-Type avtomatik
res.status(201).json(newBook); // status kod + JSON
res.status(204).end();         // tanasiz javob
res.send('<h1>HTML</h1>');     // matn/HTML
res.sendFile(path.join(dir, 'a.pdf'));
res.download('/files/hisobot.xlsx'); // yuklab olish oynasi
res.redirect(301, '/yangi-manzil');
res.set('Cache-Control', 'no-store');
res.cookie('theme', 'dark', { httpOnly: true, maxAge: 86400000 });
```



Bitta so'rovga **faqat bitta** javob yuboriladi. `res.json()` dan keyin yana `res.send()` chaqirilsa — "Cannot set headers after they are sent" xatosi. Shuning uchun javobdan keyin `return` qilish odatiy: `if (!book) return res.status(404).json(...)`.

Router — marshrutlarni modullarga ajratish

src/routes/books.routes.js

```
import { Router } from 'express';
const router = Router();

router.get('/', (req, res) => { /* barcha kitoblar */ });
router.get('/:id', (req, res) => { /* bitta kitob */ });
router.post('/', (req, res) => { /* yaratish */ });

export default router;
```

src/app.js

```
import booksRouter from './routes/books.routes.js';
import authorsRouter from './routes/authors.routes.js';

app.use('/api/books', booksRouter); // router ichidagi '/' → '/api/books'
app.use('/api/authors', authorsRouter);
```

Middleware — Express'ning yuragi

Middleware — so'rov va javob o'rtasida ishlaydigan funksiya. Express ilovasi — bu middleware'lar

zanjiri: so'rov ularning har biridan ketma-ket o'tadi.



Har bir middleware so'rovni o'zgartirishi, javob qaytarib zanjirni to'xtatishi yoki next() bilan keyingisiga uzatishi mumkin

JavaScript

```
function middleware(req, res, next) {
  // 1. req/res ni o'qish yoki o'zgartirish
  // 2. va keyin:
  next();           // → keyingi middleware'ga o'tish
  // yoki: res.status(401).json(...) – zanjirni to'xtatish
  // yoki: next(err)           – xato ishlovchisiga o'tish
}
```

Middleware turlari

JavaScript

```
// 1. Ilova darajasida – barcha so'rovlar uchun
app.use(express.json({ limit: '1mb' }));           // JSON tana
app.use(express.urlencoded({ extended: true }));   // HTML forma tanasi
app.use(express.static('public'));                 // statik fayllar
(public/index.html → /)

// 2. Yo'l bo'yicha
app.use('/api', apiRateLimiter);

// 3. Marshrut darajasida – faqat shu marshrut uchun (bir nechta bo'lishi
mumkin)
app.delete('/api/books/:id', requireAuth, requireRole('admin'), deleteBook);

// 4. Xato ishlovchi – 4 ta parametr (err, req, res, next). Doim ENG OXIRIDA
app.use((err, req, res, next) ⇒ {
  res.status(500).json({ message: 'Server xatosi' });
});
```



Tartib muhim!

Middleware'lar **yoziqlan tartibda** bajariladi. `express.json()` marshrutlardan keyin yozilsa, `req.body` `undefined` bo'ladi. 404 ishlovchisi barcha marshrutlardan keyin, xato ishlovchisi — eng oxirida.

Mashhur tayyor middleware'lar

Paket	Vazifasi
<code>cors</code>	Boshqa domendagi frontendga API'dan foydalanishga ruxsat
<code>helmet</code>	Xavfsizlik sarlavhalarini o'rnatadi
<code>morgan</code>	So'rovlarni log qilish
<code>express-rate-limit</code>	Bir IP'dan so'rovlar sonini cheklash
<code>cookie-parser</code>	Cookie'larni <code>req.cookies</code> ga o'qish
<code>multer</code>	Fayl yuklash (<code>multipart/form-data</code>)
<code>compression</code>	Javoblarni gzip bilan siqish

REST API dizayni

REST (Representational State Transfer) — API'ni loyihalash uslubi. Asosiy g'oya: hamma narsa — **resurs** (kitob, foydalanuvchi, buyurtma), har bir resurs **URL** bilan aniqlanadi, ular ustidagi amallar **HTTP metodlari** bilan ifodalanadi.

Amal	Metod va URL	Muvaffaqiyatli javob
Ro'yxat	<code>GET /api/books</code>	<code>200</code> + massiv
Bittasini olish	<code>GET /api/books/42</code>	<code>200</code> + obyekt / <code>404</code>
Yaratish	<code>POST /api/books</code>	<code>201</code> + yaratilgan obyekt
To'liq yangilash	<code>PUT /api/books/42</code>	<code>200</code>
Qisman yangilash	<code>PATCH /api/books/42</code>	<code>200</code>
O'chirish	<code>DELETE /api/books/42</code>	<code>204</code>
Ichki resurs	<code>GET /api/authors/7/books</code>	<code>200</code>

Yaxshi API qoidalari

- URL'da **otlar**, ko'plikda: `/api/books` ✓, `/api/getBooks` ✗, `/api/book/delete/5` ✗.
- Kichik harflar va defis: `/api/order-items`.
- Filtrlash, saralash, sahifalash — query orqali: `/api/books?genre=roman&sort=-price&page=2&limit=20`.
- To'g'ri status kodlar (1-dars jadvali).
- Xatolar **bir xil formatda**: `{ "error": { "message": "...", "details": [...] } }`.
- Versiyalash: `/api/v1/books` — API'ni buzuvchi o'zgarishlar kiritilganda eski mijozlar ishlashda davom etadi.
- Sahifalangan javobda meta-ma'lumot: `{ data: [...], meta: { page, limit, total } }`.

Validatsiya

Mijozdan kelgan har qanday ma'lumot — **ishonchsiz**. Validatsiyasiz API bazaga noto'g'ri ma'lumot yozadi, xatolar bilan "yiqiladi" va hujumlarga ochiq bo'ladi. Qo'lda tekshirish mumkin, lekin **Zod** kabi sxema kutubxonalari qulayroq:

Terminal

```
npm install zod
```

```
import { z } from 'zod';

const bookSchema = z.object({
  title: z.string().trim().min(1, 'Nom majburiy').max(200),
  author: z.string().trim().min(2),
  price: z.number().int().positive('Narx musbat bo\'lishi kerak'),
  year: z.number().int().min(1450).max(new Date().getFullYear()).optional(),
  genres: z.array(z.enum(['roman', 'ilmiy', 'bolalar',
    'dasturlash']))).default([]),
});

const result = bookSchema.safeParse(req.body);
if (!result.success) {
  // result.error.issues → [{ path: ['price'], message: 'Narx musbat bo\'lishi
  kerak' }, ...]
}
const cleanData = result.data; // tozalangan, standart qiymatlar qo'shilgan,
ortiqcha maydonlar olib tashlangan

// PATCH uchun — barcha maydonlar ixtiyoriy
const bookPatchSchema = bookSchema.partial();
```

Xatolarni markazlashgan boshqarish

Har bir marshrutda `try/catch` yozib, xatoni qo'lda formatlash — takrorlanuvchi kod. To'g'ri yondashuv: marshrutlar xatoni **tashlaydi (throw)**, bitta xato middleware'i ularni **bir xil formatda** javobga aylantiradi.

```
export class HttpError extends Error {
  constructor(status, message, details) {
    super(message);
    this.status = status;
    this.details = details;
  }
  static badRequest(msg = 'Noto\'g\'ri so\'rov', details) { return new
HttpError(400, msg, details); }
  static unauthorized(msg = 'Avtorizatsiya talab qilinadi') { return new
HttpError(401, msg); }
  static forbidden(msg = 'Ruxsat yo\'q') { return new HttpError(403, msg); }
  static notFound(msg = 'Topilmadi') { return new HttpError(404, msg); }
  static conflict(msg) { return new HttpError(409, msg); }
}
```

```
import { HttpError } from '../utils/errors.js';

export function notFound(req, res, next) {
  next(HttpError.notFound(`Marshrut topilmadi: ${req.method}
${req.originalUrl}`));
}

// eslint-disable-next-line no-unused-vars
export function errorHandler(err, req, res, next) {
  // Noto'g'ri JSON (express.json() tashlaydi)
  if (err.type === 'entity.parse.failed') {
    err = HttpError.badRequest('So\'rov tanasidagi JSON noto\'g\'ri');
  }

  const status = err.status ?? 500;
  if (status ≥ 500) console.error(err); // kutilmagan xatolarni log qilamiz

  res.status(status).json({
    error: {
      message: status ≥ 500 && process.env.NODE_ENV === 'production'
        ? 'Ichki server xatosi' // 🔒 production'da ichki
        : err.message,          tafsilotlarni oshkor qilmaymiz
      ...(err.details && { details: err.details }),
    },
  });
}
```



Express 5 va async xatolar

Express 5 da `async` marshrut ichida tashlangan xato yoki rad etilgan Promise avtomatik ravishda xato middleware'iga uzatiladi. Express 4 da esa buning uchun har bir handler'ni o'rash yoki `express-async-errors` paketini ishlatish kerak edi.

CORS

Frontend `http://localhost:5173` da, API `http://localhost:3000` da ishlasa, brauzer so'rovni bloklaydi (6-dars). Ruxsatni server beradi:

JavaScript

```
import cors from 'cors';

app.use(cors({
  origin: ['http://localhost:5173', 'https://kitoblar.uz'], // faqat ishonchli
manbalar
  credentials: true, // cookie
yuborishga ruxsat
}));
// ⚠ origin: '*' – ochiq API'lardan tashqari tavsiya etilmaydi
```

Loyiha tuzilmasi

Hamma kodni bitta faylga yozish 100 qatorgacha ishlaydi. Katta loyihalarda **mas'uliyatlarni ajratish** kerak:

```

kitob-api/
├── src/
│   ├── server.js           # faqat ishga tushirish (listen)
│   ├── app.js             # Express ilova: middleware'lar va router'lar
│   ├── config.js          # muhit o'zgaruvchilari bir joyda
│   ├── routes/            # URL → controller bog'lanishi
│   │   └── books.routes.js
│   ├── controllers/       # HTTP qatlami: req'dan o'qish, res'ga yozish
│   │   └── books.controller.js
│   └── services/          # biznes-mantiq (HTTP haqida hech narsa
bilmaydi)
│   └── books.service.js
│   ├── db/                # ma'lumotlar bilan ishlash
│   ├── middleware/        # auth, validate, errorHandler
│   ├── schemas/           # Zod sxemalari
│   └── utils/
├── tests/
├── .env
├── .env.example
├── .gitignore
└── package.json

```

Nega bunday? Controller "qanday so'raldi" bilan, service "nima qilish kerak" bilan, db qatlami "qayerda saqlanadi" bilan shug'ullanadi. JSON fayldan PostgreSQL'ga o'tganda faqat db qatlami o'zgaradi; service'ni HTTP'siz test qilish mumkin. `app.js` va `server.js` ning ajratilishi esa testlarda ilovani portni band qilmasdan ishga tushirishga imkon beradi (14-dars).

Amaliy misollar

MISOL 1 Kitoblar do'koni uchun to'liq CRUD REST API

8-darsdagi `JsonStore` ustiga qurilgan, yuqoridagi tuzilmaga amal qilgan to'liq API. Filtrlash, qidiruv, saralash va sahifalash ham bor.

Terminal

```
npm install express zod cors helmet morgan
```

src/schemas/book.schema.js

```
import { z } from 'zod';

export const bookSchema = z.object({
  title: z.string().trim().min(1).max(200),
  author: z.string().trim().min(2).max(100),
  price: z.number().int().positive(),
  stock: z.number().int().min(0).default(0),
  genre: z.enum(['roman', 'ilmiy', 'bolalar', 'dasturlash', 'tarix']),
  year: z.number().int().min(1450).max(new
Date().getFullYear()).optional(),
});

export const bookPatchSchema = bookSchema.partial();

export const listQuerySchema = z.object({
  q: z.string().trim().optional(),
  genre: bookSchema.shape.genre.optional(),
  minPrice: z.coerce.number().int().min(0).optional(), // coerce:
'5000' → 5000
  maxPrice: z.coerce.number().int().min(0).optional(),
  sort: z.enum(['price', '-price', 'title', '-createdAt']).default('-
createdAt'),
  page: z.coerce.number().int().min(1).default(1),
  limit: z.coerce.number().int().min(1).max(100).default(10),
});
```

src/middleware/validate.js

```
import { HttpError } from '../utils/errors.js';

// Sxema bo'yicha req.body / req.query ni tekshiruvchi middleware
"fabrikasi"
export const validate = (schema, source = 'body') ⇒ (req, res, next) ⇒ {
  const result = schema.safeParse(req[source]);
  if (!result.success) {
    const details = result.error.issues.map(i ⇒ ({ field:
i.path.join('.'), message: i.message }));
    return next(HttpError.badRequest('Validatsiya xatosi', details));
  }
  req.validated = { ...req.validated, [source]: result.data };
  next();
};
```



```
import { JsonStore } from '../db/jsonStore.js';
import { HttpError } from '../utils/errors.js';

const store = new JsonStore('data/books.json');

const SORTERS = {
  price: (a, b) => a.price - b.price,
  '-price': (a, b) => b.price - a.price,
  title: (a, b) => a.title.localeCompare(b.title),
  '-createdAt': (a, b) => b.createdAt.localeCompare(a.createdAt),
};

export async function listBooks({ q, genre, minPrice, maxPrice, sort,
page, limit }) {
  const needle = q?.toLowerCase();
  const all = await store.all({
    filter: (b) =>
      (!needle || b.title.toLowerCase().includes(needle) ||
b.author.toLowerCase().includes(needle)) &&
      (!genre || b.genre === genre) &&
      (minPrice === undefined || b.price ≥ minPrice) &&
      (maxPrice === undefined || b.price ≤ maxPrice),
    sort: SORTERS[sort],
  });
  const start = (page - 1) * limit;
  return {
    data: all.slice(start, start + limit),
    meta: { page, limit, total: all.length, pages: Math.ceil(all.length /
limit) },
  };
}

export async function getBook(id) {
  const book = await store.get(id);
  if (!book) throw HttpError.notFound('Kitob topilmadi');
  return book;
}

export async function createBook(data) {
  const duplicate = await store.all({
    filter: b => b.title.toLowerCase() === data.title.toLowerCase() &&
b.author === data.author,
```

```

});
if (duplicate.length) throw HttpError.conflict('Bu kitob allaqachon mavjud');
return store.create(data);
}

export async function updateBook(id, patch) {
  const book = await store.update(id, patch);
  if (!book) throw HttpError.notFound('Kitob topilmadi');
  return book;
}

export async function deleteBook(id) {
  if (!(await store.remove(id))) throw HttpError.notFound('Kitob topilmadi');
}

```

src/controllers/books.controller.js

```

import * as service from '../services/books.service.js';

export const list = async (req, res) => res.json(await service.listBooks(req.validated.query));
export const getOne = async (req, res) => res.json(await service.getBook(req.params.id));
export const create = async (req, res) => res.status(201).json(await service.createBook(req.validated.body));
export const update = async (req, res) => res.json(await service.updateBook(req.params.id, req.validated.body));
export const remove = async (req, res) => {
  await service.deleteBook(req.params.id);
  res.status(204).end();
};

```

src/routes/books.routes.js

```
import { Router } from 'express';
import * as ctrl from '../controllers/books.controller.js';
import { validate } from '../middleware/validate.js';
import { bookSchema, bookPatchSchema, listQuerySchema } from
'../schemas/book.schema.js';

const router = Router();

router.get('/', validate(listQuerySchema, 'query'), ctrl.list);
router.get('/:id', ctrl.getOne);
router.post('/', validate(bookSchema), ctrl.create);
router.patch('/:id', validate(bookPatchSchema), ctrl.update);
router.delete('/:id', ctrl.remove);

export default router;
```

src/app.js

```
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import morgan from 'morgan';
import booksRouter from './routes/books.routes.js';
import { notFound, errorHandler } from './middleware/errorHandler.js';

export const app = express();

app.use(helmet());
app.use(cors({ origin: process.env.CLIENT_URL ?? 'http://localhost:5173'
})));
app.use(morgan('dev'));
app.use(express.json({ limit: '100kb' })));

app.get('/api/health', (req, res) => res.json({ status: 'ok', uptime:
process.uptime() }));
app.use('/api/v1/books', booksRouter);

app.use(notFound);
app.use(errorHandler);
```

src/server.js

```
import { app } from './app.js';

const PORT = Number(process.env.PORT) || 3000;
app.listen(PORT, () => console.log(`📖 Kitob API:
http://localhost:${PORT}/api/v1/books`));
```

Sinab ko'ramiz:

Terminal

```
# Yaratish
curl -X POST localhost:3000/api/v1/books -H "Content-Type:
application/json" \
  -d '{"title":"O'tkan kunlar","author":"Abdulla
Qodiriy","price":65000,"stock":10,"genre":"roman","year":1926}'
# → 201 { "id": "c1f...", "title": "O'tkan kunlar", ... }

# Validatsiya xatosi
curl -X POST localhost:3000/api/v1/books -H "Content-Type:
application/json" -d '{"title":"","price":-5}'
# → 400 { "error": { "message": "Validatsiya xatosi", "details": [
#       { "field": "title", "message": "..."},
#       { "field": "author", "message": "..."}, ... ] } }
#   (aniq xabar matni Zod versiyasiga bog'liq; o'zbekcha xabarlarni
sxemada berish mumkin)

# Ro'yxat: qidiruv, filtr, saralash, sahifalash
curl "localhost:3000/api/v1/books?q=qodiriy&genre=roman&sort=-
price&page=1&limit=5"

# Qisman yangilash va o'chirish
curl -X PATCH localhost:3000/api/v1/books/ID -H "Content-Type:
application/json" -d '{"price":59000}'
curl -i -X DELETE localhost:3000/api/v1/books/ID      # → 204
curl -i localhost:3000/api/v1/books/ID                # → 404
```



curl o'rniga VS Code'ning **REST Client** kengaytmasi, **Thunder Client** yoki **Postman/Insomnia** dasturlaridan foydalanish qulayroq.

```
src/middleware/common.js
```

```
import { randomUUID } from 'node:crypto';

// 1. Har bir so'rovga noyob ID – loglarda bitta so'rovning izini kuzatish uchun
export function requestId(req, res, next) {
  req.id = req.get('X-Request-Id') ?? randomUUID();
  res.set('X-Request-Id', req.id);
  next();
}

// 2. Tuzilgan (structured) logger: javob yuborilgandan keyin yozadi
export function logger(req, res, next) {
  const start = process.hrtime.bigint();
  res.on('finish', () => {
    const ms = Number(process.hrtime.bigint() - start) / 1e6;
    const line = {
      time: new Date().toISOString(),
      id: req.id,
      method: req.method,
      url: req.originalUrl,
      status: res.statusCode,
      ms: Math.round(ms * 10) / 10,
      ip: req.ip,
    };
    (res.statusCode ≥ 500 ? console.error : console.log)
    (JSON.stringify(line));
  });
  next();
}

// 3. Sekin so'rovlarni aniqlash
export const slowWarning = (thresholdMs = 1000) => (req, res, next) => {
  const t = setTimeout(() => console.warn(`🐢 Sekin so'rov
  (${thresholdMs}ms): ${req.method} ${req.originalUrl}`), thresholdMs);
  res.on('finish', () => clearTimeout(t));
  next();
};
```

```

// 4. API kaliti bilan himoya (masalan, ichki xizmatlar uchun)
export const requireApiKey = (req, res, next) => {
  const key = req.get('X-API-Key');
  if (!key || key !== process.env.INTERNAL_API_KEY) {
    return res.status(401).json({ error: { message: 'API kaliti noto\'g\'ri' } });
  }
  next();
};

// 5. Juda oddiy xotiradagi rate limiter (o'rganish uchun; productionda express-rate-limit + Redis)
export function rateLimit({ windowMs = 60_000, max = 60 } = {}) {
  const hits = new Map(); // ip -> { count, reset }
  return (req, res, next) => {
    const now = Date.now();
    const entry = hits.get(req.ip) ?? { count: 0, reset: now + windowMs };
    if (now > entry.reset) Object.assign(entry, { count: 0, reset: now + windowMs });
    entry.count++;
    hits.set(req.ip, entry);

    res.set('RateLimit-Remaining', Math.max(0, max - entry.count));
    if (entry.count > max) {
      res.set('Retry-After', Math.ceil((entry.reset - now) / 1000));
      return res.status(429).json({ error: { message: 'Juda ko\'p so\'rov. Birozdan keyin urinib ko\'ring.' } });
    }
    next();
  };
}

```

src/app.js (qo'shimcha)

```
import { requestId, logger, slowWarning, requireApiKey, rateLimit } from
'./middleware/common.js';

app.use(requestId);
app.use(logger);
app.use(slowWarning(800));
app.use('/api', rateLimit({ windowMs: 60_000, max: 100 }));
app.use('/internal', requireApiKey, internalRouter);
```

Log natijasi: {"time":"2026-10-

03T10:00:00.000Z","id":"5b1e...","method":"GET","url":"/api/v1/books","status":200,"ms":4.2,""

JSON formatidagi loglarni keyinchalik monitoring tizimlarida (Grafana, Datadog) qidirish va tahlil qilish oson.

MISOL 3 Fayl yuklash: kitob muqovasi (multer)

Terminal

```
npm install multer
```



```

import { Router } from 'express';
import multer from 'multer';
import path from 'node:path';
import { randomUUID } from 'node:crypto';
import { HttpError } from '../utils/errors.js';

const ALLOWED = new Map([
  ['image/jpeg', '.jpg'],
  ['image/png', '.png'],
  ['image/webp', '.webp'],
]);

const upload = multer({
  storage: multer.diskStorage({
    destination: 'uploads/covers',
    // 🛒 Foydalanuvchi bergan nomni ISHLATMAYMIZ – tasodifiy nom +
    ishonchli kengaytma
    filename: (req, file, cb) => cb(null, randomUUID() +
ALLOWED.get(file.mimetype)),
  }),
  limits: { fileSize: 2 * 1024 * 1024, files: 1 }, // 2 MB
  fileFilter: (req, file, cb) =>
    ALLOWED.has(file.mimetype)
      ? cb(null, true)
      : cb(HttpError.badRequest('Faqat JPG, PNG yoki WEBP rasm')),
});

const router = Router();

router.post('/books/:id/cover', upload.single('cover'), async (req, res)
=> {
  if (!req.file) throw HttpError.badRequest('Rasm yuborilmadi (maydon
nomi: cover)');
  const url = `/uploads/covers/${req.file.filename}`;
  // await booksService.updateBook(req.params.id, { coverUrl: url });
  res.status(201).json({ url, size: req.file.size });
});

export default router;

```

src/app.js (qo'shimcha)

```
import fs from 'node:fs';
fs.mkdirSync('uploads/covers', { recursive: true });
app.use('/uploads', express.static('uploads', { maxAge: '7d' }));
app.use('/api/v1', uploadRouter);

// errorHandler ichida multer xatolarini ham tushunarli qilamiz:
// if (err.code === 'LIMIT_FILE_SIZE') err = HttpError.badRequest('Fayl 2 MB dan katta');
```

Terminal

```
curl -F "cover=@muqova.jpg" localhost:3000/api/v1/books/ID/cover
# → 201 { "url": "/uploads/covers/0b6e...c1.jpg", "size": 184322 }
```

Frontend tomonda: `const fd = new FormData(); fd.append('cover', input.files[0]);`
`fetch(url, { method: 'POST', body: fd })`. Production'da fayllar odatda serverning diskida emas, S3 kabi obyekt omborlarida saqlanadi.

MISOL 4 API'ni frontend bilan bog'lash: statik sahifa + fetch

Express bitta serverda ham API'ni, ham frontend fayllarni bera oladi. `public/` papkasiga oddiy sahifa qo'shamiz:

```

<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Kitob do'koni</title>
  <style>
    body { font-family: system-ui; max-width: 760px; margin: 40px auto;
padding: 0 16px; }
    form { display: grid; grid-template-columns: 2fr 2fr 1fr 1fr auto;
gap: 8px; margin-bottom: 24px; }
    input, select, button { padding: 8px; font: inherit; }
    .book { display: flex; justify-content: space-between; padding: 10px
0; border-bottom: 1px solid #eee; }
    .error { color: #b91c1c; }
  </style>
</head>
<body>
  <h1>📖 Kitoblar</h1>
  <form id="add">
    <input name="title" placeholder="Nomi" required>
    <input name="author" placeholder="Muallif" required>
    <input name="price" type="number" placeholder="Narx" required>
    <select name="genre">
      <option value="roman">Roman</option><option
value="dasturlash">Dasturlash</option>
      <option value="ilmiy">Ilmiy</option><option
value="bolalar">Bolalar</option>
    </select>
    <button>Qo'shish</button>
  </form>
  <p id="msg" class="error"></p>
  <input id="search" placeholder="🔍 Qidirish..."
style="width:100%;margin-bottom:12px">
  <div id="list"></div>

  <script type="module">
    const API = '/api/v1/books'; // bir xil origin — CORS kerak
emas
    const list = document.querySelector('#list');
    const msg = document.querySelector('#msg');

```

```

    async function request(url, options) {
        const res = await fetch(url, {
            ...options,
            headers: { 'Content-Type': 'application/json' },
        });
        if (res.status === 204) return null;
        const data = await res.json();
        if (!res.ok) {
            const details = data.error.details?.map(d => `${d.field}:
            ${d.message}`).join('; ');
            throw new Error(details || data.error.message);
        }
        return data;
    }

    async function load(q = '') {
        const { data, meta } = await request(`${API}?
        limit=50&q=${encodeURIComponent(q)}`);
        list.replaceChildren(...data.map(b => {
            const row = document.createElement('div');
            row.className = 'book';
            const info = document.createElement('span');
            info.textContent = `${b.title} - ${b.author}
            (${b.price.toLocaleString()} so'm)`;
            const del = document.createElement('button');
            del.textContent = '🗑';
            del.onclick = async () => { await request(`${API}/${b.id}`, {
            method: 'DELETE' }); load(q); };
            row.append(info, del);
            return row;
        }));
        if (!meta.total) list.textContent = 'Kitob topilmadi';
    }

    document.querySelector('#add').addEventListener('submit', async (e) =>
    {
        e.preventDefault();
        const body = Object.fromEntries(new FormData(e.target));
        body.price = Number(body.price);
        try {
            await request(API, { method: 'POST', body: JSON.stringify(body)
        });
        } catch {
            e.target.reset();
            msg.textContent = '';
        }
    });

```

```

    load();
  } catch (err) {
    msg.textContent = err.message;      // server validatsiya xabarini
    ko'rsatamiz
  }
});

let t;
document.querySelector('#search').addEventListener('input', (e) => {
  clearTimeout(t);
  t = setTimeout(() => load(e.target.value), 300);
});

load();
</script>
</body>
</html>

```

src/app.js (qo'shimcha)

```

app.use(express.static('public')); // http://localhost:3000 →
public/index.html

```

Tabriklaymiz — bu sizning birinchi **full-stack** ilovangiz: brauzer → fetch → Express → validatsiya → service → fayl ombori va orqaga. Keyingi darslarda ombor o'rnini PostgreSQL egallaydi, autentifikatsiya qo'shiladi, frontend esa React'ga ko'chadi.

Mashqlar



Mustaqil ish

1. Kitob API'siga `authors` resursini qo'shing: `GET /api/v1/authors/:id/books` muallifning barcha kitoblarini qaytarsin.
2. `POST /api/v1/orders` yarating: `{ items: [{ bookId, qty }] }` qabul qilsin, `stock` yetariliginı tekshirib, kamaytirsın; yetmasa `409`.
3. Barcha javoblarga `X-Response-Time` sarlavhasini qo'shuvchi middleware yozing.

4. `GET /api/v1/books/stats` — janrlar bo'yicha kitoblar soni, o'rtacha narx va eng qimmat kitob. (Diqqat: bu marshrut `/:id` dan **oldin** e'lon qilinishi kerak — nega?)
5. API hujjatini `README.md` da yozing: har bir endpoint, parametrlar, so'rov va javob namunalari, xato kodlari.

▼ 4-savol javobi

Express marshrutlarni yozilgan tartibda tekshiradi. Agar `/:id` oldin tursa, `/stats` so'rovi unga mos keladi va `id = 'stats'` deb qabul qilinib, "Kitob topilmadi" qaytadi.

✓ Xulosa

- Express ilovasi — middleware'lar zanjiri; `next()` boshqaruvni uzatadi, tartib muhim.
- REST: resurslar (otlar, ko'plikda) + HTTP metodlari + to'g'ri status kodlar.
- Mijoz ma'lumotini doim validatsiya qiling (Zod); xatolarni bitta middleware'da bir xil formatda qaytaring.
- Routes → Controllers → Services → DB qatlamlari kodni test qilinadigan va o'zgartirishga oson qiladi.
- `cors`, `helmet`, rate limiting, fayl yuklashda ehtiyotkorlik — xavfsiz API asoslari.



10-DARS · 14 TADAN

Ma'lumotlar bazasi: SQL va PostgreSQL

Relyatsion bazalar, jadvallar va bog'lanishlar, SQL (SELECT, JOIN, GROUP BY), indekslar, normalizatsiya, tranzaksiyalar, Node.js'dan PostgreSQL bilan ishlash, SQL injection va MongoDB bilan taqqoslash.

O'rta · ~150 daqiqa

Oldingi darsda ma'lumotlarni JSON faylda saqladik. Bu yondashuv ming foydalanuvchida "yiqiladi": qidiruv sekin, bir vaqtdagi yozuvlar to'qnashadi, "shu muallifning 2020-yildan keyingi, narxi 50 mingdan arzon kitoblari" kabi so'rovlarni har safar qo'lda yozish kerak. **Ma'lumotlar bazasi boshqaruv tizimi (DBMS)** bu muammolarni o'nlab yillik muhandislik tajribasi bilan hal qiladi. Bu darsda eng mashhur ochiq manbali relyatsion baza — **PostgreSQL** bilan ishlaymiz.

Relyatsion va NoSQL bazalar

	Relyatsion (SQL)	NoSQL (hujjatli)
Misollar	PostgreSQL, MySQL, SQLite, SQL Server	MongoDB, Firestore, CouchDB
Tuzilma	Jadvallar, qat'iy sxema (ustunlar va turlar)	JSON'ga o'xshash hujjatlar, moslashuvchan sxema
Bog'lanishlar	Kalitlar va JOIN — kuchli	Hujjat ichiga joylash yoki havola — cheklangan
Tranzaksiyalar	To'liq ACID	Cheklangan/qo'shimcha
Qachon tanlash	Aksariyat ilovalar: e-commerce, moliya, CRM, ijtimoiy tarmoqlar	Tuzilmasi tez o'zgaradigan ma'lumot, loglar, kontent katalogi



Tavsiya

Qaysi birini tanlashni bilmasangiz — **PostgreSQL** ni tanlang. U relyatsion ma'lumotlar bilan birga `JSONB` ustunlari orqali hujjatli ma'lumotlarni ham juda yaxshi saqlaydi.

Asosiy tushunchalar

- **Jadval (table)** — bir turdagi obyektlar to'plami: `users`, `books`.
- **Qator (row, yozuv)** — bitta obyekt: bitta kitob.
- **Ustun (column)** — xususiyat va uning turi: `title TEXT`, `price INTEGER`.
- **Birlamchi kalit (PRIMARY KEY)** — har bir qatorni noyob aniqlovchi ustun (odatda `id`).
- **Tashqi kalit (FOREIGN KEY)** — boshqa jadvalning birlamchi kalitiga havola. Bog'lanishlarni yaratadi va yaxlitlikni kafolatlaydi (mavjud bo'lmagan muallifga kitob yozib bo'lmaydi).
- **Sxema (schema)** — jadvallar, ustunlar, turlar va cheklovlarning umumiy tuzilmasi.

Bog'lanish turlari

Tur	Misol	Qanday amalga oshiriladi
Birga-bir (1:1)	Foydalanuvchi ↔ profil	Ikkinchi jadvalda <code>UNIQUE</code> tashqi kalit
Birga-ko'p (1:N)	Muallif → kitoblar	"Ko'p" tomonda tashqi kalit: <code>books.author_id</code>
Ko'pga-ko'p (N:M)	Kitoblar ↔ janrlar, talabalar ↔ kurslar	Oraliq (bog'lovchi) jadval: <code>book_genres(book_id, genre_id)</code>

PostgreSQL'ni o'rnatish

Terminal

```
# Eng oson usul – Docker (14-dars):
docker run --name pg -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=kitoblar -p
5432:5432 -d postgres:17

# Yoki: postgresql.org/download dan o'rnatuvchi; macOS: brew install
postgresql@17

# Konsol mijozi
psql -h localhost -U postgres -d kitoblar
# \l – bazalar    \dt – jadvallar    \d books – jadval tuzilmasi    \q – chiqish
```

Grafik interfeys afzal bo'lsa: **pgAdmin**, **DBeaver**, **TablePlus** yoki VS Code kengaytmalari. Bulutda bepul PostgreSQL: Neon, Supabase.

SQL: jadval yaratish (DDL)

SQL

```
CREATE TABLE authors (  
  id          SERIAL PRIMARY KEY,          -- avtomatik o'suvchi butun  
  son (1, 2, 3...)                          -- son (1, 2, 3...)   
  name        TEXT NOT NULL,               -- satr ichidagi ' '   
  country      TEXT DEFAULT '0''zbekiston', -- ikkilantiriladi   
  born_year    INTEGER CHECK (born_year > 0),  
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE books (  
  id          SERIAL PRIMARY KEY,  
  title       TEXT NOT NULL,  
  author_id   INTEGER NOT NULL REFERENCES authors(id) ON DELETE RESTRICT,  
  price       INTEGER NOT NULL CHECK (price ≥ 0), -- so'mda, butun son   
  stock       INTEGER NOT NULL DEFAULT 0 CHECK (stock ≥ 0),  
  isbn        TEXT UNIQUE,  
  published   DATE,  
  meta        JSONB DEFAULT '{}'::jsonb,      -- moslashuvchan   
  qo'shimcha ma'lumot  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
ALTER TABLE books ADD COLUMN pages INTEGER;  
ALTER TABLE books RENAME COLUMN pages TO page_count;  
DROP TABLE IF EXISTS temp_data;
```

Asosiy ma'lumot turlari

Tur	Ishlatilishi
INTEGER , BIGINT	Butun sonlar; SERIAL / BIGSERIAL yoki GENERATED ALWAYS AS IDENTITY — avto-ID
NUMERIC(12,2)	Aniq o'nlik sonlar — pul uchun (FLOAT emas!)

<code>TEXT</code> , <code>VARCHAR(n)</code>	Matn
<code>BOOLEAN</code>	true/false
<code>DATE</code> , <code>TIMESTAMPZ</code>	Sana; vaqt zonasi bilan sana-vaqt (doim <code>TIMESTAMPZ</code> ni tanlang)
<code>UUID</code>	Global noyob ID: <code>gen_random_uuid()</code>
<code>JSONB</code>	JSON hujjat, indekslash va so'rov bilan
<code>TEXT[]</code>	Massiv

Cheklovlar (constraints)

`NOT NULL` , `UNIQUE` , `CHECK` , `PRIMARY KEY` , `FOREIGN KEY` — bu baza darajasidagi **oxirgi himoya chizig'i**. Ilova kodida xato bo'lsa ham, baza manfiy narx yoki mavjud bo'lmagan muallifni saqlashga yo'l qo'ymaydi.

SQL: ma'lumot bilan ishlash (CRUD)

SQL

```
-- CREATE: qo'shish
INSERT INTO authors (name, born_year) VALUES
  ('Abdulla Qodiriy', 1894),
  ('Cho'lpon', 1897),
  ('Robert Martin', 1952);

INSERT INTO books (title, author_id, price, stock, published)
VALUES ('O'tkan kunlar', 1, 65000, 12, '1926-01-01')
RETURNING id, title;                                -- yaratilgan qatorni darhol
qaytaradi

-- READ: o'qish
SELECT * FROM books;
SELECT title, price FROM books WHERE price < 100000 ORDER BY price DESC LIMIT
10;
SELECT * FROM books WHERE title ILIKE '%kun%';        -- katta-kichik harfsiz
qidiruv
SELECT * FROM books WHERE author_id IN (1, 2) AND stock > 0;
SELECT * FROM books WHERE published BETWEEN '1920-01-01' AND '1940-12-31';
SELECT * FROM books WHERE isbn IS NULL;
SELECT DISTINCT country FROM authors;

-- Sahifalash: 3-sahifa, har sahifada 20 ta
SELECT * FROM books ORDER BY id LIMIT 20 OFFSET 40;

-- UPDATE: yangilash (WHERE ni UNUTMANG – aks holda BARCHA qatorlar o'zgaradi!)
UPDATE books SET price = 59000, stock = stock - 1 WHERE id = 1 RETURNING *;

-- DELETE: o'chirish (WHERE ni UNUTMANG!)
DELETE FROM books WHERE stock = 0 AND published < '1900-01-01';
```

JOIN — jadvallarni birlashtirish

Relyatsion bazalarning asosiy kuchi. Kitob va muallif nomini birga olish:

SQL

```
-- INNER JOIN: faqat ikkala jadvalda mos keladigan qatorlar
SELECT b.title, a.name AS author, b.price
FROM books b
JOIN authors a ON a.id = b.author_id
ORDER BY a.name;

-- LEFT JOIN: chap jadvalning HAMMA qatorlari (o'ng tomonda mos bo'lmasa -
NULL)
-- "Barcha mualliflar va kitoblari, kitobi yo'qlari ham"
SELECT a.name, b.title
FROM authors a
LEFT JOIN books b ON b.author_id = a.id;

-- Kitobi yo'q mualliflar
SELECT a.name
FROM authors a
LEFT JOIN books b ON b.author_id = a.id
WHERE b.id IS NULL;
```

JOIN turi	Natija
INNER JOIN	Faqat ikkala tomonda mos kelganlar (kesishma)
LEFT JOIN	Chapdagi hammasi + o'ngdan mos kelganlar
RIGHT JOIN	O'ngdagi hammasi + chapdan mos kelganlar (kam ishlatiladi)
FULL JOIN	Ikkala tomonning hammasi

Agregatsiya: GROUP BY va HAVING

SQL

```
-- Umumiy statistika
SELECT COUNT(*) AS jami, AVG(price)::int AS ortacha, MIN(price), MAX(price),
SUM(price * stock) AS ombor_qiymati
FROM books;

-- Har bir muallif bo'yicha
SELECT a.name, COUNT(b.id) AS kitoblar, COALESCE(SUM(b.stock), 0) AS nusxalar
FROM authors a
LEFT JOIN books b ON b.author_id = a.id
GROUP BY a.id, a.name
HAVING COUNT(b.id) ≥ 2           -- HAVING – guruhlardan keyin filtr (WHERE
-- guruhlashdan oldin)
ORDER BY kitoblar DESC;

-- Bajarilish tartibi: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER
BY → LIMIT
```

Subquery va CTE

SQL

```
-- O'rtacha narxdan qimmat kitoblar
SELECT title, price FROM books
WHERE price > (SELECT AVG(price) FROM books);

-- CTE (WITH) – murakkab so'rovni nomlangan qadamlarga bo'lish
WITH oylik AS (
    SELECT date_trunc('month', created_at) AS oy, SUM(total) AS daromad
    FROM orders
    WHERE status = 'paid'
    GROUP BY 1
)
SELECT oy, daromad,
       daromad - LAG(daromad) OVER (ORDER BY oy) AS oldingi_oyga_nisbatan
FROM oylik
ORDER BY oy;
```

Normalizatsiya

Normalizatsiya — ma'lumotlarni takrorlanmaydigan va ziddiyatsiz qilib jadvallarga bo'lish. Yomon dizayn:

order_id	customer_name	customer_phone	products
1	Ali Valiyev	+998901112233	Kitob A, Kitob B
2	Ali Valiyev	+998901112233	Kitob C

Muammolar: Ali telefonini o'zgartirsa, uni barcha buyurtmalarda yangilash kerak (birida unutilsa — ziddiyat); "Kitob A nechta sotildi?" degan savolga matnni parse qilmasdan javob berib bo'lmaydi. To'g'ri dizayn — har bir mavjudot alohida jadvalda, bog'lanishlar kalitlar orqali:

Matn

```
customers(id, name, phone)
orders(id, customer_id → customers, status, total, created_at)
order_items(order_id → orders, book_id → books, qty, price)  -- narx sotilgan
paytdagi qiymat!
```

Asosiy qoidalar (1NF–3NF): har bir katakda bitta qiymat (ro'yxat emas); har bir ustun to'liq birlamchi kalitga bog'liq; kalit bo'lmagan ustunlar bir-biriga bog'liq emas. Amaliy qoida: *"bitta fakt — bitta joyda"*.

 `order_items.price` ataylab takrorlangan: kitob narxi keyin o'zgarsa ham, eski buyurtmada sotilgan narx saqlanib qolishi kerak. Bu — ongli denormalizatsiya.

Indekslar

Indekssiz baza `WHERE email = '...'` so'rovi uchun **har bir qatorni** tekshiradi (Seq Scan). Indeks — kitob oxiridagi alifbo ko'rsatkichi kabi: kerakli joyni darhol topadi (B-tree, $O(\log n)$).

SQL

```
CREATE INDEX idx_books_author ON books(author_id);           -- JOIN va filtrlar uchun
CREATE UNIQUE INDEX idx_users_email ON users(lower(email)); -- katta-kichik harfsiz noyoblik
CREATE INDEX idx_orders_user_date ON orders(user_id, created_at DESC); -- kompozit

-- So'rov rejasini ko'rish
EXPLAIN ANALYZE SELECT * FROM books WHERE author_id = 5;
-- Index Scan using idx_books_author ... (actual time=0.02..0.03 ms)
```

- **PRIMARY KEY** va **UNIQUE** avtomatik indekslanadi; **tashqi kalitlar** — **yo'q**, ularni qo'lda indekslang.
- Indeks o'qishni tezlashtiradi, lekin yozishni biroz sekinlashtiradi va joy egallaydi — faqat haqiqatan so'raladigan ustunlarga qo'ying.

Tranzaksiyalar va ACID

Tranzaksiya — bir nechta amalni "**hammasi yoki hech biri**" tamoyili bilan bajarish. Klassik misol — pul o'tkazmasi: birinchi hisobdan yechildi, ikkinchisiga yozishda server o'chib qoldi — pul "yo'qoladi". Tranzaksiya buni oldini oladi.

SQL

```
BEGIN;
  UPDATE accounts SET balance = balance - 500000 WHERE id = 1;
  UPDATE accounts SET balance = balance + 500000 WHERE id = 2;
COMMIT;           -- ikkalasi birga saqlanadi
-- yoki xato bo'lsa: ROLLBACK; - hech narsa o'zgarmaydi
```

- **Atomicity** — bo'linmaslik: hammasi yoki hech biri.
- **Consistency** — izchillik: cheklovlar (`CHECK balance ≥ 0`) har doim bajariladi.
- **Isolation** — ajratilganlik: parallel tranzaksiyalar bir-birining yarim natijasini ko'rmaydi.
- **Durability** — barqarorlik: **COMMIT** dan keyin ma'lumot, hatto elektr o'chsa ham, saqlanadi.

Node.js'dan PostgreSQL bilan ishlash

Terminal

```
npm install pg
```

src/db/pool.js

```
import pg from 'pg';

// Pool – tayyor ulanishlar to'plami (har so'rovga yangi ulanish ochish qimmat)
export const pool = new pg.Pool({
  connectionString: process.env.DATABASE_URL, //
  postgres://user:pass@host:5432/db
  max: 10,
});

export const query = (text, params) => pool.query(text, params);
```

JavaScript

```
import { query } from './db/pool.js';

const { rows } = await query('SELECT id, title, price FROM books WHERE price < $1 ORDER BY price', [100000]);
console.log(rows); // [{ id: 1, title: "O'tkan kunlar", price: 65000 }, ...]

const { rows: [book] } = await query(
  'INSERT INTO books (title, author_id, price) VALUES ($1, $2, $3) RETURNING *',
  ['Kecha va kunduz', 2, 55000]
);
```

SQL injection — eng xavfli xato

JavaScript

```
// ❌ HECH QACHON: foydalanuvchi ma'lumotini SQL satriga qo'shmang
const email = req.body.email; // hujumchi yuboradi: ' OR '1'='1
await query(`SELECT * FROM users WHERE email = '${email}'`);
// Natijaviy SQL: SELECT * FROM users WHERE email = ' OR '1'='1' → BARCHA
// foydalanuvchilar!
// Yoki: '; DROP TABLE users; -- → jadval o'chiriladi

// ✅ DOIM parametrlangan so'rovlar ($1, $2 ...)
await query('SELECT * FROM users WHERE email = $1', [email]);
// Baza qiymatni hech qachon SQL kodi sifatida talqin qilmaydi
```



Qat'iy qoida

SQL'dagi har bir o'zgaruvchan qiymat — faqat `$1, $2...` parametrlari orqali. Ustun yoki saralash nomini dinamik tanlash kerak bo'lsa — oldindan belgilangan ruxsat ro'yxatidan (whitelist) foydalaning: `const col = { price: 'price', title: 'title' }[req.query.sort] ?? 'id'`.

ORM va query builder'lar

Xom SQL o'rniga ko'pincha **ORM** (Object-Relational Mapping) ishlatiladi: **Prisma**, **Drizzle**, **Sequelize**, **TypeORM**. Ular sxemani kodda tasvirlash, migratsiyalar va qulay API beradi. Lekin SQL'ni bilish baribir zarur — ORM qanday so'rov hosil qilayotganini tushunish va sekin so'rovlarni tuzatish uchun.

JavaScript

```
// Prisma misoli — xuddi shu so'rov
const books = await prisma.book.findMany({
  where: { price: { lt: 100000 }, author: { country: "O'zbekiston" } },
  include: { author: true },
  orderBy: { price: 'asc' },
});
```

Amaliy misollar

MISOL 1

Blog platformasi sxemasi va real so'rovlar

Foydalanuvchilar, postlar, izohlar, teglar (N:M) va layklar bo'lgan blog uchun to'liq sxema.

```

CREATE TABLE users (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  username    TEXT NOT NULL UNIQUE CHECK (username ~ '^[a-z0-9_]{3,30}$'),
  email       TEXT NOT NULL,
  password_hash TEXT NOT NULL,
  bio         TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX users_email_uq ON users (lower(email));

CREATE TABLE posts (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  author_id   BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title       TEXT NOT NULL CHECK (length(title) BETWEEN 3 AND 200),
  slug        TEXT NOT NULL UNIQUE,
  body        TEXT NOT NULL,
  status       TEXT NOT NULL DEFAULT 'draft' CHECK (status IN ('draft',
'published'))),
  published_at TIMESTAMPTZ,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX posts_author_idx ON posts (author_id);
CREATE INDEX posts_published_idx ON posts (published_at DESC) WHERE status
= 'published'; -- qisman indeks

CREATE TABLE comments (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  post_id     BIGINT NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
  author_id   BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  body        TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX comments_post_idx ON comments (post_id, created_at);

CREATE TABLE tags (
  id      INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name    TEXT NOT NULL UNIQUE
);

CREATE TABLE post_tags (
  -- ko'pga-ko'p bog'lovchi
  jadval
  post_id BIGINT REFERENCES posts(id) ON DELETE CASCADE,

```

```
tag_id INT REFERENCES tags(id) ON DELETE CASCADE,  
PRIMARY KEY (post_id, tag_id)  
);  
  
CREATE TABLE likes (  
  user_id BIGINT REFERENCES users(id) ON DELETE CASCADE,  
  post_id BIGINT REFERENCES posts(id) ON DELETE CASCADE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  PRIMARY KEY (user_id, post_id)          -- bir foydalanuvchi bir  
postga bir marta  
);
```

```

-- 1. Bosh sahifa lentasi: muallif, izohlar va layklar soni, teglar
SELECT p.id, p.title, p.slug, p.published_at,
       u.username AS author,
       (SELECT COUNT(*) FROM comments c WHERE c.post_id = p.id) AS
comments,
       (SELECT COUNT(*) FROM likes l WHERE l.post_id = p.id) AS likes,
       COALESCE(array_agg(t.name ORDER BY t.name) FILTER (WHERE t.id IS
NOT NULL), '{}') AS tags
FROM posts p
JOIN users u ON u.id = p.author_id
LEFT JOIN post_tags pt ON pt.post_id = p.id
LEFT JOIN tags t ON t.id = pt.tag_id
WHERE p.status = 'published'
GROUP BY p.id, u.username
ORDER BY p.published_at DESC
LIMIT 10;

-- 2. "javascript" tegidagi postlar
SELECT p.title FROM posts p
JOIN post_tags pt ON pt.post_id = p.id
JOIN tags t ON t.id = pt.tag_id
WHERE t.name = 'javascript' AND p.status = 'published';

-- 3. Oxirgi 30 kundagi eng faol 5 muallif (postlar + izohlar)
WITH activity AS (
  SELECT author_id, 'post' AS kind FROM posts WHERE created_at > now() -
interval '30 days'
  UNION ALL
  SELECT author_id, 'comment' FROM comments WHERE created_at > now() -
interval '30 days'
)
SELECT u.username,
       COUNT(*) FILTER (WHERE kind = 'post') AS posts,
       COUNT(*) FILTER (WHERE kind = 'comment') AS comments,
       COUNT(*) AS total
FROM activity a JOIN users u ON u.id = a.author_id
GROUP BY u.username
ORDER BY total DESC
LIMIT 5;

-- 4. Layk qo'yish (takroriy bo'lsa - xato emas, shunchaki e'tiborsiz)
INSERT INTO likes (user_id, post_id) VALUES (7, 42) ON CONFLICT DO

```

```
NOTHING;
```

```
-- 5. Post qidiruvi (to'liq matnli qidiruv)
SELECT title, ts_rank(to_tsvector('simple', title || ' ' || body), q) AS
rank
FROM posts, plainto_tsquery('simple', 'react hook') q
WHERE to_tsvector('simple', title || ' ' || body) @@ q
ORDER BY rank DESC;
```

`ON DELETE CASCADE` — foydalanuvchi o'chirilsa, uning postlari, izohlari va layklari ham avtomatik o'chadi. `FILTER (WHERE ...)` — bitta so'rovda bir nechta shartli hisob. Kompozit birlamchi kalit `(user_id, post_id)` takroriy laykni baza darajasida taqiqlaydi.

MISOL 2 Express API'ni JSON fayldan PostgreSQL'ga ko'chirish

9-darsdagi qatlamli arxitektura o'z samarasini beradi: **faqat service qatlami** o'zgaradi, routes va controllers tegilmaydi.

migrations/001_books.sql

```
CREATE TABLE IF NOT EXISTS books (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title       TEXT NOT NULL,
  author      TEXT NOT NULL,
  price       INTEGER NOT NULL CHECK (price > 0),
  stock       INTEGER NOT NULL DEFAULT 0 CHECK (stock ≥ 0),
  genre       TEXT NOT NULL,
  year        INTEGER,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ,
  UNIQUE (title, author)
);
CREATE INDEX IF NOT EXISTS books_genre_idx ON books (genre);
```

```

import { query } from '../db/pool.js';
import { HttpError } from '../utils/errors.js';

const SORT_SQL = {                                // whitelist — dinamik ORDER BY
  uchun xavfsiz usul
  price: 'price ASC',
  '-price': 'price DESC',
  title: 'title ASC',
  '-createdAt': 'created_at DESC',
};

const COLUMNS = `id, title, author, price, stock, genre, year,
  created_at AS "createdAt", updated_at AS "updatedAt"`;

export async function listBooks({ q, genre, minPrice, maxPrice, sort,
  page, limit }) {
  const where = [];
  const params = [];
  const add = (sql, value) => { params.push(value);
  where.push(sql.replaceAll('?', `$$${params.length}`)); };

  if (q) add('(title ILIKE ? OR author ILIKE ?)', `%${q}%`); // ikkala ?
  → bitta $n
  if (genre) add('genre = ?', genre);
  if (minPrice !== undefined) add('price ≥ ?', minPrice);
  if (maxPrice !== undefined) add('price ≤ ?', maxPrice);

  const whereSql = where.length ? `WHERE ${where.join(' AND ')}` : '';
  const offset = (page - 1) * limit;

  const [{ rows }, { rows: [{ total }] }] = await Promise.all([
    query(
      `SELECT ${COLUMNS} FROM books ${whereSql}
      ORDER BY ${SORT_SQL[sort]} LIMIT $$${params.length + 1} OFFSET
      $$${params.length + 2}`,
      [...params, limit, offset]
    ),
    query(`SELECT COUNT(*)::int AS total FROM books ${whereSql}`, params),
  ]);

  return { data: rows, meta: { page, limit, total, pages: Math.ceil(total
    / limit) } };

```



```

}

export async function getBook(id) {
  const { rows: [book] } = await query(`SELECT ${COLUMNS} FROM books WHERE
id = $1`, [id]);
  if (!book) throw HttpError.notFound('Kitob topilmadi');
  return book;
}

export async function createBook({ title, author, price, stock, genre,
year }) {
  try {
    const { rows: [book] } = await query(
      `INSERT INTO books (title, author, price, stock, genre, year)
      VALUES ($1, $2, $3, $4, $5, $6) RETURNING ${COLUMNS}`,
      [title, author, price, stock, genre, year ?? null]
    );
    return book;
  } catch (err) {
    if (err.code === '23505') throw HttpError.conflict('Bu kitob
allaqachon mavjud'); // unique_violation
    throw err;
  }
}

export async function updateBook(id, patch) {
  const allowed = ['title', 'author', 'price', 'stock', 'genre', 'year'];
  const entries = Object.entries(patch).filter(([k]) =>
allowed.includes(k));
  if (!entries.length) return getBook(id);

  const set = entries.map(([k], i) => `${k} = ${i + 2}`).join(', ');
  const { rows: [book] } = await query(
    `UPDATE books SET ${set}, updated_at = now() WHERE id = $1 RETURNING
${COLUMNS}`,
    [id, ...entries.map(([ , v]) => v)]
  );
  if (!book) throw HttpError.notFound('Kitob topilmadi');
  return book;
}

export async function deleteBook(id) {
  const { rowCount } = await query('DELETE FROM books WHERE id = $1',
[id]);

```

```
if (!rowCount) throw HttpError.notFound('Kitob topilmadi');
}
```

`add()` yordamchisi qiymatni `params` ga qo'shib, shartdagi `?` belgilarini tegishli `$n` ga almashtiradi (`q` shartida ikkala `?` bitta parametrni ishlatadi). Ustun nomlari faqat `allowed` ro'yxatidan, qiymatlar esa faqat parametrlar orqali — injection'ga joy yo'q. UUID formatida bo'lmagan `id` kelsa, PostgreSQL `22P02` xatosini beradi — uni ham `errorHandler` da 404/400 ga aylantirish mumkin.

MISOL 3 Buyurtma berish tranzaksiyasi (ombor, pool client, qulflash)

Buyurtma: (1) har bir kitob omborda yetarlimi, (2) ombordan ayirish, (3) buyurtma va uning qatorlarini yozish. Agar biror qadam muvaffaqiyatsiz bo'lsa — **hech narsa** o'zgarmasligi kerak.

SQL

```
CREATE TABLE orders (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id     BIGINT NOT NULL,
  total       INTEGER NOT NULL,
  status      TEXT NOT NULL DEFAULT 'pending',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE order_items (
  order_id BIGINT REFERENCES orders(id) ON DELETE CASCADE,
  book_id  UUID REFERENCES books(id),
  qty      INTEGER NOT NULL CHECK (qty > 0),
  price    INTEGER NOT NULL,
  PRIMARY KEY (order_id, book_id)
);
```

```

import { pool } from '../db/pool.js';
import { HttpError } from '../utils/errors.js';

// Qayta ishlatiladigan tranzaksiya yordamchisi
export async function withTransaction(fn) {
  const client = await pool.connect(); // tranzaksiya BITTA ulanishda
  bo'lishi shart
  try {
    await client.query('BEGIN');
    const result = await fn(client);
    await client.query('COMMIT');
    return result;
  } catch (err) {
    await client.query('ROLLBACK');
    throw err;
  } finally {
    client.release(); // ulanishni pool'ga qaytarish
  }
}

export function placeOrder(userId, items) { // items: [{ bookId, qty }]
  return withTransaction(async (db) => {
    const ids = items.map(i => i.bookId);

    // FOR UPDATE – bu qatorlarni tranzaksiya tugaguncha qulflaydi:
    // ikki xaridor oxirgi nusxani bir vaqtda sotib o'lolmaydi
    const { rows: books } = await db.query(
      'SELECT id, title, price, stock FROM books WHERE id = ANY($1) ORDER
BY id FOR UPDATE',
      [ids]
    );
    const byId = new Map(books.map(b => [b.id, b]));

    let total = 0;
    for (const { bookId, qty } of items) {
      const book = byId.get(bookId);
      if (!book) throw HttpError.notFound(`Kitob topilmadi: ${bookId}`);
      if (book.stock < qty) {
        throw HttpError.conflict(`${book.title}" omborda yetarli emas
(bor: ${book.stock}, so'raldi: ${qty})`);
      }
    }
  }
}

```

```

    total += book.price * qty;
  }

  const { rows: [order] } = await db.query(
    'INSERT INTO orders (user_id, total) VALUES ($1, $2) RETURNING *',
    [userId, total]
  );

  for (const { bookId, qty } of items) {
    await db.query('UPDATE books SET stock = stock - $1 WHERE id = $2',
      [qty, bookId]);
    await db.query(
      'INSERT INTO order_items (order_id, book_id, qty, price) VALUES
      ($1, $2, $3, $4)',
      [order.id, bookId, qty, byId.get(bookId).price]
    );
  }

  return { ...order, items };
});
}

```

Uchta muhim nuqta: tranzaksiya uchun `pool.query` emas, bitta `client` kerak (aks holda har bir so'rov boshqa ulanishga tushishi mumkin); `FOR UPDATE` "race condition" ni yo'q qiladi; `ORDER BY id` bilan qulflash tartibi bir xil bo'lgani uchun "deadlock" ehtimoli kamayadi. Xato tashlansa — `ROLLBACK`, ombor qiymatlari o'zgarmaydi.

MISOL 4 Taqqoslash uchun: xuddi shu model MongoDB + Mongoose'da

NoSQL yondashuvni his qilish uchun blog postini MongoDB'da modellashtiramiz. Bu yerda izohlar post hujjati **ichiga joylanadi**.

Terminal

```

npm install mongoose
# docker run -d --name mongo -p 27017:27017 mongo:8

```

```

import mongoose from 'mongoose';

const commentSchema = new mongoose.Schema({
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
  body: { type: String, required: true, maxlength: 2000 },
}, { timestamps: true });

const postSchema = new mongoose.Schema({
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true, index: true },
  title: { type: String, required: true, trim: true, minlength: 3,
maxlength: 200 },
  slug: { type: String, required: true, unique: true },
  body: String,
  tags: [{ type: String, lowercase: true }], // massiv - alohida
jadval shart emas
  status: { type: String, enum: ['draft', 'published'], default: 'draft'
},
  comments: [commentSchema], // ichma-ich
hujjatlar
  likes: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
}, { timestamps: true });

postSchema.index({ tags: 1, createdAt: -1 });
postSchema.virtual('likeCount').get(function () { return
this.likes.length; });

export const Post = mongoose.model('Post', postSchema);

```

JavaScript

```
import mongoose from 'mongoose';
import { Post } from './models/Post.js';

await mongoose.connect('mongodb://localhost:27017/blog');

// Yaratish
const post = await Post.create({
  author: userId, title: 'React hook\'lari', slug: 'react-hooks', tags:
['react', 'JavaScript'],
});

// "javascript" tegidagi nashr qilingan postlar, muallif ma'lumoti bilan
(JOIN o'xshashi)
const posts = await Post.find({ tags: 'javascript', status: 'published' })
  .populate('author', 'username')
  .sort({ createdAt: -1 })
  .limit(10)
  .lean();

// Izoh qo'shish – atomar operator
await Post.updateOne({ _id: post._id }, { $push: { comments: { author:
userId, body: 'Zo\'r!' } } });

// Layk (takrorlanmas) – $addToSet
await Post.updateOne({ _id: post._id }, { $addToSet: { likes: userId } });

// Agregatsiya: teglar bo'yicha postlar soni
const stats = await Post.aggregate([
  { $match: { status: 'published' } },
  { $unwind: '$tags' },
  { $group: { _id: '$tags', count: { $sum: 1 } } },
  { $sort: { count: -1 } },
]);
```

Jihat	PostgreSQL	MongoDB
Izohlar	Alohida jadval + JOIN	Post ichida (bitta o'qishda keladi, lekin hujjat 16 MB bilan cheklangan)
Teglar	N:M jadval	Satrlar massivi

Sxema	Baza darajasida majburiy	Mongoose (ilova darajasida)
"Foydalanuvchining barcha izohlari"	Oddiy <code>WHERE author_id</code>	Barcha postlarni skanerlash yoki agregatsiya — noqulay

Xulosa: ma'lumotlarni qanday **so'rashingiz** modelni belgilaydi. Bog'lanishlar ko'p bo'lsa — relyatsion baza tabiiyroq.

Mashqlar



Mustaqil ish

1. SQLBolt interaktiv darslarini to'liq bajaring.
2. Onlayn do'kon sxemasini loyihalang: `categories` (ichma-ich, `parent_id`), `products`, `customers`, `orders`, `order_items`, `reviews` (1–5 baho).
3. Shu sxema uchun so'rovlar: eng ko'p sotilgan 10 mahsulot; har bir toifaning o'rtacha reytingi; hech narsa sotib olmagan mijozlar; oylik daromad.
4. `EXPLAIN ANALYZE` bilan indeksli va indeksiz so'rov tezligini 100 000 qatorli jadvalda solishtiring (`generate_series` bilan test ma'lumot yarating).
5. 3-misoldagi `placeOrder` ni sinang: ikki terminaldan bir vaqtda oxirgi nusxani sotib olishga harakat qiling. `FOR UPDATE` olib tashlansa nima bo'ladi?

▼ 4-mashq uchun test ma'lumot

SQL

```
INSERT INTO books (title, author, price, stock, genre)
SELECT 'Kitob ' || g, 'Muallif ' || (g % 500), (random() * 200000 +
10000)::int, (random() * 50)::int,
      (ARRAY['roman','ilmiy','bolalar','dasturlash','tarix'])[1 + g %
5]
FROM generate_series(1, 100000) g;
```

✓ Xulosa

- Relyatsion baza: jadvallar, birlamchi va tashqi kalitlar, cheklovlar — ma'lumot yaxlitligining kafolati.
- SQL: `SELECT ... JOIN ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT .`
- Normalizatsiya: "bitta fakt — bitta joyda"; N:M uchun bog'lovchi jadval.
- Tez-tez filtrlanadigan va JOIN qilinadigan ustunlarga indeks; `EXPLAIN ANALYZE` bilan tekshiring.
- Bir nechta bog'liq yozuv — tranzaksiya ichida (`BEGIN/COMMIT/ROLLBACK`).
- Faqat parametrlangan so'rovlar — SQL injection'dan himoya.



11-DARS · 14 TADAN

Autentifikatsiya va avtorizatsiya

Parollarni xeshlash (bcrypt), sessiya va JWT, httpOnly cookie, himoyalangan marshrutlar, rollar va ruxsatlar, access/refresh tokenlar hamda frontendda tokenlar bilan ishlash.

O'rta–Yuqori · ~130 daqiqa

Deyarli har bir ilovada "Kirish" tugmasi bor. Uning ortida esa veb-xavfsizlikning eng nozik qismi yotadi: parollarni qanday saqlash, foydalanuvchini keyingi so'rovlarda qanday "tanish", kimga nimaga ruxsat berish. Bu yerdagi xato butun ma'lumotlar bazasining o'g'irlanishiga olib kelishi mumkin. Shuning uchun har bir qarorning **nima uchun** ekanini tushunamiz.

Autentifikatsiya va avtorizatsiya

	Autentifikatsiya (AuthN)	Avtorizatsiya (AuthZ)
Savol	Sen kimsan?	Senga nima qilish mumkin?
Misol	Login va parolni tekshirish	Faqat admin foydalanuvchini o'chira oladi
Xato kodi	401 Unauthorized	403 Forbidden
Ketma-ketlik	Birinchi	Keyin

Parollarni saqlash: xeshlash

Qoida №1: parolni hech qachon ochiq holda saqlamang. Baza o'g'irlansa (bu katta kompaniyalarda ham bo'ladi), barcha parollar oshkor bo'ladi — odamlar esa bir xil parolni ko'p saytda ishlatadi.

Nima uchun shifrlash emas, xeshlash?

- **Shifrlash** — ikki tomonlama: kalit bilan qayta ochish mumkin. Kalit o'g'irlansa — hamma parollar ochiladi.
- **Xeshlash** — bir tomonlama funksiya: `hash("saʼlom123")` → `"$2b$12$Kix..."`, lekin xeshdan parolni qaytarib bo'lmaydi. Tekshirishda kiritilgan parol qayta xeshlanadi va solishtiriladi.

Nima uchun MD5/SHA-256 emas?

Ular **tez** ishlash uchun yaratilgan — zamonaviy GPU sekundiga milliardlab SHA-256 hisoblaydi va oddiy parollarni lug'at hujumi bilan bir zumda topadi. Parollar uchun ataylab **sekin** va **tuzli (salted)** algoritmlar kerak:

- **bcrypt** — eng ko'p ishlatiladigan; "cost factor" (masalan 12) bilan sekinlik sozlanadi.
- **argon2id** — zamonaviy tavsiya (OWASP), xotirani ham talab qiladi.
- **Tuz (salt)** — har bir parolga qo'shiladigan tasodifiy qiymat. Bir xil parollar har xil xeshga ega bo'ladi, tayyor "rainbow table"lar ishlamaydi. bcrypt tuzni avtomatik yaratadi va xesh ichida saqlaydi.

JavaScript

```
import bcrypt from 'bcrypt'; // npm install bcrypt (yoki sof JS: bcryptjs)

const hash = await bcrypt.hash('MeningParolim2026', 12);
// '$2b$12$C6UzMDM.H6dfI/f/IKcEe05H8v.Lk3PqxW7JLw7xYcR9EJzt4P6eW'
// |   |   | 22 belgili tuz + 31 belgili xesh
// |   |   | cost = 12 (2^12 = 4096 iteratsiya)
// |   |   | algoritm versiyasi

const ok = await bcrypt.compare('MeningParolim2026', hash); // true
const bad = await bcrypt.compare('boshqa', hash); // false
```

Foydalanuvchini "eslab qolish": sessiya va token

HTTP holatsiz (1-dars) — server har bir so'rovda kim kelganini bilmaydi. Login'dan keyin mijozga "guvohnoma" beriladi va u har bir so'rov bilan uni ko'rsatadi. Ikki asosiy yondashuv bor:

1. Sessiyalar (stateful)

- 1. Login muvaffaqiyatli → server tasodifiy **sessiya ID** yaratadi va uni **serverda** (Redis yoki bazada) saqlaydi: `sid_abc → { userId: 7 }`.
- 2. ID mijozga `Set-Cookie: sid=abc; HttpOnly; Secure` orqali yuboriladi.
- 3. Brauzer har so'rovda cookie'ni avtomatik yuboradi; server ID bo'yicha foydalanuvchini topadi.
- 4. Chiqish — serverdagi yozuvni o'chirish. Darhol, ishonchli.

2. JWT tokenlar (stateless)

JWT (JSON Web Token) — server **imzolagan** va ichida ma'lumot saqlovchi satr. Server hech narsani eslab qolmaydi: imzoni tekshirish kifoya.

Matn

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI3Iiwicm9sZSI6InVzZXIiLCJleHAiOjE3OTE5ODAwMDB9.Xk3v...
```

header

payload

imzo

header: { "alg": "HS256", "typ": "JWT" }
payload: { "sub": "7", "role": "user", "iat": 1791979100, "exp": 1791980000 }
imzo: HMAC-SHA256(base64(header) + "." + base64(payload), JWT_SECRET)

 **JWT shifrlanmagan!**

Header va payload shunchaki Base64 — jwt.io da istalgan kishi o'qiy oladi. Imzo faqat **o'zgartirilmaganini** kafolatlaydi. Payload'ga parol, karta raqami kabi maxfiy ma'lumot qo'ymang.

	Sessiya	JWT
Holat qayerda	Serverda	Tokenning o'zida
Har so'rovda bazaga murojaat	Ha (Redis bilan tez)	Yo'q
Chiqish/bloklash	Darhol	Token muddati tugaguncha amal qiladi (qisqa muddat + refresh token kerak)

Bir nechta server/mikroservis	Umumiy sessiya ombori kerak	Oson — har bir servis imzoni tekshiradi
Mobil ilovalar	Mumkin	Juda qulay

Token qayerda saqlanadi?

Joy	XSS'ga qarshi	CSRF'ga qarshi	Izoh
<code>localStorage</code>	❌ Har qanday skript o'qiydi	✅ Avtomatik yuborilmaydi	Oddiy, lekin XSS bo'lsa token o'g'irlanadi
<code>HttpOnly</code> cookie	✅ JS o'qiy olmaydi	⚠️ <code>SameSite</code> kerak	Veb-ilovalar uchun tavsiya
Xotira (JS o'zgaruvchisi)	⚠️ Qisman	✅	Sahifa yangilansa yo'qoladi — refresh token bilan birga

JavaScript

```
res.cookie('access_token', token, {
  httpOnly: true, // document.cookie orqali
  // o'qib bo'lmaydi
  secure: process.env.NODE_ENV === 'production', // faqat HTTPS orqali
  sameSite: 'lax', // boshqa saytdan POST
  // so'rovlarda yuborilmaydi (CSRF himoyasi)
  maxAge: 15 * 60 * 1000, // 15 daqiqa
  path: '/',
});
```

Access va refresh tokenlar

Muammo: uzoq muddatli token o'g'irlansa — hujumchi uzoq vaqt foydalanadi; qisqa muddatli bo'lsa — foydalanuvchi har 15 daqiqada qayta login qiladi. Yechim — ikki token:

- **Access token** — qisqa muddatli (5–15 daqiqa), har bir API so'rovida ishlatiladi.

- **Refresh token** — uzoq muddatli (7–30 kun), faqat `/auth/refresh` ga yuboriladi va yangi access token olish uchun ishlatiladi. U **serverda (bazada) qayd qilinadi** — shuning uchun uni bekor qilish (logout, "barcha qurilmalardan chiqish") mumkin.
- **Rotatsiya** — har refresh'da eski refresh token bekor qilinib, yangisi beriladi. Eski token qayta ishlatilsa — o'g'irlik belgisi, foydalanuvchining barcha sessiyalari yopiladi.

Login xavfsizligi bo'yicha qoidalar

- **Umumiy xato xabari:** "Email yoki parol noto'g'ri" — "Bunday email yo'q" emas (aks holda hujumchi qaysi emaillar ro'yxatdan o'tganini biladi).
- **Rate limiting:** bir IP/akkauntga daqiqada 5–10 urinish (brute-force'ga qarshi).
- **Parol siyosati:** kamida 8–12 belgi; murakkablik qoidalaridan ko'ra uzunlik muhimroq; mashhur sizib chiqqan parollarni rad etish.
- **HTTPS** — majburiy.
- Parolni tiklash tokenlari — bir martalik, qisqa muddatli, bazada xesh ko'rinishida.
- Muhim ilovalar uchun **ikki bosqichli autentifikatsiya (2FA)**: TOTP (Google Authenticator) yoki SMS.
- Tayyor yechimlar ham bor: **OAuth** ("Google bilan kirish"), Auth0, Clerk, Supabase Auth, Better Auth. Lekin ular ortidagi tamoyillarni bilish baribir zarur.

Amaliy misollar

MISOL 1

Ro'yxatdan o'tish va login (bcrypt + JWT + httpOnly cookie)

Terminal

```
npm install bcrypt jsonwebtoken cookie-parser zod
```

migrations/002_users.sql

```
CREATE TABLE users (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  name        TEXT NOT NULL,  
  email       TEXT NOT NULL,  
  password_hash TEXT NOT NULL,  
  role        TEXT NOT NULL DEFAULT 'user' CHECK (role IN ('user',  
'editor', 'admin')),  
  token_version INTEGER NOT NULL DEFAULT 0,      -- oshirilsa barcha  
tokenlar bekor bo'ladi  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE UNIQUE INDEX users_email_uq ON users (lower(email));
```

src/config.js

```
function required(name) {  
  const v = process.env[name];  
  if (!v) throw new Error(`Muhit o'zgaruvchisi o'rnatilmagan: ${name}`);  
  return v;  
}  
  
export const config = {  
  isProd: process.env.NODE_ENV === 'production',  
  jwtSecret: required('JWT_SECRET'),          // kamida 32 belgili  
tasodifiy satr  
  accessTtl: '15m',  
  clientUrl: process.env.CLIENT_URL ?? 'http://localhost:5173',  
};  
// Kalit yaratish: node -e  
"console.log(require('crypto').randomBytes(48).toString('hex'))"
```

```

import bcrypt from 'bcrypt';
import jwt from 'jsonwebtoken';
import { query } from '../db/pool.js';
import { config } from '../config.js';
import { HttpError } from '../utils/errors.js';

const BCRYPT_COST = 12;
// Foydalanuvchi topilmasa ham bcrypt.compare bajarilishi uchun – javob
vaqti bo'yicha email borligini bilib bo'lmasin
const DUMMY_HASH = await bcrypt.hash('dummy-password', BCRYPT_COST);

const publicUser = ({ id, name, email, role }) => ({ id, name, email, role
});

export function signAccessToken(user) {
  return jwt.sign(
    { sub: String(user.id), role: user.role, tv: user.token_version },
    config.jwtSecret,
    { expiresIn: config.accessTtl, algorithm: 'HS256' }
  );
}

export async function register({ name, email, password }) {
  const passwordHash = await bcrypt.hash(password, BCRYPT_COST);
  try {
    const { rows: [user] } = await query(
      `INSERT INTO users (name, email, password_hash) VALUES ($1,
lower($2), $3)
      RETURNING id, name, email, role, token_version`,
      [name, email, passwordHash]
    );
    return user;
  } catch (err) {
    if (err.code === '23505') throw HttpError.conflict('Bu email bilan
ro\'yxatdan o\'tilgan');
    throw err;
  }
}

export async function login({ email, password }) {
  const { rows: [user] } = await query('SELECT * FROM users WHERE email =
lower($1)', [email]);

```

```
    const ok = await bcrypt.compare(password, user?.password_hash ??
DUMMY_HASH);
    if (!user || !ok) throw HttpError.unauthorized('Email yoki parol
noto\'g\'ri');
    return user;
}

export { publicUser };
```



```

import { Router } from 'express';
import { z } from 'zod';
import { validate } from '../middleware/validate.js';
import { requireAuth } from '../middleware/auth.js';
import * as auth from '../services/auth.service.js';
import { config } from '../config.js';
import { rateLimit } from '../middleware/common.js';

const router = Router();

const registerSchema = z.object({
  name: z.string().trim().min(2).max(60),
  email: z.string().trim().email(),
  password: z.string().min(8, 'Parol kamida 8 belgi').max(72), // bcrypt
  // 72 baytdan keyingisini e'tiborsiz qoldiradi
});
const loginSchema = z.object({ email: z.string().trim().email(), password:
z.string().min(1) });

const cookieOptions = {
  httpOnly: true,
  secure: config.isProd,
  sameSite: 'lax',
  maxAge: 15 * 60 * 1000,
};

function sendSession(res, user, status = 200) {
  res.cookie('access_token', auth.signAccessToken(user), cookieOptions);
  res.status(status).json({ user: auth.publicUser(user) });
}

const authLimiter = rateLimit({ windowMs: 15 * 60_000, max: 10 }); // 15
daqiqada 10 urinish

router.post('/register', authLimiter, validate(registerSchema), async
(req, res) => {
  const user = await auth.register(req.validated.body);
  sendSession(res, user, 201);
});

router.post('/login', authLimiter, validate(loginSchema), async (req, res)
=> {

```

```
const user = await auth.login(req.validated.body);
sendSession(res, user);
});

router.post('/logout', (req, res) => {
  res.clearCookie('access_token', { ...cookieOptions, maxAge: undefined });
  res.status(204).end();
});

router.get('/me', requireAuth, (req, res) => {
  res.json({ user: auth.publicUser(req.user) });
});

export default router;
```

src/app.js (qo'shimcha)

```
import cookieParser from 'cookie-parser';
import authRouter from './routes/auth.routes.js';

app.use(cors({ origin: config.clientUrl, credentials: true })); // cookie
uchun credentials SHART
app.use(cookieParser());
app.use('/api/v1/auth', authRouter);
```

```
src/middleware/auth.js
```

```
import jwt from 'jsonwebtoken';
import { query } from '../db/pool.js';
import { config } from '../config.js';
import { HttpError } from '../utils/errors.js';

export async function requireAuth(req, res, next) {
  // Cookie (brauzer) yoki Authorization sarlavhasi (mobil ilova, Postman)
  const header = req.get('authorization');
  const token = req.cookies?.access_token
    ?? (header?.startsWith('Bearer ') ? header.slice(7) : null);

  if (!token) return next(HttpError.unauthorized());

  let payload;
  try {
    payload = jwt.verify(token, config.jwtSecret, { algorithms: ['HS256']
}); // algoritmni aniq ko'rsatamiz
  } catch (err) {
    return next(HttpError.unauthorized(
      err.name === 'TokenExpiredError' ? 'Sessiya muddati tugagan' :
      'Token yaroqsiz'
    ));
  }

  // Foydalanuvchi o'chirilgan yoki "barcha qurilmalardan chiqish"
  qilingan bo'lishi mumkin
  const { rows: [user] } = await query(
    'SELECT id, name, email, role, token_version FROM users WHERE id =
    $1',
    [payload.sub]
  );

  if (!user || user.token_version !== payload.tv) return
  next(HttpError.unauthorized('Sessiya bekor qilingan'));

  req.user = user;
  next();
}
```

```
// Ixtiyoriy autentifikatsiya: mehmon ham kira oladi, lekin login qilgan  
bo'lsa req.user bo'ladi  
export function optionalAuth(req, res, next) {  
  if (!req.cookies?.access_token && !req.get('authorization')) return  
  next();  
  // Token yaroqsiz bo'lsa ham xato bermaymiz – mehmon sifatida davom  
  etadi  
  return requireAuth(req, res, () => next());  
}
```

Endi postlar API'sida egalikni tekshiramiz. **Muhim:** ID'ni so'rov tanasidan emas, `req.user` dan
olamiz — aks holda foydalanuvchi boshqa birov nomidan post yaratishi mumkin.

```

import { Router } from 'express';
import { requireAuth } from '../middleware/auth.js';
import { query } from '../db/pool.js';
import { HttpError } from '../utils/errors.js';

const router = Router();

router.post('/', requireAuth, async (req, res) => {
  const { title, body } = req.body; // (validatsiya - 9-
  // darsdagidek)
  const { rows: [post] } = await query(
    'INSERT INTO posts (author_id, title, body) VALUES ($1, $2, $3)
    RETURNING *',
    [req.user.id, title, body] // ✅ author_id -
    // tokendan, mijozdan emas
  );
  res.status(201).json(post);
});

router.patch('/:id', requireAuth, async (req, res) => {
  const { rows: [post] } = await query('SELECT author_id FROM posts WHERE
  id = $1', [req.params.id]);
  if (!post) throw HttpError.notFound('Post topilmadi');

  const isOwner = String(post.author_id) === String(req.user.id);
  if (!isOwner && req.user.role !== 'admin') {
    throw HttpError.forbidden('Faqat muallif yoki admin tahrirlay oladi');
  }

  const { rows: [updated] } = await query(
    'UPDATE posts SET title = COALESCE($2, title), body = COALESCE($3,
    body) WHERE id = $1 RETURNING *',
    [req.params.id, req.body.title, req.body.body]
  );
  res.json(updated);
});

export default router;

```



IDOR — eng keng tarqalgan avtorizatsiya xatosi

`GET /api/orders/1053` — tizimga kirgan foydalanuvchi URL'dagi raqamni o'zgartirib, **boshqa bironing** buyurtmasini ko'rsa — bu "Insecure Direct Object Reference". Har bir resursga murojaatda "bu resurs shu foydalanuvchigami?" degan tekshiruv bo'lishi shart:

```
WHERE id = $1 AND user_id = $2 .
```

MISOL 3 Rollar va ruxsatlar (RBAC)

Har bir rolga emas, har bir **ruxsatga** tekshirish yozish kodni moslashuvchan qiladi: yangi rol qo'shish faqat jadvalni o'zgartirishni talab qiladi.

src/auth/permissions.js

```
export const PERMISSIONS = {
  user: ['post:create', 'post:update:own', 'post:delete:own',
'comment:create'],
  editor: ['post:create', 'post:update:any', 'post:publish',
'comment:create', 'comment:delete:any'],
  admin: ['*'], // hamma narsa
};

export function can(user, permission) {
  const list = PERMISSIONS[user?.role] ?? [];
  return list.includes('*') || list.includes(permission);
}
```

src/middleware/authorize.js

```
import { can } from '../auth/permissions.js';
import { HttpError } from '../utils/errors.js';

export const requirePermission = (...perms) => (req, res, next) => {
  if (!req.user) return next(HttpError.unauthorized());
  if (!perms.some(p => can(req.user, p))) {
    return next(HttpError.forbidden(`Bu amal uchun ruxsat kerak:
    ${perms.join(' yoki ')}`));
  }
  next();
};
```

Foydalanish

```
import { requireAuth } from '../middleware/auth.js';
import { requirePermission } from '../middleware/authorize.js';
import { can } from '../auth/permissions.js';

// Marshrut darajasida
router.post('/:id/publish', requireAuth,
  requirePermission('post:publish'), publishPost);
router.get('/admin/users', requireAuth, requirePermission('*'),
  listUsers);

// Resurs darajasida (egalik bilan)
router.delete('/:id', requireAuth, async (req, res) => {
  const post = await getPost(req.params.id);
  const allowed = can(req.user, 'post:delete:any')
    || (can(req.user, 'post:delete:own') && post.author_id ===
req.user.id);
  if (!allowed) throw HttpError.forbidden();
  await deletePost(post.id);
  res.status(204).end();
});

// Admin foydalanuvchi rolini o'zgartiradi va uning eski tokenlarini bekor
qiladi
router.patch('/admin/users/:id/role', requireAuth, requirePermission('*'),
  async (req, res) => {
    const { rows: [u] } = await query(
      'UPDATE users SET role = $2, token_version = token_version + 1 WHERE
id = $1 RETURNING id, role',
      [req.params.id, req.body.role]
    );
    res.json(u);
  });
```

Frontendda ham xuddi shu `can()` funksiyasidan foydalanib, ruxsat bo'lmagan tugmalarni yashirish mumkin — lekin bu faqat qulaylik uchun. **Haqiqiy tekshiruv doim serverda.**

MISOL 4**Frontend: login formasi, sessiyani tekshirish va 401'ni boshqarish**

httpOnly cookie ishlatilganda frontend tokenni umuman "ko'rmaydi" — brauzer uni o'zi yuboradi. Faqat `credentials: 'include'` kerak.

```

const API = 'http://localhost:3000/api/v1';

export async function api(path, { method = 'GET', body } = {}) {
  const res = await fetch(API + path, {
    method,
    credentials: 'include',          // 🍪 cookie'ni boshqa origin'dagi
API'ga yuborish
    headers: body ? { 'Content-Type': 'application/json' } : {},
    body: body ? JSON.stringify(body) : undefined,
  });

  if (res.status === 401 && path !== '/auth/me' && path !== '/auth/login')
  {
    // Sessiya tugagan – foydalanuvchini login sahifasiga, qaytish manzili
    bilan
    location.href = `/login.html?
next=${encodeURIComponent(location.pathname)}`;
    return new Promise(() => {});    // yo'naltirish paytida kod davom
    etmasin
  }

  const data = res.status === 204 ? null : await res.json();
  if (!res.ok) throw Object.assign(new Error(data?.error?.message ??
'Xatolik'), { status: res.status, data });
  return data;
}

export const auth = {
  me: () => api('/auth/me').then(d => d.user).catch(() => null),
  login: (email, password) => api('/auth/login', { method: 'POST', body: {
email, password } }),
  register: (data) => api('/auth/register', { method: 'POST', body: data
}),
  logout: () => api('/auth/logout', { method: 'POST' }),
};

```

```

<form id="login">
  <h1>Kirish</h1>
  <input name="email" type="email" autocomplete="email"
placeholder="Email" required>
  <input name="password" type="password" autocomplete="current-password"
placeholder="Parol" required>
  <button>Kirish</button>
  <p id="error" role="alert"></p>
</form>

<script type="module">
  import { auth } from './auth.js';

  const form = document.querySelector('#login');
  const error = document.querySelector('#error');

  // Allaqachon kirgan bo'lsa — to'g'ridan-to'g'ri ichkariga
  if (await auth.me()) location.replace('/dashboard.html');

  form.addEventListener('submit', async (e) => {
    e.preventDefault();
    const btn = form.querySelector('button');
    btn.disabled = true;
    btn.textContent = 'Tekshirilmoqda...';
    error.textContent = '';

    try {
      const { email, password } = Object.fromEntries(new FormData(form));
      await auth.login(email, password);
      const next = new URLSearchParams(location.search).get('next');
      // 🔒 Open redirect'dan himoya: faqat o'z saytimizdagi yo'llarga
      location.href = next?.startsWith('/') && !next.startsWith('//') ?
next : '/dashboard.html';
    } catch (err) {
      error.textContent = err.status === 429
        ? 'Juda ko\'p urinish. 15 daqiqadan so\'ng qayta urinib ko\'ring.'
        : err.message;
      form.elements.password.value = '';
      form.elements.password.focus();
    } finally {
      btn.disabled = false;
      btn.textContent = 'Kirish';
    }
  });

```

```
}  
});  
</script>
```

client/dashboard.html (himoyalangan sahifa)

```
<header>  
  <span id="hello"></span>  
  <button id="logout">Chiqish</button>  
</header>  
<script type="module">  
  import { auth } from './auth.js';  
  
  const user = await auth.me();  
  if (!user) location.replace('/login.html?next=/dashboard.html');  
  else {  
    document.querySelector('#hello').textContent = `Salom, ${user.name}!  
    (${user.role})`;   
    document.querySelector('#logout').onclick = async () => {  
      await auth.logout();  
      location.replace('/login.html');  
    };  
  }  
</script>
```



Cookie va turli domenlar

Frontend `localhost:5173`, API `localhost:3000` bo'lsa ham ular "same-site" (bir xil `localhost`), shuning uchun `SameSite=Lax` cookie ishlaydi. Production'da frontend va API'ni bitta domen ostida (`example.uz` va `api.example.uz`) yoki bitta origin'da (`/api` proksi orqali, 13-dars) joylash eng oson yo'l.

Mashqlar



Mustaqil ish

1. Refresh token mexanizmini qo'shing: `refresh_tokens(id, user_id, token_hash, expires_at, revoked_at)` jadvali, `POST /auth/refresh` (rotatsiya bilan), `POST /auth/logout-all`.
2. "Parolni unutdim": `POST /auth/forgot` bir martalik token yaratib (bazada xeshi bilan, 30 daqiqa muddat) konsolga havola chiqarsin; `POST /auth/reset` uni tekshirib, parolni almashtirsin va `token_version` ni oshirsin.
3. Login urinishlarini `login_attempts` jadvalida qayd qiling; 5 ta muvaffaqiyatsiz urinishdan keyin akkauntni 15 daqiqaga bloklang.
4. jwt.io saytida o'z tokeningizni oching va payload'ni o'zgartirib ko'ring. Server uni nega qabul qilmaydi?
5. `GET /api/orders/:id` marshrutida IDOR zaifligini ataylab yarating, so'ng uni tuzating va farqni test bilan isbotlang.



Xulosa

- AuthN — "kimsan?" (401), AuthZ — "nima qila olasan?" (403).
- Parollar faqat bcrypt/argon2 bilan xeshlangan holda saqlanadi.
- JWT imzolangan, lekin shifrlanmagan; qisqa muddat + refresh token; algoritmi aniq ko'rsating.
- Veb-ilovalarda tokenni `HttpOnly; Secure; SameSite` cookie'da saqlang.
- Foydalanuvchi ID'si doim tokendan; har bir resursda egalikni tekshiring (IDOR).
- Ruxsatlar asosidagi RBAC kodni moslashuvchan qiladi; frontend tekshiruvi faqat UX uchun.



12-DARS · 14 TADAN

React asoslari

Komponentlar, JSX, props, state, hodisalar, ro'yxatlar va key, formalar, useEffect va ma'lumot yuklash, custom hook'lar, Context va React Router.

O'rta · ~150 daqiqa

5-darsdagi Todo ilovasida biz "holat → render" arxitekturasini qo'lda qurdik: holat o'zgarsa, `render()` funksiyasi DOM'ni qayta chizdi. Ilova kattalashgan sari bu qiyinlashadi: qaysi qismni yangilash kerak? Holat bir necha joyda bo'lsa-chi? **React** — aynan shu muammoni hal qiladigan kutubxona: siz *holat qanday ko'rinishga mos kelishini* tasvirlaysiz, React esa DOM'ni o'zi samarali yangilaydi. Bugun React — frontend vakansiyalarining eng katta qismida talab qilinadigan texnologiya.

React'ning asosiy g'oyalari

- **Deklarativlik** — "qanday o'zgartirish" (`appendChild`, `classList.add`) emas, "qanday ko'rinishi kerak" deb yozasiz: `UI = f(state)`.
- **Komponentlar** — interfeys mustaqil, qayta ishlatiladigan bo'laklardan quriladi: `<Header>`, `<ProductCard>`, `<Cart>`.
- **Bir yo'nalishli ma'lumot oqimi** — ma'lumot ota komponentdan farzandga `props` orqali tushadi; farzand o'zgarishni callback orqali "yuqoriga" xabar qiladi.
- **Virtual DOM va reconciliation** — holat o'zgarganda React yangi "virtual" daraxtni eskisi bilan solishtiradi va haqiqiy DOM'ga faqat farqni yozadi.

Loyiha yaratish (Vite)

Terminal

```
npm create vite@latest kitob-client -- --template react
cd kitob-client
npm install
npm run dev          # http://localhost:5173
```

Matn

```
kitob-client/
├── index.html          # yagona HTML sahifa – <div id="root"></div>
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx        # kirish nuqtasi: React'ni #root ga ulaydi
    ├── App.jsx         # ildiz komponent
    ├── index.css
    └── components/
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

JSX

JSX — JavaScript ichida HTML'ga o'xshash sintaksis. Brauzer uni tushunmaydi — Vite uni oddiy funksiya chaqiruvlariga aylantiradi: `<h1 className="t">Salom</h1>` → `React.createElement('h1', { className: 't' }, 'Salom')`.

React (JSX)

```
function Greeting() {
  const user = { name: 'Malika', avatar: '/m.jpg', unread: 3 };
  const isMorning = new Date().getHours() < 12;

  return (
    <div className="greeting">                                {/* class emas — className
    */}
      <img src={user.avatar} alt={user.name} />              {/* {} ichida istalgan JS
    ifoda */}
      <h2>{isMorning ? 'Xayrli tong' : 'Salom'}, {user.name}!</h2>
      {user.unread > 0 && <span className="badge">{user.unread}</span>}
      <label htmlFor="q">Qidiruv</label>                      {/* for emas — htmlFor
    */}
      <input id="q" style={{ width: 200, marginTop: 8 }} />  {/* style —
    obyekt, camelCase */}
    </div>
  );
}
```

JSX qoidalari:

- Komponent **bitta** ildiz element qaytaradi. Ortiqcha `div` kerak bo'lmasa — fragment: `<...>`.
- Barcha teglar yopilishi shart: `<img />`, `<br />`, `<input />`.
- Atributlar camelCase: `onClick`, `tabIndex`, `className` (lekin `aria-*` va `data-*` o'zgarmaydi).
- `{}` ichida faqat **ifoda** (expression): `if` / `for` emas — ternary, `&&`, `.map()`.
- React `{}` ichidagi matnni avtomatik ekranlaydi — XSS'dan himoya (`dangerouslySetInnerHTML` bundan mustasno, nomi ham ogohlantiradi).

Komponentlar va props

Komponent — **katta harf bilan boshlanadigan** va JSX qaytaradigan funksiya. **Props** — komponentga tashqaridan beriladigan ma'lumotlar (HTML atributlari kabi). Props **faqat o'qish uchun**: komponent o'z props'ini o'zgartirmaydi.

React (JSX)

```
function Avatar({ src, name, size = 48 }) { // destructuring +
  standart qiymat
  return <img className="avatar" src={src} alt={name} width={size} height=
    {size} />;
}

function UserCard({ user, onFollow, children }) {
  return (
    <article className="card">
      <Avatar src={user.photo} name={user.name} size={64} />
      <h3>{user.name}</h3>
      {children} /* teglar orasidagi
mazmun */
      <button onClick={() => onFollow(user.id)}>Kuzatish</button>
    </article>
  );
}

// Foydalanish
<UserCard user={{ id: 1, name: 'Bobur', photo: '/b.jpg' }} onFollow={(id) =>
  console.log(id)}>
  <p>Frontend dasturchi, Samarqand</p>
</UserCard>
```



Komponentlarga qanday bo'lish kerak?

Dizayn maketiga qarab, har bir mustaqil "blok"ni to'rtburchak bilan o'rab chiqing. Qoida: bitta komponent — bitta vazifa. Komponent 150–200 qatordan oshsa yoki ichida takrorlanuvchi qism bo'lsa — bo'lish vaqti keldi.

State — komponent xotirasi

Oddiy o'zgaruvchi (`let count = 0`) har render'da qaytadan yaratiladi va uning o'zgarishi ekranni yangilamaydi. **State** — React eslab qoladigan va o'zgarganda komponentni **qayta render qiladigan** qiymat.

React (JSX)

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);    // [joriy qiymat, o'zgartiruvchi
  funksiya]

  return (
    <div>
      <p>Bosildi: {count} marta</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(0)} disabled={count === 0}>No'lga</button>
    </div>
  );
}
```

State bilan ishlashning muhim qoidalari

React (JSX)

```
// 1. State'ni TO'G'RIDAN-TO'G'RI o'zgartirmang – yangi obyekt/massiv yarating
(immutability)
const [todos, setTodos] = useState([]);
todos.push(newTodo); // ❌ React o'zgarishni sezmaydi
setTodos([...todos, newTodo]); // ✅ qo'shish
setTodos(todos.filter(t => t.id !== id)); // ✅ o'chirish
setTodos(todos.map(t => t.id === id ? { ...t, done: !t.done } : t)); // ✅ yangilash

const [form, setForm] = useState({ name: '', email: '' });
setForm({ ...form, email: 'yangi@mail.uz' }); // ✅ obyektning bitta maydonini

// 2. Yangi qiymat eskisiga bog'liq bo'lsa – funksional ko'rinish
setCount(c => c + 1);
setCount(c => c + 1); // ikkalasi ham ishlaydi → +2
// setCount(count + 1); setCount(count + 1); → faqat +1 (ikkalasi ham eski "count" ni ko'radi)

// 3. State yangilanishi asinxron: set'dan keyin darhol eski qiymat
setCount(5);
console.log(count); // hali eski qiymat – yangisi keyingi render'da

// 4. Hisoblanadigan qiymatni state'da saqlamang – render paytida hisoblang
const doneCount = todos.filter(t => t.done).length; // ✅ alohida state emas
```

Hodisalar

React (JSX)

```
function SearchBox({ onSearch }) {
  const [q, setQ] = useState('');

  function handleSubmit(e) {
    e.preventDefault();
    onSearch(q.trim());
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={q}
        onChange={(e) => setQ(e.target.value)}
        onKeyDown={(e) => e.key === 'Escape' && setQ('')}
      />
      <button type="submit">Izlash</button>
    </form>
  );
}

// ⚠️ onClick={handleClick}   ✅ funksiyani BERAMIZ
//   onClick={handleClick()}   ❌ render paytida CHAQIRILADI
```

Shartli render va ro'yxatlar

React (JSX)

```
function ProductList({ products, loading, error }) {
  if (loading) return <Spinner />; // erta qaytish
  if (error) return <p className="error">{error}</p>;
  if (products.length === 0) return <p>Mahsulot topilmadi</p>;

  return (
    <ul className="grid">
      {products.map((p) => (
        <li key={p.id}> {/* key – majburiy va
barqaror */}
          <ProductCard product={p} />
        </li>
      ))}
    </ul>
  );
}
```



key nima uchun kerak?

`key` React'ga ro'yxatdagi qaysi element qaysi ekanini aytadi: element qo'shilsa, o'chirilsa yoki tartib o'zgarsa, React to'g'ri DOM tugunlarini qayta ishlatadi. `key` sifatida **massiv indeksini** ishlatmang (agar ro'yxat tartibi o'zgarsa) — input'lardagi qiymatlar "aralashib ketadi". Ma'lumotning barqaror `id` sini ishlateng.

Formalar: boshqariladigan komponentlar

React (JSX)

```
function RegisterForm({ onSubmit }) {
  const [values, setValues] = useState({ name: '', email: '', plan: 'free',
agree: false });
  const [errors, setErrors] = useState({});

  // Barcha maydonlar uchun bitta ishlovchi – name atributi orqali
  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setValues(v => ({ ...v, [name]: type === 'checkbox' ? checked : value }));
  }

  function validate(v) {
    const e = {};
    if (v.name.trim().length < 2) e.name = 'Ism juda qisqa';
    if (!/^\\S+@\\S+\\.\\S+$/\\.test(v.email)) e.email = 'Email noto\\'g\\'ri';
    if (!v.agree) e.agree = 'Shartlarga rozilik kerak';
    return e;
  }

  function handleSubmit(e) {
    e.preventDefault();
    const errs = validate(values);
    setErrors(errs);
    if (Object.keys(errs).length === 0) onSubmit(values);
  }

  return (
    <form onSubmit={handleSubmit} noValidate>
      <input name="name" value={values.name} onChange={handleChange}
placeholder="Ism" />
      {errors.name && <small className="error">{errors.name}</small>}

      <input name="email" value={values.email} onChange={handleChange}
placeholder="Email" />
      {errors.email && <small className="error">{errors.email}</small>}

      <select name="plan" value={values.plan} onChange={handleChange}>
        <option value="free">Bepul</option>
        <option value="pro">Pro</option>
      </select>
    </form>
  );
}
```

```

    </select>

    <label>
      <input type="checkbox" name="agree" checked={values.agree} onChange=
{handleChange} />
      Shartlarga roziman
    </label>
    {errors.agree && <small className="error">{errors.agree}</small>}

    <button>Ro'yxatdan o'tish</button>
  </form>
);
}

```

Boshqariladigan (controlled) input'da qiymatning yagona "haqiqat manbai" — React state. Shu tufayli istalgan payt qiymatni o'qish, o'zgartirish va validatsiya qilish oson. Katta formalar uchun **React Hook Form** + **Zod** kutubxonalari mashhur.

State'ni yuqoriga ko'tarish (lifting state up)

Ikki komponent bitta ma'lumotdan foydalanishi kerak bo'lsa (masalan, mahsulotlar ro'yxati va savat), state ularning **eng yaqin umumiy otasiga** ko'chiriladi. Ota ma'lumotni props orqali beradi, farzandlar esa o'zgarishni callback orqali so'raydi.

```

function Shop() {
  const [cart, setCart] = useState([]); // umumiy state – otada

  const addToCart = (product) =>
    setCart(c => {
      const found = c.find(i => i.id === product.id);
      return found
        ? c.map(i => i.id === product.id ? { ...i, qty: i.qty + 1 } : i)
        : [...c, { ...product, qty: 1 }];
    });

  return (
    <div className="shop">
      <ProductList onAdd={addToCart} /> /* o'zgartiradi */
      <CartSummary items={cart} /> /* o'qiydi */
    </div>
  );
}

```

useEffect — tashqi dunyo bilan sinxronlash

Render **sof** bo'lishi kerak: faqat JSX hisoblaydi. Server so'rovi, taymer, `document.title`, hodisa tinglovchisi kabi "yon ta'sirlar" (side effects) `useEffect` ichida bajariladi — u render ekranga chiqqandan **keyin** ishlaydi.

React (JSX)

```
import { useEffect, useState } from 'react';

useEffect(() => {
  // ... effekt
  return () => { /* tozalash (cleanup): keyingi effektdan oldin va komponent
o'chganda */ };
}, [dep1, dep2]); // bog'liqliklar massivi

// [] — faqat birinchi render'dan keyin bir marta
// [userId] — birinchi marta va userId o'zgarganda
// massivsiz — HAR render'dan keyin (kamdan-kam kerak)
```

React (JSX)

```
function Clock() {
  const [now, setNow] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setNow(new Date()), 1000);
    return () => clearInterval(id); // tozalanmasa — "xotira oqishi"
    va ikki baravar taymer
  }, []);

  useEffect(() => {
    document.title = now.toLocaleTimeString('uz-UZ');
  }, [now]);

  return <time>{now.toLocaleTimeString('uz-UZ')}</time>;
}
```



StrictMode'da effekt ikki marta ishlaydi

Ishlab chiqish rejimida React tozalash funksiyasi to'g'ri yozilganini tekshirish uchun effektни ataylab "o'rnatish → tozalash → qayta o'rnatish" qiladi. Network panelida ikkita so'rovni ko'rsangiz — bu xato emas. Production'da bir marta ishlaydi.



Effekt kerak bo'lmagan holatlar

Props yoki state'dan hisoblanadigan qiymat uchun effekt + state ishlatmang — shunchaki render paytida hisoblang. Foydalanuvchi harakatiga javob (tugma bosilganda so'rov yuborish) — hodisa ishlovchisida, effektda emas.

Boshqa muhim hook'lar

React (JSX)

```
import { useRef, useMemo, useCallback, useReducer } from 'react';

// useRef – render'lar orasida saqlanadigan, lekin o'zgarganda render
// QILMAYDIGAN qiymat; DOM elementga havola
function AutoFocusInput() {
  const inputRef = useRef(null);
  useEffect(() => inputRef.current.focus(), []);
  return <input ref={inputRef} />;
}

// useMemo – og'ir hisob natijasini keshlash
const sorted = useMemo(
  () => [...products].sort((a, b) => a.price - b.price),
  [products]
);

// useCallback – funktsiyani keshlash (memo'langan farzandga beriladigan
// callback uchun)
const handleAdd = useCallback((p) => setCart(c => [...c, p]), []);

// useReducer – murakkab state mantiqi uchun (Redux uslubi)
function reducer(state, action) {
  switch (action.type) {
    case 'add': return [...state, action.item];
    case 'remove': return state.filter(i => i.id !== action.id);
    case 'clear': return [];
    default: throw new Error(`Noma'lum amal: ${action.type}`);
  }
}

const [items, dispatch] = useReducer(reducer, []);
dispatch({ type: 'add', item: { id: 1, name: 'Kitob' } });
```

Hook qoidalari: (1) faqat komponent yoki boshqa hook'ning **yuqori darajasida** chaqiriladi — `if`, sikl yoki ichki funktsiyada emas; (2) faqat React funktsiyalarida. Sabab: React hook'larni chaqirilish **tartibi** bo'yicha taniydi.

Context — "props drilling"siz ma'lumot uzatish

Joriy foydalanuvchi, til yoki mavzu (theme) kabi ma'lumotlar o'nlab komponentga kerak. Ularni har bir darajadan props orqali o'tkazish ("prop drilling") noqulay. **Context** ma'lumotni daraxtning istalgan chuqurligiga to'g'ridan-to'g'ri yetkazadi.

src/context/ThemeContext.jsx

```
import { createContext, useContext, useEffect, useState } from 'react';

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState(() => localStorage.getItem('theme') ?? 'light');

  useEffect(() => {
    document.documentElement.dataset.theme = theme;
    localStorage.setItem('theme', theme);
  }, [theme]);

  const toggle = () => setTheme(t => (t === 'light' ? 'dark' : 'light'));

  return <ThemeContext.Provider value={{ theme, toggle }}>{children}</ThemeContext.Provider>;
}

export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme ThemeProvider ichida ishlatilishi kerak');
  return ctx;
}

// main.jsx: <ThemeProvider><App /></ThemeProvider>
// Istalgan joyda:
// const { theme, toggle } = useTheme();
// <button onClick={toggle}>{theme === 'light' ? '🌙' : '☀️'}</button>
```

React Router — sahifalar

React ilovasi — **SPA (Single Page Application)**: bitta HTML, sahifalar orasidagi o'tish esa JavaScript bilan, serverga yangi HTML so'ramasdan bajariladi.

Terminal

```
npm install react-router
```

```

import { BrowserRouter, Routes, Route, Link, NavLink, useParams, useNavigate,
Navigate } from 'react-router';

export default function App() {
  return (
    <BrowserRouter>
      <nav>
        <NavLink to="/" end>Bosh sahifa</NavLink>
        <NavLink to="/books">Kitoblar</NavLink>
        <NavLink to="/cart">Savat</NavLink>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/books" element={<BookList />} />
        <Route path="/books/:id" element={<BookDetails />} />
        <Route path="/cart" element={<RequireAuth><Cart /></RequireAuth>} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

function BookDetails() {
  const { id } = useParams(); // /books/42 → id = '42'
  const navigate = useNavigate();
  // ... id bo'yicha kitobni yuklash
  return <button onClick={() ⇒ navigate(-1)}>< Orqaga</button>;
}

function RequireAuth({ children }) {
  const user = useAuth(); // 13-darsda yoziladi
  return user ? children : <Navigate to="/login" replace />;
}

```

Amaliy misollar

MISOL 1 Mahsulotlar katalogi va savat (komponentlar, props, lifting state)

src/Shop.jsx

```
import { useState } from 'react';

const PRODUCTS = [
  { id: 1, name: 'Simsiz quloqchin', price: 450000, emoji: '🎧' },
  { id: 2, name: 'Aqlli soat', price: 1200000, emoji: '🕒' },
  { id: 3, name: 'Powerbank 20000', price: 320000, emoji: '🔋' },
  { id: 4, name: 'Mexanik klaviatura', price: 890000, emoji: '⌨' },
];

const som = (n) => n.toLocaleString('uz-UZ') + " so'm";

function ProductCard({ product, qtyInCart, onAdd }) {
  return (
    <article className="product">
      <div className="emoji">{product.emoji}</div>
      <h3>{product.name}</h3>
      <p>{som(product.price)}</p>
      <button onClick={() => onAdd(product)}>
        {qtyInCart ? `Savatda: ${qtyInCart} ✓ (+1)` : 'Savatga'}
      </button>
    </article>
  );
}

function CartItem({ item, onChangeQty, onRemove }) {
  return (
    <li className="cart-item">
      <span>{item.emoji} {item.name}</span>
      <div className="qty">
        <button onClick={() => onChangeQty(item.id, item.qty - 1)} aria-label="Kamaytirish">-</button>
        <span>{item.qty}</span>
        <button onClick={() => onChangeQty(item.id, item.qty + 1)} aria-label="Ko'paytirish">+</button>
      </div>
    </li>
  );
}
```

```

        <b>{som(item.price * item.qty)}</b>
        <button onClick={() => onRemove(item.id)} aria-
label="O'chirish">x</button>
      </li>
    );
  }

function Cart({ items, onChangeQty, onRemove, onClear }) {
  const total = items.reduce((s, i) => s + i.price * i.qty, 0);
  // hisoblanadigan qiymat
  const count = items.reduce((s, i) => s + i.qty, 0);
  const delivery = total === 0 || total >= 1_000_000 ? 0 : 25_000;

  if (items.length === 0) return <aside className="cart"><h2>🛒 Savat
bo'sh</h2></aside>;

  return (
    <aside className="cart">
      <h2>🛒 Savat ({count})</h2>
      <ul>
        {items.map(item => (
          <CartItem key={item.id} item={item} onChangeQty={onChangeQty}
onRemove={onRemove} />
        ))}
      </ul>
      <p>Yetkazish: {delivery ? som(delivery) : 'BEPUL'}</p>
      {delivery > 0 && <small>Yana {som(1_000_000 - total)} xarid
qilsangiz – bepul yetkazish</small>}
      <h3>Jami: {som(total + delivery)}</h3>
      <button onClick={onClear}>Tozalash</button>
    </aside>
  );
}

export default function Shop() {
  const [cart, setCart] = useState([]);

  const add = (product) => setCart(c =>
    c.some(i => i.id === product.id)
      ? c.map(i => (i.id === product.id ? { ...i, qty: i.qty + 1 } : i))
      : [...c, { ...product, qty: 1 }])
  );

  const changeQty = (id, qty) => setCart(c =>
    qty <= 0 ? c.filter(i => i.id !== id) : c.map(i => (i.id === id ? {

```



```

...i, qty } : i))
);
const remove = (id) => setCart(c => c.filter(i => i.id !== id));

return (
  <div className="shop">
    <section className="products">
      {PRODUCTS.map(p => (
        <ProductCard
          key={p.id}
          product={p}
          qtyInCart={cart.find(i => i.id === p.id)?.qty}
          onAdd={add}
        />
      ))}
    </section>
    <Cart items={cart} onChangeQty={changeQty} onRemove={remove}
onClear={() => setCart([])} />
  </div>
);
}

```

src/index.css (qo'shimcha)

```

.shop { display: grid; grid-template-columns: 1fr 340px; gap: 24px;
padding: 24px; align-items: start; }
.products { display: grid; grid-template-columns: repeat(auto-fill,
minmax(180px, 1fr)); gap: 16px; }
.product { border: 1px solid #e5e7eb; border-radius: 14px; padding: 16px;
text-align: center; }
.emoji { font-size: 48px; }
.cart { position: sticky; top: 16px; border: 1px solid #e5e7eb; border-
radius: 14px; padding: 16px; }
.cart ul { list-style: none; padding: 0; }
.cart-item { display: grid; grid-template-columns: 1fr auto auto auto;
gap: 8px; align-items: center; padding: 6px 0; }
.qty { display: flex; gap: 6px; align-items: center; }
@media (max-width: 800px) { .shop { grid-template-columns: 1fr; } }

```

Bu 4-darsdagi savat mantiqining React versiyasi. Savat state'i `Shop` da, chunki uni ham `ProductCard` (savatda nechta borligini ko'rsatish uchun), ham `Cart` ishlatadi. `total` state emas — har render'da hisoblanadi.

MISOL 2

Todo ilovasi React'da: `useReducer` + `localStorage`

5-darsdagi 150 qatorlik ilovani React'da qayta yozamiz va farqni his qilamiz.

```

import { useEffect, useReducer, useState } from 'react';

function todosReducer(todos, action) {
  switch (action.type) {
    case 'added':
      return [{ id: crypto.randomUUID(), text: action.text, done: false },
...todos];
    case 'toggled':
      return todos.map(t => (t.id === action.id ? { ...t, done: !t.done }
: t));
    case 'edited':
      return todos.map(t => (t.id === action.id ? { ...t, text:
action.text } : t));
    case 'deleted':
      return todos.filter(t => t.id !== action.id);
    case 'clearedDone':
      return todos.filter(t => !t.done);
    default:
      throw new Error('Noma\'lum amal: ' + action.type);
  }
}

const load = () => {
  try { return JSON.parse(localStorage.getItem('todos-react')) ?? []; }
catch { return []; }
};

const FILTERS = {
  all: { label: 'Barchasi', fn: () => true },
  active: { label: 'Faol', fn: t => !t.done },
  done: { label: 'Bajarilgan', fn: t => t.done },
};

export default function TodoApp() {
  const [todos, dispatch] = useReducer(todosReducer, undefined, load);
  // lazy init
  const [filter, setFilter] = useState('all');
  const [text, setText] = useState('');

  useEffect(() => {
    localStorage.setItem('todos-react', JSON.stringify(todos));
  }, [todos]);

```

```

const visible = todos.filter(FILTERS[filter].fn);
const left = todos.filter(t => !t.done).length;

function handleSubmit(e) {
  e.preventDefault();
  if (!text.trim()) return;
  dispatch({ type: 'added', text: text.trim() });
  setText('');
}

return (
  <main className="app">
    <h1>📝 Vazifalar</h1>
    <form onSubmit={handleSubmit}>
      <input value={text} onChange={e => setText(e.target.value)}
placeholder="Yangi vazifa..." autoFocus />
      <button disabled={!text.trim()}>Qo'shish</button>
    </form>

    <div className="filters">
      {Object.entries(FILTERS).map(([key, f]) => (
        <button key={key} className={filter === key ? 'active' : ''}
onClick={() => setFilter(key)}>
          {f.label}
        </button>
      ))}
    </div>

    <ul>
      {visible.map(todo => (
        <TodoItem key={todo.id} todo={todo} dispatch={dispatch} />
      ))}
      {visible.length === 0 && <li className="empty">Hozircha vazifa
yo'q 🤖</li>}
    </ul>

    <footer>
      <span>{left} ta faol vazifa</span>
      {todos.some(t => t.done) && (
        <button onClick={() => dispatch({ type: 'clearedDone'
})}>Bajarilganlarni tozalash</button>
      )}
    </footer>
  </main>
)

```

```

    </main>
  );
}

function TodoItem({ todo, dispatch }) {
  const [editing, setEditing] = useState(false);
  const [draft, setDraft] = useState(todo.text);

  function finish(save) {
    if (save && draft.trim()) dispatch({ type: 'edited', id: todo.id,
text: draft.trim() });
    else setDraft(todo.text);
    setEditing(false);
  }

  return (
    <li className={todo.done ? 'done' : ''}>
      <input type="checkbox" checked={todo.done} onChange={() =>
dispatch({ type: 'toggled', id: todo.id })} />
      {editing ? (
        <input
          className="edit"
          value={draft}
          autoFocus
          onChange={e => setDraft(e.target.value)}
          onBlur={() => finish(true)}
          onKeyDown={e => { if (e.key === 'Enter') finish(true); if (e.key
=== 'Escape') finish(false); }}
        />
      ) : (
        <span className="text" onClick={() => setEditing(true)}>
{todo.text}</span>
      )}
      <button onClick={() => dispatch({ type: 'deleted', id: todo.id })}
aria-label="O'chirish">x</button>
    </li>
  );
}

```

Taqqoslang: qo'lda `createElement`, `replaceChildren`, delegatsiya, "qaysi elementni yangilash kerak" degan savollar yo'qoldi. Reducer — sof funksiya: uni React'siz ham test qilish mumkin. Tahrirlash holati (`editing`) har bir element ichida — mahalliy (local) state.

Custom hook — `use` bilan boshlanadigan va ichida boshqa hook'lardan foydalanadigan funksiya. U komponentlar orasida **mantiqni** (UI'ni emas) qayta ishlatishga imkon beradi.

src/hooks/useFetch.js

```
import { useEffect, useState } from 'react';

export function useFetch(url) {
  const [state, setState] = useState({ data: null, loading: Boolean(url),
  error: null });

  useEffect(() => {
    if (!url) return;
    const controller = new AbortController();
    setState(s => ({ ...s, loading: true, error: null }));

    fetch(url, { signal: controller.signal })
      .then(res => {
        if (!res.ok) throw new Error(`Server xatosi: ${res.status}`);
        return res.json();
      })
      .then(data => setState({ data, loading: false, error: null }))
      .catch(err => {
        if (err.name !== 'AbortError') setState({ data: null, loading:
false, error: err.message });
      });

    // url o'zgarsa yoki komponent yopilsa – eski so'rovni bekor qilamiz
    (race condition'dan himoya)
    return () => controller.abort();
  }, [url]);

  return state;
}
```

src/hooks/useDebounce.js

```
import { useEffect, useState } from 'react';

export function useDebounce(value, delay = 400) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounced;
}
```

```

import { useState } from 'react';
import { useFetch } from '../hooks/useFetch';
import { useDebounce } from '../hooks/useDebounce';

export default function UserSearch() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounce(query, 400);
  const { data: users, loading, error } =
    useFetch('https://jsonplaceholder.typicode.com/users');
  const [selectedId, setSelectedId] = useState(null);
  const { data: posts, loading: postsLoading } = useFetch(
    selectedId ? `https://jsonplaceholder.typicode.com/posts?
    userId=${selectedId}` : null
  );

  const q = debouncedQuery.trim().toLowerCase();
  const filtered = users?.filter(u =>
    u.name.toLowerCase().includes(q) || u.email.toLowerCase().includes(q)
  ) ?? [];

  return (
    <div className="user-search">
      <input
        type="search"
        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Ism yoki email bo'yicha qidirish..."
      />

      {loading && <p>Yuklanmoqda... </p>}
      {error && <p className="error">⚠️ {error} </p>}}

      <div className="columns">
        <ul>
          {filtered.map(u => (
            <li key={u.id}>
              <button
                className={u.id === selectedId ? 'active' : ''}
                onClick={() => setSelectedId(u.id)}
              >
                <b>{u.name}</b><br /><small>{u.email}</small>
              </button>
            )
          )}
        </ul>
      </div>
    </div>
  );
}

```



```

        </li>
      )})
      {!loading && filtered.length === 0 && <li>Hech kim
topilmadi</li>}
    </ul>

    <section>
      {!selectedId && <p>Postlarini ko'rish uchun foydalanuvchini
tanlang</p>}
      {postsLoading && selectedId && <p>Postlar yuklanmoqda...</p>}
      {posts?.map(p => (
        <article key={p.id}>
          <h4>{p.title}</h4>
          <p>{p.body}</p>
        </article>
      )})}
    </section>
  </div>
</div>
);
}

```



Real loyihalarda server ma'lumotlarini yuklash uchun **TanStack Query** (React Query) kutubxonasi ishlatiladi: keshlash, qayta yuklash, "stale-while-revalidate", mutatsiyalar va xatolarni boshqarishni tayyor holda beradi. `useFetch` esa uning ishlash tamoyilini tushunish uchun ajoyib mashq.

```
src/hooks/useLocalStorage.js
```

```
import { useEffect, useState } from 'react';

export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try {
      const saved = localStorage.getItem(key);
      return saved !== null ? JSON.parse(saved) : initialValue;
    } catch {
      return initialValue;
    }
  });

  useEffect(() => {
    try { localStorage.setItem(key, JSON.stringify(value)); } catch { /*
to'lgan yoki taqiqlangan */ }
    }, [key, value]);

    // Boshqa brauzer yorlig'ida o'zgarsa – sinxronlash
    useEffect(() => {
      const onStorage = (e) => {
        if (e.key === key && e.newValue !== null)
          setValue(JSON.parse(e.newValue));
      };
      window.addEventListener('storage', onStorage);
      return () => window.removeEventListener('storage', onStorage);
    }, [key]);

    return [value, setValue];
  }
```

src/context/FavoritesContext.jsx

```
import { createContext, useContext } from 'react';
import { useLocalStorage } from '../hooks/useLocalStorage';

const FavoritesContext = createContext(null);

export function FavoritesProvider({ children }) {
  const [ids, setIds] = useLocalStorage('favorites', []);
  const toggle = (id) => setIds(list => (list.includes(id) ? list.filter(x
=> x !== id) : [...list, id]));
  const isFavorite = (id) => ids.includes(id);
  return (
    <FavoritesContext.Provider value={{ ids, toggle, isFavorite }}>
      {children}
    </FavoritesContext.Provider>
  );
}

export const useFavorites = () => useContext(FavoritesContext);
```

```

import { BrowserRouter, Routes, Route, NavLink, Link, useParams } from
'react-router';
import { FavoritesProvider, useFavorites } from
'./context/FavoritesContext';
import { useFetch } from './hooks/useFetch';

const API = 'https://jsonplaceholder.typicode.com';

function Header() {
  const { ids } = useFavorites(); // props'siz - Context
  orqali
  return (
    <header className="header">
      <NavLink to="/" end>📄 Maqolalar</NavLink>
      <NavLink to="/favorites">★ Saralangan ({ids.length})</NavLink>
    </header>
  );
}

function FavoriteButton({ id }) {
  const { isFavorite, toggle } = useFavorites();
  const fav = isFavorite(id);
  return (
    <button onClick={() => toggle(id)} aria-pressed={fav} title={fav ?
'Saralanganidan olish' : 'Saralash'}>
      {fav ? '★' : '☆'}
    </button>
  );
}

function PostList({ onlyFavorites = false }) {
  const { data: posts, loading } = useFetch(`${API}/posts?_limit=20`);
  const { ids } = useFavorites();
  if (loading) return <p>Yuklanmoqda...</p>;
  if (!posts) return <p>Maqolalarni yuklab bo'lmadi.</p>;
  const list = onlyFavorites ? posts.filter(p => ids.includes(p.id)) :
posts;
  if (!list.length) return <p>Hozircha saralangan maqola yo'q.</p>;
  return (
    <ul className="posts">
      {list.map(p => (
        <li key={p.id}>

```

```

        <FavoriteButton id={p.id} />
        <Link to={` /posts/${p.id}`}>{p.title}</Link>
      </li>
    )}
  </ul>
);
}

function PostPage() {
  const { id } = useParams();
  const { data: post, loading, error } = useFetch(`${API}/posts/${id}`);
  const { data: comments } = useFetch(`${API}/posts/${id}/comments`);
  if (loading) return <p>Yuklanmoqda... </p>;
  if (error) return <p>Maqola topilmadi. <Link to="/">Bosh sahifaga</Link>
</p>;
  return (
    <article>
      <Link to="/">< Barcha maqolalar</Link>
      <h1>{post.title} <FavoriteButton id={post.id} /></h1>
      <p>{post.body}</p>
      <h3>Izohlar ({comments?.length ?? '...'})</h3>
      {comments?.map(c => <blockquote key={c.id}><b>{c.email}</b>:
{c.body}</blockquote>)}
    </article>
  );
}

export default function App() {
  return (
    <FavoritesProvider>
      <BrowserRouter>
        <Header />
        <main>
          <Routes>
            <Route path="/" element={<PostList />} />
            <Route path="/favorites" element={<PostList onlyFavorites />}
/>

            <Route path="/posts/:id" element={<PostPage />} />
            <Route path="*" element={<h2>404 – sahifa topilmadi</h2>} />
          </Routes>
        </main>
      </BrowserRouter>
    </FavoritesProvider>
  );
}

```

```
);  
}
```

Yulduzcha bosilganda `Header` dagi hisoblagich ham darhol yangilanadi — garchi ular daraxtning turli shoxlarida bo'lsa ham. Saralanganlar `useLocalStorage` tufayli sahifa yangilanganda ham, boshqa yorliqda ham saqlanadi.

Mashqlar



Mustaqil ish

1. react.dev/learn rasmiy qo'llanmasidagi "Tic-Tac-Toe" o'yinini yasang.
2. 1-misoldagi savatni `CartContext` ga ko'chiring va `useLocalStorage` bilan saqlang; `Header`'da savatdagi mahsulotlar sonini ko'rsating.
3. 6-darsdagi ob-havo ilovasini React'da qayta yozing: `useDebounce`, `useFetch`, yuklanish va xato holatlari bilan.
4. `useOnlineStatus()` hook'ini yozing (`navigator.onLine`, `online` / `offline` hodisalari) va internet uzilganda banner ko'rsating.
5. Ko'p bosqichli forma (wizard): 3 qadam, "Orqaga/Keyingi" tugmalari, har qadamda validatsiya, oxirida umumiy ko'rinish.



Xulosa

- React: `UI = f(state)`; komponentlar — JSX qaytaruvchi funksiyalar.
- Props — tashqaridan, faqat o'qish uchun; state — ichki xotira, o'zgarsa qayta render.
- State'ni o'zgartirmang — yangi massiv/obyekt yarating; hisoblanadigan qiymatni state'da saqlamang.
- Ro'yxatlarda barqaror `key`; formalarda boshqariladigan input'lar.
- `useEffect` — tashqi tizimlar bilan sinxronlash, cleanup bilan; custom hook'lar — mantiqni qayta ishlatish.
- Context — global ma'lumot; React Router — SPA sahifalari.



13-DARS · 14 TADAN

Yakuniy loyiha: full-stack "Eslatmalar" ilovasi

Barcha bilimlarni birlashtiramiz: PostgreSQL + Express API + JWT autentifikatsiya + React frontend. Loyihalash, backend, frontend va ularni bog'lash bosqichma-bosqich.

Yuqori · ~4–6 soat amaliyot

Endi barcha bo'laklarni bitta haqiqiy ilovaga yig'amiz. **"Eslatmalar" (Notes)** — foydalanuvchilar ro'yxatdan o'tadi, shaxsiy eslatmalar yaratadi, ularni teglar bilan belgilaydi, qidiradi, mahkamlaydi (pin) va arxivlaydi. Loyiha kichik ko'rinadi, lekin unda real ishdagi barcha muammolar bor: sxema dizayni, API, autentifikatsiya, egalik, validatsiya, frontend holati, xatolar, CORS va proksi.



Qanday o'qish kerak

Bu dars — qo'llanma emas, **amaliy ish**. Har bir faylni o'zingiz yarating va har bosqich oxirida ilovani ishga tushirib tekshiring. Tushunmagan joyingiz bo'lsa — tegishli darsga qayting (havolalar matnda).

1-bosqich: Loyihalash

Funksional talablar

- Ro'yxatdan o'tish, kirish, chiqish; sessiya sahifa yangilanganda saqlanadi.
- Eslatma: sarlavha, matn, rang, teglar, "mahkamlangan" va "arxivlangan" belgilari.
- Eslatmalarni yaratish, tahrirlash, o'chirish (CRUD); faqat **o'z** eslatmalarini ko'rish.
- Matn bo'yicha qidiruv, teg bo'yicha filtr, arxiv bo'limi; mahkamlanganlar tepada.

Arxitektura



API shartnomasi (contract)

Frontend va backend o'rtasidagi "kelishuv"ni oldindan yozib qo'yish — ikki tomonni parallel ishlab chiqish imkonini beradi.

Metod va yo'l	Tana / query	Javob
POST /api/auth/register	{ name, email, password }	201 { user } + cookie
POST /api/auth/login	{ email, password }	200 { user } + cookie
POST /api/auth/logout	—	204
GET /api/auth/me	—	200 { user } / 401
GET /api/notes	?q=&tag=&archived=false	200 { data: Note[] }
POST /api/notes	{ title, body, color, tags[] }	201 Note
PATCH /api/notes/:id	istalgan maydonlar	200 Note / 404
DELETE /api/notes/:id	—	204 / 404
GET /api/tags	—	200 [{ name, count }]

Papka tuzilmasi

Matn

```
notes-app/  
├── server/  
│   ├── package.json  
│   ├── .env / .env.example  
│   ├── db/schema.sql  
│   └── src/  
│       ├── server.js  app.js  config.js  db.js  
│       ├── middleware/  auth.js  validate.js  errors.js  
│       └── routes/      auth.routes.js  notes.routes.js  
└── client/  
    ├── package.json  vite.config.js  index.html  
    └── src/  
        ├── main.jsx  App.jsx  api.js  index.css  
        ├── context/AuthContext.jsx  
        ├── pages/    LoginPage.jsx  NotesPage.jsx  
        └── components/  NoteCard.jsx  NoteEditor.jsx  TagFilter.jsx
```

2-bosqich: Ma'lumotlar bazasi

server/db/schema.sql

```
CREATE EXTENSION IF NOT EXISTS pgcrypto; -- gen_random_uuid() uchun (PG 13+
da o'rnatilgan)

CREATE TABLE IF NOT EXISTS users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL CHECK (length(name) BETWEEN 2 AND 60),
  email       TEXT NOT NULL,
  password_hash TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX IF NOT EXISTS users_email_uq ON users (lower(email));

CREATE TABLE IF NOT EXISTS notes (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title       TEXT NOT NULL DEFAULT '' CHECK (length(title) ≤ 200),
  body       TEXT NOT NULL DEFAULT '' CHECK (length(body) ≤ 20000),
  color       TEXT NOT NULL DEFAULT 'default',
  tags        TEXT[] NOT NULL DEFAULT '{}',
  pinned      BOOLEAN NOT NULL DEFAULT false,
  archived    BOOLEAN NOT NULL DEFAULT false,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Eng ko'p so'rov: "foydalanuvchining arxivlanmagan eslatmalari,
mahkamlanganlar va yangilari tepada"
CREATE INDEX IF NOT EXISTS notes_user_list_idx ON notes (user_id, archived,
pinned DESC, updated_at DESC);
CREATE INDEX IF NOT EXISTS notes_tags_idx ON notes USING GIN (tags); --
massiv ichida qidiruv uchun
```

Teglar uchun alohida N:M jadval o'rniga PostgreSQL massivi (`TEXT[]`) tanlandi: teglar faqat bitta foydalanuvchiga tegishli, alohida xususiyatlari yo'q, va GIN indeksi `tags @> ARRAY['ish']` so'rovini tezlashtiradi. Bu — loyihaning ehtiyojiga qarab qilingan ongli tanlov (10-dars).

Terminal

```
docker run -d --name notes-db -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=notes
-p 5432:5432 postgres:17
psql postgres://postgres:secret@localhost:5432/notes -f server/db/schema.sql
```

3-bosqich: Backend (Express API)

Terminal

```
mkdir -p notes-app/server && cd notes-app/server
npm init -y && npm pkg set type=module
npm install express pg bcrypt jsonwebtoken cookie-parser zod helmet morgan
npm pkg set scripts.dev="node --watch --env-file=.env src/server.js"
npm pkg set scripts.start="node src/server.js"
```

server/.env.example

```
PORT=3000
DATABASE_URL=postgres://postgres:secret@localhost:5432/notes
JWT_SECRET=bu-yerga-48-baytlik-tasodifiy-kalit
NODE_ENV=development
```

server/src/config.js

```
const required = (name) => {
  if (!process.env[name]) throw new Error(`.env da ${name} yo'q`);
  return process.env[name];
};

export const config = {
  port: Number(process.env.PORT) || 3000,
  databaseUrl: required('DATABASE_URL'),
  jwtSecret: required('JWT_SECRET'),
  isProd: process.env.NODE_ENV === 'production',
};
```

server/src/db.js

```
import pg from 'pg';
import { config } from './config.js';

export const pool = new pg.Pool({ connectionString: config.databaseUrl });
export const query = (text, params) => pool.query(text, params);
```

server/src/middleware/errors.js

```
export class HttpError extends Error {
  constructor(status, message, details) {
    super(message);
    this.status = status;
    this.details = details;
  }
}

export function notFound(req, res, next) {
  next(new HttpError(404, `Topilmadi: ${req.method} ${req.originalUrl}`));
}

export function errorHandler(err, req, res, _next) {
  if (err.type === 'entity.parse.failed') err = new HttpError(400, 'JSON noto\'g\'ri');
  if (err.code === '22P02') err = new HttpError(404, 'Topilmadi'); // noto'g'ri UUID
  const status = err.status ?? 500;
  if (status ≥ 500) console.error(err);
  res.status(status).json({
    error: {
      message: status ≥ 500 && process.env.NODE_ENV === 'production' ? 'Server xatosi' : err.message,
      ...(err.details && { details: err.details }),
    },
  });
}
```

```
import { HttpError } from './errors.js';

export const validate = (schema, source = 'body') => (req, res, next) => {
  const r = schema.safeParse(req[source]);
  if (!r.success) {
    return next(new HttpError(400, 'Validatsiya xatosi',
      r.error.issues.map(i => ({ field: i.path.join('.'), message: i.message
    })))));
  }
  req.valid = r.data;
  next();
};
```

```
import jwt from 'jsonwebtoken';
import { config } from '../config.js';
import { HttpError } from './errors.js';

export const COOKIE = 'notes_token';

export const cookieOptions = {
  httpOnly: true,
  secure: config.isProd,
  sameSite: 'lax',
  maxAge: 7 * 24 * 60 * 60 * 1000, // 7 kun (soddalik uchun refresh
token'siz)
  path: '/',
};

export const signToken = (user) =>
  jwt.sign({ sub: user.id, name: user.name }, config.jwtSecret, { expiresIn:
'7d', algorithm: 'HS256' });

export function requireAuth(req, res, next) {
  const token = req.cookies[COOKIE];
  if (!token) return next(new HttpError(401, 'Tizimga kiring'));
  try {
    const payload = jwt.verify(token, config.jwtSecret, { algorithms: ['HS256']
});
    req.userId = payload.sub;
    next();
  } catch {
    next(new HttpError(401, 'Sessiya muddati tugagan'));
  }
}
```

```

import { Router } from 'express';
import bcrypt from 'bcrypt';
import { z } from 'zod';
import { query } from '../db.js';
import { validate } from '../middleware/validate.js';
import { HttpError } from '../middleware/errors.js';
import { COOKIE, cookieOptions, signToken, requireAuth } from
'../middleware/auth.js';

const router = Router();
const DUMMY = await bcrypt.hash('dummy', 12);

const registerSchema = z.object({
  name: z.string().trim().min(2, 'Ism kamida 2 harf').max(60),
  email: z.string().trim().toLowerCase().email('Email noto\'g\'ri'),
  password: z.string().min(8, 'Parol kamida 8 belgi').max(72),
});
const loginSchema = z.object({
  email: z.string().trim().toLowerCase().email(),
  password: z.string().min(1),
});

const login = (res, user, status = 200) => {
  res.cookie(COOKIE, signToken(user), cookieOptions);
  res.status(status).json({ user: { id: user.id, name: user.name, email:
user.email } });
};

router.post('/register', validate(registerSchema), async (req, res) => {
  const { name, email, password } = req.valid;
  const hash = await bcrypt.hash(password, 12);
  try {
    const { rows: [user] } = await query(
      'INSERT INTO users (name, email, password_hash) VALUES ($1, $2, $3)
RETURNING id, name, email',
      [name, email, hash]
    );
    login(res, user, 201);
  } catch (err) {
    if (err.code === '23505') throw new HttpError(409, 'Bu email band');
    throw err;
  }
}

```

```

});

router.post('/login', validate(loginSchema), async (req, res) => {
  const { email, password } = req.valid;
  const { rows: [user] } = await query('SELECT * FROM users WHERE lower(email)
= $1', [email]);
  const ok = await bcrypt.compare(password, user?.password_hash ?? DUMMY);
  if (!user || !ok) throw new HttpError(401, 'Email yoki parol noto\'g\'ri');
  login(res, user);
});

router.post('/logout', (req, res) => {
  const { maxAge, ...opts } = cookieOptions;
  res.clearCookie(COOKIE, opts).status(204).end();
});

router.get('/me', requireAuth, async (req, res) => {
  const { rows: [user] } = await query('SELECT id, name, email FROM users WHERE
id = $1', [req.userId]);
  if (!user) throw new HttpError(401, 'Foydalanuvchi topilmadi');
  res.json({ user });
});

export default router;

```



```

import { Router } from 'express';
import { z } from 'zod';
import { query } from '../db.js';
import { validate } from '../middleware/validate.js';
import { HttpError } from '../middleware/errors.js';
import { requireAuth } from '../middleware/auth.js';

const router = Router();
router.use(requireAuth); // - bu router'dagi BARCHA
marshrutlar himoyalangan

const COLORS = ['default', 'yellow', 'green', 'blue', 'pink', 'purple'];
const tag =
z.string().trim().toLowerCase().min(1).max(30).regex(/^[\p{L}\p{N}_-]+$\/u,
'Teg: harf, raqam, _ yoki -');

const noteSchema = z.object({
  title: z.string().trim().max(200).default(''),
  body: z.string().max(20000).default(''),
  color: z.enum(COLORS).default('default'),
  tags: z.array(tag).max(10).default([]).transform(t => [...new Set(t)]), //
takrorlarni olib tashlash
  pinned: z.boolean().default(false),
  archived: z.boolean().default(false),
}).refine(n => n.title || n.body, { message: 'Sarlavha yoki matn bo\'lishi
kerak' });

const patchSchema = z.object({
  title: z.string().trim().max(200),
  body: z.string().max(20000),
  color: z.enum(COLORS),
  tags: z.array(tag).max(10).transform(t => [...new Set(t)]),
  pinned: z.boolean(),
  archived: z.boolean(),
}).partial().strict(); // strict - noma'lum maydon (masalan user_id)
yuborilsa xato

const listSchema = z.object({
  q: z.string().trim().max(100).optional(),
  tag: tag.optional(),
  archived: z.enum(['true', 'false']).default('false').transform(v => v ===
'true'),

```

```

});

const COLS = `id, title, body, color, tags, pinned, archived,
              created_at AS "createdAt", updated_at AS "updatedAt"`;

// GET /api/notes?q=&tag=&archived=
router.get('/', validate(listSchema, 'query'), async (req, res) => {
  const { q, tag, archived } = req.valid;
  const params = [req.userId, archived];
  let sql = `SELECT ${COLS} FROM notes WHERE user_id = $1 AND archived = $2`;

  if (q) {
    params.push(`%${q}%`);
    sql += ` AND (title ILIKE ${params.length} OR body ILIKE
    ${params.length})`;
  }
  if (tag) {
    params.push([tag]);
    sql += ` AND tags @> ${params.length}`;
  }
  sql += ` ORDER BY pinned DESC, updated_at DESC LIMIT 500`;

  const { rows } = await query(sql, params);
  res.json({ data: rows });
});

router.post('/', validate(noteSchema), async (req, res) => {
  const n = req.valid;
  const { rows: [note] } = await query(
    `INSERT INTO notes (user_id, title, body, color, tags, pinned, archived)
    VALUES ($1, $2, $3, $4, $5, $6, $7) RETURNING ${COLS}`,
    [req.userId, n.title, n.body, n.color, n.tags, n.pinned, n.archived]
  );
  res.status(201).json(note);
});

router.patch('/:id', validate(patchSchema), async (req, res) => {
  const entries = Object.entries(req.valid);
  if (!entries.length) throw new HttpError(400, '0\'zgartirish uchun maydon
  yo\'q');

  // Ustun nomlari – faqat sxema ruxsat bergan kalitlar (strict), qiymatlar –
  parametrklar
  const set = entries.map(([k], i) => `${k} = ${i + 3}`).join(', ');

```

```

const { rows: [note] } = await query(
  `UPDATE notes SET ${set}, updated_at = now()
   WHERE id = $1 AND user_id = $2                                -- 🔒 egalik tekshiruvi
  (IDOR'ga qarshi)
   RETURNING ${COLS}` ,
  [req.params.id, req.userId, ...entries.map(([, v]) => v)]
);
if (!note) throw new HttpError(404, 'Eslatma topilmadi');
res.json(note);
});

router.delete('/:id', async (req, res) => {
  const { rowCount } = await query('DELETE FROM notes WHERE id = $1 AND user_id
= $2', [req.params.id, req.userId]);
  if (!rowCount) throw new HttpError(404, 'Eslatma topilmadi');
  res.status(204).end();
});

export default router;

// Teglar statistikasi alohida router'da
export const tagsRouter = Router().get('/', requireAuth, async (req, res) => {
  const { rows } = await query(
    `SELECT t AS name, COUNT(*)::int AS count
     FROM notes, unnest(tags) AS t
     WHERE user_id = $1 AND NOT archived
     GROUP BY t ORDER BY count DESC, t`,
    [req.userId]
  );
  res.json(rows);
});

```

Diqqat: `WHERE id = $1 AND user_id = $2` — boshqa foydalanuvchining eslatmasini tahrirlashga urinish `403` emas, `404` qaytaradi. Bu ataylab: hujumchiga hatto "bunday ID mavjud" degan ma'lumotni ham bermaymiz.

```
import express from 'express';
import helmet from 'helmet';
import morgan from 'morgan';
import cookieParser from 'cookie-parser';
import authRouter from './routes/auth.routes.js';
import notesRouter, { tagsRouter } from './routes/notes.routes.js';
import { notFound, errorHandler } from './middleware/errors.js';

export const app = express();

app.set('trust proxy', 1); // Nginx/hosting proksisi ortida
to'g'ri IP va HTTPS uchun
app.use(helmet());
app.use(morgan('dev'));
app.use(express.json({ limit: '100kb' }));
app.use(cookieParser());

app.get('/api/health', (req, res) => res.json({ ok: true }));
app.use('/api/auth', authRouter);
app.use('/api/notes', notesRouter);
app.use('/api/tags', tagsRouter);

app.use('/api', notFound);
app.use(errorHandler);
```

server/src/server.js

```
import { app } from './app.js';
import { config } from './config.js';
import { pool } from './db.js';

await pool.query('SELECT 1'); // bazaga ulanishni ishga
tushishda tekshiramiz
const server = app.listen(config.port, () => console.log(`🚀 API:
http://localhost:${config.port}`));

// To'g'ri to'xtash (graceful shutdown) – joriy so'rovlar tugashini kutamiz
for (const signal of ['SIGINT', 'SIGTERM']) {
  process.on(signal, () => {
    console.log(`${signal} – to'xtatilmoqda...`);
    server.close(() => pool.end().then(() => process.exit(0)));
  });
}
```

Backend'ni tekshirish

Terminal

```
cp .env.example .env # va JWT_SECRET ni to'ldiring
npm run dev

# -c / -b : cookie'ni faylga saqlash va yuborish (brauzer kabi)
curl -c jar.txt -X POST localhost:3000/api/auth/register -H 'Content-Type:
application/json' \
  -d '{"name":"Sardor","email":"sardor@mail.uz","password":"parol12345"}'

curl -b jar.txt -X POST localhost:3000/api/notes -H 'Content-Type:
application/json' \
  -d '{"title":"Xaridlar","body":"Non, sut, tuxum","tags":
["uy","bozor"],"color":"yellow"}'

curl -b jar.txt "localhost:3000/api/notes?q=sut"
curl -b jar.txt localhost:3000/api/tags
curl -i localhost:3000/api/notes # cookie'siz → 401
```

4-bosqich: Frontend (React)

Terminal

```
cd .. # notes-app/  
npm create vite@latest client -- --template react  
cd client && npm install && npm install react-router
```

client/vite.config.js

```
import { defineConfig } from 'vite';  
import react from '@vitejs/plugin-react';  
  
export default defineConfig({  
  plugins: [react()],  
  server: {  
    // /api bilan boshlanadigan so'rovlar Express'ga uzatiladi:  
    // brauzer uchun hammasi localhost:5173 – CORS ham, cookie muammosi ham  
    yo'q  
    proxy: { '/api': 'http://localhost:3000' },  
  },  
});
```

```

export class ApiError extends Error {
  constructor(status, message, details) {
    super(message);
    this.status = status;
    this.details = details;
  }
}

export async function api(path, { method = 'GET', body, signal } = {}) {
  const res = await fetch('/api' + path, {
    method,
    signal,
    credentials: 'same-origin',
    headers: body ? { 'Content-Type': 'application/json' } : undefined,
    body: body ? JSON.stringify(body) : undefined,
  });
  if (res.status === 204) return null;
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new ApiError(res.status, data.error?.message ?? 'Tarmoq xatosi', data.error?.details);
  return data;
}

export const authApi = {
  me: () => api('/auth/me'),
  login: (body) => api('/auth/login', { method: 'POST', body }),
  register: (body) => api('/auth/register', { method: 'POST', body }),
  logout: () => api('/auth/logout', { method: 'POST' }),
};

export const notesApi = {
  list: (params, signal) => api('/notes?' + new URLSearchParams(params), { signal }),
  create: (note) => api('/notes', { method: 'POST', body: note }),
  update: (id, patch) => api(`/notes/${id}`, { method: 'PATCH', body: patch }),
  remove: (id) => api(`/notes/${id}`, { method: 'DELETE' }),
  tags: () => api('/tags'),
};

```

```
import { createContext, useContext, useEffect, useState } from 'react';
import { authApi } from '../api';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [checking, setChecking] = useState(true); // sahifa ochilganda sessiyani tekshirish

  useEffect(() => {
    authApi.me()
      .then(d => setUser(d.user))
      .catch(() => setUser(null))
      .finally(() => setChecking(false));
  }, []);

  const value = {
    user,
    checking,
    login: async (creds) => setUser((await authApi.login(creds)).user),
    register: async (data) => setUser((await authApi.register(data)).user),
    logout: async () => { await authApi.logout(); setUser(null); },
  };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export const useAuth = () => useContext(AuthContext);
```



```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router';
import { AuthProvider, useAuth } from '../context/AuthContext';
import LoginPage from '../pages/LoginPage';
import NotesPage from '../pages/NotesPage';

function Protected({ children }) {
  const { user, checking } = useAuth();
  if (checking) return <div className="center">⌚</div>;
  return user ? children : <Navigate to="/login" replace />;
}

function GuestOnly({ children }) {
  const { user, checking } = useAuth();
  if (checking) return null;
  return user ? <Navigate to="/" replace /> : children;
}

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<GuestOnly><LoginPage /></GuestOnly>} />
          <Route path="/" element={<Protected><NotesPage /></Protected>} />
          <Route path="/archive" element={<Protected><NotesPage archived /></Protected>} />
          <Route path="*" element={<Navigate to="/" replace />} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  );
}
```

```

import { useState } from 'react';
import { useAuth } from '../context/AuthContext';

export default function LoginPage() {
  const { login, register } = useAuth();
  const [mode, setMode] = useState('login'); // 'login' | 'register'
  const [form, setForm] = useState({ name: '', email: '', password: '' });
  const [error, setError] = useState(null);
  const [busy, setBusy] = useState(false);

  const onChange = (e) => setForm(f => ({ ...f, [e.target.name]: e.target.value }));

  async function onSubmit(e) {
    e.preventDefault();
    setBusy(true);
    setError(null);
    try {
      mode === 'login'
        ? await login({ email: form.email, password: form.password })
        : await register(form);
      // muvaffaqiyatli – GuestOnly avtomatik "/" ga yo'naltiradi
    } catch (err) {
      setError(err.details?.map(d => d.message).join('. ') || err.message);
    } finally {
      setBusy(false);
    }
  }

  return (
    <main className="auth">
      <form onSubmit={onSubmit} className="auth-card">
        <h1>📖 Eslatmalar</h1>
        <div className="tabs">
          <button type="button" className={mode === 'login' ? 'active' : ''}
            onClick={() => setMode('login')}>Kirish</button>
          <button type="button" className={mode === 'register' ? 'active' : ''}
            onClick={() => setMode('register')}>Ro'yxatdan o'tish</button>
        </div>

        {mode === 'register' && (
          <input name="name" placeholder="Ismingiz" value={form.name} onChange=

```

```

{onChange} required autoComplete="name" />
    })
    <input name="email" type="email" placeholder="Email" value={form.email}
onChange={onChange} required autoComplete="email" />
    <input name="password" type="password" placeholder="Parol (kamida 8
belgi)" value={form.password} onChange={onChange}
        required minLength={mode === 'register' ? 8 : undefined}
        autoComplete={mode === 'login' ? 'current-password' : 'new-
password'} />

    {error && <p className="error" role="alert">{error}</p>}
    <button className="primary" disabled={busy}>
        {busy ? '...' : mode === 'login' ? 'Kirish' : 'Hisob yaratish'}
    </button>
</form>
</main>
);
}

```

```

import { useState } from 'react';

export const COLORS = ['default', 'yellow', 'green', 'blue', 'pink', 'purple'];

// Yangi eslatma yaratish va mavjudini tahrirlash uchun bitta komponent
export default function NoteEditor({ initial, onSave, onCancel }) {
  const [title, setTitle] = useState(initial?.title ?? '');
  const [body, setBody] = useState(initial?.body ?? '');
  const [color, setColor] = useState(initial?.color ?? 'default');
  const [tagsText, setTagsText] = useState(initial?.tags?.join(', ') ?? '');
  const [error, setError] = useState(null);
  const [saving, setSaving] = useState(false);

  async function submit(e) {
    e.preventDefault();
    if (!title.trim() && !body.trim()) return setError('Sarlavha yoki matn yozing');
    const tags = tagsText.split(',').map(t => t.trim().toLowerCase()).filter(Boolean);
    setSaving(true);
    try {
      await onSave({ title, body, color, tags });
      if (!initial) { setTitle(''); setBody(''); setTagsText(''); setColor('default'); }
      setError(null);
    } catch (err) {
      setError(err.details?.map(d => `${d.field}: ${d.message}`).join('; ') || err.message);
    } finally {
      setSaving(false);
    }
  }

  return (
    <form className={`editor note-${color}`} onSubmit={submit}>
      <input placeholder="Sarlavha" value={title} onChange={e => setTitle(e.target.value)} maxLength={200} />
      <textarea placeholder="Eslatma matni..." rows={initial ? 8 : 3} value={body} onChange={e => setBody(e.target.value)} />
      <input placeholder="Teglar: ish, g'oyalar" value={tagsText} onChange={e => setTagsText(e.target.value)} />
      <div className="editor-bar">

```

```

    <div className="colors" role="radiogroup" aria-label="Rang">
      {COLORS.map(c => (
        <button key={c} type="button" role="radio" aria-checked={c ===
color} aria-label={c}
          className={`swatch note-${c} ${c === color ? 'selected' :
''}`} onClick={() => setColor(c)} />
      ))}
    </div>
    {onCancel && <button type="button" onClick={onCancel}>Bekor</button>}
    <button className="primary" disabled={saving}>{saving ? 'Saqlanmoqda...'
: 'Saqlash'}</button>
  </div>
  {error && <p className="error">{error}</p>}
</form>
);
}

```

```

export default function NoteCard({ note, onEdit, onUpdate, onDelete, onTagClick
}) {
  const date = new Date(note.updatedAt).toLocaleString('uz-UZ', { dateStyle:
'medium', timeStyle: 'short' });
  return (
    <article className={`note note-${note.color}`}>
      <button className="pin" title={note.pinned ? 'Bo\'shatish' :
'Mahkamlash'} aria-pressed={note.pinned}
        onClick={() => onUpdate(note.id, { pinned: !note.pinned })}>
        {note.pinned ? '📌' : '📌'}
      </button>
      <div className="note-content" onClick={() => onEdit(note)} role="button"
tabIndex={0}
        onKeyDown={e => e.key === 'Enter' && onEdit(note)}>
        {note.title && <h3>{note.title}</h3>}
        <p>{note.body}</p>
      </div>
      {note.tags.length > 0 && (
        <div className="tags">
          {note.tags.map(t => <button key={t} className="tag" onClick={() =>
onTagClick(t)}>#{t}</button>)}
        </div>
      )}
      <footer>
        <small>{date}</small>
        <span>
          <button onClick={() => onUpdate(note.id, { archived: !note.archived
})} title={note.archived ? 'Arxivdan chiqarish' : 'Arxivlash'}>
            {note.archived ? '🗑️' : '📁'}
          </button>
          <button onClick={() => onDelete(note)} title="O'chirish">🗑️</button>
        </span>
      </footer>
    </article>
  );
}

```

```

import { useCallback, useEffect, useState } from 'react';
import { NavLink } from 'react-router';
import { useAuth } from '../context/AuthContext';
import { notesApi } from '../api';
import NoteCard from '../components/NoteCard';
import NoteEditor from '../components/NoteEditor';

function useDebounce(value, ms = 300) {
  const [v, setV] = useState(value);
  useEffect(() => { const t = setTimeout(() => setV(value), ms); return () =>
clearTimeout(t); }, [value, ms]);
  return v;
}

export default function NotesPage({ archived = false }) {
  const { user, logout } = useAuth();
  const [notes, setNotes] = useState([]);
  const [tags, setTags] = useState([]);
  const [status, setStatus] = useState('loading'); // 'loading' | 'ready'
  | 'error'
  const [q, setQ] = useState('');
  const [tag, setTag] = useState(null);
  const [editing, setEditing] = useState(null);
  const query = useDebounce(q);

  const loadTags = useCallback(() => notesApi.tags().then(setTags).catch(() =>
{}), []);

  // Filtr o'zgarganda ro'yxatni qayta yuklash; eski so'rovni bekor qilish
  useEffect(() => {
    const ctrl = new AbortController();
    setStatus('loading');
    const params = { archived: String(archived) };
    if (query) params.q = query;
    if (tag) params.tag = tag;
    notesApi.list(params, ctrl.signal)
      .then(({ data }) => { setNotes(data); setStatus('ready'); })
      .catch(err => { if (err.name !== 'AbortError') setStatus('error'); });
    return () => ctrl.abort();
  }, [archived, query, tag]);

  useEffect(() => { loadTags(); }, [loadTags]);

```

```

const sortNotes = (list) =>
  [...list].sort((a, b) => (b.pinned - a.pinned) ||
b.updatedAt.localeCompare(a.updatedAt));

async function create(data) {
  const note = await notesApi.create(data);
  setNotes(n => sortNotes([note, ...n]));
  loadTags();
}

// Optimistik yangilash: interfeys darhol o'zgaradi, xato bo'lsa – orqaga
qaytariladi
async function update(id, patch) {
  const prev = notes;
  setNotes(n => n.map(x => (x.id === id ? { ...x, ...patch } : x)));
  try {
    const saved = await notesApi.update(id, patch);
    setNotes(n => {
      const next = n.map(x => (x.id === id ? saved : x));
      // arxiv holati o'zgargan bo'lsa – joriy ro'yxatdan chiqaramiz
      return sortNotes(next.filter(x => x.archived === archived));
    });
    if ('tags' in patch || 'archived' in patch) loadTags();
  } catch (err) {
    setNotes(prev);
    alert(`Saqlanmadi: ${err.message}`);
  }
}

async function remove(note) {
  if (!confirm(`"${note.title || 'Sarlavhasiz'}" eslatmasi o'chirilsinmi?`))
return;
  const prev = notes;
  setNotes(n => n.filter(x => x.id !== note.id));
  try {
    await notesApi.remove(note.id);
    loadTags();
  } catch (err) {
    setNotes(prev);
    alert(err.message);
  }
}

```



```

async function saveEdit(data) {
  await update(editing.id, data);
  setEditing(null);
}

return (
  <div className="layout">
    <header className="topbar">
      <h1>📖 Eslatmalar</h1>
      <input type="search" placeholder="Qidirish..." value={q} onChange={e =>
setQ(e.target.value)} />
      <span className="user">{user.name}</span>
      <button onClick={logout}>Chiqish</button>
    </header>

    <aside className="sidebar">
      <NavLink to="/" end>📖 Eslatmalar</NavLink>
      <NavLink to="/archive">📁 Arxiv</NavLink>
      <h4>Teglar</h4>
      {tags.length === 0 && <small>Teg yo'q</small>}
      {tags.map(t => (
        <button key={t.name} className={t.name === tag ? 'active' : ''}
onClick={() => setTag(t.name === tag ? null : t.name)}>
          #{t.name} <small>{t.count}</small>
        </button>
      ))}
    </aside>

    <main className="main">
      {!archived && <NoteEditor onSave={create} />}

      {tag && <p className="filter-info">Filtr: <b>#{tag}</b> <button
onClick={() => setTag(null)}>x</button></p>}
      {status === 'loading' && notes.length === 0 && <p
className="muted">Yuklanmoqda...</p>}
      {status === 'error' && <p className="error">Eslatmalarni yuklab
bo'lmadi. Sahifani yangilang.</p>}
      {status === 'ready' && notes.length === 0 && (
        <p className="muted">{query || tag ? 'Hech narsa topilmadi.' :
archived ? 'Arxiv bo\'sh.' : 'Birinchi eslatmangizni yozing 🖋️'}</p>
      )}

      <section className="grid">
        {notes.map(note => (

```

```

        <NoteCard key={note.id} note={note} onEdit={setEditing} onUpdate=
{update} onDelete={remove} onTagClick={setTag} />
      )}
    </section>
  </main>

  {editing && (
    <dialog open className="modal" onKeyDown={e => e.key === 'Escape' &&
setEditing(null)}>
      <NoteEditor initial={editing} onSave={saveEdit} onCancel={() =>
setEditing(null)} />
    </dialog>
  )}
</div>
);
}

```

```

:root { --bg: #f8fafc; --card: #fff; --text: #0f172a; --muted: #64748b; --
border: #e2e8f0; --primary: #4f46e5; }
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: var(--bg);
color: var(--text); }
button { font: inherit; cursor: pointer; border: 1px solid var(--border);
background: var(--card); border-radius: 8px; padding: 6px 10px; }
button.primary { background: var(--primary); color: #fff; border-color: var(--
primary); }
input, textarea { font: inherit; width: 100%; padding: 10px; border: 1px solid
var(--border); border-radius: 8px; background: transparent; }
.error { color: #b91c1c; } .muted { color: var(--muted); } .center { display:
grid; place-items: center; height: 100vh; }

/* Auth */
.auth { min-height: 100vh; display: grid; place-items: center; }
.auth-card { width: min(380px, 92vw); display: grid; gap: 12px; background:
var(--card); padding: 28px; border-radius: 16px; box-shadow: 0 10px 40px rgb(0
0 0 / .06); }
.tabs { display: flex; gap: 6px; } .tabs button { flex: 1; } .tabs .active {
border-color: var(--primary); color: var(--primary); }

/* Layout */
.layout { display: grid; grid-template: "top top" auto "side main" 1fr / 220px
1fr; min-height: 100vh; }
.topbar { grid-area: top; display: flex; gap: 12px; align-items: center;
padding: 10px 20px; background: var(--card); border-bottom: 1px solid var(--
border); position: sticky; top: 0; z-index: 5; }
.topbar h1 { font-size: 1.2rem; margin: 0; white-space: nowrap; } .topbar input
{ max-width: 480px; }
.user { margin-left: auto; color: var(--muted); white-space: nowrap; }
.sidebar { grid-area: side; display: flex; flex-direction: column; gap: 4px;
padding: 16px; }
.sidebar a { padding: 8px 10px; border-radius: 8px; color: inherit; text-
decoration: none; }
.sidebar a.active, .sidebar button.active { background: #eef2ff; color: var(--
primary); }
.sidebar button { text-align: left; border: 0; background: none; display: flex;
justify-content: space-between; }
.main { grid-area: main; padding: 20px; }

/* Notes */

```

```

.editor { display: grid; gap: 8px; max-width: 600px; margin: 0 auto 24px;
padding: 14px; border-radius: 14px; border: 1px solid var(--border); box-
shadow: 0 2px 10px rgb(0 0 0 / .04); }
.editor input, .editor textarea { border: 0; } .editor textarea { resize:
vertical; }
.editor-bar { display: flex; gap: 8px; align-items: center; } .colors {
display: flex; gap: 4px; margin-right: auto; }
.swatch { width: 24px; height: 24px; border-radius: 50%; padding: 0; }
.swatch.selected { outline: 2px solid var(--text); }
.grid { columns: 260px; column-gap: 16px; } /* "masonry" uslubidagi
ustunlar */
.note { break-inside: avoid; margin-bottom: 16px; padding: 14px; border-radius:
14px; border: 1px solid var(--border); position: relative; }
.note h3 { margin: 0 0 6px; padding-right: 28px; } .note p { white-space: pre-
wrap; margin: 0; max-height: 240px; overflow: hidden; }
.note-content { cursor: pointer; }
.pin { position: absolute; top: 8px; right: 8px; border: 0; background: none; }
.note footer { display: flex; justify-content: space-between; align-items:
center; margin-top: 10px; color: var(--muted); }
.note footer button { border: 0; background: none; opacity: .6; } .note:hover
footer button { opacity: 1; }
.tags { display: flex; flex-wrap: wrap; gap: 4px; margin-top: 8px; }
.tag { font-size: 12px; padding: 2px 8px; border-radius: 999px; background:
rgb(0 0 0 / .06); border: 0; }
.note-default { background: var(--card); } .note-yellow { background: #fef9c3;
} .note-green { background: #dcfce7; }
.note-blue { background: #dbeafe; } .note-pink { background: #fce7f3; } .note-
purple { background: #ede9fe; }
.modal { position: fixed; inset: 0; margin: auto; border: 0; border-radius:
16px; padding: 0; width: min(640px, 94vw); box-shadow: 0 20px 80px rgb(0 0 0 /
.3); }
.modal .editor { margin: 0; border: 0; }
@media (max-width: 760px) {
  .layout { grid-template: "top" auto "side" auto "main" 1fr / 1fr; }
  .sidebar { flex-direction: row; overflow-x: auto; } .sidebar h4 { display:
none; }
  .topbar { flex-wrap: wrap; } .topbar input { order: 3; max-width: none; }
}

```

5-bosqich: Ishga tushirish va tekshirish

Terminal

```
# 1-terminal — baza (agar ishlayotgan bo'lsa)
docker start notes-db

# 2-terminal — backend
cd notes-app/server && npm run dev

# 3-terminal — frontend
cd notes-app/client && npm run dev
# → http://localhost:5173
```

Qo'lda test ssenariysi

1. Ro'yxatdan o'ting → eslatmalar sahifasiga avtomatik o'tasiz.
2. Teglar bilan 3–4 eslatma yarating; bittasini mahkamlang — u tepaga chiqadi.
3. Qidiruvga yozing — 300ms dan keyin natija filtrlanadi (Network panelida so'rovlar sonini kuzating).
4. Teg ustiga bosing — filtr; arxivlang — "Arxiv" bo'limida paydo bo'ladi.
5. Sahifani yangilang — tizimda qolasiz (httpOnly cookie). DevTools → Application → Cookies: `notes_token` bor, lekin konsolda `document.cookie` uni ko'rsatmaydi.
6. Boshqa brauzerda ikkinchi foydalanuvchi bilan kiring — birinchisining eslatmalari ko'rinmaydi.
7. Backend'ni o'chirib, eslatmani mahkamlashga urining — optimistik o'zgarish orqaga qaytadi va xato chiqadi.

6-bosqich: Production uchun yig'ish

Production'da ikkita alohida server shart emas: Express React'ning build qilingan statik fayllarini o'zi bera oladi.

Terminal

```
cd client && npm run build # → client/dist/ (minifikatsiya qilingan HTML, JS, CSS)
```

server/src/app.js (qo'shimcha, errorHandler'dan oldin)

```
import path from 'node:path';

const dist = path.resolve(import.meta.dirname, '../..../client/dist');
app.use(express.static(dist, { maxAge: '1y', index: false }));
// SPA: /api'dan tashqari barcha GET so'rovlarga index.html (React Router o'zi
// hal qiladi)
app.get(/^(!\s/api).*/ (req, res) => res.sendFile(path.join(dist,
'index.html')));
```

Endi `NODE_ENV=production npm start` — bitta port, bitta origin, bitta deploy. Keyingi darsda uni internetga chiqaramiz.

Loyihani kengaytirish g'oyalari

Keyingi qadamlar (portfolio uchun)

1. **Markdown:** eslatma matnini `marked` + `DOMPurify` bilan formatlangan ko'rsatish (XSS'dan himoyani unutmang!).
2. **Sahifalash:** 500 ta limit o'rniga kursor asosida (`?before=updatedAt`) cheksiz scroll.
3. **Ulashish:** eslatmani boshqa foydalanuvchi bilan "faqat o'qish" huquqi bilan bo'lishish (`note_shares` jadvali, N:M).
4. **Rasm biriktirish:** multer + S3 (9-dars, 3-misol).
5. **Real vaqt:** bir xil akkaunt ikki qurilmada ochiq bo'lsa, o'zgarish WebSocket (socket.io) orqali darhol ko'rinsin.
6. **Testlar:** API uchun supertest, komponentlar uchun Vitest + Testing Library (14-dars).
7. **TypeScript:** server va klientga turlarni qo'shing — umumiy `Note` turi ikkala tomonda.

Xulosa

- Avval loyihalash: talablar, arxitektura, API shartnomasi, sxema — keyin kod.
- Backend: config → db → middleware (auth, validate, errors) → routes; har bir so'rovda egalik (`user_id = $2`).

- Frontend: api qatlami → AuthContext → himoyalangan marshrutlar → sahifalar → komponentlar.
- Vite proksi ishlab chiqishda, Express static production'da — bitta origin, oddiy cookie autentifikatsiyasi.
- Optimistik yangilash, debounce, AbortController — tez va ishonchli interfeys uchun.



Deploy, xavfsizlik va testlash

Ilovani internetga chiqarish (PaaS, VPS, Docker, Nginx, HTTPS), CI/CD, OWASP asosiy zaifliklari (XSS, SQLi, CSRF), unumdorlik hamda avtomatik testlar (node:test, Supertest, Vitest).

Yuqori · ~150 daqiqa

Kompyuteringizda ishlaydigan ilova hali mahsulot emas. Uni haqiqiy foydalanuvchilarga yetkazish uchun: (1) internetdagi serverga **joylash (deploy)**, (2) hujumlardan **himoyalash**, (3) har bir o'zgarishdan keyin hech narsa buzilmaganini **avtomatik tekshirish** kerak. Bu dars full-stack dasturchini "kod yozuvchi"dan "mahsulot yetkazuvchi"ga aylantiradi.

Deploy: muhitlar va asosiy tushunchalar

- **Development** — sizning kompyuteringiz: batafsil xatolar, avtomatik qayta yuklash.
- **Staging** — production'ning nusxasi, sinov uchun.
- **Production** — haqiqiy foydalanuvchilar: minifikatsiya, kesh, xatolar yashirilgan, HTTPS majburiy.

12-Factor App tamoyillarining eng muhimlari: konfiguratsiya muhit o'zgaruvchilarida; ilova holatsiz (fayllar va sessiyalar diskda emas — baza/S3/Redis'da); loglar `stdout` ga; build va ishga tushirish bosqichlari ajratilgan; dev va prod iloji boricha o'xshash.

Hosting turlari

Tur	Misollar	Afzalligi	Kamchiligi	Qachon
Statik hosting	GitHub Pages, Netlify, Vercel, Cloudflare Pages	Bepul, tez (CDN), sozlash shart emas	Faqat frontend	Portfolio, landing, React SPA

PaaS	Render, Railway, Fly.io, Heroku	Git push → deploy; baza, HTTPS tayyor	Qimmatroq, kamroq nazorat	Birinchi loyihalar, startaplar
VPS	DigitalOcean, Hetzner, AWS EC2, mahalliy provayderlar	To'liq nazorat, arzon	Hammasini o'zingiz sozlaysiz va himoyalaysiz	O'rganish, maxsus talablar
Konteyner platformalar	AWS ECS, Google Cloud Run, Kubernetes	Masshtablash	Murakkab	Katta tizimlar

Docker asoslari

Docker ilovani barcha bog'liqliklari (Node versiyasi, paketlar, tizim kutubxonalari) bilan birga **konteynerga** o'raydi. "Mening kompyuterimda ishlayapti" muammosi yo'qoladi: konteyner har qayerda bir xil ishlaydi.

- **Image** — ilovaning "qolipi" (faqat o'qish uchun), `Dockerfile` asosida quriladi.
- **Container** — image'ning ishlab turgan nusxasi.
- **Volume** — konteyner o'chsa ham saqlanadigan ma'lumot (baza fayllari).
- **Docker Compose** — bir nechta konteynerni (API + baza) bitta faylda tasvirlash.

Terminal

```
docker build -t notes-app .
docker run -d -p 3000:3000 --env-file .env --name notes notes-app
docker ps                # ishlayotgan konteynerlar
docker logs -f notes     # loglar
docker exec -it notes sh  # konteyner ichiga kirish
docker stop notes && docker rm notes
```

Nginx, PM2 va HTTPS (VPS uchun)

VPS'da odatda Node.js ilovasi to'g'ridan-to'g'ri 80/443-portga qo'yilmaydi. Uning oldida **Nginx** turadi — **teskari proksi (reverse proxy)**:



/etc/nginx/sites-available/notes

```
server {
    server_name notes.example.uz;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade;           # WebSocket uchun
        proxy_set_header Connection "upgrade";
    }

    client_max_body_size 5m;
    gzip on;
    gzip_types text/css application/javascript application/json image/svg+xml;
}
```

Terminal

```
# PM2 — Node jarayonlarini boshqaruvchi: yiqilsa qayta ishga tushiradi, server
qayta yuklansa ham
npm install -g pm2
pm2 start src/server.js --name notes -i max      # CPU yadrolari soniga teng
nusxa (cluster)
pm2 save && pm2 startup                          # tizim ishga tushganda
avtomatik
pm2 logs notes
pm2 reload notes                                # to'xtamasdan (zero-downtime)
qayta yuklash

# Bepul HTTPS sertifikat (Let's Encrypt) — avtomatik yangilanadi
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d notes.example.uz
```



VPS xavfsizligi minimum

SSH'da parol bilan kirishni o'chiring (faqat kalit), root bilan kirishni taqiqlang, firewall (`ufw allow OpenSSH; ufw allow 'Nginx Full'; ufw enable`), avtomatik xavfsizlik yangilanishlari, PostgreSQL portini (5432) tashqariga ochmang.

CI/CD — avtomatik tekshirish va deploy

CI (Continuous Integration) — har bir push/PR'da testlar va linter avtomatik ishlaydi. **CD (Continuous Delivery/Deployment)** — `main` ga merge qilingan kod avtomatik deploy qilinadi. GitHub'da buning uchun **GitHub Actions** bor.

.github/workflows/ci.yml

```
name: CI
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres: # testlar uchun haqiqiy baza
        image: postgres:17
        env:
          POSTGRES_PASSWORD: test
          POSTGRES_DB: notes_test
        ports: ['5432:5432']
        options: >-
          --health-cmd pg_isready --health-interval 5s --health-timeout 5s --
health-retries 10
        env:
          DATABASE_URL: postgres://postgres:test@localhost:5432/notes_test
          JWT_SECRET: test-secret-test-secret-test-secret
          NODE_ENV: test
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
          cache-dependency-path: server/package-lock.json
      - run: npm ci
        working-directory: server
      - run: psql "$DATABASE_URL" -f db/schema.sql
        working-directory: server
      - run: npm test
        working-directory: server
```

GitHub'da **Settings** → **Branches** → **Branch protection** orqali "CI yashil bo'lmasa merge qilib bo'lmaydi" qoidasini yoqing. Render/Railway kabi platformalar esa `main` yangilanganda o'zi avtomatik deploy qiladi.

Veb-xavfsizlik: OWASP Top 10 asoslari

OWASP — veb-xavfsizlik bo'yicha xalqaro tashkilot; uning "Top 10" ro'yxati eng xavfli zaifliklarni sanab o'tadi. Ularning ko'pchiligini oldingi darslarda uchratdik — endi tizimlashtiramiz.

Zaiflik	Mohiyati	Himoya	Dars
Broken Access Control (IDOR)	Boshqa birovning ma'lumotiga kirish	Har so'rovda egalik/ruxsat tekshiruvi	11, 13
Injection (SQLi)	Ma'lumot kod sifatida bajariladi	Parametrlangan so'rovlar, ORM	10
XSS	Hujumchi skripti boshqa foydalanuvchi brauzerida	<code>textContent</code> , React ekranlashi, DOMPurify, CSP	5, 12
CSRF	Boshqa sayt sizning cookie'ingiz bilan so'rov yuboradi	<code>SameSite</code> cookie, CSRF token, GET'da o'zgartirmaslik	11
Cryptographic Failures	Ochiq parollar, HTTP	bcrypt/argon2, HTTPS, maxfiy kalitlar <code>.env</code> da	8, 11
Security Misconfiguration	Stack trace'lar ko'rinadi, standart parollar, ochiq portlar	Production'da xatolarni yashirish, <code>helmet</code> , firewall	9
Vulnerable Components	Eskirgan paketlardagi ma'lum zaifliklar	<code>npm audit</code> , Dependabot	8
Authentication Failures	Brute-force, zaif parollar	Rate limit, 2FA, umumiy xato xabarlari	11
SSRF	Server foydalanuvchi bergan URL'ga so'rov yuboradi (ichki tarmoqqa)	URL'larni whitelist bilan cheklash	—

XSS chuqurroq va Content Security Policy

JavaScript

```
// Hujum: izoh sifatida yuboriladi
const comment = `>`;

element.innerHTML = comment; // ❌ skript bajariladi
element.textContent = comment; // ✅ matn sifatida ko'rinadi
<p>{comment}</p> // ✅ React avtomatik ekranlaydi
<div dangerouslySetInnerHTML={{ __html: comment }} /> // ❌
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(markdownHtml) }} />
// ✅ tozalangan

// Yana bir yashirin XSS: javascript: havolalar
<a href={user.website}>Sayt</a> // website = "javascript:alert(1)" bo'lsa xavfli!
// → URL'ni tekshiring: faqat http(s):// bilan boshlanganlarga ruxsat
```

CSP (Content-Security-Policy) sarlavhasi — "zaxira himoya": XSS sodir bo'lsa ham, brauzer faqat ruxsat etilgan manbalardagi skriptlarni bajaradi. `helmet()` uni standart sozlamalar bilan qo'yadi:

JavaScript

```
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"], // inline <script> va boshqa
      // domenlar bloklanadi
      imgSrc: ["'self'", 'data:', 'https://cdn.example.uz'],
      connectSrc: ["'self'"],
    },
  },
}));
```

CSRF

Siz `bank.uz` ga kirgansiz (cookie bor). `evil.com` sahifasida yashirin forma `bank.uz/transfer` ga POST yuboradi — brauzer cookie'ni **avtomatik qo'shadi**. Himoya: (1) cookie'larda `SameSite=Lax` yoki `Strict`

(zamonaviy brauzerlarda Lax standart); (2) holatni o'zgartiruvchi amallar faqat POST/PUT/PATCH/DELETE; (3) qo'shimcha himoya uchun CSRF tokeni yoki `Origin` sarlavhasini tekshirish.

Xavfsizlik nazorat ro'yxati (deploy oldidan)

- ☐ HTTPS majburiy, HTTP → HTTPS yo'naltirish
- ☐ `.env` git'da yo'q, production kalitlari kuchli va noyob
- ☐ `NODE_ENV=production`, xato javoblarida stack trace yo'q
- ☐ `helmet`, CORS faqat o'z domeningiz, `express.json({ limit })`
- ☐ Login va ro'yxatdan o'tishda rate limit
- ☐ Barcha kirish ma'lumotlari validatsiya qilinadi; SQL faqat parametrlar bilan
- ☐ Har bir resursda egalik tekshiruvi
- ☐ Cookie: `HttpOnly; Secure; SameSite`
- ☐ `npm audit` toza; Dependabot yoqilgan
- ☐ Bazaning zaxira nusxasi (backup) avtomatik va tiklash sinab ko'rilgan

Unumdorlik (performance)

Qatlam	Usullar
Frontend	Build (minifikatsiya, tree-shaking), kod bo'lish (<code>React.lazy</code>), rasmlarni siqish (WebP/AVIF, <code>loading="lazy"</code> , o'lcham), shriftlarni kamaytirish, CDN
Tarmoq	gzip/brotli, HTTP/2, statik fayllarga uzoq <code>Cache-Control</code> (fayl nomida xesh bilan)
Backend	Event loop'ni bloklamaslik, ulanishlar pool'i, sahifalash, keraksiz ma'lumotni yubormaslik
Baza	Indeksklar, <code>EXPLAIN ANALYZE</code> , N+1 so'rovlardan qochish, kesh (Redis)

O'lchash: Chrome DevTools → **Lighthouse** va **Performance** panellari; asosiy ko'rsatkichlar — **Core Web Vitals**: LCP (asosiy mazmun qancha tez ko'rinadi, < 2.5s), INP (bosishga javob tezligi, < 200ms), CLS (sahifa "sakrashi", < 0.1).



N+1 muammosi

10 ta postni olib, keyin har biri uchun muallifni alohida so'rash = 1 + 10 so'rov. 1000 ta post bo'lsa — 1001 so'rov. Yechim: bitta `JOIN` yoki `WHERE id = ANY($1)` bilan hammasini bir so'rovda olish.

Testlash

Qo'lda tekshirish har safar 10 daqiqa oladi va bir narsani unutasiz. Avtomatik testlar sekundlarda ishlaydi va o'zgarishdan keyin "eski funksiyalar buzilmadimi?" degan savolga ishonchli javob beradi. Ular, shuningdek, kodni qo'rqmasdan qayta yozish (refactoring) imkonini beradi.

Test piramidasi

- **Unit testlar** (ko'p, tez) — alohida funksiyalar: `hisobla(savat)`, reducer, validatsiya sxemasi.
- **Integratsion testlar** (o'rtacha) — bir necha qism birga: API marshruti + baza.
- **E2E testlar** (kam, sekin) — haqiqiy brauzerda foydalanuvchi ssenariysi: Playwright, Cypress.

Yaxshi test **AAA** tuzilishiga ega: **Arrange** (tayyorlash) → **Act** (bajarish) → **Assert** (tekshirish). U amalga oshirishni emas, **xatti-harakatni** tekshiradi.

Amaliy misollar

MISOL 1

Unit testlar: Node.js'ning o'rnatilgan test runner'i

Node 20+ da qo'shimcha kutubxonasiz test yozish mumkin. 4-darsdagi savat funksiyasini test qilamiz.

src/cart.js

```
export const BEPUL_YETKAZISH = 1_000_000;
export const YETKAZISH_NARXI = 25_000;

export function hisobla(items, { promo } = {}) {
  if (!Array.isArray(items)) throw new TypeError('items massiv bo\'lishi kerak');
  const oraliq = items.reduce((s, { narx, soni, chegirma = 0 }) => {
    if (soni < 0 || narx < 0) throw new RangeError('Manfiy qiymat');
    return s + Math.round(narx * (1 - chegirma)) * soni;
  }, 0);
  const promodan = promo === 'BAH0R10' ? Math.round(oraliq * 0.9) :
oraliq;
  const yetkazish = promodan === 0 || promodan ≥ BEPUL_YETKAZISH ? 0 :
YETKAZISH_NARXI;
  return { oraliq, yetkazish, jami: promodan + yetkazish };
}
```

```

import { describe, it } from 'node:test';
import assert from 'node:assert/strict';
import { hisobla, YETKAZISH_NARXI } from '../src/cart.js';

describe('hisobla()', () => {
  it('bo\'sh savat – hammasi nol', () => {
    assert.deepEqual(hisobla([]), { oraliq: 0, yetkazish: 0, jami: 0 });
  });

  it('kichik xaridga yetkazish narxi qo\'shiladi', () => {
    // Arrange
    const items = [{ narx: 100_000, soni: 2 }];
    // Act
    const r = hisobla(items);
    // Assert
    assert.equal(r.oralik, 200_000);
    assert.equal(r.yetkazish, YETKAZISH_NARXI);
    assert.equal(r.jami, 225_000);
  });

  it('1 000 000 dan oshsa yetkazish bepul (chegara qiymati)', () => {
    assert.equal(hisobla([{ narx: 1_000_000, soni: 1 }]).yetkazish, 0);
    assert.equal(hisobla([{ narx: 999_999, soni: 1 }]).yetkazish,
YETKAZISH_NARXI);
  });

  it('mahsulot chegirmasi va promokod birga qo\'llanadi', () => {
    const r = hisobla([{ narx: 500_000, soni: 2, chegirma: 0.1 }], {
promo: 'BAHOR10' });
    assert.equal(r.oralik, 900_000);
    assert.equal(r.jami, 810_000 + YETKAZISH_NARXI);
  });

  it(' noto\'g\'ri kirishda xato tashlaydi', () => {
    assert.throws(() => hisobla(null), TypeError);
    assert.throws(() => hisobla([{ narx: 100, soni: -1 }]), /Manfiy/);
  });
});

```

Terminal

```
node --test
# ▶ hisobla()
#   ✓ bo'sh savat — hammasi nol (0.4ms)
#   ✓ kichik xaridga yetkazish narxi qo'shiladi (0.1ms)
#   ✓ 1 000 000 dan oshsa yetkazish bepul (chegara qiymati) (0.1ms)
#   ✓ mahsulot chegirmasi va promokod birga qo'llanadi (0.1ms)
#   ✓ noto'g'ri kirishda xato tashlaydi (0.2ms)
# i tests 5 pass 5 fail 0

node --test --watch # fayl o'zgarganda qayta
node --test --experimental-test-coverage # kod qamrovi
```

E'tibor bering: **chegara qiymatlari** (999 999 va 1 000 000), bo'sh kirish va xato holatlari alohida tekshirildi — xatolar aynan shu yerlarda yashirinadi.

MISOL 2 API integratsion testlari: Supertest + haqiqiy baza

13-darsdagi Notes API'ni test qilamiz. `app.js` va `server.js` alohida bo'lgani uchun (9-dars) testda port band qilinmaydi.

Terminal

```
npm install -D supertest
npm pkg set scripts.test="node --env-file=.env.test --test"
```

```

import { describe, it, before, after, beforeEach } from 'node:test';
import assert from 'node:assert/strict';
import request from 'supertest';
import { app } from '../src/app.js';
import { pool } from '../src/db.js';

// Har bir test toza bazada ishlaydi
beforeEach(() => pool.query('TRUNCATE users, notes CASCADE'));
after(() => pool.end());

// Yordamchi: ro'yxatdan o'tib, cookie'li "agent" qaytaradi (brauzerga
o'xshab cookie saqlaydi)
async function signup(email = 'ali@test.uz') {
  const agent = request.agent(app);
  await agent.post('/api/auth/register')
    .send({ name: 'Ali', email, password: 'parol12345' })
    .expect(201);
  return agent;
}

describe('Auth', () => {
  it('ro'yxatdan o'tish httpOnly cookie qaytaradi', async () => {
    const res = await request(app).post('/api/auth/register')
      .send({ name: 'Vali', email: 'vali@test.uz', password: 'parol12345'
    })
      .expect(201);
    assert.equal(res.body.user.email, 'vali@test.uz');
    assert.equal(res.body.user.password_hash, undefined); // xesh
    hech qachon qaytmasin!
    assert.match(res.headers['set-cookie'][0], /HttpOnly/);
  });

  it('takroriy email - 409', async () => {
    await signup('a@test.uz');
    await request(app).post('/api/auth/register')
      .send({ name: 'Boshqa', email: 'A@TEST.UZ', password: 'parol12345'
    }) // katta harf ham!
      .expect(409);
  });

  it('noto'lg'ri parol - 401 va umumiy xabar', async () => {
    await signup('b@test.uz');

```

```

const res = await request(app).post('/api/auth/login')
  .send({ email: 'b@test.uz', password: 'xato-parol' })
  .expect(401);
assert.equal(res.body.error.message, 'Email yoki parol noto\'g\'ri');
});

describe('Notes', () => {
  it('cookie\'siz - 401', async () => {
    await request(app).get('/api/notes').expect(401);
  });

  it('CRUD to\'liq sikli', async () => {
    const agent = await signup();

    const { body: created } = await agent.post('/api/notes')
      .send({ title: 'Test', body: 'Matn', tags: ['Ish', 'ish'] })
      .expect(201);
    assert.deepEqual(created.tags, ['ish']); //
    kichik harf + takrorsiz

    const { body: list } = await agent.get('/api/notes?
q=matn').expect(200);
    assert.equal(list.data.length, 1);

    const { body: updated } = await
agent.patch(`/api/notes/${created.id}`)
      .send({ pinned: true }).expect(200);
    assert.equal(updated.pinned, true);

    await agent.delete(`/api/notes/${created.id}`).expect(204);
    await agent.get('/api/notes').expect(200).then(r =>
assert.equal(r.body.data.length, 0));
  });

  it('🔒 boshqa foydalanuvchining eslatmasini o\'zgartirib bo\'lmaydi
(IDOR)', async () => {
    const ali = await signup('ali@test.uz');
    const hacker = await signup('hacker@test.uz');
    const { body: note } = await ali.post('/api/notes').send({ title:
'Maxfiy' }).expect(201);

    await hacker.patch(`/api/notes/${note.id}`).send({ title: 'Buzildi'
}).expect(404);
  });

```

```

    await hacker.delete(`/api/notes/${note.id}`).expect(404);
    const { body } = await hacker.get('/api/notes').expect(200);
    assert.equal(body.data.length, 0);
  });

  it('noma\'lum maydon (user_id) qabul qilinmaydi', async () => {
    const agent = await signup();
    const { body: note } = await agent.post('/api/notes').send({ title:
'X' });
    await agent.patch(`/api/notes/${note.id}`).send({ user_id: '00000000-
0000-0000-0000-000000000000' }).expect(400);
  });

  it('validatsiya: bo\'sh eslatma - 400 va tafsilotlar', async () => {
    const agent = await signup();
    const res = await agent.post('/api/notes').send({ title: '', body: ''
}).expect(400);
    assert.ok(Array.isArray(res.body.error.details));
  });
});

```

Xavfsizlik talablari ham test sifatida yozildi: endi kimdir kelajakda `AND user_id = $2` ni tasodifan o'chirib yuborsa, CI darhol qizarib, xatoni ko'rsatadi.

MISOL 3 React komponentlarini testlash: Vitest + Testing Library

Terminal

```

cd client
npm install -D vitest jsdom @testing-library/react @testing-library/user-
event @testing-library/jest-dom

```

client/vite.config.js (qo'shimcha)

```
export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: './src/test-setup.js',
  },
});
// src/test-setup.js: import '@testing-library/jest-dom/vitest';
```

```

import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { vi } from 'vitest';
import NoteEditor from './NoteEditor';

describe('<NoteEditor />', () => {
  it('bo\'sh eslatmani saqlamaydi va xabar ko\'rsatadi', async () => {
    const onSave = vi.fn();
    render(<NoteEditor onSave={onSave} />);

    await userEvent.click(screen.getByRole('button', { name: 'Saqlash'
}));

    expect(onSave).not.toHaveBeenCalled();
    expect(screen.getByText('Sarlavha yoki matn
yozing')).toBeInTheDocument();
  });

  it('teglarni ajratib, tozalab yuboradi va formani tozalaydi', async ()
=> {
    const onSave = vi.fn().mockResolvedValue();
    const user = userEvent.setup();
    render(<NoteEditor onSave={onSave} />);

    await user.type(screen.getByPlaceholderText('Sarlavha'), 'Xaridlar');
    await user.type(screen.getByPlaceholderText(/Teglar/), ' Uy , bozor,,
');
    await user.click(screen.getByRole('radio', { name: 'green' }));
    await user.click(screen.getByRole('button', { name: 'Saqlash' }));

    expect(onSave).toHaveBeenCalledWith({ title: 'Xaridlar', body: '',
color: 'green', tags: ['uy', 'bozor'] });
    expect(screen.getByPlaceholderText('Sarlavha')).toHaveValue('');
  });

  it('server xatosini ko\'rsatadi', async () => {
    const onSave = vi.fn().mockRejectedValue(new Error('Tarmoq xatosi'));
    render(<NoteEditor onSave={onSave} initial={{ title: 'A', body: '',
tags: [] }} />);
    await userEvent.click(screen.getByRole('button', { name: 'Saqlash'
}));
    expect(await screen.findByText('Tarmoq xatosi')).toBeInTheDocument();
  });
});

```



```
});  
});
```

Terminal

```
npx vitest          # watch rejimi  
npx vitest run      # bir marta (CI uchun)
```

Testing Library falsafasi: komponentni foydalanuvchi ko'rgandek test qiling — CSS sinflari yoki ichki state orqali emas, **rol**, **yozuv va matn** orqali (`getByRole('button', { name: 'Saqlash' })`). Shu tufayli testlar accessibility'ni ham bilvosita tekshiradi.

MISOL 4

Docker Compose bilan deploy va Render'ga joylash

A) Ko'p bosqichli Dockerfile (frontend + backend bitta image'da)

notes-app/Dockerfile

```
# 1-bosqich: React'ni build qilish
FROM node:22-alpine AS client
WORKDIR /app/client
COPY client/package*.json ./
RUN npm ci
COPY client/ ./
RUN npm run build

# 2-bosqich: faqat production bog'liqliklari bilan server
FROM node:22-alpine
ENV NODE_ENV=production
WORKDIR /app/server
COPY server/package*.json ./
RUN npm ci --omit=dev
COPY server/ ./
COPY --from=client /app/client/dist /app/client/dist
# root emas - xavfsizroq
USER node
EXPOSE 3000
HEALTHCHECK CMD wget -q0- http://localhost:3000/api/health || exit 1
CMD ["node", "src/server.js"]
```

notes-app/.dockerignore

```
**/node_modules
**/.env
**/dist
.git
```

```

services:
  db:
    image: postgres:17
    restart: unless-stopped
    environment:
      POSTGRES_DB: notes
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./server/db/schema.sql:/docker-entrypoint-initdb.d/schema.sql:ro
# birinchi ishga tushishda
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 10

  app:
    build: .
    restart: unless-stopped
    depends_on:
      db:
        condition: service_healthy
    environment:
      DATABASE_URL: postgres://postgres:${DB_PASSWORD}@db:5432/notes #
      JWT_SECRET: ${JWT_SECRET}
      PORT: 3000
    ports:
      - "3000:3000"

volumes:
  pgdata:

```

Terminal

```
# .env (compose uchun): DB_PASSWORD=... JWT_SECRET=...
docker compose up -d --build
docker compose logs -f app
docker compose exec db psql -U postgres notes -c "SELECT count(*) FROM
notes;"

# Zaxira nusxa
docker compose exec db pg_dump -U postgres notes > backup-$(date +%F).sql
```

VPS'da shu buyruqlarni ishga tushirib, oldiga Nginx + certbot qo'ysangiz — ilova tayyor.

B) Render.com'ga joylash (PaaS — eng oson yo'l)

1. Loyihani GitHub'ga push qiling (7-dars).
2. Render'da **New → PostgreSQL** — bepul baza yarating, "Internal Database URL" ni nusxalang. `schema.sql` ni `psql` orqali bir marta ishga tushiring.
3. **New → Web Service** → repozitoriyani tanlang → Runtime: **Docker** (yuqoridagi Dockerfile ishlatiladi).
4. **Environment:** `DATABASE_URL`, `JWT_SECRET` (yangi, kuchli kalit), `NODE_ENV=production`.
5. **Health Check Path:** `/api/health`. Deploy!
6. Bir necha daqiqadan so'ng ilova `https://notes-xxxx.onrender.com` da HTTPS bilan ishlaydi. Har bir `git push` — avtomatik qayta deploy.
7. O'z domeningiz: Render → Settings → Custom Domain, domen DNS'ida `CNAME` yozuvi (1-dars).



Deploy'dan keyin

Xatolarni kuzatish (Sentry), uptime monitoring (UptimeRobot — sayt o'chsa xabar beradi), loglarni ko'rish va bazaning muntazam zaxira nusxalari — bularsiz production ilova "ko'r" holda ishlaydi.

Mashqlar



Yakuniy topshiriqlar

1. 13-darsdagi loyihani Docker Compose bilan ishga tushiring, so'ng Render (yoki Railway) ga deploy qiling va havolani portfolio'ngizga qo'shing.
2. Loyihaga GitHub Actions CI qo'shing: lint + server testlari + client testlari. Ataylab xato commit qilib, CI qizarishini ko'ring.
3. Lighthouse auditidan o'tkazing va barcha 4 toifada 90+ ballga erishing.
4. OWASP nazorat ro'yxati bo'yicha o'z ilovangizni tekshiring; topilgan har bir muammo uchun test yozing va tuzating.
5. Playwright bilan bitta E2E test yozing: ro'yxatdan o'tish → eslatma yaratish → qidirish → chiqish.



Qo'llanma yakuni

Tabriklaymiz! Siz web qanday ishlashidan boshlab, HTML/CSS/JavaScript, Node.js, Express, PostgreSQL, autentifikatsiya, React, deploy, xavfsizlik va testlashgacha bo'lgan to'liq yo'lni bosib o'tdingiz. Bu — junior full-stack dasturchidan kutiladigan bilimlar poydevori.

Keyingi qadamlar:

- **TypeScript** — katta loyihalarda xatolarni kod yozish paytidayoq ushlash.
- **Next.js** — React uchun full-stack freymvork (server rendering, API marshrutlari).
- **Tailwind CSS** yoki komponent kutubxonalari — tezroq dizayn.
- **Redis, WebSocket, navbatlar** — real vaqt va yuqori yuklama.
- Eng muhimi: **loyihalar qiling**. Har bir yangi loyiha — qo'llanmadagi bilimlarning chuqurlashuvi. 3–5 ta sifatli, deploy qilingan va GitHub'da toza kodli loyiha — eng yaxshi rezyume.